

**Hardware and Software**  
**Engineered to Work Together**



# Oracle Database 12c: New Features for Administrators

Student Guide – Volume I

D77758GC20

Edition 2.0 | November 2014 | D87748

Learn more from Oracle University at [oracle.com/education/](http://oracle.com/education/)

## Authors

Dominique Jeunot  
Jean-François Verrier

## Technical Contributors and Reviewers

Andy Rivenes  
James Spiller  
Donna Keesling  
Maria Billings  
Lachlan Williams  
Peter Fusek  
Mark Fuller  
Gregg Christman  
Dimpri Sarmah  
Kevin Jernigan  
Branislav Valny  
Frank Fu  
Joel Goodman  
Gerlinde Frenzen  
Harald Van Breederode  
Hermann Baer  
Jim Stenoish  
Mark Drake  
Beda Hammerschmidt  
Prabhaker Gongloor  
Patrick Wheeler  
Maria Colgan  
Jesse Kamp  
Paul Needham  
Pat Huey  
Roy F Swonger  
Ron Soltani  
Sue Lee  
Sharath Bhujani

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

## Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

### U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

## Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

## Editors

Anwasha Ray  
Malavika Jinka  
Smita Kommini

## Graphic Designers

Seema Bopaiah  
Maheshwari Krishnamurthy

## Publishers

Jobi Varghese  
Pavithran Adka  
Joseph Fernandez

# Contents

## 1 Introduction

- Overview 1-2
- Oracle Database Innovation 1-3
- Enterprise Cloud Computing 1-4
- Oracle Database 12c New and Enhanced Features 1-5

## 2 Enterprise Manager Cloud Control and Other Tools

- Oracle Database 12c New and Enhanced Features 2-2
- Objectives 2-3
- Key Challenges for Administrators 2-4
- Enterprise Manager Cloud Control 2-5
- Cloud Control Components 2-7
- Components and Communication Flow 2-8
- Oracle Management Repository 2-9
- Controlling the Enterprise Manager Cloud Control Framework 2-10
- Starting the Enterprise Manager Cloud Control Framework 2-11
- Stopping the Enterprise Manager Cloud Control Framework 2-12
- Different Target Types 2-13
- Target Discovery 2-14
- Enterprise Manager Cloud Control 2-15
- User Interface 2-16
- Security: Overview 2-17
- Managing Securely with Credentials 2-18
- Distinguishing Credentials 2-19
- Quiz 2-21
- Enterprise Manager Database Express Architecture 2-22
- Configuring EM Database Express 2-23
- Home Page 2-24
- Menus 2-25
- Quiz 2-26
- Database Configuration Assistant 2-27
- Oracle SQL Developer: Connections 2-28
- Oracle SQL Developer: DBA Actions 2-29
- Quiz 2-30
- Summary 2-31
- Practice 2-32

### **3 Module – Multitenant Container Database and Pluggable Databases**

Basics of Multitenant Container Database and Pluggable Databases 3-2

Oracle Database 12c New and Enhanced Features 3-3

Objectives 3-4

Challenges 3-5

Oracle Database in 11g Release 2 3-6

New Multitenant Architecture: Benefits 3-7

Other Benefits of Multitenant Architecture 3-9

Configurations 3-11

Multitenant Container Database 3-12

Pristine Installation 3-13

Adding User Data 3-14

Separating SYSTEM and User Data 3-15

SYSTEM Objects in the USER Container 3-16

Naming the Containers 3-17

Provisioning a Pluggable Database 3-18

Interacting Within Multitenant Container Database 3-19

Multitenant Container Database Architecture 3-20

Containers 3-21

Questions: Root Versus PDBs 3-22

Questions: PDBs Versus Root 3-23

Terminology 3-24

Common and Local Users 3-25

Common and Local Privileges and Roles 3-26

Shared and Non-Shared Objects 3-27

Data Dictionary Views 3-28

Impacts 3-29

Quiz 3-31

Summary 3-34

Practice 3-35

### **4 Creating Container Databases and Pluggable Databases**

Oracle Database 12c New and Enhanced Features 4-2

Objectives 4-3

Goals 4-4

Tools 4-5

Steps to Create a Container Database 4-6

Creating a Container Database: Using SQL\*Plus 4-7

Creating a Container Database: Using DBCA 4-9

New Clause: SEED FILE\_NAME\_CONVERT 4-10

New Clause: ENABLE PLUGGABLE DATABASE 4-11

After CDB Creation: What Is New in CDBs 4-12



- Data Dictionary Views: DBA\_xxx 4-13
- Data Dictionary Views: CDB\_xxx 4-14
- Data Dictionary Views: Examples 4-15
- Data Dictionary Views: V\$xxx Views 4-16
- After CDB Creation: To-Do List 4-17
- Automatic Diagnostic Repository 4-18
- Automatic Diagnostic Repository: alert.log File 4-19
- Quiz 4-20
- Practice 4-22
- Provisioning New Pluggable Databases 4-23
- Tools 4-24
- Method 1: Create New PDB from PDB\$SEED 4-25
- Steps: With Location Clauses 4-26
- Steps: Without Location Clauses 4-28
- Synchronization 4-29
- Method 2: Plug a Non-CDB into a CDB 4-30
- Plug a Non-CDB in to CDB Using DBMS\_PDB 4-32
- Method 3: Clone Local PDBs 4-33
- Method 3: Clone a Non-CDB or Remote PDB 4-34
- Method 4: Plug Unplugged PDB in to CDB 4-35
- Method 4: Flow 4-36
- Plug Sample Schemas PDB: Using DBCA 4-38
- Dropping a PDB 4-39
- Migrating pre-12.1 Databases to 12.1 CDB 4-40
- Quiz 4-41
- Summary 4-43
- Practice 4-44

## **5 Managing Multitenant Container Databases and Pluggable Databases**

- Oracle Database 12c New and Enhanced Features 5-2
- Objectives 5-3
- Connection 5-4
- Connection with SQL\*Developer 5-7
- Switching Connections 5-8
- Starting Up a CDB Instance 5-9
- Mounting a CDB 5-10
- Opening a CDB 5-11
- Opening a PDB 5-12
- Closing a PDB 5-13
- Shutting Down a CDB Instance 5-14
- Automatic PDB Opening 5-15
- Changing PDB Open Mode 5-16

Changing PDB Mode: With SQL Developer 5-17  
 Modifying PDB Settings 5-18  
 Instance Parameter Change Impact 5-19  
 Instance Parameter Change Impact: Example 5-20  
 Quiz 5-21  
 Summary 5-23  
 Practice 5-24

## **6 Managing Tablespaces and Users in CDB and PDBs**

Oracle Database 12c New and Enhanced Features 6-2  
 Objectives 6-3  
 Tablespaces in PDBs 6-4  
 Creating Permanent Tablespaces in a CDB 6-5  
 Assigning Default Tablespaces 6-6  
 Creating Local Temporary Tablespaces 6-7  
 Assigning Default Temporary Tablespaces 6-8  
 Users, Roles, and Privileges 6-9  
 Local Users, Roles, and Privileges 6-10  
 Creating a Local User 6-11  
 Common Users 6-12  
 Creating Users 6-13  
 Common and Local Schemas/Users 6-14  
 Common and Local Privileges 6-15  
 Granting and Revoking Privileges 6-16  
 Creating Common and Local Roles 6-17  
 Granting Common or Local Privileges/Roles to Roles 6-18  
 Granting Common and Local Roles to Users 6-19  
 Creating Shared and Non-Shared Objects 6-20  
 Restriction on Definer's Rights 6-21  
 Quiz 6-22  
 Summary 6-24  
 Practice 6-25

## **7 Backup, Recovery, and Flashback CDBs and PDBs**

Oracle Database 12c New and Enhanced Features 7-2  
 Objectives 7-3  
 New Syntax and Clauses in RMAN 7-5  
 CDB Backup: Whole CDB Backup 7-6  
 CDB Backup: Partial CDB Backup 7-7  
 PDB Backup: Whole PDB Backup 7-8  
 PDB Backup: Partial PDB Backup 7-9  
 PDB Backup: User-Managed Hot PDB Backup 7-10

Practice 7-11  
 Recovery 7-12  
 Instance Failure 7-13  
 NOARCHIVELOG Mode 7-14  
 Media Failure: CDB Temporary File Recovery 7-15  
 Media Failure: PDB Temporary File Recovery 7-16  
 Media Failure: Control File Loss 7-17  
 Media Failure: Redo Log File Loss 7-18  
 Media Failure: Root SYSTEM or UNDO Data File 7-19  
 Media Failure: Root SYSAUX Data File 7-20  
 Media Failure: PDB Data File 7-21  
 Media Failure: PITR 7-22  
 Flashback CDB 7-24  
 Special Situations 7-26  
 Quiz 7-27  
 Summary 7-29  
 Practice 7-30

## **8 Module – Automatic Data Optimization and Storage Enhancements**

Heat Map, Automatic Data Optimization, and Online Data File and Partition Move 8-2  
 Oracle Database 12c New and Enhanced Features 8-3  
 Objectives 8-4  
 ILM Challenges and Solutions 8-5  
 ILM Components 8-6  
 ILM Challenges 8-7  
 Solutions 8-8  
 Components 8-10  
 What Is Automatic Data Optimization? 8-12  
 Data Classification Levels 8-13  
 Heat Map and ADO 8-14  
 Enabling Heat Map Segment-Level Statistics 8-15  
 DBA\_HEAT\_MAP\_SEGMENT View 8-16  
 Block-Level Statistics 8-17  
 Extent-Level Statistics 8-18  
 Defining Automatic Detection Conditions 8-19  
 Defining Automatic Actions 8-20  
 Compression Scopes and Types 8-21  
 Creating Compression Policies Tablespace and Group 8-22  
 Creating Compression Policies Segment and Row 8-23  
 Creating Storage Tiering Policy 8-25  
 Storage Tiering: Priority 8-26  
 Storage Tiering: READ ONLY 8-27

- Policy Relying on Function 8-28
- Multiple SEGMENT Policies on a Segment 8-29
- Only One Single ROW Policy on a Segment 8-31
- Policy Inheritance 8-32
- Displaying Policies DBA\_ILMPOLICIES/DBA\_ILMDATAMOVEMENTPOLICIES 8-33
- Displaying Policies DBA\_ILMDATAMOVEMENTPOLICIES 8-34
- Preparing Evaluation and Execution 8-35
- Customizing Evaluation and Execution 8-36
- Monitoring Evaluation and Execution 8-37
- ADO DDL 8-39
- Turning ADO Off and On 8-40
- Stop Activity Tracking and Clean Up Heat Map Statistics 8-41
- Specific Situations of Activity Tracking 8-42
- Quiz 8-44
- Online Move Data File 8-46
- Compression 8-47
- REUSE and KEEP 8-48
- States 8-49
- Compatibilities 8-50
- Flashback Database 8-51
- Online Move Partition 8-52
- Online Move Partition: Benefits 8-53
- Online Move Partition: Compress 8-54
- Quiz 8-55
- Summary 8-56
- Practice 8-57

## **9 In-Database Archiving and Temporal**

- Oracle Database 12c New and Enhanced Features 9-2
- Objectives 9-3
- Archiving Challenges 9-4
- Archiving Solutions 9-5
- In-Database Archiving: HCC 9-6
- Archiving Challenges and Solutions 9-8
- In-Database Archiving 9-10
- ORA\_ARCHIVE\_STATE column 9-11
- Session Visibility Control 9-12
- Disable Row-Archival 9-13
- Quiz 9-14
- PERIOD FOR Clause Concept 9-16
- Filtering on Valid-Time Columns: Example 1 9-17
- Filtering on Valid-Time Columns: Example 2 9-18

DBMS\_FLASHBACK\_ARCHIVE 9-19  
 Quiz 9-20  
 Temporal History Enhancements: FDA Optimization 9-21  
 Temporal History Enhancements: User Context Metadata 9-22  
 Summary 9-23  
 Practice 9-24

## **10 Module – Security**

Auditing 10-2  
 Oracle Database 12c New and Enhanced Features 10-3  
 Objectives 10-4  
 Types of Auditing 10-5  
 Audit Trail Implementation 10-7  
 Oracle Database 12c Auditing 10-9  
 Security and Performance: Audit Architecture 10-10  
 Consolidation 10-11  
 Data Pump Audit Policy 10-12  
 Unified Audit Implementation 10-13  
 Quiz 10-15  
 Security 10-17  
 Simplicity: Audit Policy 10-18  
 Step 1: Creating the Audit Policy 10-19  
 Step 2: Enabling/Disabling the Audit Policy 10-21  
 Viewing the Audit Policy 10-22  
 Using Predefined Audit Policies 10-23  
 Including Application Context Data 10-24  
 Dropping the Audit Policy 10-25  
 Audit Cleanup 10-26  
 Quiz 10-27  
 Summary 10-28  
 Practice 10-29

## **11 Privileges**

Oracle Database 12c New and Enhanced Features 11-2  
 Objectives 11-3  
 Major Challenges 11-4  
 New Administrative Privileges 11-5  
 OS Authentication and OS Groups 11-6  
 Password Authentication for SYSBACKUP 11-8  
 Oracle Database Vault Data Protection and Administration Privileged Users 11-10  
 Quiz 11-11  
 New System Privilege: PURGE DBA\_RECYCLEBIN 11-13

|                                            |       |
|--------------------------------------------|-------|
| Privilege Analysis                         | 11-14 |
| Privilege Analysis Flow                    | 11-15 |
| Used Privileges Results                    | 11-17 |
| Compare Used and Unused Privileges         | 11-18 |
| Listing Captures                           | 11-19 |
| Dropping an Analysis                       | 11-20 |
| Quiz                                       | 11-21 |
| Privilege Checking During PL/SQL Calls     | 11-22 |
| New Privilege Checking During PL/SQL Calls | 11-23 |
| INHERIT (ANY) PRIVILEGES Privileges        | 11-24 |
| Privilege Checking with New BEQUEATH Views | 11-25 |
| Quiz                                       | 11-26 |
| Summary                                    | 11-28 |
| Practice                                   | 11-29 |

## **12 Oracle Data Redaction**

|                                                                             |       |
|-----------------------------------------------------------------------------|-------|
| Oracle Database 12c New and Enhanced Features                               | 12-2  |
| Objectives                                                                  | 12-3  |
| Oracle Data Redaction: Overview                                             | 12-4  |
| Oracle Data Redaction and Operational Activities                            | 12-6  |
| Available Redaction Methods                                                 | 12-7  |
| Oracle Data Redaction: Examples                                             | 12-8  |
| What Is a Redaction Policy?                                                 | 12-9  |
| Managing Redaction Policies                                                 | 12-11 |
| Applying a Redaction Policy to a Table or View                              | 12-12 |
| Full Redaction: Examples                                                    | 12-13 |
| Partial Redaction: Examples                                                 | 12-14 |
| Regular Expression                                                          | 12-15 |
| Modifying the Redaction Policy                                              | 12-16 |
| Exempting Users from Redaction Policies                                     | 12-17 |
| Using Oracle Data Redaction with Other Oracle Database Security Solutions   | 12-18 |
| Oracle Database Security Features                                           | 12-19 |
| Best Practices: Preventing Unauthorized Policy Modifications and Exemptions | 12-21 |
| Best Practices: Considerations                                              | 12-22 |
| Summary                                                                     | 12-23 |
| Practice 12: Overview                                                       | 12-24 |

## **13 Module – High Availability**

|                                               |      |
|-----------------------------------------------|------|
| Recovery Manager: New Features                | 13-2 |
| Oracle Database 12c New and Enhanced Features | 13-3 |
| Objectives                                    | 13-4 |
| Separation of DBA Duties                      | 13-5 |

|                                                 |       |
|-------------------------------------------------|-------|
| Using SQL in RMAN                               | 13-6  |
| Backing Up and Restoring Very Large Files       | 13-7  |
| RMAN Duplication Enhancements                   | 13-8  |
| Duplicating an Active Database                  | 13-9  |
| What Is New?                                    | 13-10 |
| NOOPEN Option                                   | 13-11 |
| Duplicating Multitenant Container Databases     | 13-12 |
| Recovering Databases with Third-Party Snapshots | 13-13 |
| Quiz                                            | 13-14 |
| Transporting Data Across Platforms              | 13-15 |
| Data Transport                                  | 13-16 |
| Transporting Database: Process Steps - 1        | 13-17 |
| Transporting Database: Process Steps - 2        | 13-18 |
| Transporting Tablespace: Process Steps - 1      | 13-19 |
| Transporting Tablespace: Process Steps - 2      | 13-20 |
| Quiz                                            | 13-21 |
| Table Recovery                                  | 13-22 |
| Recovering Tables from Backups                  | 13-23 |
| Table Recovery: Graphical Overview              | 13-24 |
| Specifying the Recovery Point-in-Time           | 13-25 |
| Process Steps of Table Recovery - 1             | 13-26 |
| Customization                                   | 13-27 |
| Quiz                                            | 13-28 |
| Summary                                         | 13-29 |
| Practice 13: Overview                           | 13-30 |

## **14 Module – Manageability**

|                                                            |       |
|------------------------------------------------------------|-------|
| Real-Time Database Operation Monitoring                    | 14-2  |
| Oracle Database 12c New and Enhanced Features              | 14-3  |
| Objectives                                                 | 14-4  |
| Real-Time Database Operation Monitoring: Overview          | 14-5  |
| Use Cases                                                  | 14-6  |
| Current Tools                                              | 14-7  |
| Defining a DB Operation                                    | 14-8  |
| Scope of a Composite DB Operation                          | 14-9  |
| Database Operation Concepts                                | 14-10 |
| Identifying a Database Operation                           | 14-11 |
| Enabling Monitoring of Database Operations                 | 14-12 |
| Identifying, Starting, and Completing a Database Operation | 14-13 |
| Monitoring the Progress of a Database Operation            | 14-14 |
| Monitoring Load Database Operations                        | 14-15 |
| Monitoring Load Database Operation Details                 | 14-16 |



Reporting Database Operations Using Views 14-17  
 Reporting Database Operations by Using Functions 14-19  
 Database Operation Tuning 14-21  
 Quiz 14-22  
 Summary 14-24  
 Practice 14: Overview 14-25

## **15 Emergency Monitoring, Real-Time ADDM, Compare Period ADDM, and ASH Analytics**

Oracle Database 12c New and Enhanced Features 15-2  
 Objectives 15-3  
 Emergency Monitoring: Challenges 15-4  
 Emergency Monitoring: Goals 15-5  
 Real-Time ADDM: Challenges 15-7  
 Real-Time ADDM: Goals 15-8  
 Flow 15-10  
 Using the DBMS\_ADDM Package 15-11  
 Quiz 15-12  
 AWR Compare Periods Report 15-13  
 Method: Preserved Snapshot Sets 15-14  
 What Is Missing? 15-15  
 Compare Period ADDM: Analysis 15-16  
 Workload Commonality 15-17  
 Comparison Modes 15-18  
 Report: Configuration 15-19  
 Report: Finding 15-21  
 Using the DBMS\_ADDM Package 15-22  
 Quiz 15-24  
 ASH: Overview 15-25  
 Top Activity Page 15-26  
 ASH Analytics Page: Activity 15-27  
 Summary 15-28  
 Practice 15: Overview 15-29

## **16 ADR and Network Enhancements**

Oracle Database 12c New and Enhanced Features 16-2  
 Objectives 16-3  
 Automatic Diagnostic Repository 16-4  
 ADR File Types 16-5  
 ADR Files: Location 16-6  
 ADR Files: DDL and DEBUG Log Files 16-7  
 New ADRCI Command 16-8  
 Network Performance: Compression 16-9

Setting Up Compression 16-10  
 Session Data Unit (SDU) Size 16-11  
 Setting SDU Size 16-12  
 Quiz 16-13  
 Summary 16-14  
 Practice 16: Overview 16-15

## **17 Module – Performance**

In-Memory Column Store 17-2  
 Oracle Database 12c New and Enhanced Features 17-3  
 Objectives 17-4  
 Goals of In-Memory Column Store 17-5  
 Benefits 17-7  
 Overview 17-8  
 Row Store Versus Column Store: 2D Vision 17-10  
 In-Memory Column Unit 17-11  
 In-Memory Column Store Cache Versus Buffer Cache 17-12  
 Dual Format In Memory 17-13  
 No More Indexes Issues 17-14  
 Process 17-15  
 Deploying IM Column Store 17-16  
 Deploying IM Column Store: Objects Setting 17-18  
 Deploying IM Column Store: Columns Setting 17-19  
 Objects Candidates for IM Column Store 17-20  
 Columns Candidates for IM Column Store 17-22  
 Defining IM Column Store Priority 17-23  
 Segments Populated into the IM Column Store 17-24  
 Defining IM Column Store Compression 17-25  
 IM Column Store Compression Advisor 17-27  
 Computing Compression Ratio 17-29  
 Default In-Memory Setting 17-30  
 Impact of Changing In-Memory Attributes 17-31  
 Moving or Splitting In-Memory Segments 17-32  
 INMEMORY Inheritance 17-33  
 After Objects Settings 17-34  
 Retrieving CREATE DDL Statement of In-Memory Objects 17-35  
 Quiz 17-36  
 Query Benefits 17-38  
 Testing and Comparing Query Performance 17-39  
 Queries on In-Memory Tables: Simple Predicate 17-40  
 MINMAX Pruning Statistics 17-41  
 IM Column Store Statistics 17-42

Execution Plan: TABLE ACCESS IN MEMORY FULL 17-43  
 Queries on In-Memory Tables: Join 17-44  
 Execution Plan: JOIN FILTER CREATE / USE 17-46  
 Queries on In-Memory and Non-In-Memory Tables 17-47  
 Queries on In-Memory and Non-In-Memory Columns 17-48  
 DMLs and In-Memory Column Store 17-49  
 Recommendations 17-50  
 Views 17-51  
 Interaction with Other Products 17-52  
 Optimizer 17-53  
 IM Column Store and RAC 17-55  
 IM Column Store and Data Pump 17-57  
 Data Pump TRANSFORM Names 17-58  
 Summary 17-59  
 Practice 17: Overview 17-60

## **18 In-Memory Caching**

Oracle Database 12c New and Enhanced Features 18-2  
 Objectives 18-3  
 Full Database In-Memory Caching 18-4  
 Setting Up Force Full Database Caching 18-6  
 Monitoring Full Database In-Memory Caching 18-8  
 In-Memory Parallel Query Before Automatic Big Table Caching 18-9  
 Automatic Big Table Caching 18-11  
 Configuring Automatic Big Table Caching 18-12  
 Using Automatic Big Table Caching 18-14  
 Monitoring Automatic Big Table Caching 18-15  
 Summary 18-17  
 Practice 18: Overview 18-18

## **19 SQL Tuning Enhancements**

Oracle Database 12c New and Enhanced Features 19-2  
 Objectives 19-3  
 Road Map 19-4  
 SQL Plan Baseline: Architecture 19-5  
 SQL Plan Management: Overview 19-7  
 Adaptive SQL Plan Management 19-8  
 Automatically Evolving SQL Plan Baseline 19-9  
 SQL Management Base Enhancements 19-10  
 Quiz 19-11  
 Lesson Road Map 19-12  
 Adaptive Execution Plans 19-13

|                                                            |       |
|------------------------------------------------------------|-------|
| Dynamic Plans                                              | 19-14 |
| Dynamic Plan: Adaptive Process                             | 19-15 |
| Dynamic Plans: Example                                     | 19-16 |
| Reoptimization: Statistics Feedback                        | 19-17 |
| Statistics Feedback: Monitoring Query Executions           | 19-18 |
| Statistics Feedback: Reparsing Statements                  | 19-19 |
| Automatic Reoptimization                                   | 19-20 |
| Quiz                                                       | 19-22 |
| Lesson Road Map                                            | 19-23 |
| SQL Plan Directives                                        | 19-24 |
| Creating SQL Plan Directives                               | 19-25 |
| Using SQL Plan Directives                                  | 19-26 |
| SQL Plan Directives: Example                               | 19-27 |
| Online Statistics Gathering for Bulk-Load                  | 19-28 |
| Concurrent Statistics Enhancements in Oracle Database 12c  | 19-29 |
| Statistics for Global Temporary Tables                     | 19-30 |
| Histogram Enhancements                                     | 19-32 |
| Top Frequency Histograms                                   | 19-33 |
| Hybrid Histograms                                          | 19-34 |
| Hybrid Histograms: Example                                 | 19-35 |
| Extended Statistics Enhancements                           | 19-36 |
| Capturing Column Group Usage                               | 19-37 |
| Capturing Column Group Usage: Running the Workload         | 19-38 |
| Creating Column Groups Detected During Workload Monitoring | 19-40 |
| Automatic Dynamic Sampling                                 | 19-41 |
| Quiz                                                       | 19-42 |
| Summary                                                    | 19-43 |
| Practice                                                   | 19-44 |

## **20 Resource Manager and Other Performance Enhancements**

|                                               |       |
|-----------------------------------------------|-------|
| Oracle Database 12c New and Enhanced Features | 20-2  |
| Objectives                                    | 20-3  |
| Resource Manager and Pluggable Databases      | 20-4  |
| Managing Resources Between PDBs               | 20-5  |
| CDB Resource Plan Basics: Share               | 20-6  |
| CDB Resource Plan Basics: Limits              | 20-8  |
| CDB Resource Plan: Full Example               | 20-10 |
| Creating a CDB Resource Plan                  | 20-11 |
| Setting Default Directives                    | 20-12 |
| Viewing CDB Resource Plan Directives          | 20-13 |
| Maintaining a CDB Resource Plan               | 20-14 |
| Managing Resources Within a PDB               | 20-15 |

|                                                               |       |
|---------------------------------------------------------------|-------|
| Managing PDB Resource Plans                                   | 20-16 |
| Putting It Together                                           | 20-17 |
| Considerations                                                | 20-18 |
| Runaway Queries and Resource Manager                          | 20-19 |
| Controlling IM Column Store Repopulation Resource Consumption | 20-21 |
| Default UNIX/Linux Architecture                               | 20-22 |
| Multi-Process Multi-Threaded UNIX/Linux Architecture          | 20-23 |
| Multi-Process Multi-Threaded Architecture: Benefits and Setup | 20-24 |
| Multi-Process Multi-Threaded Architecture: Considerations     | 20-25 |
| Multi-Process Multi-Threaded Architecture: Monitoring         | 20-26 |
| Database Smart Flash Cache Enhancements                       | 20-27 |
| Enabling and Disabling Flash Devices                          | 20-28 |
| In-Memory PQ Algorithm: Benefits                              | 20-29 |
| Smart Flash Cache: New Statistics                             | 20-30 |
| Temporary Undo: Overview                                      | 20-31 |
| Temporary Undo: Benefits and Setup                            | 20-32 |
| Temporary Undo Monitoring                                     | 20-33 |
| Limiting the Size of the Program Global Area                  | 20-34 |
| Summary                                                       | 20-35 |
| Practice 20: Overview                                         | 20-36 |

## **21 Tables, Indexes, and Online Operations Enhancements**

|                                                       |       |
|-------------------------------------------------------|-------|
| Oracle Database 12c New and Enhanced Features         | 21-2  |
| Objectives                                            | 21-3  |
| Why Multiple Indexes on the Same Set of Columns?      | 21-4  |
| Creating Multiple Indexes on the Same Set of Columns  | 21-5  |
| Quiz                                                  | 21-7  |
| Invisible and Hidden Columns in SQL*Plus              | 21-8  |
| SET COLINVISIBLE and DESCRIBE Commands                | 21-9  |
| Quiz                                                  | 21-10 |
| Online Redefinition: Tables with VPD                  | 21-11 |
| Online Redefinition: dml_lock_timeout                 | 21-12 |
| Advanced Row Compression: New Feature Name and Syntax | 21-13 |
| LOB Compression: New Name                             | 21-14 |
| Using the Compression Advisor                         | 21-15 |
| Enhanced Online DDL Capabilities                      | 21-16 |
| DROP INDEX / CONSTRAINT                               | 21-17 |
| Index UNUSABLE                                        | 21-18 |
| SET UNUSED Column                                     | 21-19 |
| Summary                                               | 21-20 |
| Practice 21: Overview                                 | 21-21 |

## **22 Module – Miscellaneous**

- Oracle Data Pump, SQL\*Loader, and External Tables 22-2
- Oracle Database 12c New and Enhanced Features 22-3
- Objectives 22-4
- Full Transportable Export/Import: Overview 22-5
- Full Transportable Export/Import: Usage 22-6
- Full Transportable Export/Import: Example 22-8
- Transporting a Database Over the Network: Example 22-9
- Disabling Logging for Oracle Data Pump Import 22-10
- Exporting Views as Tables 22-11
- Specifying the Encryption Password 22-13
- Compressing Tables During Import 22-14
- Creating SecureFile LOBs During Import 22-15
- Quiz 22-16
- SQL\*Loader Support for Direct-Path Loading of Identity Columns 22-17
- SQL\*Loader and External Table Enhancements 22-18
- SQL\*Loader Express Mode 22-19
- Summary 22-21
- Practice 22: Overview 22-22

## **23 Partitioning Enhancements**

- Oracle Database 12c New and Enhanced Features 23-2
- Objectives 23-3
- Enhancements to Reference Partitioning 23-4
- Interval Reference Partitioning 23-5
- TRUNCATE TABLE CASCADE 23-6
- Multipartition Maintenance Operations 23-7
- Adding Multiple Partitions 23-8
- Creating a Range-Partitioned Table 23-9
- Adding Multiple Partitions 23-10
- Truncating Multiple Partitions 23-11
- Dropping Multiple Partitions 23-12
- Splitting into Multiple Partitions 23-13
- Splitting into Multiple Partitions Rules 23-14
- Splitting into Multiple Partitions: Examples 23-15
- Merging Multiple Range Partitions 23-16
- Merging List and System Partitions 23-17
- Quiz 23-18
- Partitioned Indexes: Review 23-19
- Partial Indexes for Partitioned Tables 23-20
- Partial Index Creation on a Table 23-21
- Specifying the INDEXING Clause at the Partition and Subpartition Levels 23-22

|                                                                               |       |
|-------------------------------------------------------------------------------|-------|
| Creating a Partial Local or Global Index                                      | 23-23 |
| Explain Plan: LOCAL INDEX ROWID                                               | 23-24 |
| Explain Plan: GLOBAL INDEX ROWID                                              | 23-25 |
| Affected Data Dictionary Views: Overview                                      | 23-26 |
| Asynchronous Global Index Maintenance                                         | 23-27 |
| DBMS_PART Package                                                             | 23-28 |
| Global Index Maintenance Optimization During Partition Maintenance Operations | 23-29 |
| Quiz                                                                          | 23-30 |
| Summary                                                                       | 23-31 |
| Practice 23: Overview                                                         | 23-32 |

## **24 SQL Enhancements and Migration Assistant for Unicode**

|                                               |       |
|-----------------------------------------------|-------|
| Oracle Database 12c New and Enhanced Features | 24-2  |
| Objectives                                    | 24-3  |
| Increased Length Limits of Data Types         | 24-4  |
| Configuring Database for Extended Data Type   | 24-5  |
| Using VARCHAR2, NVARCHAR2, and RAW Data Types | 24-6  |
| Database Migration Assistant for Unicode      | 24-7  |
| SecureFiles                                   | 24-8  |
| SQL Row-Limiting Clause                       | 24-9  |
| SQL Row-Limiting Clause: Examples             | 24-10 |
| Quiz                                          | 24-11 |
| Summary                                       | 24-13 |
| Practice 25: Overview                         | 24-14 |

## **A New Processes, Views, Parameters, Packages, and Privileges**

|                                                       |      |
|-------------------------------------------------------|------|
| Instance and Database                                 | A-2  |
| Multitenant Architecture: General Architecture Poster | A-3  |
| CDB and PDB                                           | A-4  |
| Heat Map and ADO                                      | A-6  |
| In-Database Archiving and Temporal Validity           | A-8  |
| Security: Auditing                                    | A-9  |
| Security: Privilege Analysis                          | A-10 |
| Security: Privilege Analysis and New Privileges       | A-11 |
| Security: Oracle Data Redaction                       | A-12 |
| HA: Flashback Data Archive                            | A-13 |
| Manageability: Database Operations                    | A-14 |
| Manageability: ADDM                                   | A-16 |
| Performance: In-Memory Column Store                   | A-17 |
| Performance: Full Database In-Memory Caching          | A-19 |
| Performance: Automatic Big Table Caching              | A-20 |
| Performance: SQL Tuning                               | A-21 |

Performance: Resource Manager A-22  
 Performance: Multi-Process Multi-Threaded A-23  
 Performance: Database Smart Flash Cache A-24  
 Performance: Temporary UNDO A-25  
 Performance: Online Operations A-26  
 Miscellaneous: Partitioning A-27  
 Miscellaneous: SQL A-28  
 Appendix C: Data Comparison A-29

## **B Pluggable Databases: Other Creation Methods**

Plugging a Non-CDB into CDB Using Data Pump B-2  
 Plugging a Non-CDB into CDB Using Replication B-3  
 Cloning PDBs Across CDBs B-4  
 Plugging Unplugged PDB: Using SQL Developer B-5

## **C Schema and Data Change Management**

Objectives C-2  
 Database Lifecycle Management Pack: New Features C-3  
 Change Management Pack Features C-4  
 Change Management Pack Components C-5  
 Dictionary Baselines C-6  
 Dictionary Comparisons C-8  
 Dictionary Synchronization C-10  
 Comparing Change Propagation and 11g SQL Scripts C-11  
 Database Lifecycle Management Pack Schema Change Plans C-12  
 Change Requests C-14  
 Schema Synchronization C-16  
 Database Lifecycle Management Pack Data Comparisons C-18  
 DBMS\_COMPARISON C-19  
 Flow C-21  
 Guidelines C-22  
 Creating a Data Comparison C-24  
 Comparison Job and Results C-25  
 Results: Reference Only Rows C-26  
 Results: Candidate-Only Rows C-27  
 Results: Non-Identical Rows C-28  
 Quiz C-29  
 Summary C-31  
 Practice C-32

## **D New and Enhanced Features in Other Courses**

Further Information D-2  
 Suggested Oracle University ILT Courses D-3





# 1

## Introduction

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Overview

- This course focuses on the features of Oracle Database 12c including those of the patchset 1 that are applicable to database administration.
- Previous experience with Oracle databases (particularly Oracle Database 11g) is required for a full understanding of many of the new features.
- Hands-on practices emphasize functionality rather than test knowledge.



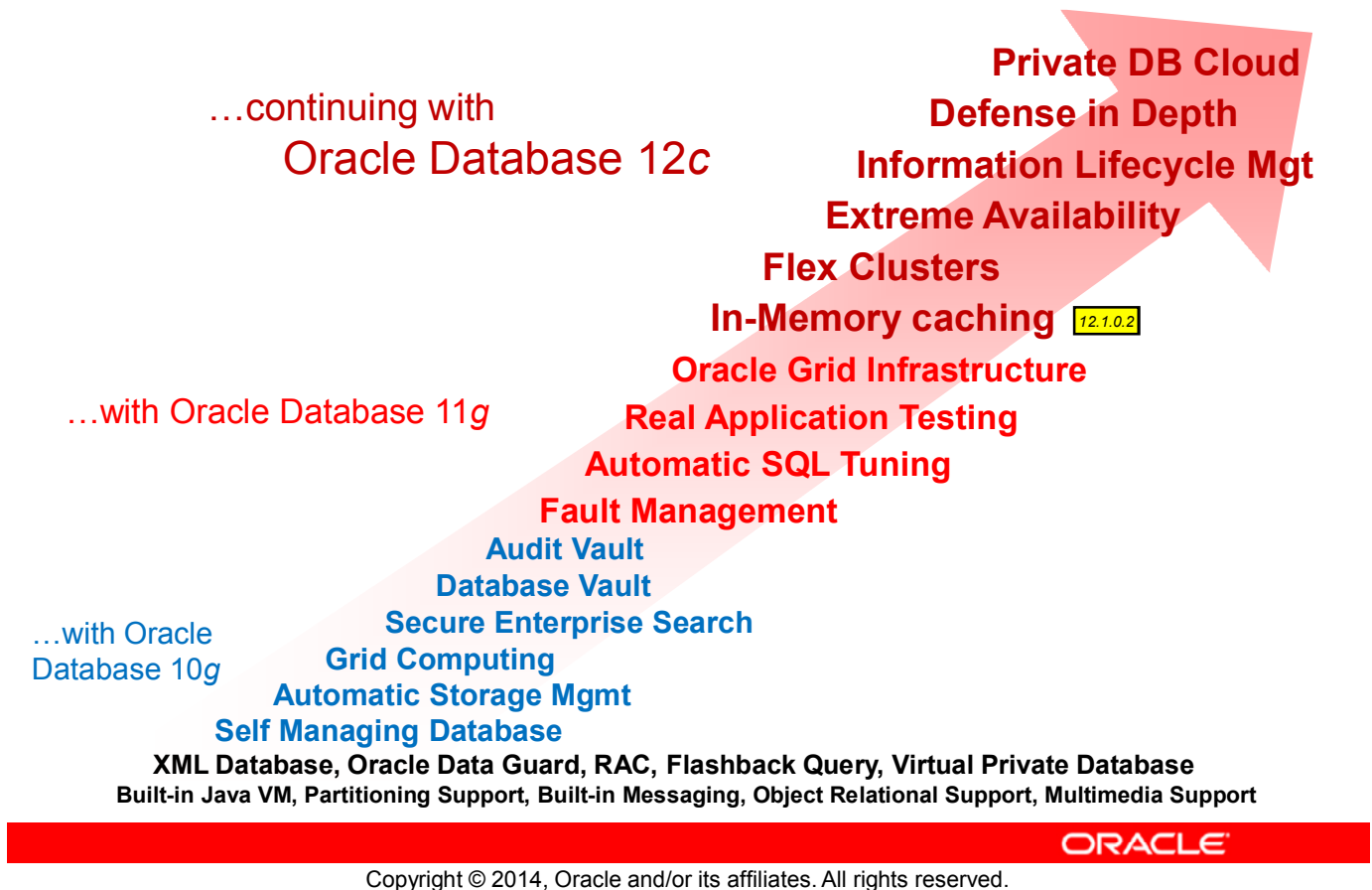
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This course is designed to introduce you to the new features of Oracle Database 12c Release 1 Patchset 1 (12.1.0.2) that are applicable to the work usually performed by database administrators and related personnel. The course covers all features and enhancements. The course does not attempt to provide every detail about a feature or cover aspects of a feature that were available in previous releases (except when defining the context for a new feature or comparing past behavior with current behavior). Consequently, the course is most useful to you if you have administered other versions of Oracle databases, particularly Oracle Database 11g. Even with this background, you should not expect to be able to implement all of the features discussed in the course without supplemental reading, especially the Oracle Database 12c documentation.

The course consists of instructor-led lessons and demonstrations, in addition to many hands-on practices that enable you to see for yourself how certain new features behave. As with the course content in general, these practices are designed to introduce you to the fundamental aspects of a feature. They are not intended to test your knowledge of unfamiliar syntax or provide an opportunity for you to examine every nuance of a new feature. The length of this course precludes such activity. Consequently, you are strongly encouraged to use the provided scripts to complete the practices rather than struggle with unfamiliar syntax.

# Oracle Database Innovation

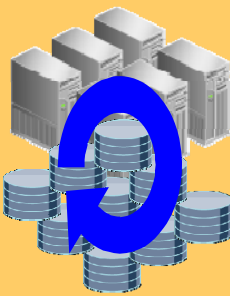


As a result of its early focus on innovation, Oracle has maintained the lead in the industry with a large number of trend-setting products.

Some of the marquee areas in the Oracle Database 12c release are the following:

- Private Database Cloud
- Defense in Depth included Oracle Data Redaction, Real Application Security
- Information Lifecycle Management (ILM), which includes hot/cold data classification, declarative compression and tiering, In-database Archiving and Temporal Validity.
- Flex Clusters
- Extreme Availability, which includes Data Guard Far-Sync and Application Continuity
- Lower Cost Migrations
- Performance and Ease of Use, which includes “just in time” optimizations, In-Memory Column Store, Full Database In-Memory Caching, and Automatic Big Table Caching

# Enterprise Cloud Computing

|                                                                                                                        |                                                                                                                                 |                                                                                                                                |                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>RAC clusters for availability</p> |  <p>Grids of low-cost hardware and storage</p> |  <p>Managing change across the enterprise</p> |  <p>Enterprise Manager Cloud Control and database consolidation across the enterprise</p> |
|                                      |                                                |                                               |                                                                                           |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 10g was the first database management system designed for grid computing.

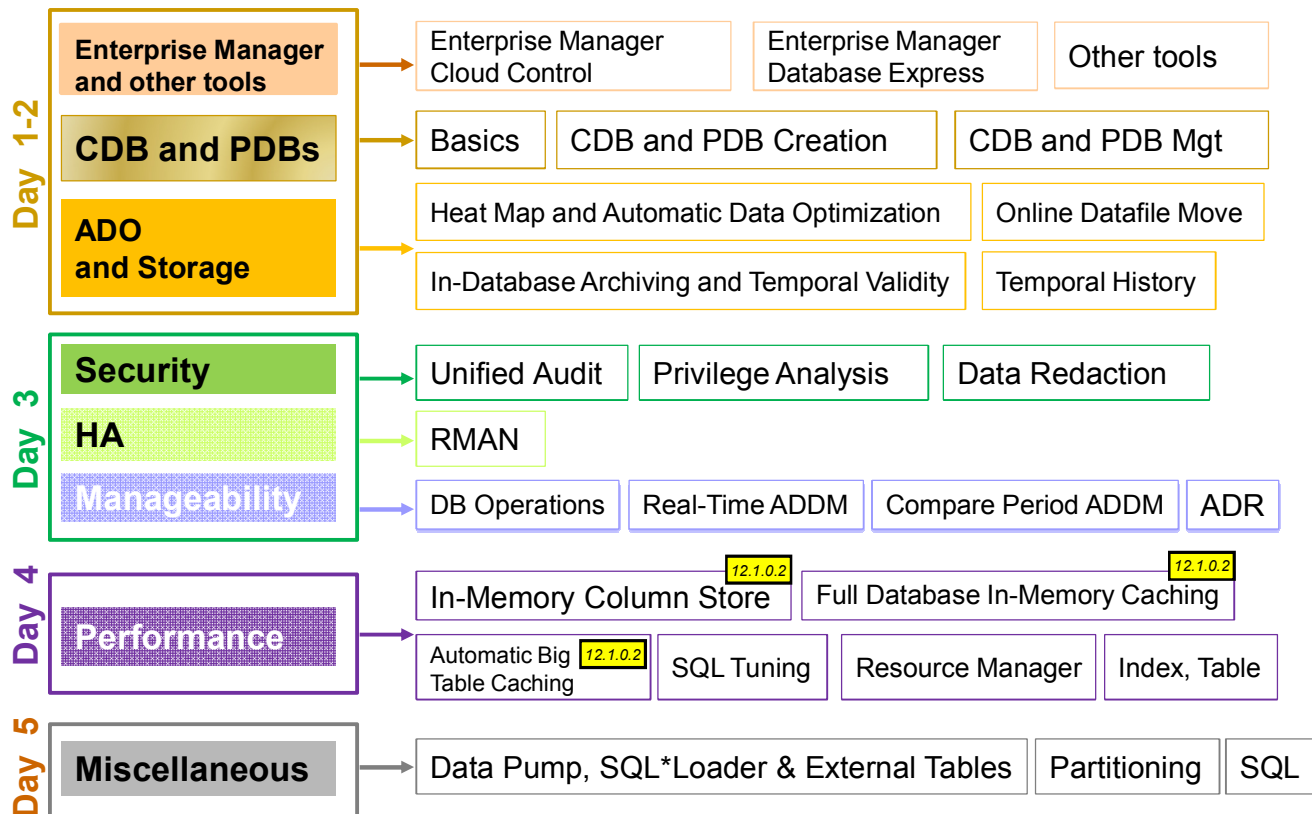
Oracle Database 11g consolidates and extends Oracle's unique ability to deliver the benefits of grid computing, transforming data centers from silos of isolated system resources to shared pools of servers and storage.

Oracle Database 12c and Enterprise Manager Cloud Control are designed for cloud computing. Cloud computing creates a complete, pre-integrated, off-the-shelf private cloud solution that allows you to quickly transform the enterprise data center into a private cloud.

The key benefits are the following:

- Reduce servers sprawl and improve CPU utilization by consolidating on fewer servers
- Reduce the amount of time a DBA spends installing and configuring databases, by automating deployment of standard database configurations
- Single console to manage entire Cloud life cycle—plan, set up, deliver, operate
- Prevent resource hogging by setting quotas for individual users
- Forecast future resource needs by analyzing trending reports
- Compute chargeback based on performance and configuration metrics

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Features, Options, and Schedule

**Enterprise Manager Cloud Control:** Oracle Enterprise Manager is Oracle's integrated enterprise IT management product line, and provides the industry's first complete cloud lifecycle management solution.

**Oracle Multitenant:** Oracle Multitenant is a new option in Oracle Database 12c. The multitenant architecture enables an Oracle database to contain a portable set of schemas, objects, and related structures that appears logically to an application as a separate database. This self-contained collection is called a pluggable database (PDB). A multitenant container database (CDB) contains one or several PDBs.

**Information Lifecycle Management:** One challenge facing each organization is to understand how its data evolves and grows, monitor how its usage changes over time, and decide how long it should survive, while adhering to all the rules and regulations that now apply to that data. Information Lifecycle Management (ILM) is designed to address these issues, with a combination of processes, policies, software, and hardware so that the appropriate technology can be used for each stage in the life cycle of the data. ILM offers new features including heat Map, Automatic Data Optimization and enhancements to In-Database Archiving, including the new Temporal Validity feature.

## Defense in Depth

- Unified auditing makes audit information available in a uniform format and centralized in a single place accessible only by privileged users.
- New administrative privileges help separation of duties.
- Privilege Analysis contributes to identify privilege usage and to decide which privileges must be revoked.
- Oracle Data Redaction prevents the display of sensitive data by performing masking in each application, defining redaction at the database level and displaying redacted data regardless of the access method, even to DBAs.

## High Availability

RMAN enhancements include recovering a table or table partitions from existing backups, using backup sets for cross-platform data transport, connecting with the new `SYSCONNECT` privilege, and many more.

## Manageability

- Database Operation monitoring extends and generalizes Real Time SQL Monitoring.

## Performance

- The In-Memory Column Store feature brings the solution for accelerating database-driven business decision-making to real-time speeds. The In-Memory Column Store feature enables objects to be stored in memory in a new format known as the columnar format. This format enables scans, joins, and aggregates to perform much faster than the traditional on-disk formats, providing fast reporting and DML performance for both OLTP and DW environments. This is particularly useful for analytic applications that operate on few columns returning many rows rather than for OLTP operating on few rows returning many columns. The DBA has to define which segments to be loaded “into the in-memory cache, such as hot tables, partitions and more precisely more frequently accessed columns. There are other new In-Memory features available with Oracle Database 12c R1 PS1.
- SQL tuning is essentially based on performance automation with Adaptive SQL Plan Management, Adaptive Execution Plans, and Optimizer Statistics Management.
- Real-Time ADDM and Emergency Monitoring detect that the system is sick and analyzes the root causes of the hanging situation to help you find the appropriate action.
- In a situation where the performance has decreased or increased, Compare Period Advisor identifies what has changed and why and which changes may have impacted the performance between two periods of time. This helps you take appropriate actions upon relevant analysis.
- Resource Manager is now adapted to multitenant container databases and pluggable databases to share CPU and other resources among the different containers in a database.

There are also many other enhancements about Oracle Data Pump, SQL\*Loader, partitioning and online operations, and SQL covered in the course.

Subjects like Automatic Storage Management, Real Application Cluster, Data Guard, Real Application Testing and many other subjects of expertise cannot be presented in this course. You can find references to these courses, demonstrations, and OBEs in the appendix titled “New and Enhanced Features in Other Courses.”

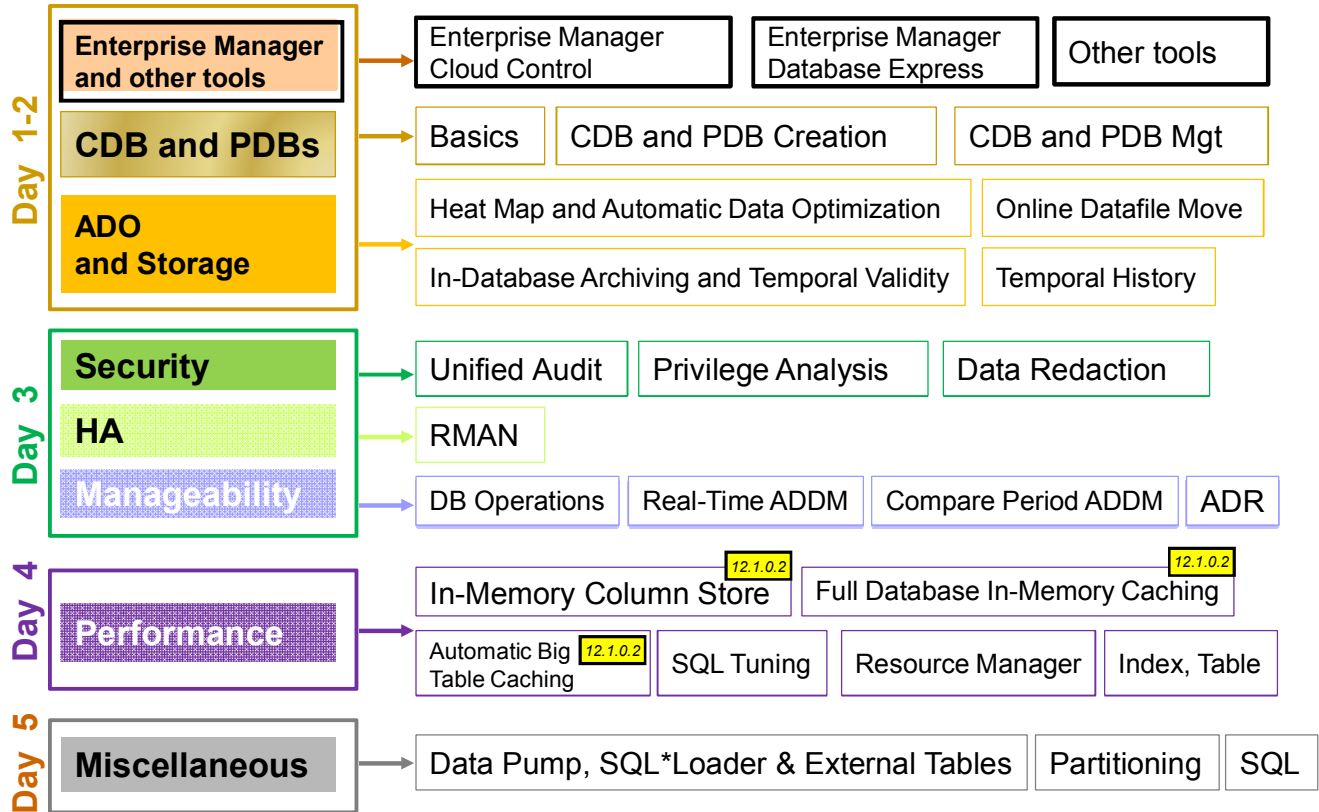
# Enterprise Manager Cloud Control and Other Tools

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Enterprise Manager:** Enterprise Manager Database Control is no longer available in Oracle Database 12c. For Oracle Database 12.1, both Enterprise Manager Cloud Control 12c and Enterprise Manager Database Express 12c provide some functionality against 12.1 databases.

# Objectives

After completing this lesson, you should be able to:

- Describe the different components of Cloud Control
- Explain the architecture of Cloud Control
- List the target types managed by Cloud Control
- Explore the Oracle Enterprise Manager Cloud Control interface
- Describe Oracle Enterprise Manager Database Express navigation
- Use DBCA to create new types of databases
- Explore the Oracle SQL Developer interface

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of Oracle Enterprise Manager Cloud Control and Database Express installation and usage, refer to the following guides in the Oracle documentation:

- *Oracle Enterprise Manager Cloud Control Basic Installation Guide 12c Release 1*
- *Oracle Enterprise Manager Cloud Control Advanced Installation and Configuration Guide 12c Release 1*
- *Oracle Enterprise Manager Cloud Control Administrator's Guide 12c Release 1*
- *Oracle Enterprise Manager Licensing Information 12c Release 1*

## Key Challenges for Administrators

As the composition of the data center broadens into the cloud environment, the challenges to manage it also increase. The key challenges for managing a data center include:

- Monitoring performance and availability
- Resolving problems quickly
- Containing operating costs
- Aligning IT with business priorities

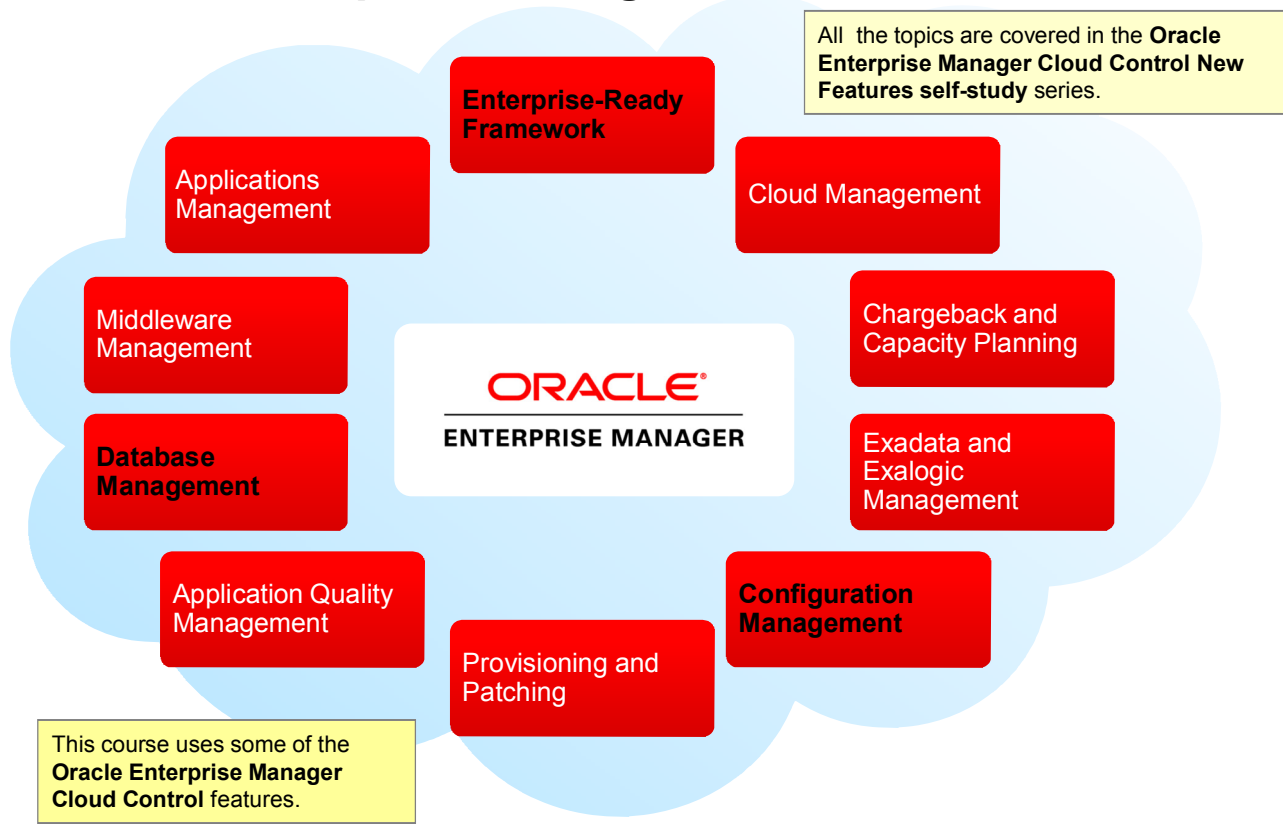
The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

As the data center grows with the growth in business, so do the challenges. An administrator is faced with challenges that include:

- Monitoring high levels of performance and availability of applications
- Identifying and resolving problems quickly and effectively
- Enabling IT professionals to use resources effectively, thereby reducing costs
- Aligning IT with business priorities to ensure that businesses are agile enough to meet the changing needs

# Enterprise Manager Cloud Control



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Key Objectives in Enterprise Manager Cloud Control Design

- Designing a management framework that is capable of providing next-generation functionality
- Enhancing application-to-disk manageability
- Providing a complete enterprise private cloud solution

Enterprise Manager Cloud Control includes the following features:

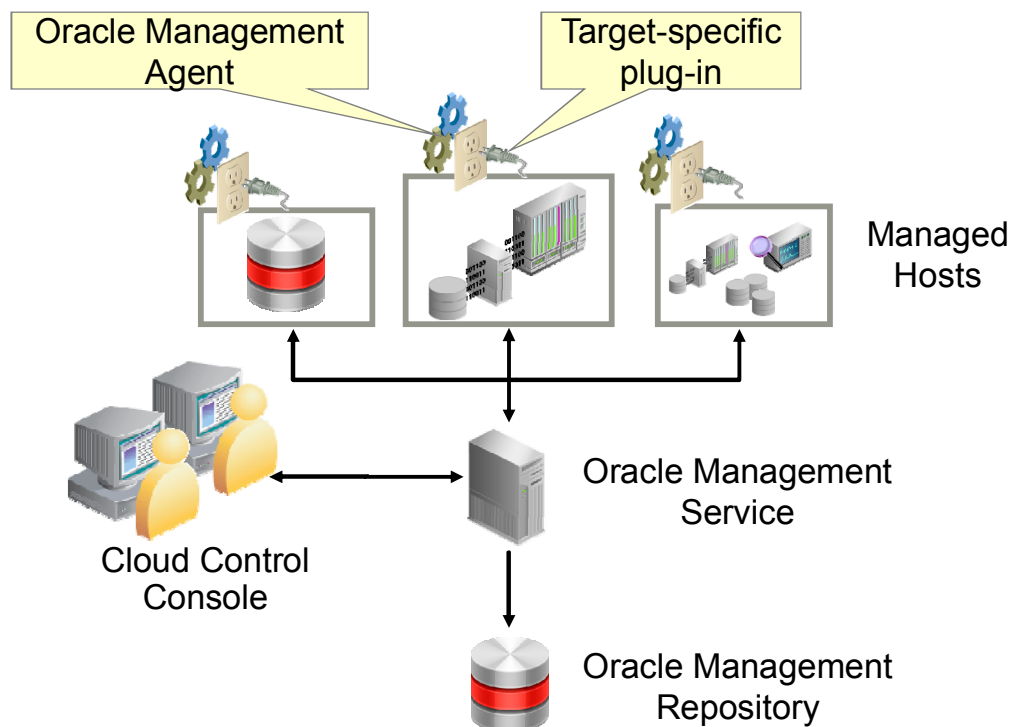
- **Enterprise-Ready Framework:** Provides modular and extensible architecture, target plug-ins, self-updateable entities, integrated Support Workbench, and centralized incident console
- **Cloud Management:** Provides complete cloud lifecycle management
- **Chargeback and Capacity Planning:** Provides chargeback based on target types, and uses Automatic Workload Repository (AWR) Warehouse to consolidate AWR reports from multiple databases across the enterprise
- **Exadata and Exalogic Management:** Provides an integrated view of the hardware and software in an Exadata machine, and complete lifecycle management for Exalogic systems
- **Configuration Management:** Provides an integrated set of tools, agent-less discovery, integration with My Oracle Support, and custom configuration capabilities

- **Provisioning and Patching:** Provides profiles for provisioning known configurations, user-defined deployment procedures, and a software library integrated with self-updating capabilities
- **Application and Quality Management:** Database Replay, Application Server Replay, Real Application Testing integrated with Data Masking, and test database management including Application Data Model
- **Database Management:** Provides management of Oracle Database systems, including performance management and change lifecycle management. Some aspects of Database Management are covered in detail in this course.
- **Middleware Management:** Provides management of Fusion Middleware systems
- **Applications Management:** Provides management of Fusion Applications

**Note:** For a complete understanding of Oracle Enterprise Manager Cloud Control usage, refer to the following sources of information:

- *Using Oracle Enterprise Manager Cloud Control* course
- *Oracle Enterprise Manager Cloud Control: Install and Upgrade* course
- *Oracle Enterprise Manager Cloud Control New Features Self-Study* series
- *Oracle Enterprise Manager 12c* demonstrations under Oracle Learning Library (URL [www.oracle.com/goto/oll](http://www.oracle.com/goto/oll))

# Cloud Control Components



ORACLE

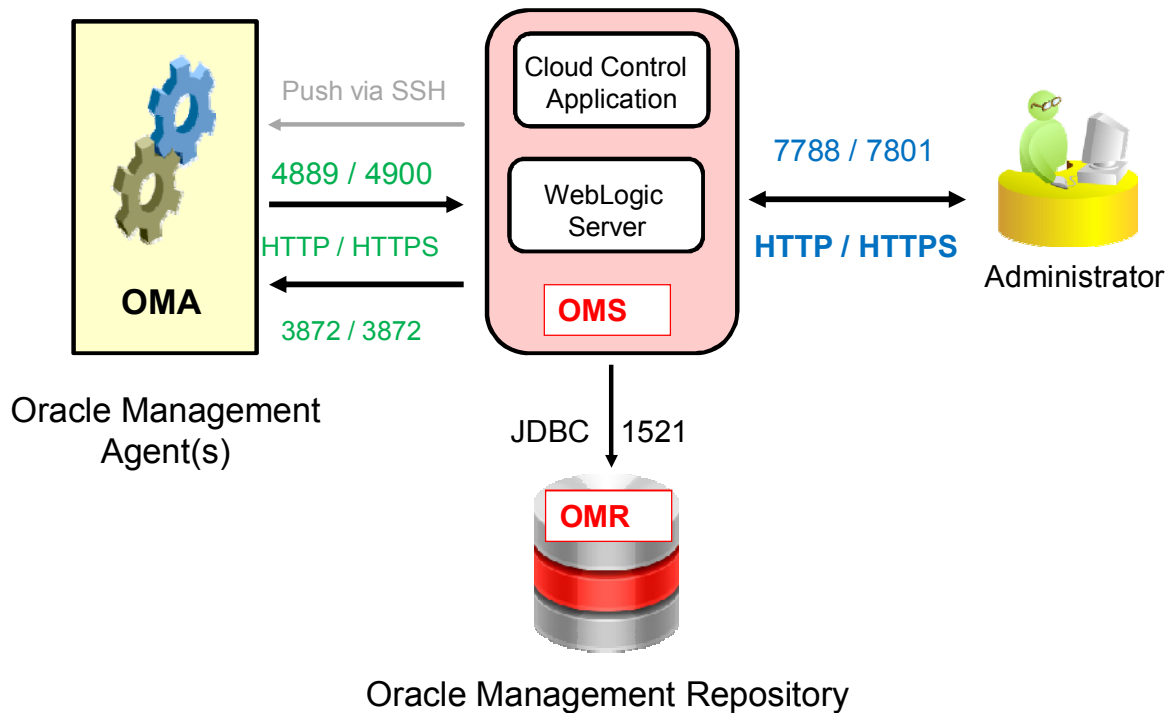
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enterprise Manager Cloud Control is composed of four main components as illustrated:

- Oracle Management Repository (OMR)
- Oracle Management Service (OMS)
- Oracle Management Agent (OMA or agent) with target-specific plug-ins
- Cloud Control Console

The Oracle Management Agent runs on hosts, gathering metric data about those host environments as well as using plug-ins to monitor availability, configuration, and performance and to manage targets running on the host. The agents communicate with the Oracle Management Service to upload metric data collected by them and their plug-ins. In turn, the OMS stores the data it collects in the Oracle Management Repository where it can be accessed by the OMS for automated and manual reporting and monitoring. The OMS also communicates with the agents to orchestrate the management of their monitored targets. Along with coordinating the agents, the OMS runs the Cloud Control Console webpages that are used by administrators and users to report on, monitor, and manage the computing environment that is visible to Cloud Control via the agents and their plug-ins.

## Components and Communication Flow



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The communication flow between the Cloud Control components is illustrated using directional arrows. Communication between the Agent and the OMS, and the OMS and the console is bi-directional. All the ports shown and listed in the slide are default values that can be changed during installation, either by the installer as it searches for available ports, or explicitly by you. You can also change ports after installation.

- The Agent uploads data to the OMS via HTTP on port 4889 or via HTTPS on port 4900. (Designed to be able to work with WAN.)
- The OMS communicates with the Agent via HTTP or HTTPS on port 3872.
- The reason for the separate ports for OMS to OMA communications is that they can communicate asynchronously and simultaneously to one another.
- The OMS communicates with the OMR via JDBC on port 1521. Although the OMR will return data to the OMS, this is not considered to be a separate communication between the two; hence, the flow is shown to be unidirectional from OMS to OMR.
- Cloud Control console users access the Cloud Control webpages via HTTPS on port 7801 or via HTTP on port 7788.

Knowing the ports used in your Cloud Control installation is important, especially if you are managing hosts behind firewalls or where other network restrictions apply, because communication will need to be allowed on these ports and in the directions shown.

# Oracle Management Repository

Oracle Management Repository (OMR):

- Resides in an Oracle database
- Includes schema objects belonging to `SYSMAN`
- Must be installed in a pre-existing database
- Can be installed in a RAC database

**Note:** Uses a restricted-use license of the Oracle Database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The OMR is installed in an Oracle database as a group of approximately 4,000 schema objects belonging to the `SYSMAN` user stored in three tablespaces: `MGMT_ECM_DEPOT_TS`, `MGMT_TABLESPACE`, and `MGMT_AD4J_TS`. These schema objects contain information about Enterprise Manager Cloud Control users and administrators, targets and applications that are monitored and managed by Enterprise Manager Cloud Control, and groups, systems, incidents, and other Enterprise Manager Cloud Control artifacts. The OMR is created during installation in a pre-existing database, and for scalability requirements, it can be installed in a Real Application Clusters (RAC) database. In this case, customers must license the second node for the database, and both nodes require an Oracle Real Application Clusters license.

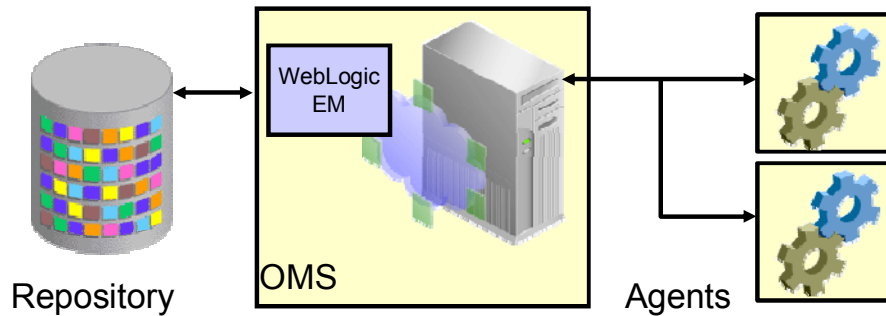
The database used to house the OMR should not be used for any other applications for the following reasons:

- Enterprise Manager Cloud Control usage of the database must not have to compete with any other usage.
- Using the OMR database for other applications may restrict your ability to upgrade and patch the OMR schema and database as required.
- Enterprise Manager Cloud Control includes a restricted-use database license that can be used for the OMR only.

**Note:** Refer to *Oracle Enterprise Manager Licensing Information 12c Release 1*, Section *Enterprise Manager Restricted-Use License*.



# Controlling the Enterprise Manager Cloud Control Framework



| Component Control Utilities |       |       |
|-----------------------------|-------|-------|
| Repository                  | OMS   | Agent |
| sqlplus or<br>srvctl        | emctl | emctl |
| lsnrctl                     |       |       |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Each component of the Enterprise Manager Cloud Control framework has its own utility or utilities that can be used to monitor, start, and stop the component. In many cases, these utilities also provide some capability to configure the component beyond the simple start-and-stop functionality.

RAC databases require the use of the Server Control commands; for single instances, there is a choice between SQL\*Plus and Server Control. Server Control is usable when Oracle Restart is installed and the database is registered with the OLR.

To start and stop the listener, either use the Server Control utility or the `lsnrctl` command.

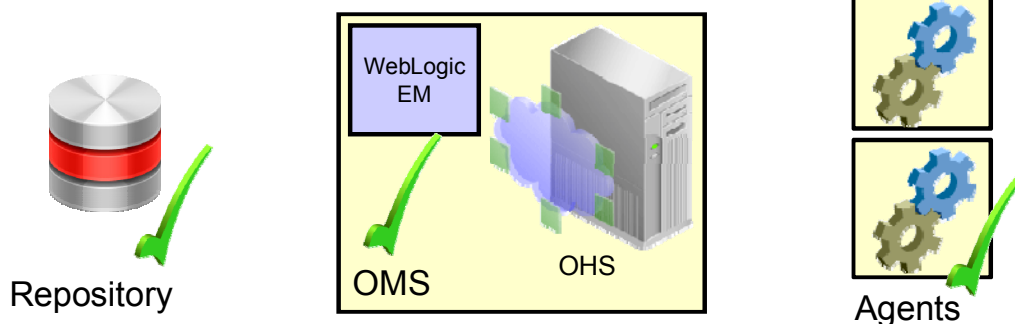
Examples:

```
srvctl stop database -d orcl -o immediate
srvctl start database -d orcl -o open
```

# Starting the Enterprise Manager Cloud Control Framework

To start the Cloud Control framework, perform the following steps:

1. Start the repository database listener.
2. Start the repository database instance.
3. Start the OMS.
4. Start the agent on the OMS/repository server.
5. Start the agents on the managed servers.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To start the whole Enterprise Manager Cloud Control framework, perform the following steps:

1. Start the repository listener:

```
$ORACLE_HOME/bin/lsnrctl start
```

2. Start the repository database instance:

```
$ORACLE_HOME/bin/sqlplus / as sysdba
```

```
SQL> startup
```

3. Start the OMS (including OHS and WebLogic Managed Server):

```
$OMS_HOME/bin/emctl start oms
```

4. Start the agent (on OMS/repository host):

```
$AGENT_HOME/bin/emctl start agent
```

5. Start the agent on the managed servers:

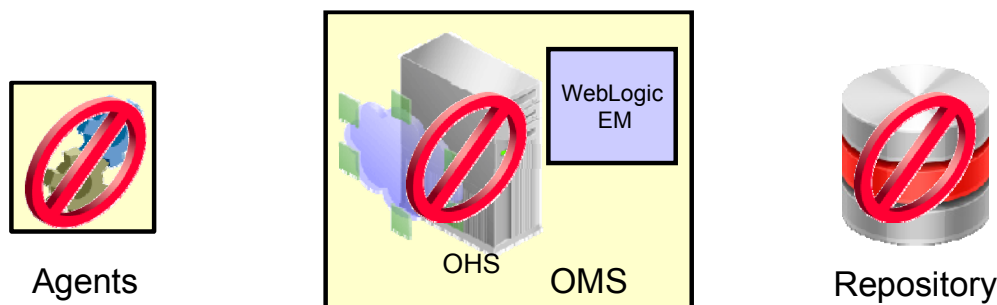
```
$AGENT_HOME/bin/emctl start agent
```

**Note:** Use the SRVCTL command if you have a RAC instance for the repository.

## Stopping the Enterprise Manager Cloud Control Framework

To stop the Enterprise Manager Cloud Control framework, perform the following steps:

1. Stop the agents on managed servers.
2. Stop the agent on the OMS/repository server.
3. Stop the OMS.
4. Stop the repository database instance.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To stop the whole Enterprise Manager Cloud Control framework, perform the following steps:

1. Stop the agent on the managed servers:  
\$AGENT\_HOME/bin/emctl stop agent
2. Stop the agent (on OMS/repository host):  
\$AGENT\_HOME/bin/emctl stop agent
3. Stop the OMS (including OHS and WebLogic Managed Server):  
\$OMS\_HOME/bin/emctl stop oms
4. Stop the repository database instance:  
\$ORACLE\_HOME/bin/sqlplus / as sysdba  
SQL> shutdown immediate

**Note:** Use the SRVCTL command if you have a RAC instance for the repository.

## Different Target Types

Enterprise Manager Cloud Control can monitor, administer, maintain, and manage many different types of targets including:

- Oracle Databases
- Oracle Database Listener
- Oracle Fusion Middleware products
- Oracle Application Server
- Oracle WebLogic Server
- Oracle applications, including E-Business Suite, SOA, Siebel, and PeopleSoft
- Exadata and Exalogic
- Cloud Control Components: OMR and OMS
- Third-party products



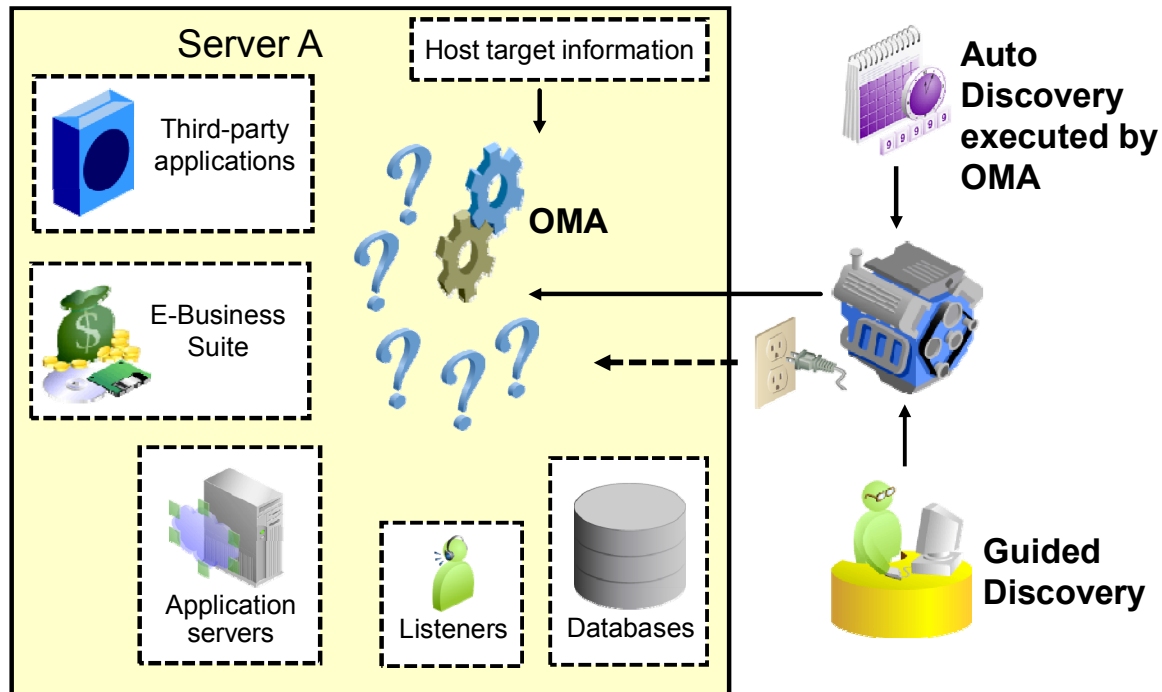
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Targets are the entities that Enterprise Manager Cloud Control manages. To do so, it uses target-type-specific plug-ins and host-specific agents.

Enterprise Manager Cloud Control can monitor, administer, maintain, and manage different types of targets as listed in the slide. As your environment changes, you can add and remove targets from Enterprise Manager Cloud Control as needed. The commonly used Oracle targets (including Enterprise Manager Cloud Control components, such as the OMR and OMS) are predefined as part of the base Enterprise Manager Cloud Control product, but Enterprise Manager Cloud Control has an open API that enables you to create custom targets.

# Target Discovery



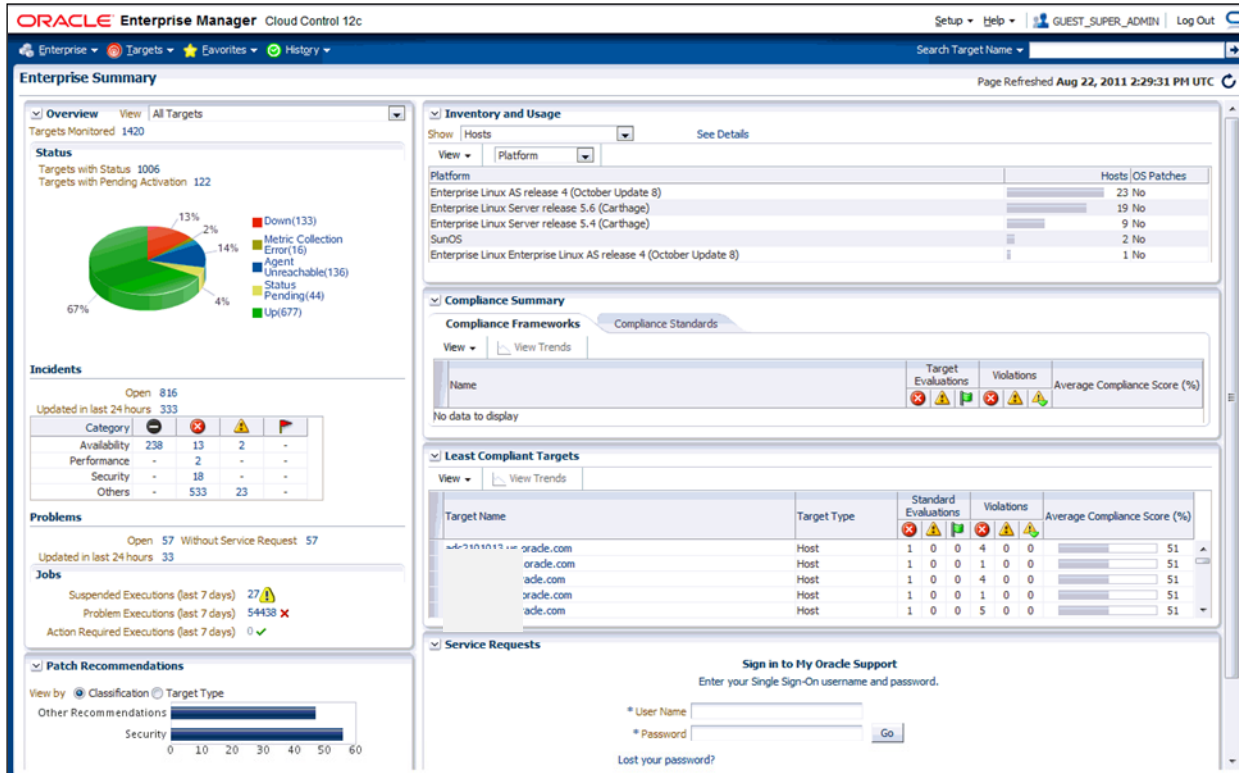
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After the Agent has been installed on a host, it needs to look for targets that it can manage. As an Enterprise Manager Cloud Control administrator, you can guide that process from the Enterprise Manager Cloud Control console pages. Guided discovery allows you to nominate a family of target types that you want to search for, such as database and listeners, and then the agents where you want that search to be executed. If any new targets are discovered, the appropriate plug-in will be pushed from the OMS if it is not already installed on the agent, the target will be recorded in the OMR, and monitoring will begin.

You can also configure auto discovery to run at regular intervals and get an agent to search for known targets unattended, allowing you to review the results at a later stage and promote discovered targets to become managed targets.

# Enterprise Manager Cloud Control



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The image in the slide is of the Enterprise Summary page of Oracle Enterprise Manager Cloud Control. The user interface (UI) functionality includes:

- Information displayed in graphs and tables
- Summary information with drilldown capability to relevant details
- User-selected home page from a predefined set, or based on any page in the console
- Menu-driven navigation
- Global target search
- History and favorites
- Customizable target home pages (per-user basis)

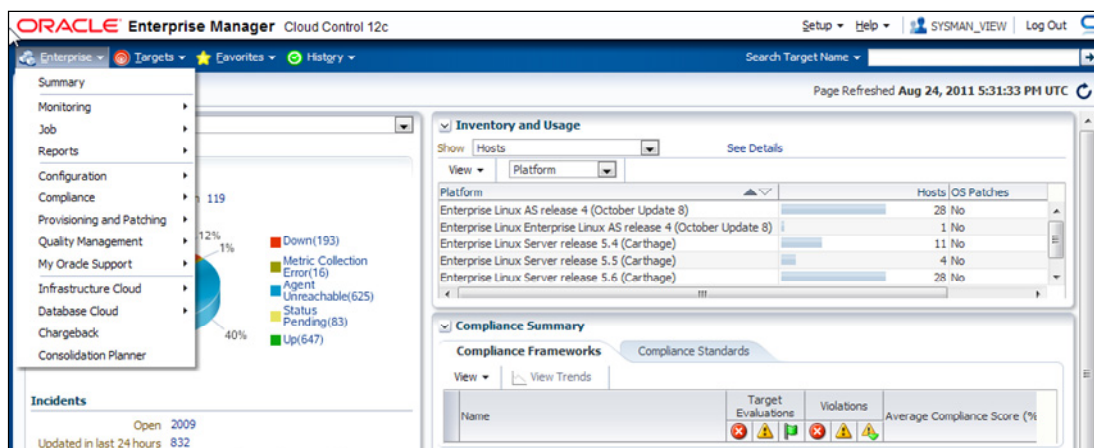
## User Interface

### Setting your home page:

- Predefined home page based on roles
- Any page

### Menu-based navigation:

- Make any page a favorite for quick access.



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

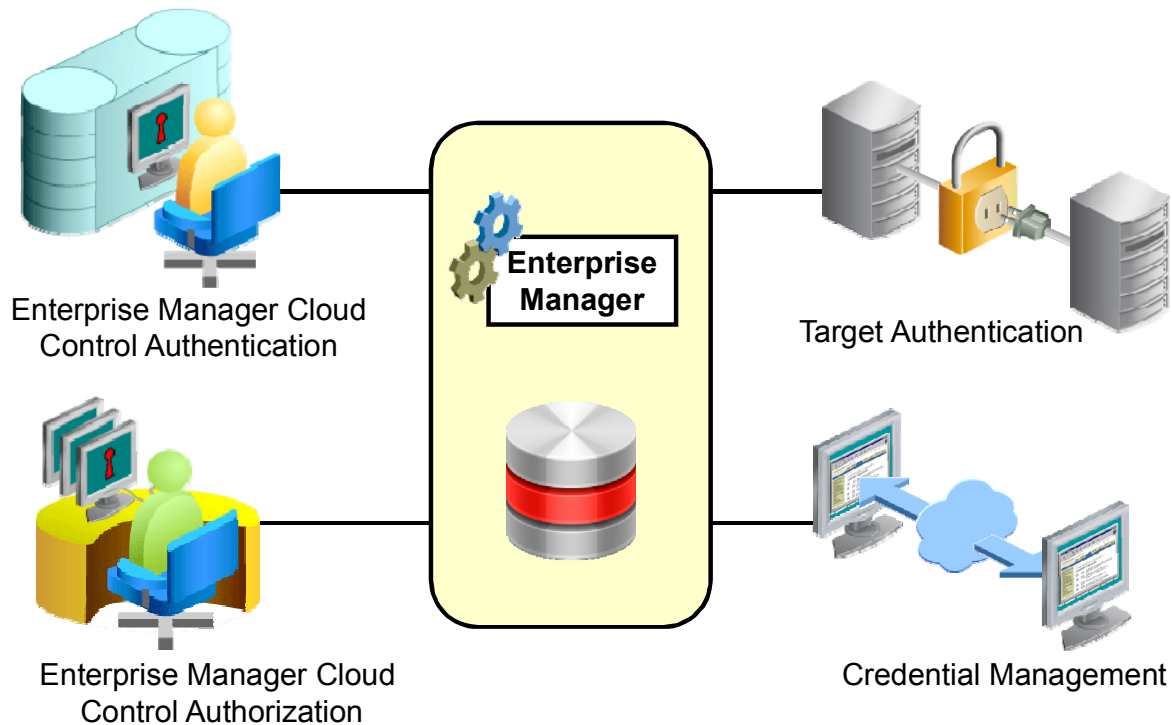
## User Interface Enhancements

The user interface in Enterprise Manager Cloud Control has been rewritten in Application Development Framework (ADF). When you log in to the new UI, use drop-down menus to navigate from one place to another in the product.

- Choose your own home page: When you first log in to Enterprise Manager, you are provided with a selection of pre-defined home pages based on roles. If you are managing databases, you can choose the database home page. If those are not suitable, you can select any page in the UI to be your home page instead.
- Mark any page as a “favorite” for quick access. Because you manage certain targets frequently, you mark these target home pages as favorites in much the same way you mark a favorite in a browser. However, because the favorites you mark in Enterprise Manager are stored in the repository, you can move from client machine to client machine and your favorites are still available to you.

For information about how to customize your Enterprise Manager Cloud Control console, follow the demonstration *Oracle Enterprise Manager 12c: Console Overview and Customization* under Oracle Learning Library.

# Security: Overview



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Enterprise Ready Framework: Security

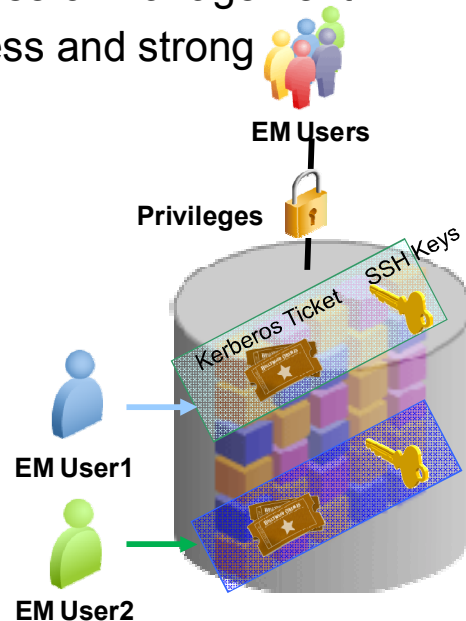
The Enterprise Manager security system can be divided into four parts (as shown in the graphic):

- Enterprise Manager Cloud Control authentication
- Enterprise Manager Cloud Control authorization and privileges
- Credential management (host credentials can be defined here)
- Target authentication (host credentials can be used here)



# Managing Securely with Credentials

- Centralized credential store for ease of management
- Support for managing passwordless and strong authentication credentials
  - Kerberos tickets
  - SSH keys
- Reuse and sharing among users (without disclosing the sensitive content of credentials)
- Controlled and protected access
- Support for sudo/powerbroker



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Security with Credentials

As a management tool that handles a lot of scripts and powerful actions like patching, Enterprise Manager has to work with a lot of credentials for hosts, databases, and a range of other objects. Managing all these credentials can be a real challenge.

- The centralized store facilitates this task because you can name and store credentials there.
- Passwordless and strong authentication credentials are supported, such as the Kerberos tickets and SSH key pairs.
- Credentials can be reused and shared among users (without disclosing the sensitive content like a password). Users are granted access to these credentials by the use of privileges, and so they can be reused without knowing what the contents of the credentials themselves are.
- Access to the credentials is controlled and protected by privileges.
- The Enterprise Manager credential subsystem enables you to securely store credentials as preferences or operation credentials, which can then be used to perform different system management activities. Enterprise Manager also supports sudo/powerbroker-based impersonation.

# Distinguishing Credentials

- Named credentials
- Preferred credentials
- Default credentials
- Access level:
  - **View:** Access to use the credential
  - **Edit:** To change the credential (including changing its name and password)
  - **Full:** For complete access (including the ability to delete the credential)
- Usage classification: Job, collection, and monitoring

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Types of Credentials

As the Enterprise Manager administrator, you can also store credentials (username/password, a public key-private key pair, or an X509v3 certificate) as **named credentials** in Enterprise Manager to use when performing operations like running jobs, patching, and other system management tasks. Objects refer or point to named credentials. They are “placeholders” to facilitate, for example, the changing of passwords.

You can store, access, and modify a fixed number of username/password–based credentials as **preferred credentials** to simplify access to managed targets by storing target login credentials in the Management Repository.

**Default credentials** can be set for a particular target type and will be available for all the targets of the target type.

The three levels of access that can be granted are:

- **View access:** To use the credentials
- **Edit access:** To change the credentials, including changing its name and password
- **Full access:** For complete access, including the ability to delete the named credential

Credentials can also be classified by their usage, such as job credentials (used by the job system), collection credentials, and monitoring credentials (used by OMA).

Core concepts and definitions:

- **Credential type** is the type of authentication supported by a target type. For example, a host can support a username/password–based authentication, public key authentication, or Kerberos authentication. Various authentication schemes are supported, including native agent authentication and SSH.
- **Credential set** is a placeholder for a credential. Credential sets can be used to decouple credentials from the system that uses a credential. A credential set enables you to change its mapping to named credentials for a target without editing the system that uses the credential. For example, you could have a credential set for patching tasks.
- **Credential store** is a logical store for all the named credentials of an Enterprise Manager administrator.

Defining credentials by:

- **Credential name:** The credential is referenced by using the name of the credential in the credential store.
- **Credential set:** The credential is referenced by using the credential set name and the target name. The lookup gets the credential associated with the credential set name and target name.
- **Direct value:** The credential is specified by providing the values of the attributes. This reference does not refer to a credential in the credential store.

For information about how to set credentials, follow the demonstrations:

- *Oracle Enterprise Manager 12c: Create and Use Named Credentials* under Oracle Learning Library
- *Oracle Enterprise Manager 12c: Create SSH Key Named Credentials* under Oracle Learning Library
- *OBE Enterprise Ready Framework: Create and Use Credentials*

## Quiz

Which targets can be managed by using Enterprise Manager Cloud Control?

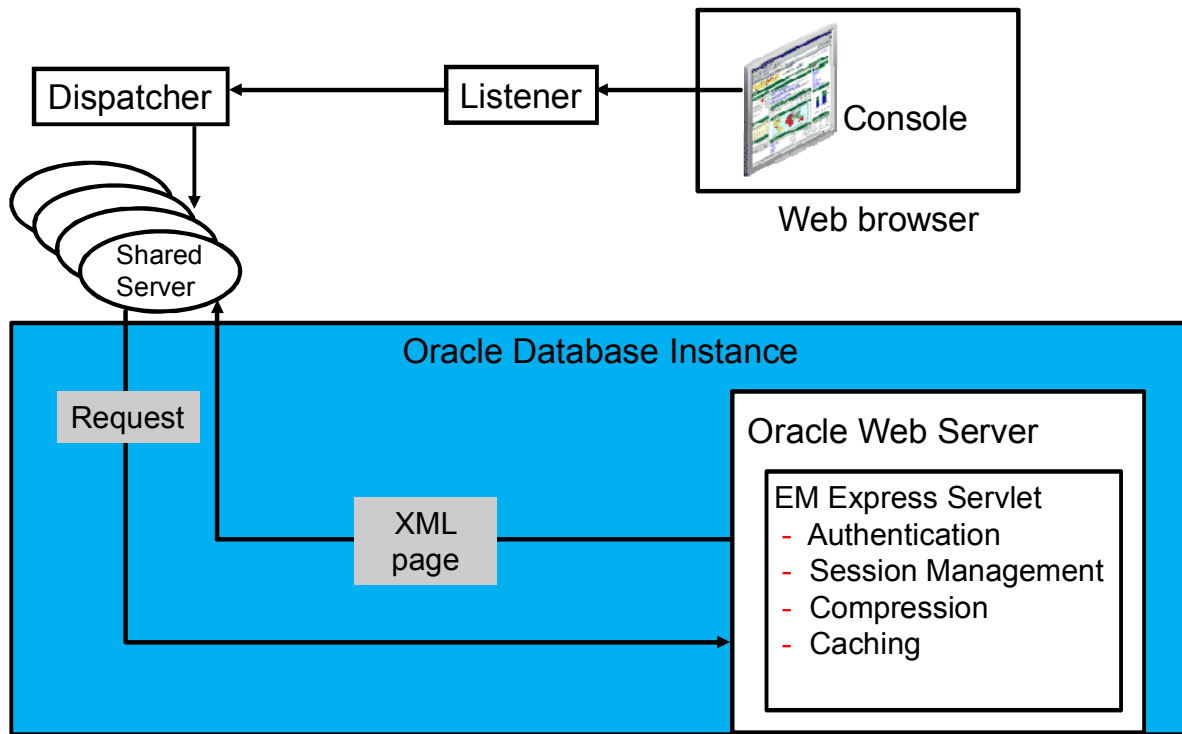
- a. Hosts
- b. Databases
- c. Application servers
- d. Web applications
- e. OMS and OMR
- f. All of the above

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: f**

# Enterprise Manager Database Express Architecture



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enterprise Manager (EM) Database Express is a lightweight administration tool. It provides an out-of-box web-based management solution for a single Oracle database (or database cluster), including performance monitoring, configuration management, administration, diagnostics, and tuning. The goal of EM Database Express is to expose key Enterprise Manager Database features out of box with database installation, with low development cost and small footprint inside database. EM Database Express has a small footprint of 50-100 MB. It runs inside the database with minimal CPU and memory overhead because the database only runs SQL calls but the UI rendering is done in the browser itself.

EM Express uses a web-based console communicating with the built-in web server available in XML DB. As requests from the console are processed, the EM Express servlet handles the requests, including authentication, Session Management, compression, and caching. The servlet processes requests for reports and returns XML pages that are rendered by the web browser. Often a single request is made per page to reduce the number of round-trips to the database.

# Configuring EM Database Express

Configure an HTTP or HTTPs listener port for each database instance:

- Verify the `DISPATCHERS` parameter.

```
dispatchers=(PROTOCOL=TCP) (SERVICE=sampleXDB)
```

- Use `DBMS_XDB_CONFIG.setHTTPsPort` procedure.

```
exec DBMS_XDB_CONFIG.setHTTPsPort(5500)
```

- Connect to EM Database Express console with:

```
https://hostname:5500/em
```

- Use a different port for each instance.
- Browser requires Flash plug-in.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

With Oracle Database 12c, EM Database Control is no longer available and is replaced by EM Database Express. EM Database Express is configurable with a single click in Database Configuration Assistant (DBCA).

EM Database Express requires that the XMLDB components are installed. All Oracle version 12.1.0 databases will have XMLDB installed.

To activate EM Database Express in a database, verify that the `DISPATCHERS` initialization parameter has at least one dispatcher configured for the XMLDB service with the TCP protocol. Use the `DBMS_XDB_CONFIG.setHTTPsPort` (secured with SSL) or `DBMS_XDB_CONFIG.setHTTPPort` procedure to configure a port used on the server. Connect to the EM Database Express console with the URL shown in the slide if you configured a secured connection. If you configured the port by using the `setHTTPPort` procedure, use a URL like `http://hostname:5500/em`.

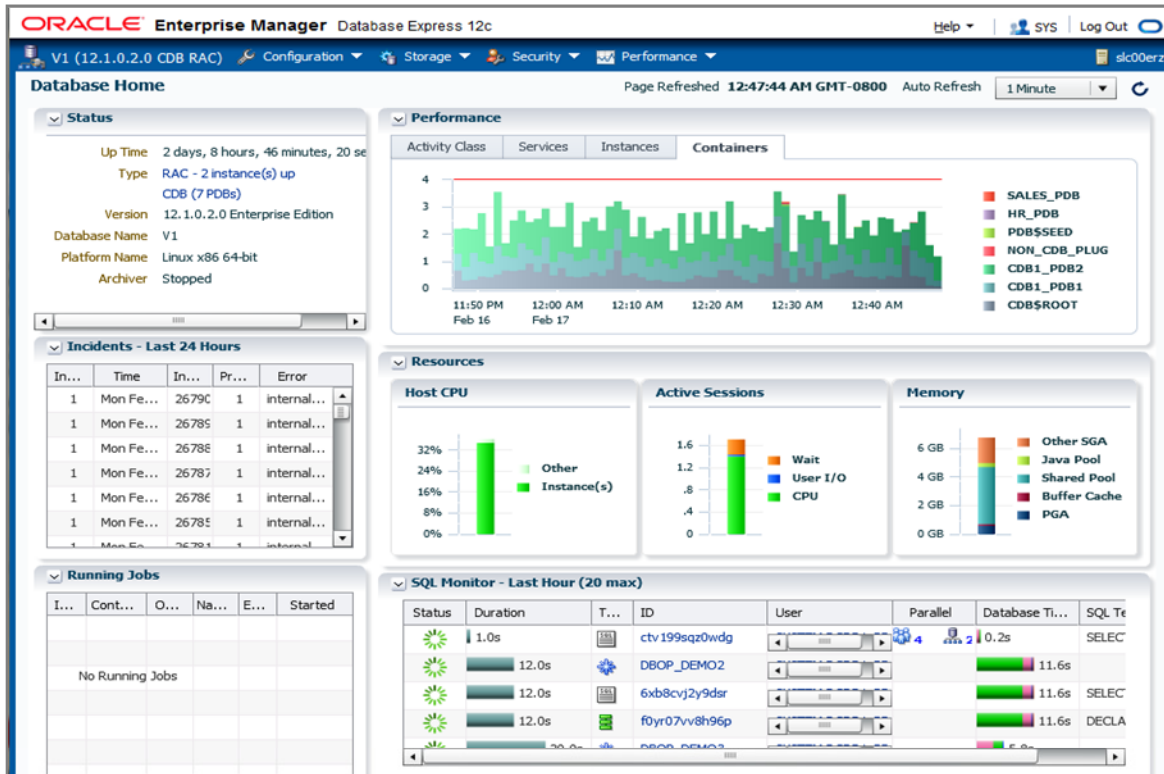
If you have multiple database instances to monitor on the same machine, each one requires a different port. To find the port used for each database instance, use either of the following statements depending on the procedure used for the configuration:

```
SQL> SELECT dbms_xdb_config.gethttpsport FROM DUAL;
```

```
SQL> SELECT dbms_xdb_config.gethttpport FROM DUAL;
```

EM Database Express uses the Shockwave Flash (SWF) files, so the web browser must have the Flash plug-in installed.

# Home Page

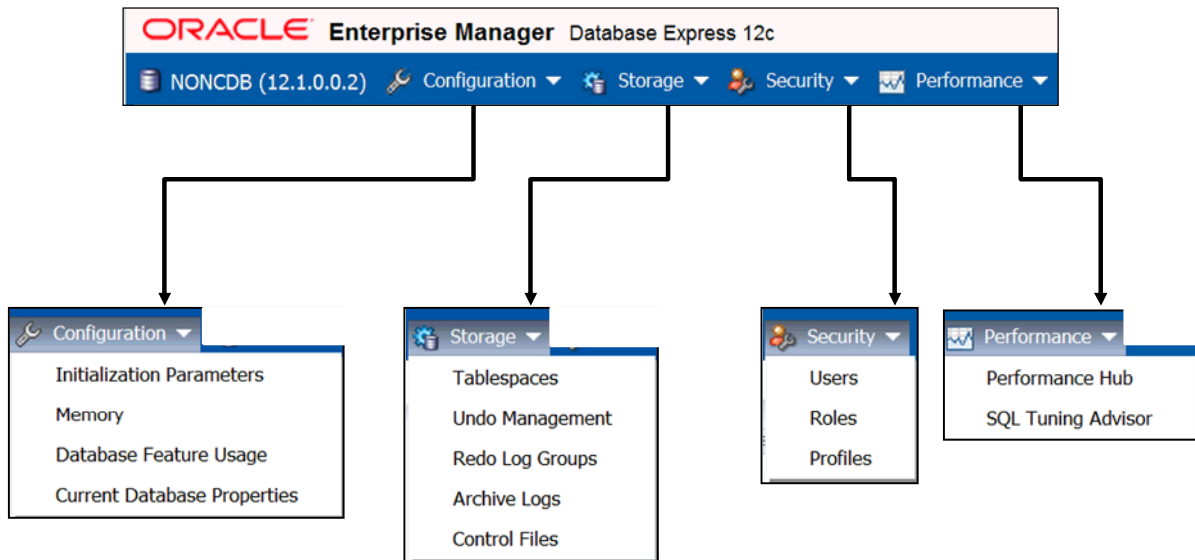


Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The EM Database Express Home page presents an overall view of the database instance status and activity.

EM Database Express is built on the Common Reporting Framework in the database. Because of this, EM Database Express is available only when the database is open. This means that EM Database Express cannot be used to start up the database. Other operations that require that the database change state, such as enable or disable ARCHIVELOG mode, are also not available in EM Database Express.

# Menus



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The menu layout for EM Database Express is shown in the slide. There are four main menu items: Configuration, Storage, Security, and Performance. The menu selections for each of the main items are shown.

The Configuration menu includes: Initialization Parameters, Memory, Database Feature Usage, and Current Database Properties. The Storage menu includes Tablespaces, Undo Management, Redo Log Groups, Archive Logs, and Control Files. The Security menu includes: Users, Roles, and Profiles. The Performance menu includes: Performance Hub and SQL Tuning Advisor.

In each of the menu areas, the menu selection directs you to a page that allows you to manage a particular area. For example, the Configuration > Initialization Parameters selection displays a page that allows you to search, view, and modify current and SPFILE initialization parameters. On many of the pages, when you select an action such as Create Tablespace, you get a pop-up dialog box that allows you to specify parameters, and then it creates a SQL command to perform the action. You may view the SQL command before you submit it, or you can copy and paste the SQL command.

**Note:** For a complete understanding of Oracle Enterprise Manager Database Express usage, refer to the following source of information:

- *Performance New Features for Oracle Database 12c* self-study series



## Quiz

Enterprise Manager Database Express provides database administration functions for:

- a. A single-instance database
- b. Cluster database instances

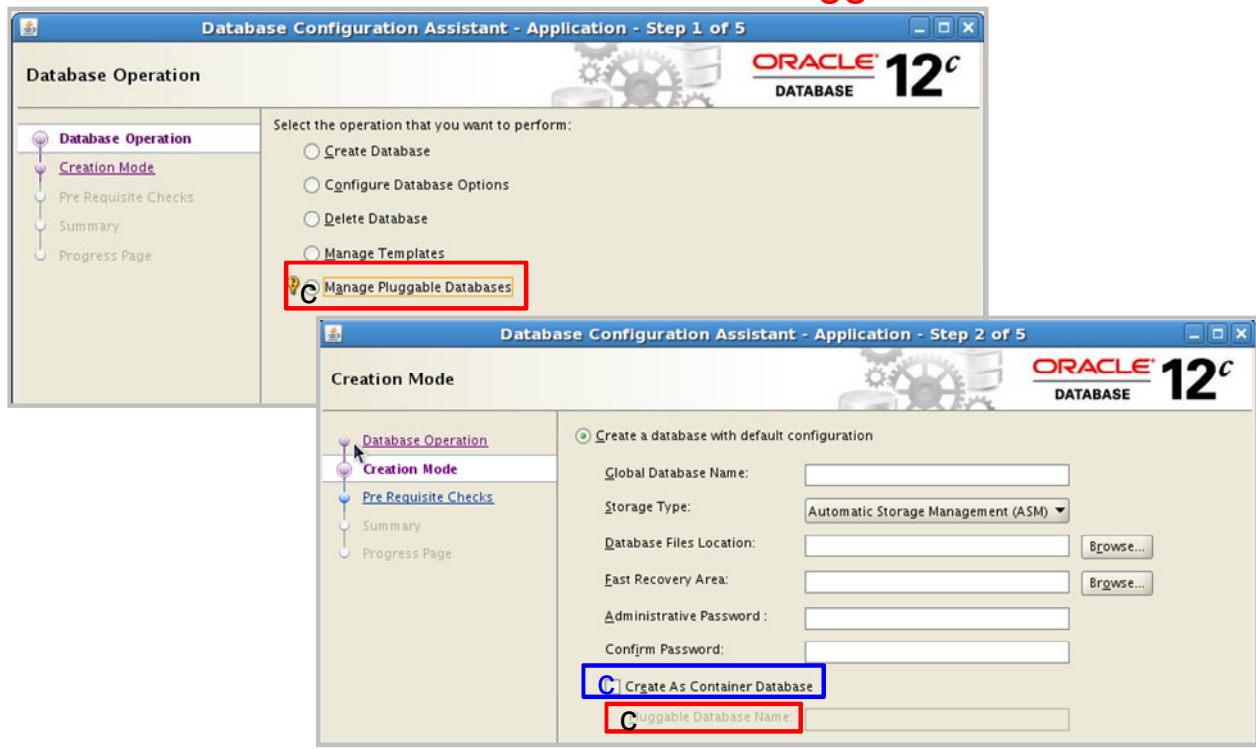
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a, b**

# Database Configuration Assistant

Multitenant container database and Pluggable database



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

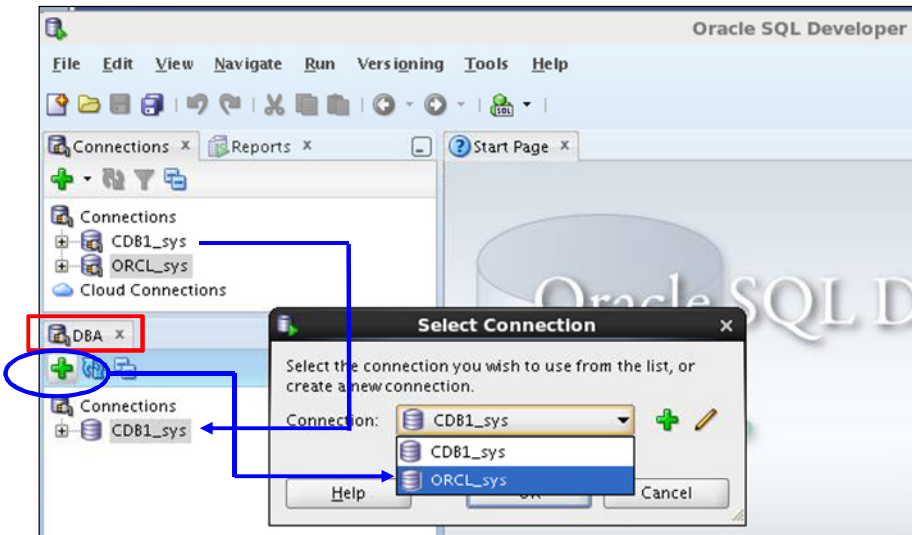
The Database Configuration Assistant in Oracle Database 12c allows the creation and management of new types of database:

- Multitenant container database
- Pluggable database

The creation and management of these new types of database are covered in the following module titled “Multitenant Container Databases and Pluggable Databases.”

## Oracle SQL Developer: Connections

Perform DBA operations in the **DBA navigator** by using **DBA connections**:



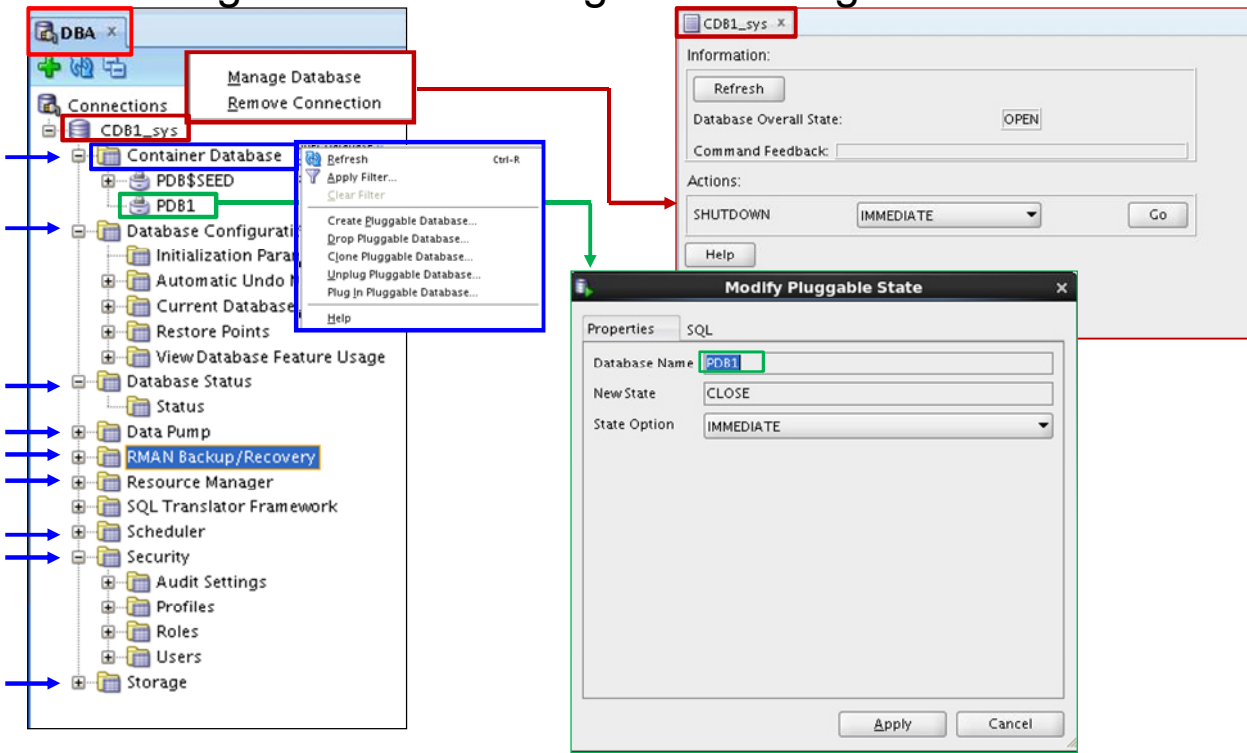
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle SQL Developer is a tool that allows stand-alone graphical browsing and development of database schema objects, as well as execution of database administrative tasks. SQL Developer enables users with database administrator privileges to view and edit certain information relevant to DBAs and perform DBA operations. To perform DBA operations, use the DBA navigator, which is similar to the Connections navigator in that it has nodes for all defined database connections. If the DBA navigator is not visible, select View, then DBA. You should add only connections for which the associated database user has DBA privileges or at least privileges for the desired DBA navigator operations on the specified database.

# Oracle SQL Developer: DBA Actions

Performing DBA tasks through **DBA** navigator:



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBA operations that can be performed are the following:

- Pluggable database startup/shutdown
- Database configuration: Initialization Parameters, Automatic Undo Management, Current Database Properties, Restore Points, View Database Feature Usage
- Database status view
- Data Pump Export and Import jobs
- RMAN Backup/Recovery actions
- Resource Manager configuration
- Scheduler setting
- Security configuration like audit settings, profiles, roles, users
- Storage configuration for archive logs, control files, data files, redo log groups, tablespaces, and temporary tablespace groups

## Quiz

Oracle SQL Developer allows database administration operations.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Summary

In this lesson, you should have learned how to:

- Access and customize the Oracle Enterprise Manager Cloud Control interface
- Set named credentials
- Navigate in Oracle Enterprise Manager Database Express
- Use DBCA with new functionalities
- Use the SQL\*Developer interface

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 2-1: Accessing Enterprise Manager Cloud Control
- 2-2: Adding a database instance as a new target monitored by EM CC
- 2-3: Creating a new named credential
- 2-4: Using the named credential

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

View the “Oracle Enterprise Manager 12c: Console Overview and Customization” demonstration (unless your instructor just demonstrated those topics) (8 mins).

Optional demonstrations about related topics are available:

- Oracle Enterprise Manager 12c: Create an Enterprise Manager Administrator (6 mins)
- Oracle Enterprise Manager 12c: Create and Use Named Credentials (6 mins)
- Oracle Enterprise Manager 12c: Create SSH Key Named Credentials (3 mins)
- Oracle Enterprise Manager 12c: Discover and Promote Unmanaged Hosts and Targets

Optional Oracle by Example (OBEs) about related topics are available:

- Oracle Enterprise Manager 12c Enterprise Ready Framework: Create and Use Credentials (60 mins)
- Oracle Enterprise Manager 12c Enterprise Ready Framework: Create a Super Administrator Account (5 mins)

# **Module**

## **Multitenant Container Database and Pluggable Databases**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



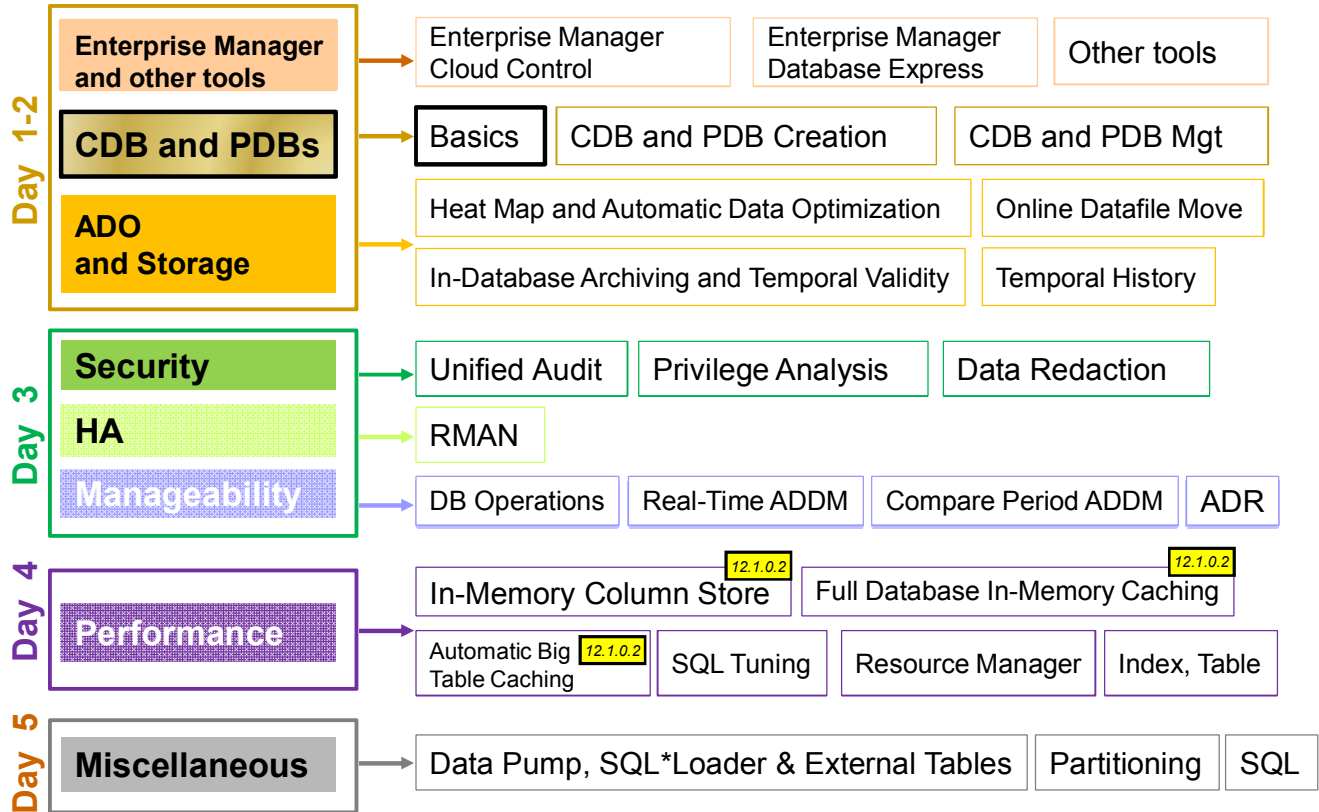
# 3

## Basics of Multitenant Container Database and Pluggable Databases

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Database Consolidation:** Oracle Multitenant is a new option in Oracle Database 12c. The multitenant architecture enables an Oracle database to contain a portable set of schemas, objects, and related structures that appears logically to an application as a separate database.

# Objectives

After completing this lesson, you should be able to:

- Describe the multitenant architecture
- Describe the root and pluggable database containers
- Differentiate the root from a pluggable database
- Explain pluggable database plugging
- List impacts in various areas

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of the multitenant architecture and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series* demonstrations:
  - *Multitenant Architecture*
  - *Oracle Database 12c Multitenant Architecture Diagram Overview*
  - *Basics of CDB and PDB Architecture*
- *Oracle By Example (OBE)*:
  - *Oracle Database 12c Multitenant OBEs*
  - *Performing Basic Tasks in Oracle Multitenant*

Refer to MOS notes:

- *Oracle Multitenant Option - 12c : Frequently Asked Questions (Doc ID 1511619.1)*

Refer to the "Oracle Multitenant" white paper under Oracle Technology

(<http://www.oracle.com/technetwork/database/multitenant/overview/multitenant-wp-12c-2078248.pdf>)

## Challenges

Many Oracle customers have large numbers of “departmental” applications built on Oracle RDBMS. They:

- Do NOT use a significant percentage of the hardware on which they are deployed
- Have instance and storage overhead preventing large numbers of “departmental” databases from being placed on the same physical and storage server
- Are NOT sufficiently complex to require 100% of the attention of a full time administrator
- Do require significant time to patch or upgrade all applications

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

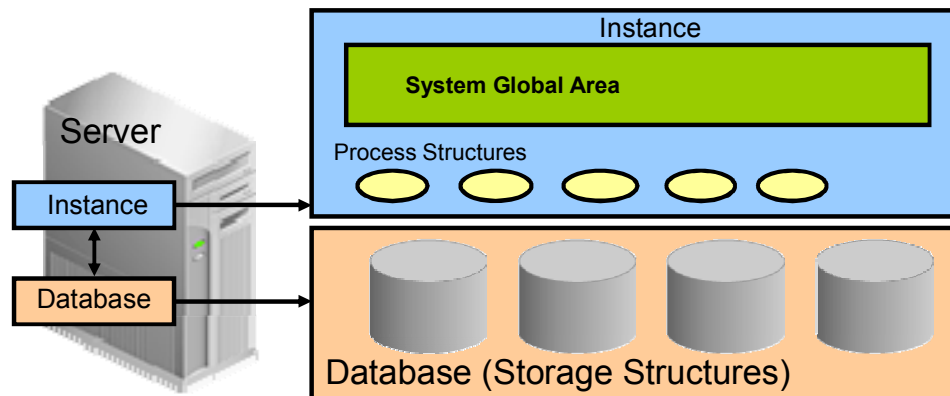
Oracle Database 12c brings a new architecture that lets you have many *pluggable databases* inside a single Oracle Database occurrence. What is the benefit of using the Oracle Multitenant new option in Oracle Database 12c?

Currently, many Oracle customers have large numbers of “departmental” applications built on Oracle RDBMS.

- These applications rarely use a significant percentage of the hardware on which they are deployed. A significant number of instances, and amount of storage allocation for all these small databases prevent these from being placed on the same physical and storage server.
- Moreover, they are typically not sufficiently complex to require 100% of the attention of a full time administrator.
- To better exploit hardware and DBA resources, customers would prefer to have most of these departmental applications consolidated onto a single Oracle RDBMS deployment.

The Oracle Multitenant option allows DBAs to consolidate large numbers of small departmental database applications into a single, larger RDBMS installation.

## Oracle Database in 11g Release 2



Multiple non-CDBs share nothing:

- Too many background processes
- High shared/process memory
- Many copies of Oracle metadata

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle 11g database, the only kind of database that is supported is a non-CDB. The old architecture is referred to as the non-CDB architecture—the term *non-CDB* will be used as a shorthand for an occurrence of a pre-12.1 database that uses the pre-12.1 architecture—requires its own instance and, therefore, its own background processes, memory allocation for the SGA, and needs to store the Oracle metadata in its data dictionary.

When you have to administer small departmental database applications, you have to create as many databases as applications and therefore multiply the number of instances, consequently the number of background processes, memory allocation for the SGAs, and provision enough storage for all data dictionaries of these databases.

When you need to upgrade your applications to a new version, you have to upgrade each database, which is time-consuming for the DBA.

## New Multitenant Architecture: Benefits

- Operates **multiple databases in a centrally managed platform** to lower costs:
  - Less instance overhead
  - Less storage cost
- Reduces DBA resources costs and maintains security
  - No application changes
  - **Fast and easy provisioning**
    - Time saving for patching and upgrade
    - Separation of duties between:
      - Different application administrators
      - Application administrators and DBA
      - Users within application
- **Provides isolation**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Consolidating many non-CDB databases onto a single platform reduces instance overhead, avoids redundant copies of data dictionaries, and consequently storage allocation, and benefits from fast provisioning, time saving upgrading, better security through separation of duties and application isolation. The new 12c database that consolidates databases together is a multitenant container database or CDB, and a database consolidated within a CDB, a pluggable database or PDB.

*DBA resource costs are reduced with:*

- *No application change and very fast provisioning:* A new database can be provisioned quickly. A clone of a populated database can be created quickly. A populated database can be quickly unplugged from its CDB on one platform and quickly plugged into a CDB on a different platform. A non-CDB can quickly be plugged into a CDB.
- *Fast upgrade and patching of the Oracle Database version:* The cost (time taken and human effort needed) to upgrade many PDBs is the cost of upgrading a single Oracle Database occurrence. You can also upgrade a single PDB by unplugging it and plugging it into a CDB at a different Oracle database version.

The multitenant architecture maintains:

- *Secure separation of duties*: The administrator of an application can do all the required tasks by connecting to the particular PDB that implements its back end. However, someone who connects to a PDB cannot see other PDBs. To manage PDBs as entities (for example, to create or drop or unplug or plug one), the system administrator needs to connect to the CDB. For these specific tasks, new privileges need to be granted.
- *Isolation of applications* that may not be achieved manually unless by using Database Vault, for example. A good example of isolation is dictionary separation enabling Oracle database to manage the multiple PDBs separately from each other and from the CDB itself.

## Other Benefits of Multitenant Architecture

- Ensures **full backwards-compatibility** with non-CDBs
- Fully operates with RAC and Data Guard
- Is supported by Enterprise Manager
- Is integrated with Resource Manager
- Allows central management and administration of multiple databases
  - Backups or disaster recovery
  - Patching and upgrades

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

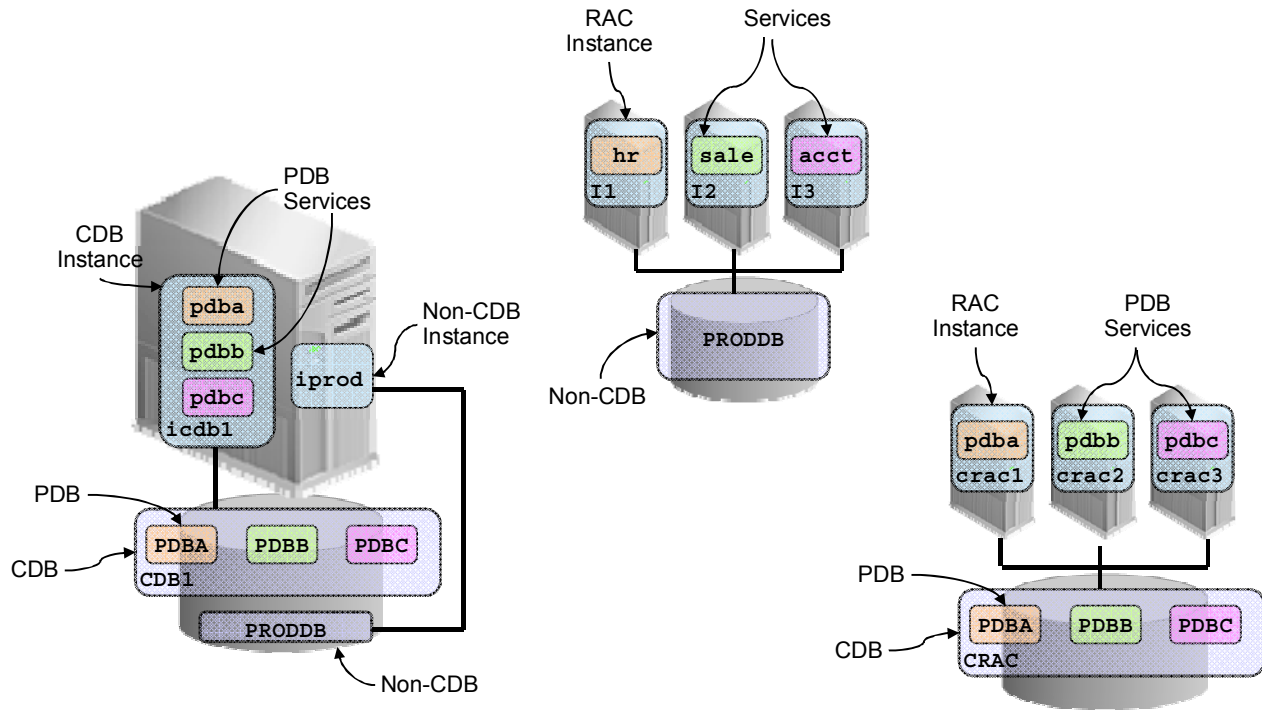
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- The multitenant architecture ensures the backwards-compatibility principle. An example is the data dictionary views. The `DBA_OBJECTS` view shows the same results in a PDB as in a non-CDB for a particular application.
- The multitenant architecture is designed to be fully interoperable with RAC. Each instance in a RAC opens the CDB as a whole. A session sees only the single PDB it connects to.
- Enterprise Manager integrates CDBs and models the separation of duties of the CDB administrator and the PDB administrator.
  - A CDB can be defined as a target. An Enterprise Manager user can be given the credentials to act as a CDB administrator in such a target.
  - A PDB can be set up as a subtarget of a CDB target. An Enterprise Manager user can be given the credentials to act as a PDB administrator in such a target. An Enterprise Manager user that has been set up with the credentials to act as a PDB administrator for a particular PDB is able to connect to that one PDB and is unaware of the existence of peer PDBs in the same CDB. Moreover, when the intention is to carry out the duties of an application administrator, this Enterprise Manager user is unaware that the environment is a CDB and not a non-CDB.



- Resource Manager is extended with new between-PDB capabilities to allow the management of resources between the PDBs within a CDB. The backward-compatibility principle implies that Resource Manager must function in exactly the same way within a PDB as it does in a non-CDB.
- When you upgrade a whole CDB with n PDBs, you achieve the effect of upgrading n non-CDBs for the cost of upgrading one non-CDB.

# Configurations



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Which are the possible instance/database configurations in Oracle Database 12c?

In Oracle Database 11g, each database instance is associated with one and only one database. In a RAC environment, several instances can be associated to a database.

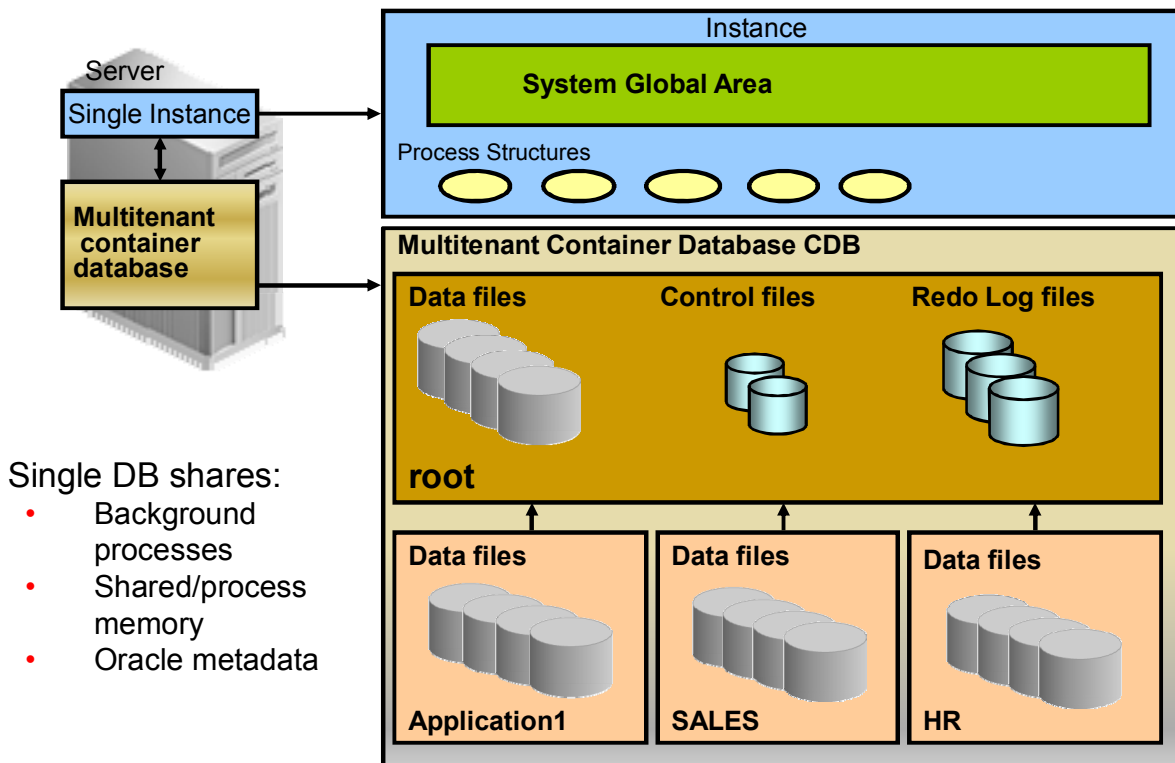
In Oracle Database 12c, an instance is associated with an entire CDB.

If there are multiple databases on the same server, there is a separate and distinct instance for each non-CDB or CDB. An instance cannot be shared between a non-CDB and CDB.

In Oracle Database 12c, there are three possible configuration options:

- **Multitenant configuration:** Typically more than one PDB per CDB, but can hold zero, one, or many PDBs at any one time, taking advantage of the full capabilities of the new architecture, which requires the licensed Oracle Multitenant option
- **Single-tenant configuration:** The special case of the new architecture, which does not require the licensed option
- **Non-CDB:** The old Oracle Database 11g architecture

# Multitenant Container Database



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The slide is an example of the consolidation of three applications that were deployed into three distinct non-CDBs into a single one. The graphic in the slide shows a multitenant container database with four containers: the root, and three pluggable databases. Each pluggable database has its own dedicated application, and is managed either by its own DBA or by the container administrator that is the `SYS` user of the root container, a common user. This common `SYS` user can manage the root container and every pluggable database.

A pluggable database is a set of database schemas that appears logically to users and applications as a separate database. But at the physical level, the multitenant container database has a database instance and database files, just as a non-CDB does. Nothing changes: neither the client code, nor the database objects.

It is easy to plug non-CDBs in to a CDB. A CDB avoids redundancy of:

- Background processes
- Memory allocation
- Oracle metadata in several data dictionaries

A CDB grouping several applications ends up with one instance, consequently one set of background processes, one SGA allocation, and one data dictionary in the root container, common for all PDBs, each PDB maintaining its own application data dictionary.

When applications need to be patched or upgraded, the maintenance operation is performed only once on the CDB and consequently all applications are updated at the same time.

## Pristine Installation

After the initial database creation, the only objects are Oracle-supplied objects.

| OBJ\$ | TAB\$ | SOURCE\$ |     |
|-------|-------|----------|-----|
|       |       |          |     |
|       |       |          |     |
|       |       |          |     |
|       |       |          |     |
|       |       |          |     |
|       |       |          |     |
|       |       |          | ... |
|       |       |          |     |
|       |       |          |     |

Oracle system data

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

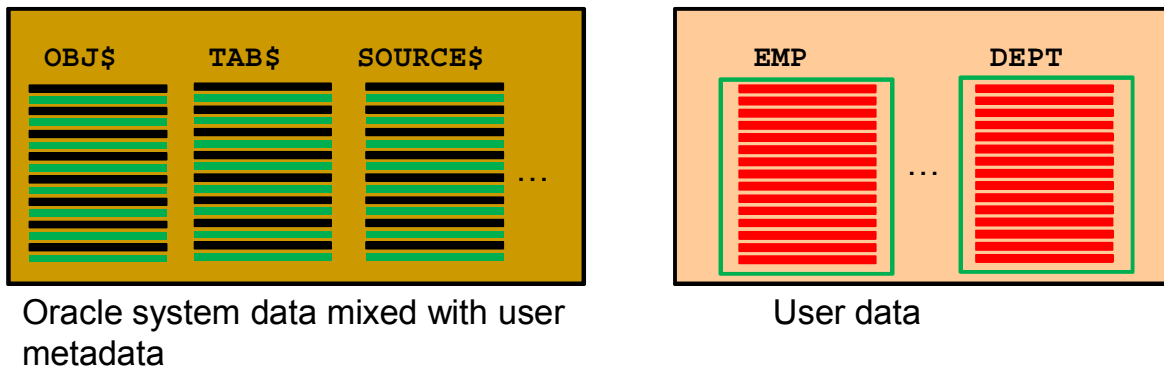
How is Oracle metadata shared between several applications in a non-CDB?

Immediately after the creation of the Oracle database, the only objects in the data dictionary are the Oracle-supplied objects. At this point, there is no user data.

The only schemas in the database are those required by the database system.

## Adding User Data

In a non-CDB, user data is added. The metadata is mixed with the Oracle-supplied data in the data dictionary.



ORACLE

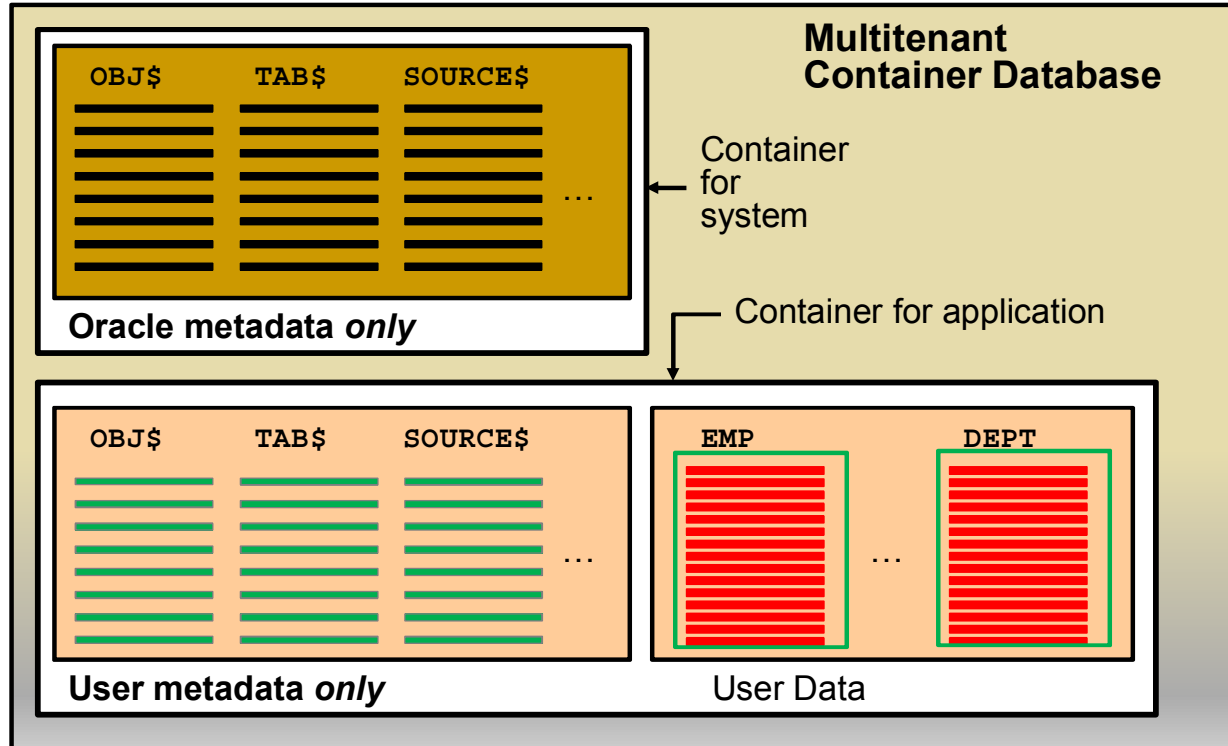
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the non-CDB, users and user data are added.

The best practice is to add this data in tablespaces dedicated to user data. Storing the data in separate tablespaces helps to protect, secure, and transport the data more easily. The metadata, though, is all mixed together in the data dictionary:

- Object definitions
- User definitions
- PL/SQL code
- Other user-created objects

## Separating SYSTEM and User Data



ORACLE

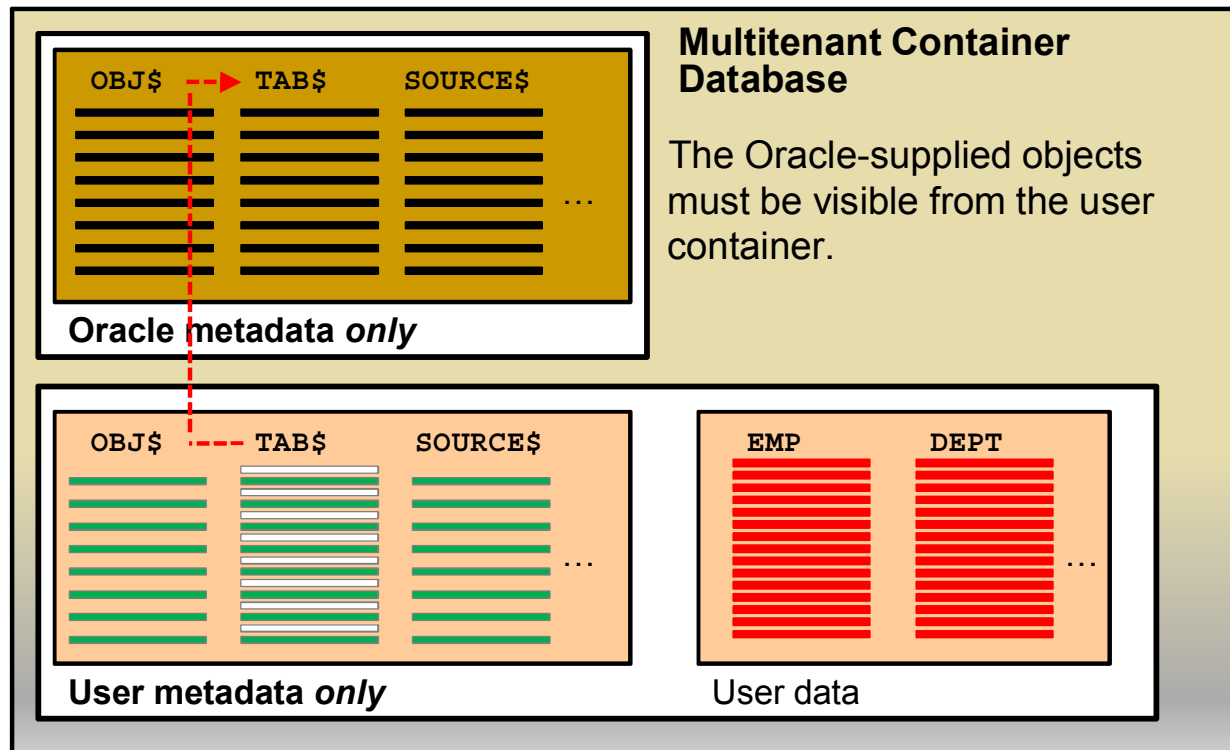
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle 12c, the concept of containers is introduced to separate the Oracle-supplied objects and the user data, including the metadata, into distinct containers.

This can be thought of as a horizontal partitioning. There is a SYSTEM tablespace in each container holding a data dictionary.

- There is a dictionary in the Oracle metadata-only container that has the metadata for the Oracle-supplied objects.
- There is a dictionary in the user container holding the user metadata.

## SYSTEM Objects in the USER Container



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

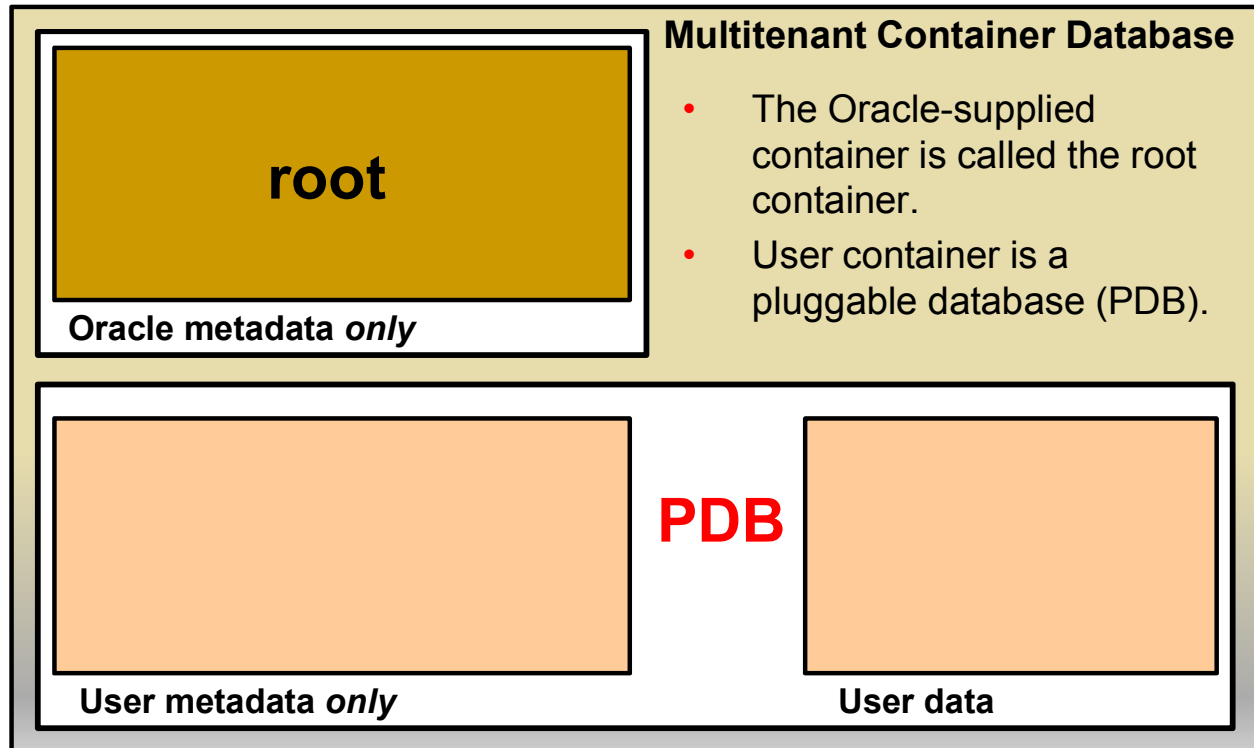
One of the goals of the multitenant architecture is that each container has a one-to-one relationship with an application.

Separating the metadata is the first step, the second is allowing the application or users inside the “user” container to access the Oracle-supplied objects.

The Oracle objects could have been duplicated in each user container, but that takes a lot of space, and would require every user container to be upgraded each time an Oracle-supplied object changes, for example, with patches.

A simple solution is to provide pointers from the user container to the Oracle-supplied objects to allow these “system” objects to be accessed without duplicating them in the user container. The user container has the pieces it needs to be a complete environment for a database application. The application can run in the container just as it does in a non-CDB.

## Naming the Containers



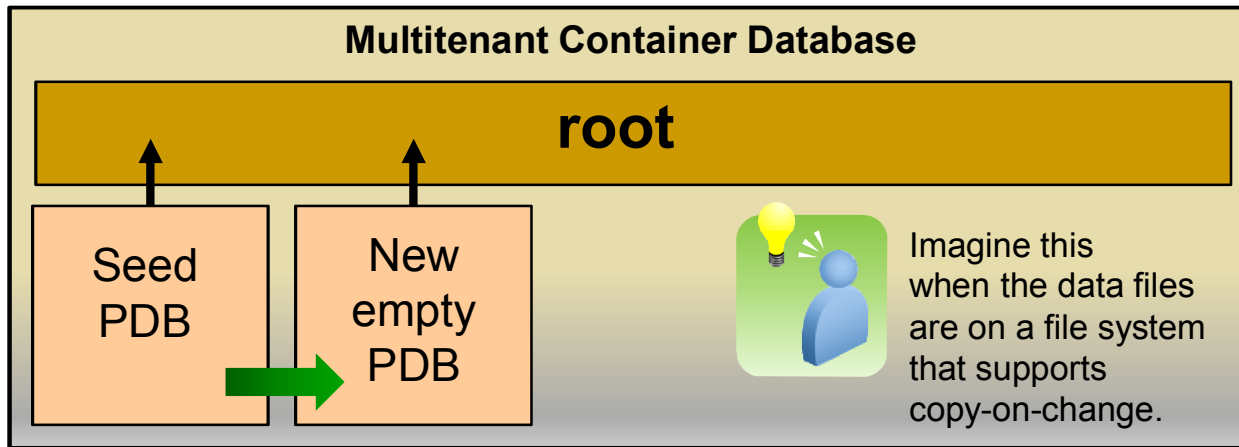
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle-supplied objects reside in a container called the root container (named CDB\$ROOT). The user container is called a pluggable database (PDB) and has the name you give it when creating it or plugging it into the CDB.



# Provisioning a Pluggable Database



Different methods:

- Create new PDB from `PDB$SEED` pluggable database.
- Plug in a non-CDB.
- Clone a non-CDB into a CDB, or a PDB into the same or another CDB.
- Plug an unplugged PDB into a CDB.

ORACLE

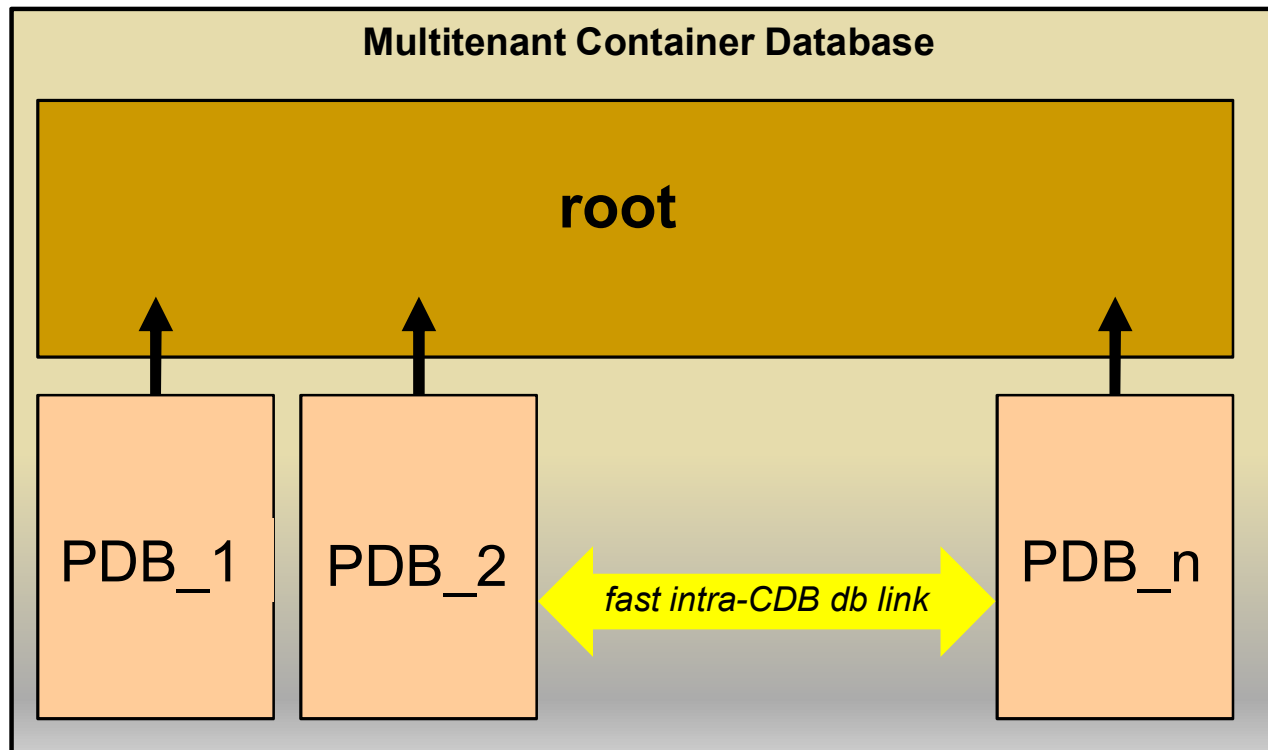
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To create a new PDB, use the provided seed PDB. This seed container is named `PDB$SEED` and is a part of every CDB. When you create a new PDB, the seed PDB is cloned and gives the new PDB the name you specify. This operation is very fast. It is measured in seconds. The time is mostly to copy the files.

There are four methods to provision pluggable databases:

- Create a new PDB from `PDB$SEED` pluggable database: for example, for a brand new application implementation.
- Create a new PDB from a non-CDB: plug the non-CDBs in a CDB as PDBs, as part of migration strategy. It is also a good way to consolidate the non-CDBs into a CDB.
- Clone a PDB from another PDB into the same or another CDB: an example of this method is application testing.
- Plug an unplugged PDB into a CDB, for example, instead of upgrading a multitenant container database from one release to another, you can unplug a pluggable database from one Oracle Database release, and then plug it in to a newly created multitenant container database from a higher release.

## Interacting Within Multitenant Container Database



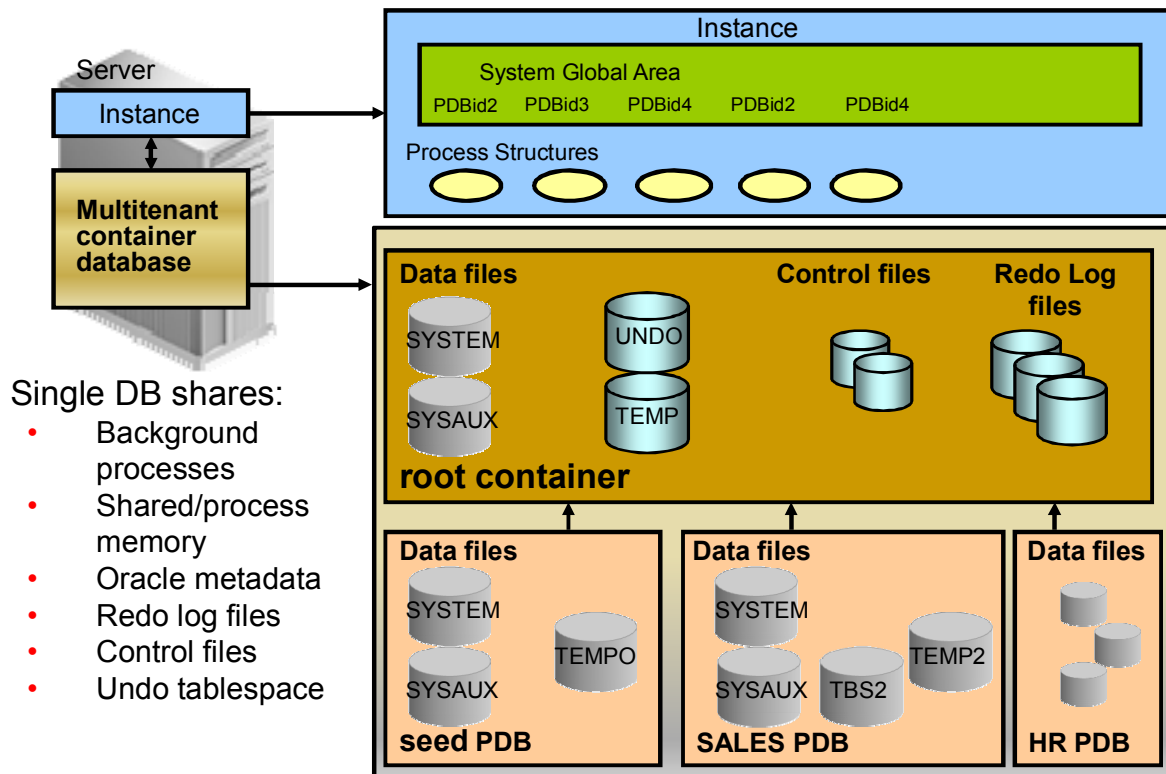
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

With a multitenant architecture that holds several PDBs, these once separated non-CDBs may now reside in a single instance, sharing memory, disk, and CPU resources, but maintaining application separation.

These databases shared data by using database links. The database link still works, but now because the "link" communication does not leave the instance, the link is very fast.

# Multitenant Container Database Architecture



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The graphic in the slide shows a CDB with four containers: the root, the seed, and two PDBs. The two applications use a single instance and are maintained separately.

At the physical level, the CDB has a database instance and database files, just as a non-CDB does.

- The redo log files are common for the whole CDB. The information it contains is annotated with the identity of the PDB where a change occurs. Oracle GoldenGate is enhanced to understand the format of the redo log for a CDB. All PDBs in a CDB share the ARCHIVELOG mode of the CDB.
- The control files are common for the whole CDB. The control files are updated to reflect any additional tablespace and data files of plugged PDBs.
- There is only one UNDO tablespace per CDB instance.
- The root or a PDB can have only one default temporary tablespace or tablespace group. Each PDB can have temporary tablespaces for use by local or common users in the PDB.
- Each container has its own data dictionary stored in its proper `SYSTEM` tablespace, containing its own metadata, and a `SYSAUX` tablespace.
- The PDBs can create tablespaces within the PDB according to application needs.
- Each data file is associated with a specific container, named `CON_ID`.

# Containers

Two types of containers in `V$CONTAINERS`:

- The root container:
  - The first container created at CDB creation
  - Mandatory
  - Oracle system-supplied common objects and metadata
  - Oracle system-supplied common users and roles
- Pluggable database containers (PDBs):
  - A container for an application:
    - Tablespaces (permanent and temporary)
    - Schemas / objects / privileges
    - Created / cloned / unplugged / plugged
  - Particular seed PDB:
    - `PDB$SEED` provides fast provisioning of a new PDB
  - Limit of 253 PDBs in a CDB including the seed
  - Limit of 1024 services in a CDB

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To summarize, a CDB is an Oracle database that contains the root and eventually several pluggable databases.

What is a pluggable database (PDB) in a CDB? A PDB is the lower part of the horizontally partitioned data dictionary plus the user's quota-consuming data.

A non-CDB cannot contain PDBs.

The multitenant architecture enables an Oracle database to contain a portable collection of schemas, schema objects, and nonschema objects that appears to an Oracle Net client as a separate database. For the PDBs to exist and work, the CDB requires a particular type of container, the root container, generated at the creation of the CDB. The root is a system-supplied container that stores common users, which are users that can connect to multiple containers, and system-supplied metadata and data. For example, the source code for system-supplied PL/SQL packages is stored in the root. When the root container exists, you can create containers of the other type, the PDB.

There is only one seed PDB in a CDB. The seed PDB is a system-supplied template that is used to create new PDBs.

A CDB can contain up to 253 PDBs including the seed. 252 user-defined PDBs can therefore be created.

The `V$CONTAINERS` view displays all containers including the root.



## Questions: Root Versus PDBs

**CDBA**

What belongs to the CDB and not to a specific container?

- Control files and redo log files



What is in the root that is not in PDBs?

- UNDO tablespace (for each instance)



- System-supplied metadata
- Shared Oracle-supplied data
  - PL/SQL Oracle-supplied packages (DBMS\_SQL ...)
  - PDBs service names

Table SYS.OBJ\$

| NAME   | TYPE |
|--------|------|
| TAB\$  | 2    |
| USER\$ | 2    |

Table SYS.SERVICE\$

| NAME      |
|-----------|
| PDB_SALES |
| PDB_HR    |

- CDB dictionary views providing information across PDBs
- CDB Resource Manager plan

Views CDB\_xxx

| TABLE_NAME | CON_ID |
|------------|--------|
| EMPLOYEES  | 1      |
| TEST       | 2      |

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

What belongs to the CDB and not to a specific container?

- Control files and redo log files

What is in the root that does not exist in PDBs?

- An UNDO tablespace for each instance of a RAC database
- Oracle-supplied metadata
- Shared Oracle-supplied data
- CDB views providing information across PDBs
- CDB resource manager plan allowing resource management between PDBs within a CDB

**Note:** A multitenant container database administrator (CDBA) is responsible for administering the CDB.



## Questions: PDBs Versus Root

**PDBA**

What is in a PDB that is not in the root nor in another PDB?

- Application tablespaces  **HR\_TBS**
- Local temporary tablespaces  **TEMP1**
- Local users   and local roles  
  - Local users connect to the PDB where they exist.

- Non-shared local metadata

| NAME      | TYPE |
|-----------|------|
| EMPLOYEES | 2    |
| JOBS      | 2    |

Table SYS.OBJ\$

- Non-shared application data with other PDBs

Table HR.EMPLOYEES

| EMP_NAME |
|----------|
| SMITH    |
| JOHN     |

- PDB Resource Manager plan

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

What is in a PDB that is not in the root?

- Application tablespaces
- Local temporary tablespaces
- Local users and local roles
- Non-shared local metadata
- PDB resource manager plan allowing resource management within PDB

**Note:** A pluggable database administrator (PDBA) is responsible for administering its own PDB.

# Terminology

- Common versus Local:
  - Users
  - Roles
  - Privileges
- CDB versus PDB level:
  - CDB Resource Manager plan versus PDB RM plan
  - Unified audit at CDB or PDB level
  - XStream at CDB or PDB level

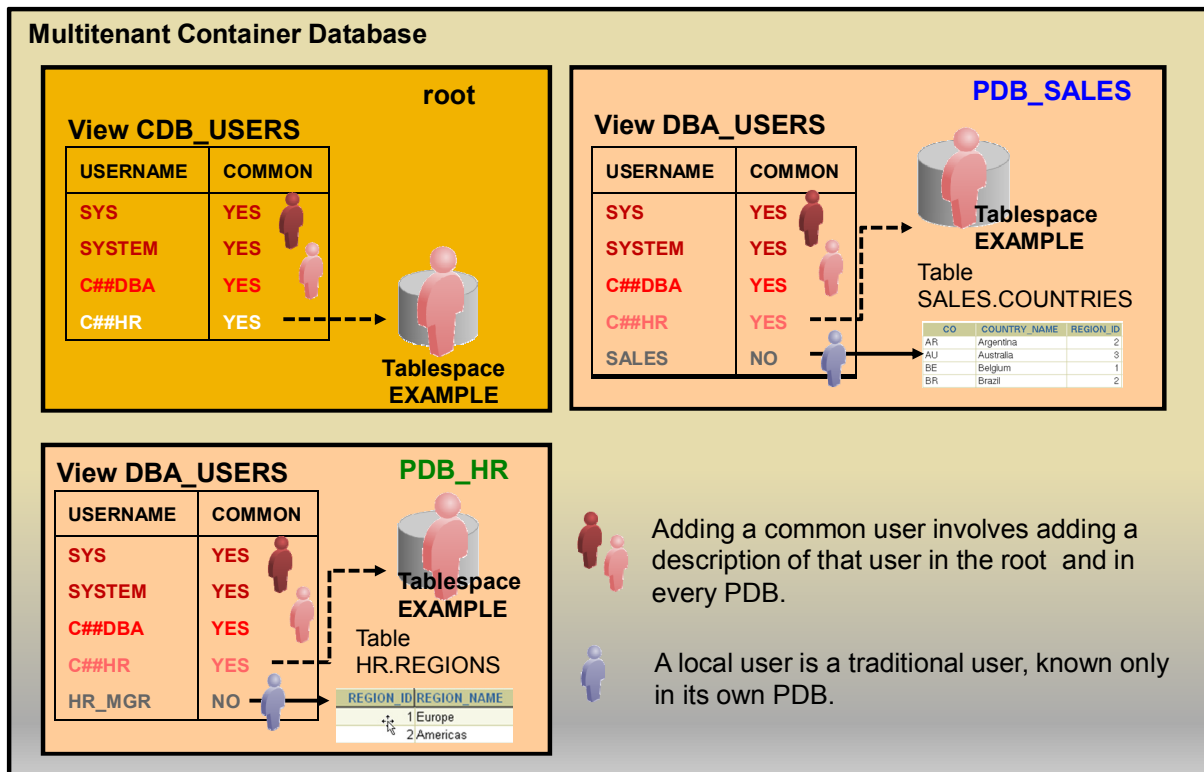
The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There is a new terminology for new entities: common versus local entities, such as:

- Common users versus local users
- Common roles versus local roles
- Common privileges versus local privileges, privileges granted commonly for all containers versus privileges granted locally within a PDB
- CDB resource management versus PDB resource management. CDB resource management works at CDB level, and PDB resource management at PDB level.
- XStream Out is only available at CDB level and XStream In only at PDB level. XStream consists of Oracle Database components and application programming interfaces (APIs) that enable client applications to receive data changes from an Oracle database and send data changes to an Oracle database. These data changes can be shared between Oracle databases and other systems. The other systems include non-Oracle databases, non-RDBMS Oracle products, file systems, and third party software applications. XStream consists of two major features: XStream Out and XStream In. XStream Out provides Oracle Database components and APIs that enable you to share data changes made to an Oracle database with other systems. XStream In provides Oracle Database components and APIs that enable you to share data changes made to other systems with an Oracle database.

# Common and Local Users



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Local User

A user in a non-CDB maps to a *local user* in a PDB.

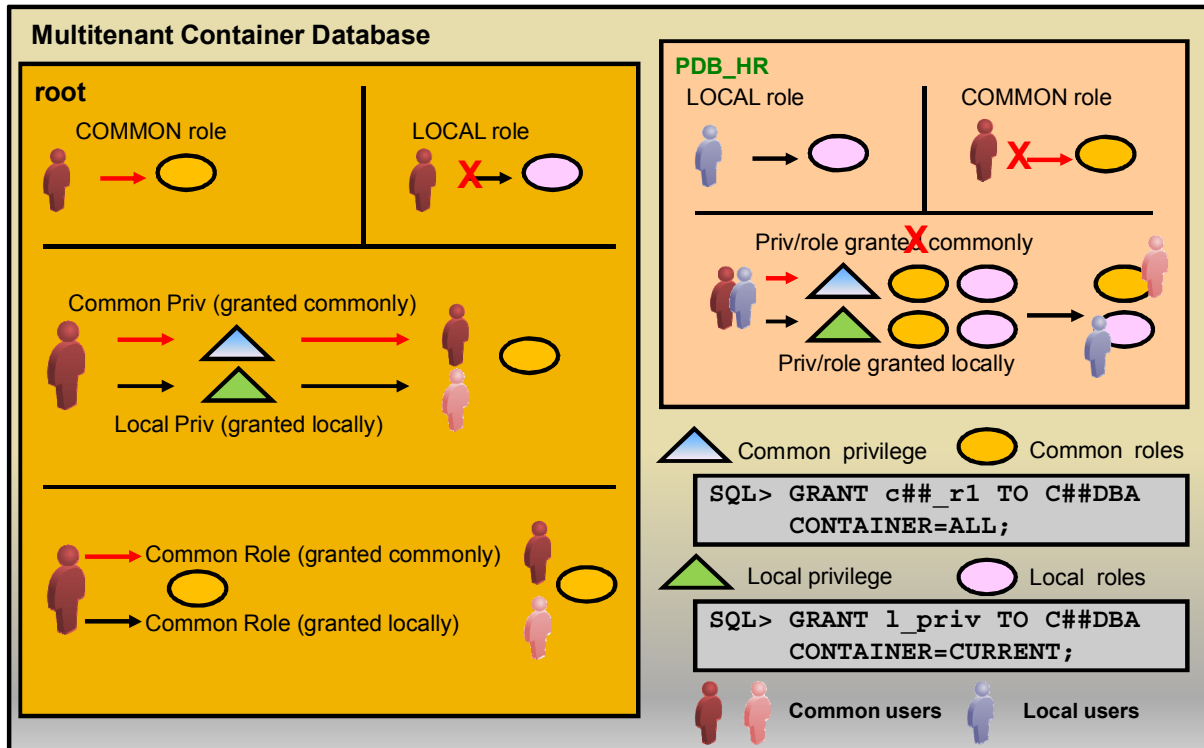
- A local user is defined in the PDB's own data dictionary—and so is not known outside of that PDB.
- A local user can connect only to the PDB where it is defined.
- A local user is specific to a particular PDB and owns a schema in this PDB.
- According to the privileges granted, a user can work on the application data within the PDB or with other PDBs' application using database links. Moreover, there cannot be any local users defined in the root.

## Common User

- A *common user* is defined in the root's data dictionary.
- Only common users can be defined in the root: Creating a common user allows the CDB administrator to create at once a user that is replicated in each PDB.
- A common user is known, not only where it is defined in the root, but also in every PDB that belongs to the CDB.
- A common user can perform administrative tasks specific to the root or PDBs, such as plugging and unplugging PDBs, starting up the CDB, or opening a PDB when granted the proper privileges.



# Common and Local Privileges and Roles



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Local Roles

A role in a non-CDB maps to a *local role* in a PDB. A local role is defined in the PDB's own data dictionary—and so it is not known outside of that PDB and can only be used within its PDB.

## Common Roles

Besides the local role, you can create common roles that are defined in every container. This way, it is easy to create at once a role that is replicated in all PDBs. It is at the creation time that you specify the nature of the role: local or common. Common roles as well as common users can only be created in the root by common users. Moreover, there cannot be any local roles defined in the root. All Oracle-supplied predefined roles are common roles.

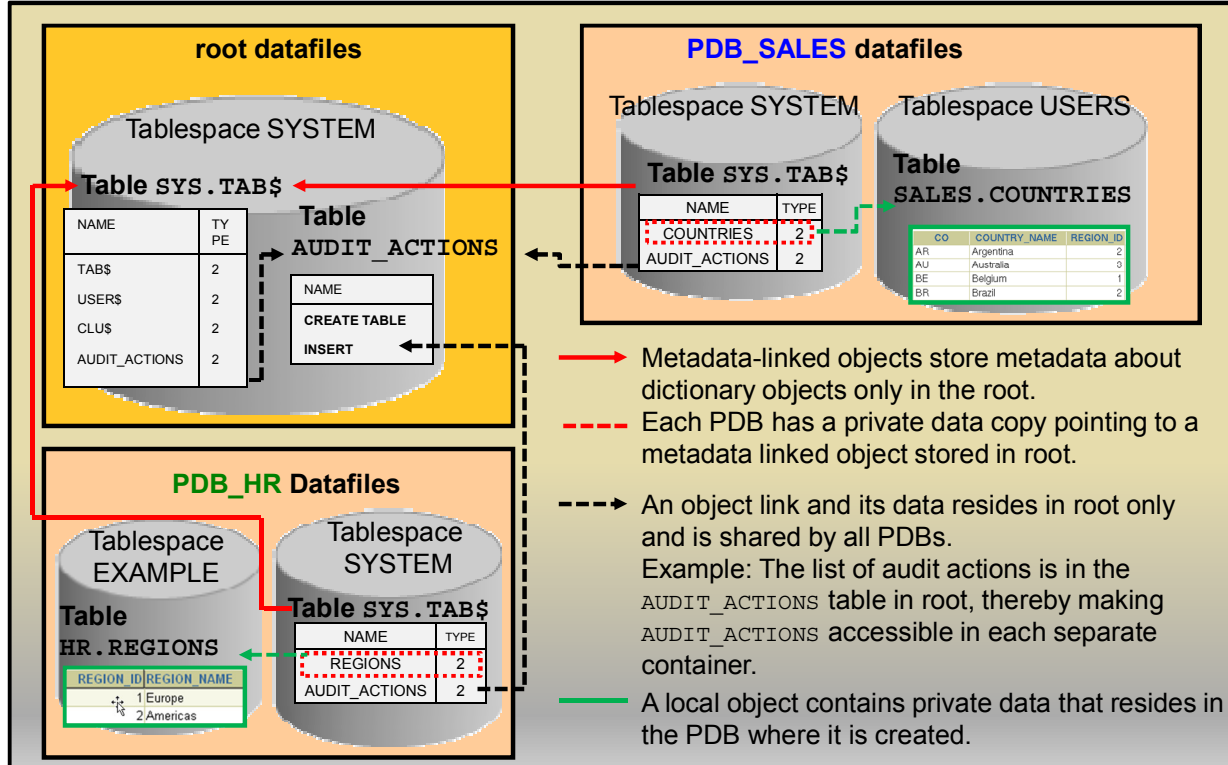
## Local and Common Privileges

The privileges are commonly referred to as local or common privileges, but to be more precise a privilege is granted either locally with the `CONTAINER=CURRENT` clause or commonly with the `CONTAINER=ALL` clause.

The same rule applies to roles: common roles can be granted commonly or locally to common users or roles. Common roles may contain privileges that apply across the CDB, that is, commonly granted for all containers, and can also contain locally granted privileges that apply only to an individual PDB, whereas local roles do not contain any commonly granted privileges.

# Shared and Non-Shared Objects

## Multitenant Container Database



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a non-CDB, all objects are local objects. Common objects are being introduced in 12c to avoid having to store redundant representations of data and metadata across a CDB and to simplify the process of upgrading a CDB. Shared objects exist only in Oracle-supplied schemas.

A shared object may be metadata-linked or object-linked.

- Metadata-linked objects store metadata about dictionary objects only in the root. Each PDB has a private data copy of an object pointing to a metadata link stored in the root.
- An object-linked object and its data resides in the root only and is shared by all PDBs.

### Example

The list of audit actions is in the `SYS.AUDIT_ACTIONS` table in the root, thereby making `AUDIT_ACTIONS` accessible in each separate container.

## Data Dictionary Views

**CDB\_xxx** All objects in the multitenant container database across all PDBs

**DBA\_xxx** All of the objects in a container or pluggable database

**ALL\_xxx** Objects accessible by the current user

**USER\_xxx** Objects owned by the current user

```
SQL> SELECT view_name FROM dba_views WHERE view_name like 'CDB%';
```

- CDB\_pdb\$ : All PDBS within CDB
- CDB\_tablespace\$ : All tablespaces within CDB
- CDB\_users\$ : All users within CDB (common and local)

DBA dictionary views providing information within PDB:

```
SQL> SELECT table_name FROM dict WHERE table_name like 'DBA%';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For backwards-compatibility, DBA views show the same results in a PDB as in a non-CDB: DBA\_OBJECTS shows the objects that exist in the PDB from which you run the query. This implies, in turn, that though the PDB and the root have separate data dictionaries, each data dictionary view in a PDB shows results fetched from both of these data dictionaries. The DBA\_xxx views in the root shows, even in a populated CDB, only the Oracle-supplied system, as is seen in a freshly created non-CDB. This is another advantage of the new architecture. To support the duties of the CDB administrator, a new family of data dictionary views is supported with names like CDB\_xxx. Each DBA\_xxx view has a CDB\_xxx view counterpart with an extra column, *Con\_ID*, that shows from which container the listed facts originate. Query the CDB\_xxx views from the root and from any PDB. The CDB\_xxx views are useful when queried from the root because the results from a particular CDB\_xxx view are the union of the results from the DBA\_xxx view counterpart over the root and all currently open PDBs. When a CDB\_xxx view is queried from a PDB, it shows only the information that it shows in its DBA\_xxx view counterpart. If you connect to the root and query CDB\_USERS, you get the list of users, common and local, of each container. Now if you query DBA\_USERS, you get the list of common users (you know that in the root, only common users exist). Now if you connect to a PDB, and query CDB\_USERS or DBA\_USERS, you get the same list of users, common and local, of the PDB.

The same backward-compatibility principle applies also to each of the familiar v\$views.

## Impacts

- One character set for all PDBs (Unicode recommended)
- PDB initialization parameters but a single SPFILE
- No PDB qualified database object names
  - ~~SELECT \* FROM HR:apps.tab1~~
  - Use DB Links: SELECT \* FROM [apps.tab1@HR](#)
- Oracle Data Guard at CDB level
- Oracle Database Vault per PDB only
- One master key per PDB to encrypt PDB data
- Unified audit both at CDB and PDB levels
- Oracle Scheduler
- Oracle GoldenGate
- Oracle Streams
- Oracle XStream both at CDB and PDB levels

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- There is only one character set and one single spfile for the CDB. The different parameter values of each PDB are stored in a dictionary table.
- There is no PDB qualified object name. To access an object in another PDB, use a DB link.
- Oracle Data Guard works exactly the same in a CDB as it does in non-CDBs. This feature is not to be available on a per-PDB basis. Open of a physical standby CDB will always open the root in READ ONLY mode and this also means that no PDB may be open in a mode other than READ ONLY.
- In Oracle Database Vault, each PDB has its own Database Vault metadata. Database Vault constructs like realms are isolated within a PDB.
- Each PDB has its own master key used to encrypt data in the PDB. The master key must be transported from the source database wallet to the target database wallet when a PDB is moved from one host to another. For column encryption, each PDB maintains its own `ENC$`, which is not a metadata-linked object.

- A unified audit configuration is visible and enforced across all PDBs. This provides the ability to create audit policies used by all PDBs and audit policies used exclusively for each PDB. An audit configuration that is not enforced across all PDBs means it applies only within a PDB and is not visible outside it. A unified audit framework enables administrators to avoid configuring auditing separately for each container.
- In general, all scheduler objects created by the user can be exported/imported into the PDB using data pump. Predefined scheduler objects will not get exported, and that means that any changes made to these objects by the user will have to be made once again when the database has been imported into the pluggable database. However, this is how import/export works currently. A job defined in a PDB runs only if a PDB is open.
- XStream is a programmatic interface to allow a client application access to the changes in the database, known as XStream Outbound Server. XStream Inbound Server allows a client application to feed changes into the database and takes advantage of the apply process available in the database. Oracle GoldenGate is the logical replication and XStream is licensed via the Oracle GoldenGate license. Capturing changes from the database must always be from the root, due to the single unified redo log for the CDB. The XStream outbound can be configured to capture changes from a PDB or the entire CDB. Applying changes via Oracle GoldenGate is done per PDB. An XStream inbound server is configured to apply changes into a specific PDB and performs all of its work within the context of the PDB.
- Oracle Streams continues to be supported in the Oracle Database 12c non-CDBs but is no longer enhanced for new database features or data types. Oracle Streams cannot be used with CDBs.

## Quiz

Which of the following is true?

- a. Oracle-supplied metadata resides only in the root container.
- b. The seed PDB can sometimes be opened for very particular operations.
- c. A PDB can have the same name in different CDBs.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Quiz

Which of the following are true?

- a. There is only one `SYSTEM` tablespace per CDB.
- b. There is only one instance per PDB.
- c. There is a set of redo log files per PDB.
- d. There is only one `UNDO` tablespace per CDB instance.
- e. There is one `SYSAUX` tablespace per PDB.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: d, e**

## Quiz

You can create common users in a PDB.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**



## Summary

In this lesson, you should have learned how to:

- Describe the multitenant architecture
- Describe the root and pluggable database containers
- Differentiate the root from a pluggable database
- Explain pluggable database plugging
- List impacts in various areas

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

### 3-1: Exploring the CDB processes, files, and CDB\_XXX views

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



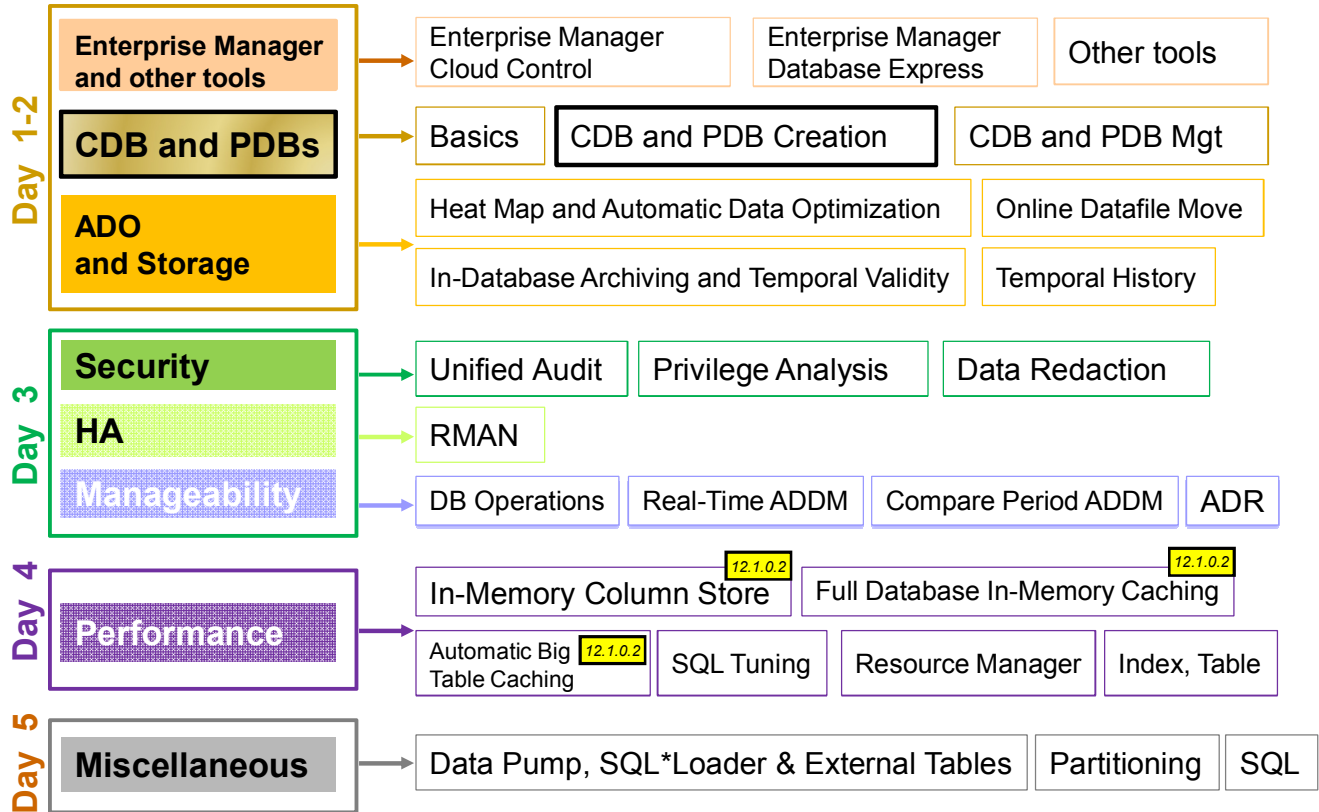
# 4

## Creating Container Databases and Pluggable Databases

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Objectives

After completing this lesson, you should be able to:

- Configure and create a CDB
- Create a PDB from `PDB$SEED`
- Plug or clone a non-CDB into a CDB
- Clone a local or remote PDB into a CDB
- Unplug and plug a PDB from one CDB to another CDB
- Explore the instance
- Explore the structure of PDBs
- Drop a PDB
- Migrate a pre-12.1 non-CDB database to CDB

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Note:** For a complete understanding of Oracle Pluggable Database new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – "DBMS\_PDB" chapter*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series* demonstrations about CDBs and PDBs:
  - *Cloning a PDB using ACFS Snapshots*
- *Oracle By Example (OBE)*:
  - *Unplugging and Plugging Pluggable Databases*
  - *Cloning Pluggable Databases*
  - *Adopting a Non-CDB as a Pluggable Database in a Multitenant Container Database*
  - *Managing Pluggable Databases (PDBs) Using SQL Developer*
- *Oracle White Paper "Upgrading to Oracle Database 12c"*  
(<http://www.oracle.com/technetwork/database/upgrade/upgrading-oracle-database-wp-12c-1896123.pdf>)

## Goals

Create a container database:

- To consolidate many pre-12.1 non-CDBs into a single, larger database
- To prepare a container
  - For plugging any future new application
  - For testing applications
  - For diagnosing application performance
- To simplify and reduce time for patching and upgrade

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Why create container databases?

There are several reasons for creating a container database as explained in the slide.

# Tools

|                                             | SQL*Plus | OUI | DBCA | EM Cloud Control | EM Database Express | SQL Developer  | DBUA |
|---------------------------------------------|----------|-----|------|------------------|---------------------|----------------|------|
| Create a new CDB or PDB                     | Yes      | Yes | Yes  | Yes (PDB only)   | Yes (PDB only)      | Yes (PDB only) |      |
| Explore CDB instance, architecture and PDBs | Yes      |     |      | Yes              | Yes                 | Yes            |      |
| Upgrade a 12.1 CDB to 12.x CDB              |          |     |      | Yes              |                     |                | Yes  |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are different tools to create and upgrade container databases. As shown in the slide, you can create a new CDB or new PDBs by using either SQL\*Plus or Database Configuration Assistant (DBCA) or during the installation of Oracle Database 12c. SQL Developer and EM Cloud Control allow you to create pluggable databases.

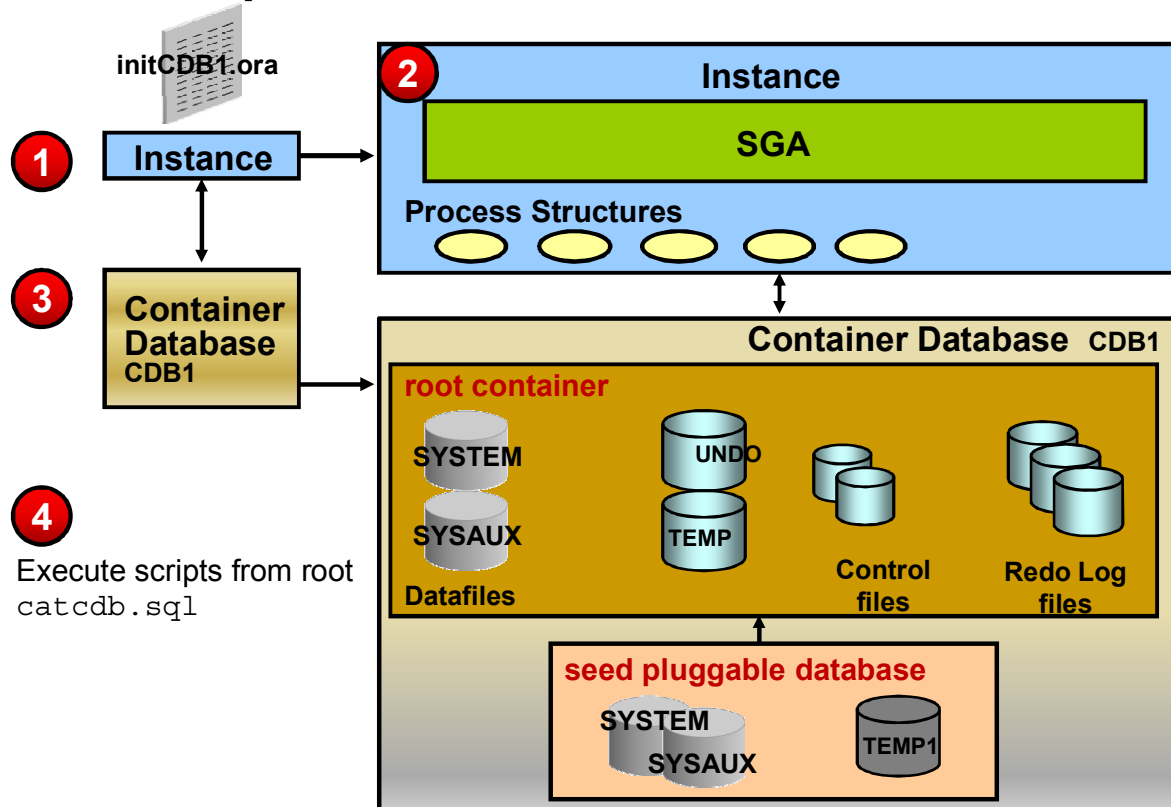
After they are created, use views to explore the instance, database architecture, files, and pluggable databases of the CDB. Query views directly with `SELECT` statements by using SQL\*Plus, or indirectly by using GUI tools such as Enterprise Manager or SQL Developer.

You can upgrade a 12.1 CDB to a 12.x CDB with Enterprise Manager or Database Upgrade Assistant (DBUA).

**Note:** EM Database Express does not provide the capability to create a CDB but it can be used to create PDBs and explore PDBs architecture or CDBs structures by using different port configurations.



# Steps to Create a Container Database



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The steps required to create a new CDB, by using either DBCA or SQL\*Plus, are the same.

- The first step, as for any database, non-CDB or CDB, consists of configuring an instance with an `init.ora` parameter file.
- The second step consists of starting the instance.
- The third step is the creation of the CDB by using the `CREATE DATABASE` command with a new clause `ENABLE PLUGGABLE DATABASE` specifying that the database is a container database and not a non-CDB. The operation creates the root container with the control files during the mount phase and the redo log files and root data files during the open phase. The root data files are used for the `SYSTEM` tablespace containing the Oracle-supplied metadata and data dictionary, and the `SYSAUX` tablespace for AWR. It also creates the seed pluggable database with its own data files used for the `SYSAUX` and `SYSTEM` tablespaces. You may use the new clause `SEED FILE_NAME_CONVERT` to rename the data files of the seed pluggable database while copying from the root container. The clause creates the seed pluggable database and its own data files. The seed data files are copied from the root data files to another location. The seed data files can be used as templates for future PDBs creation. If you omit this clause, Oracle Managed Files determines the names and locations of the seed's files.
- The fourth step is the creation of the catalog and other components with the execution of the `catcdb.sql` script connected to the root container.

# Creating a Container Database: Using SQL\*Plus

## 1. Instance startup:

- a. Set ORACLE\_SID=CDB1
- b. Set in initCDB1.ora:
  - Set CONTROL\_FILES to CDB control file names.
  - Set DB\_NAME to CDB name.
  - Set ENABLE\_PLUGGABLE\_DATABASE to TRUE.

```
SQL> CONNECT / AS SYSDBA
SQL> STARTUP NOMOUNT
```

## 2. Create the database:

```
SQL> CREATE DATABASE CDB1 ENABLE PLUGGABLE DATABASE ...
      SEED FILE_NAME_CONVERT ('/oracle/dbs', '/oracle/seed');
```

- CDB\$ROOT container
- PDB\$SEED pluggable database

## 3. Run the catcdb.sql script.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The detailed steps to create a new CDB by using SQL\*Plus are the following:

1. Before starting the instance, prepare an init<SID>.ora parameter file with the usual parameters: DB\_NAME, CONTROL\_FILES if OMF is not used, DB\_BLOCK\_SIZE. The global database name of the root is the global database name of the CDB. A new parameter is required to define that the instance started is ready for a CDB and not a non-CDB creation. The ENABLE\_PLUGGABLE\_DATABASE parameter must be set to TRUE.  
Set the ORACLE\_SID environment variable. Launch SQL\*Plus, connect as an OS authenticated user belonging to the DBA OS group, and execute the STARTUP NOMOUNT command.
2. Create the CDB by using the CREATE DATABASE command with the new clause ENABLE PLUGGABLE DATABASE. The clause specifies that the database is a CDB and not a non-CDB. This creates the root container and the seed pluggable database. You may use another clause SEED FILE\_NAME\_CONVERT to specify the location of the seed's files. If you omit the clause, OMF determines the names and locations of the seed's files. The FILE\_NAME\_CONVERT specifies the source directory of the root data files copied to target seed directory. Omit the SEED FILE\_NAME\_CONVERT clause if you use the new init.ora parameter PDB\_FILE\_NAME\_CONVERT, mapping names of the root data files to the seed data files. In the example, the /oracle/dbs and /oracle/seed directories must exist. The character set defined in the statement is still the single one for the CDB.

3. Run the `catcdb.sql` SQL script. This script installs all of the components required by a CDB.

# Creating a Container Database: Using DBCA

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating the database with DBCA requires to select the “Create As Container Database” check box, else the database is created as a non-CDB.

You also have to provide a pluggable database name when using the “Create a Database with Default Configuration” check box. If you use the “Advanced Mode” check box, you can create an empty container database with only the root and seed containers.

In Advanced Mode, you can register the CDB with Enterprise Manager Cloud Control and set passwords for SYS and SYSTEM users.

## New Clause: SEED FILE\_NAME\_CONVERT

CREATE DATABASE new clauses:

```
SQL> CREATE DATABASE cdb1
      USER SYS IDENTIFIED BY p1 USER SYSTEM IDENTIFIED BY p2
      LOGFILE GROUP 1 ('/u01/app/oradata/CDB1/redo1a.log',
                     '/u02/app/oradata/CDB1/redo1b.log') SIZE 100M,
      GROUP 2 ('/u01/app/oradata/CDB1/redo2a.log',
              '/u02/app/oradata/CDB1/redo2b.log') SIZE 100M
      CHARACTER SET AL32UTF8 NATIONAL CHARACTER SET AL16UTF16
      EXTENT MANAGEMENT LOCAL DATAFILE
                     '/u01/app/oradata/CDB1/system01.dbf' SIZE 325M
      SYSAUX DATAFILE '/u01/app/oradata/CDB1/sysaux01.dbf' SIZE 325M
      DEFAULT TEMPORARY TABLESPACE tempts1
      TEMPFILE '/u01/app/oradata/CDB1/temp01.dbf' SIZE 20M
      UNDO TABLESPACE undotbs
      DATAFILE '/u01/app/oradata/CDB1/undotbs01.dbf' SIZE 200M
      ENABLE PLUGGABLE DATABASE
      SEED FILE_NAME_CONVERT =
          ('/u01/app/oradata/CDB1',
           '/u01/app/oradata/CDB1/seed');
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can find in the slide an example of a full CREATE DATABASE statement.

What is new compared to the traditional non-CDB CREATE DATABASE statement?

The first important clause required if you want the database to be a container database is **ENABLE PLUGGABLE DATABASE**, and one way to declare the directory for the seed data files is to use the **SEED FILE\_NAME\_CONVERT** clause. The **FILE\_NAME\_CONVERT** specifies the source directory of the root data files copied to target seed directory.

The root `/u01/app/oradata/CDB1` directory and seed `/u01/app/oradata/CDB1/seed` directory must exist.

## New Clause: ENABLE PLUGGABLE DATABASE

Without SEED FILE\_NAME\_CONVERT:

- OMF: DB\_CREATE\_FILE\_DEST='/u01/app/oradata'
- Or new instance parameter:  
PDB\_FILE\_NAME\_CONVERT =  
'/u01/app/oradata/CDB1','/u01/app/oradata/seed'

```
SQL> CONNECT / AS SYSDBA
SQL> STARTUP NOMOUNT

SQL> CREATE DATABASE cdb2
      USER SYS IDENTIFIED BY p1 USER SYSTEM IDENTIFIED BY p2
      EXTENT MANAGEMENT LOCAL
      DEFAULT TEMPORARY TABLESPACE temp
      UNDO TABLESPACE undotbs
      DEFAULT TABLESPACE users
      ENABLE PLUGGABLE DATABASE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Oracle Managed Files

If you do not use explicit data file names, use Oracle Managed Files (OMF):

- Set the DB\_CREATE\_FILE\_DEST instance parameter with the value of the destination directory of the data files of the SYSTEM, SYSAUX, UNDO, and USERS tablespaces specified in the statement. Oracle chooses default sizes and properties for all data files, control files, and redo log files. In the example, the /u01/app/oradata directory must exist.

### PDB\_FILE\_NAME\_CONVERT Instance Parameter

If you do not use the SEED FILE\_NAME\_CONVERT clause, use a new instance parameter:

- PDB\_FILE\_NAME\_CONVERT instance parameter maps names of existing files (the root data files in our case) to new file names (the seed data files in this case). In the example, both /u01/app/oradata/CDB1 and /u01/app/oradata/seed directories must exist.

## After CDB Creation: What Is New in CDBs

A CDB has new characteristics compared to non-CDBs:

- **Two containers:**
  - The **root** (CDB\$ROOT)
  - The **seed** PDB (PDB\$SEED)
- **Several services:** One per container
  - Name of root service = name of the CDB (cdb1)
- **Common users** in root and seed: SYS, SYSTEM ...
- **Common privileges** granted to common users
- **Pre-defined common roles**
- **Tablespaces and data files** associated to each container:
  - **root:** SYSTEM (system-supplied metadata and no user data), SYSAUX, TEMP
  - **seed:** SYSTEM, SYSAUX, TEMP

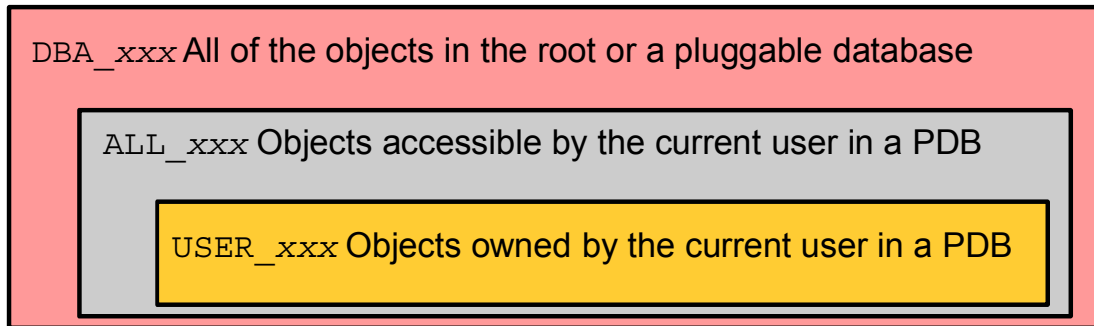
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When the CDB is created, there are new CDB components and objects such as:

- Two containers: The root container and the seed PDB
- As many services as containers: The service name for the root container is the CDB name given at the CDB creation concatenated with the domain name. Each new PDB is assigned a service name: the service name is the PDB name given at PDB creation concatenated with domain name. If you create or plug a PDBtest PDB, its service name would be PDBtest concatenated with domain name. You can find all service names maintained in a CDB in the CDB\_SERVICES view. To connect to the CDB, you connect to the root, by using either local OS authentication or the root service name. For example, if you set ORACLE\_SID to the CDB instance name and use CONNECT / AS SYSDBA, you are connected to the root under common SYS user granted system privileges to manage and maintain all PDBs. To connect to a desired PDB, use either easyconnect or tnsnames.ora file (for example, CONNECT username/password@net\_service\_name).
- Common users: SYS, SYSTEM, created in all containers, the root and the seed
- Common privileges granted to all users in all containers
- Pre-defined common roles in all containers, the root and the seed
- Tablespaces: SYSTEM, SYSAUX, TEMP associated to each container

## Data Dictionary Views: DBA\_xxx



DBA dictionary views providing information within PDB:

```
SQL> SELECT table_name FROM dict WHERE table_name like 'DBA%';
```

- DBA\_tablespaces: All tablespaces of the PDB
- DBA\_data\_files: All data files of the PDB
- DBA\_tables: All tables in the PDB
- DBA\_users: All common and local users of the PDB

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For backward-compatibility, DBA views show the same results in a PDB as in a non-CDB. For example, the DBA\_OBJECTS view shows the objects that exist in the PDB from which you run the query.

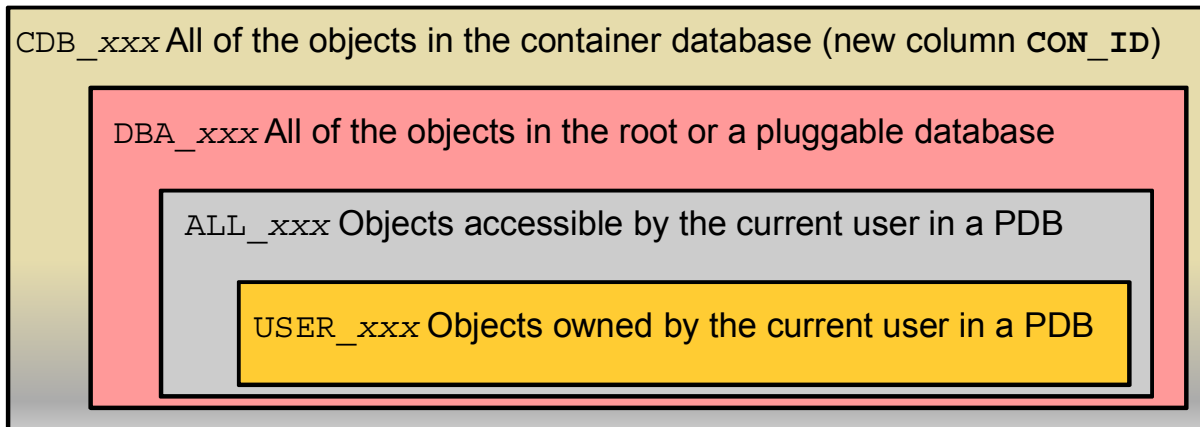
- In the root, DBA\_xxx views show only objects contained in the root.
- In a PDB, the DBA\_xxx views show only objects contained in the PDB.

Examples of DBA views:

- While connected to the root, you query DBA\_USERS. You get the list of common users created from the root (in the root, only common users exist).
- While connected to a PDB, you query DBA\_USERS. You get the list of users, common and local, of the PDB.



## Data Dictionary Views: CDB\_xxx



CDB dictionary views provide information across PDBs:

```
SQL> SELECT view_name FROM dba_views WHERE view_name like 'CDB%';
```

- CDB\_pdb\$ : All PDBS within the CDB
- CDB\_tablespaces: All tablespaces within the CDB
- CDB\_data\_files: All data files within the CDB
- CDB\_users: All users within the CDB (common and local)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a CDB, for every DBA\_\* view, a CDB\_\* view is defined.

- In the root, CDB views can be used to obtain information about tables, tablespaces, users, privileges, parameters, PDBs, and other types of objects contained in the root and all PDBs.
- In a PDB, the CDB\_\* views show objects visible through a corresponding DBA\_\* view only.

In addition to all the columns found in a given DBA\_\* view, the corresponding CDB\_\* view also contains the CON\_ID column, which identifies a container whose data a given CDB\_\* row represents. In a non-CDB, the value of a CON\_ID column is 0. In a CDB, the value can be either 1 used for rows containing data pertaining to the root only or n where n is the applicable container ID.

Examples of CDB views:

- Connected to the root and querying CDB\_USERS, you get the list of users, common and local, of each container.
- Connected to a PDB and querying CDB\_USERS or DBA\_USERS, you get the same list of users, common and local, of the PDB.
- Connected to the root and querying the CDB\_PDBS view, you get the list of all PDBs. Querying the CDB\_TABLESPACES view, you get the list of all tablespaces of all PDBs.

## Data Dictionary Views: Examples

- Comparisons:

- 1 SQL> CONNECT / AS SYSDBA  
SQL> SELECT role, common, con\_id FROM cdb\_roles;
- 2 SQL> SELECT role, common FROM dba\_roles;
- 3 SQL> CONNECT sys@PDB1 AS SYSDBA  
SQL> SELECT role, common, con\_id FROM cdb\_roles;
- 4 SQL> SELECT role, common FROM dba\_roles;

- Access to data in v\$ or GV\$ views showing data from multiple PDBs can be secured using privilege.

```
SQL> SELECT name, open_mode FROM v$pdb;
```

| NAME      | OPEN_MODE  |
|-----------|------------|
| PDB\$SEED | READ ONLY  |
| PDB1      | READ WRITE |
| PDB2      | READ WRITE |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the slide, there are some examples to make comparisons between DBA\_XXX and CDB\_XXX views.

1. In the first example, connected to the root and querying CDB\_ROLES, you get the list of roles, common and local, of each container. Note that the new column CON\_ID displays the container the role belongs to.
2. In the second example, querying DBA\_ROLES, you get all common roles of the root only (there cannot be any local roles in the root).
3. In the third example, connected to the PDB1 and querying CDB\_ROLES, you get the list of roles, common and local, of the container you are connected to. The CON\_ID displays the same value in all rows.
4. In the fourth example, querying DBA\_ROLES, you get the same list except that there is no CON\_ID column (useless in such queries).

The same backward-compatibility principle implies also to each of the familiar v\$views.

# Data Dictionary Views: v\$xxx Views

SGA accessed by all containers: v\$ views and **CON\_ID** column

```
SQL> SELECT distinct status, con_id FROM v$bh order by 2;
```

| STATUS | CON_ID |            |
|--------|--------|------------|
| -----  | -----  |            |
| cr     | 1      | → root     |
| free   | 1      |            |
| xcur   | 1      |            |
| xcur   | 2      | → seed PDB |
| cr     | 3      | → PDB1 PDB |
| xcur   | 3      |            |

```
SQL> select OBJECT_ID, ORACLE_USERNAME, LOCKED_MODE, CON_ID
from V$LOCKED_OBJECT;
```

| OBJECT_ID | ORACLE_USERNAME | LOCKED_MODE | CON_ID |            |
|-----------|-----------------|-------------|--------|------------|
| -----     | -----           | -----       | -----  |            |
| 83711     | SYS             | 3           | 3      | ← PDB1 PDB |
| 83710     | DOM             | 3           | 4      | ← PDB2 PDB |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the slide, there are some examples of v\$xxx views. The new column CON\_ID in v\$xxx views displays how the SGA is accessed by any PDB within the CDB.

In the first example, the v\$BH view provides the list of distinct status of the block buffers currently in the buffer cache. The blocks are clearly identified in the CON\_ID column, each block being accessed by a specific container. 1 stands for the root container, 2 for the seed container, and 3 by the PDB1 pluggable database.

In the second example, the v\$LOCKED\_OBJECT view provides the list of locks currently held on objects in different PDBs. The locks are clearly identified in the CON\_ID column as locked by a specific container; 3 stands for one PDB and 4 for another PDB.

## After CDB Creation: To-Do List

After CDB creation, the CDBA has to:

- Set a separate default tablespace for the root and for each PDB
- Set a default temporary tablespace for each container
- Start the listener
- Plug non-CDBs
- Test startup/shutdown procedures
- Define default PDB state to automate PDBs opening 12.1.0.2
- Create backup and recovery procedures

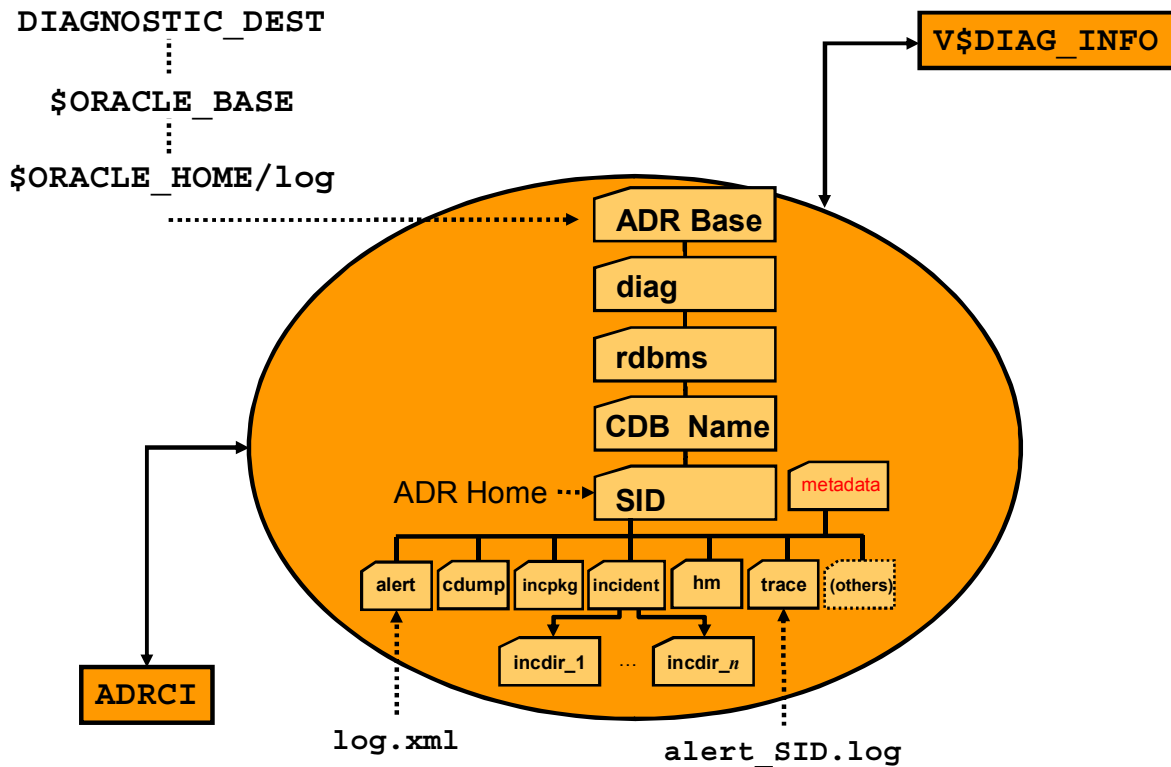
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After the CDB is created, the container database administrator (CDBA) has to complete the following administrative tasks:

- Set a separate default tablespace for the root and for each PDB. In the root, there should not be any user data.
- Set a default temporary tablespace for each container.
- Start the listener.
- Plug non-CDBs if your initial plan was to consolidate several non-CDBs into a single one.
- Test startup and shutdown procedures.
- Define default PDB state to automate PDBs opening after CDB startups.
- Create backup and recovery procedures.

# Automatic Diagnostic Repository



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Since Oracle Database 11g, Release 1, all traces, incident dumps and packages, the alert log, Health Monitor reports, core dumps, and more files are stored in the ADR, a file-based repository for database diagnostic data. It has a unified directory structure across multiple instances and multiple products stored outside of any database. It is, therefore, available for problem diagnosis when the database is down. Nothing is changed with the arrival of container databases. Each instance of each product stores diagnostic data underneath its own ADR home directory. Each CDB, linked to a single instance, stores trace files in the same ADR home directory.

Its location is set by the `DIAGNOSTIC_DEST` initialization parameter. If this parameter is omitted or left null, the database sets `DIAGNOSTIC_DEST` upon startup as follows: if the environment variable `ORACLE_BASE` is set, `DIAGNOSTIC_DEST` is set to `$ORACLE_BASE`. If the environment variable `ORACLE_BASE` is not set, `DIAGNOSTIC_DEST` is set to `$ORACLE_HOME/log`.

## Automatic Diagnostic Repository: alert.log File

The alert\_cdb1.log shows new DDL statements.

```
CREATE DATABASE cdb1
...
ENABLE PLUGGABLE DATABASE
SEED
FILE_NAME_CONVERT=('/u01/app/oradata/CDB1','/u01/app/oradata
/seed');

CREATE PLUGGABLE DATABASE pdb1 ... ;
ALTER PLUGGABLE DATABASE pdb1 UNPLUG INTO ... ;
ALTER PLUGGABLE DATABASE ALL OPEN ;
ALTER PLUGGABLE DATABASE pdb2 CLOSE IMMEDIATE ;
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The alert.log file as a CDB shows new DDL statements such as:

- CREATE PLUGGABLE DATABASE
- ALTER PLUGGABLE DATABASE
- DROP PLUGGABLE DATABASE

## Quiz

Which is a characteristic of the seed pluggable database of a CDB?

- a. It is always kept in READ ONLY mode.
- b. It is not a container.
- c. The seed can be dropped.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Quiz

You create a CDB. What is true about the seed pluggable database?

- a. Copy the seed data files yourself.
- b. Use the new clause `SEED FILE_NAME_CONVERT` in the `CREATE DATABASE` statement.
- c. The seed pluggable database is not required.
- d. The seed pluggable database does not require data files.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**



# Practice

## 4-1: Creating a CDB with no PDBs

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Provisioning New Pluggable Databases

Different methods:

- Create a new PDB from the seed PDB.
- Plug or clone a non-CDB into a CDB.
- Clone:
  - A local PDB into the same CDB
  - A remote PDB into a CDB
- Plug an unplugged PDB into another CDB.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are different methods to provision new PDBs in a CDB.

- Create a new PDB from the seed, the PDB\$SEED PDB, for example for a new application implementation. This type of PDB creation is nearly instantaneous.
- Plug non-CDBs in a CDB as PDBs, as part of migration strategy. It is also a good way to consolidate several non-CDBs into a CDB. Cloning non-CDBs in a CDB is a good way to keep the non-CDB and therefore have the opportunity to compare the performance between the new PDB and the original non-CDB or at least wait until you consider that the PDB can work appropriately.
- Clone a PDB from another PDB of the same CDB. For example, you want to test the application patch of your production. You first clone your production application in a cloned PDB and patch the cloned PDB to test.  
You can also clone a PDB from another PDB of another CDB: this case is interesting when you wish to diagnose performance issues or perform performance regression tests on your application. This cannot be done in parallel of the production in the same database.
- Plug an unplugged PDB into another CDB. For example, you have to upgrade a PDB to the latest Oracle version, but you do not want to apply it on all PDBs. Instead of upgrading a CDB from one release to another, you can unplug a PDB from one Oracle Database release, and then plug it into a newly created CDB from a higher release.

## Tools

To provision new PDBs, you can use:

- SQL\*Plus
- SQL Developer
- Enterprise Manager Cloud Control
- Enterprise Manager Database Express
- DBCA
  - Copy from seed
  - By unplugging/plugging method

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

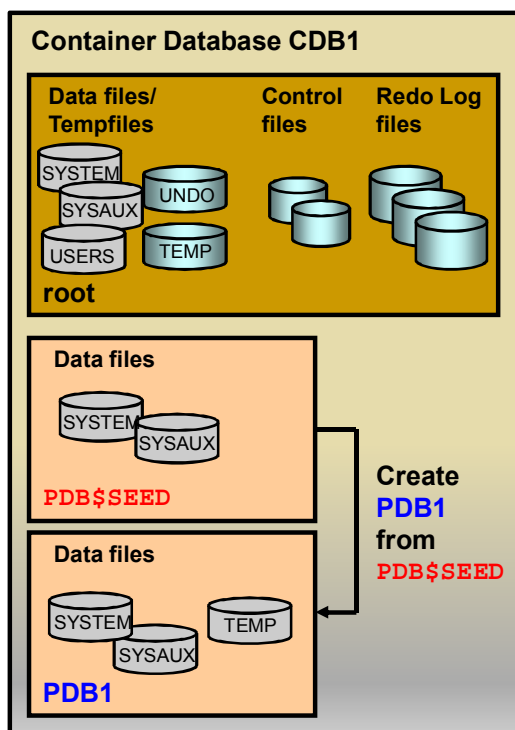
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are different tools to provision new PDBs in a CDB.

- SQL\*Plus
- SQL Developer
- Enterprise Manager Cloud Control
- Enterprise Manager Database Express

To create a new PDB from seed or by the unplug/plugin method, you can also use DBCA.

## Method 1: Create New PDB from PDB\$SEED



- Copies the data files from PDB\$SEED data files
- Creates SYSTEM and SYSAUX tablespaces
- Creates a full catalog including metadata pointing to Oracle-supplied objects
- Creates a temporary tablespace, TEMP
- Creates common users:
  - Superuser SYS
  - SYSTEM
- Creates a local user (PDBA) granted local PDB\_DBA role
- Creates a new default service

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The creation of a new PDB from the seed is nearly instantaneous. The operation copies the data files from the READ ONLY seed PDB to the target directory defined in the `CREATE PLUGGABLE DATABASE` statement.

It creates tablespaces such as SYSTEM to store a full catalog including metadata pointing to Oracle-supplied objects, SYSAUX for local auxiliary data, and TEMP for temporary data.

It creates default schemas and common users that exist in seed PDB, SYS who continues to have all superuser privileges and SYSTEM who can administer the PDB.

It creates a local user (the PDBA) granted a local PDB\_DBA role. Until the PDB SYS user grants privileges to the local PDB\_DBA role, the new PDBA cannot perform any other operation than connecting to the PDB.

A new default service is also created for the PDB.

## Steps: With Location Clauses

Connect to the root as a common user with the CREATE PLUGGABLE DATABASE privilege:

- Use **FILE\_NAME\_CONVERT**:

```
SQL> CREATE PLUGGABLE DATABASE pdb1
      ADMIN USER admin1 IDENTIFIED BY p1 ROLES=(CONNECT)
      FILE_NAME_CONVERT = ('PDB$SEEDdir', 'PDB1dir');
```

- Use **CREATE\_FILE\_DEST**: 12.1.0.2

```
SQL> CREATE PLUGGABLE DATABASE pdb2
      ADMIN USER admin2 IDENTIFIED BY p2 ROLES=(CONNECT)
      CREATE_FILE_DEST = 'PDB2dir';
```

- Use views to verify:

```
SQL> CONNECT / AS SYSDBA
SQL> SELECT * FROM cdb_pdb$;
SQL> SELECT * FROM cdb_tablespace$;
SQL> SELECT * FROM cdb_data_files;
SQL> CONNECT sys@pdb1 AS SYSDBA
SQL> CONNECT admin1@pdb1
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The steps to create a new PDB from the seed are the following:

1. Connect to the root as a common user with the CREATE PLUGGABLE DATABASE privilege.
2. Execute the CREATE PLUGGABLE DATABASE statement. The ADMIN USER clause defines the PDBA user created in the PDB with the CONNECT and empty PDB\_DBA roles.
  - If you use Oracle Managed Files (OMF), there are two methods to complete:
    - No clause is describing the location of the new files. This implies that the DB\_CREATE\_FILE\_DEST instance parameter is set to a target directory for the data files of the new PDB.
    - The CREATE\_FILE\_DEST clause defines a default location for data files in the PDB, overriding the default location of Oracle-managed data files specified for the CDB. A value of NONE disables OMF for the PDB. Using this clause implicitly sets the DB\_CREATE\_FILE\_DEST parameter in the PDB with scope=SPFILE.
  - If you do not use OMF, the FILE\_NAME\_CONVERT clause designates first the source directory of the seed files and second the destination directory for the PDB data files.

3. When the statement completes, use views to verify that the PDB is correctly created.
  - The `CDB_PDBS` view displays the list of the PDBs, the `CDB_TABLESPACES` view displays the list of the tablespaces of the new PDB (`SYSTEM`, `SYSAUX`).
  - Connect to the PDB under the `SYS` user who exists in any PDB or under `admin1`.

## Steps: Without Location Clauses

Create a new PDB from seed without clauses:

- Traditional OMF parameter:

**DB\_CREATE\_FILE\_DEST** = '/CDBS/CDB1/**pdb1**'

```
SQL> CREATE PLUGGABLE DATABASE pdb1
      ADMIN USER pdb1_admin IDENTIFIED BY p1
      ROLES= (CONNECT) ;
```

- New parameter: **PDB\_FILE\_NAME\_CONVERT** =  
'/u01/app/oradata/CDB1/seed','CDBS/CDB1/**pdb2**'

```
SQL> CREATE PLUGGABLE DATABASE pdb2
      ADMIN USER pdb2_admin IDENTIFIED BY p2
      ROLES= (CONNECT) ;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you use OMF or **PDB\_FILE\_NAME\_CONVERT**, first connect to the root of the CDB as a SYS.

- With OMF, set in `init.ora` the **DB\_CREATE\_FILE\_DEST** instance parameter to a target directory for the data files of the new PDB.
- Without OMF, set **PDB\_FILE\_NAME\_CONVERT** new instance parameter to both the source directory of the seed data files and the target directory for the new PDB data files.

Both `/CDBS/CDB1/pdb1` and `/CDBS/CDB1/pdb2` directories must exist.

Then use the `cdb_pdfs` view to verify that the new PDB and its tablespaces exist:

```
SQL> SELECT * FROM cdb_pdfs;
```

```
SQL> SELECT * FROM cdb_tablespaces;
```

**Note:** Oracle Database 12.1.0.2 introduces three new clauses of the `CREATE PLUGGABLE DATABASE` statement:

- USER\_TABLESPACES** clause specifies which tablespaces are available in the new PDB. For example, when you move a non-CDB to a PDB, you can list one or more tablespaces to include.
- ENABLE|DISABLE FORCE LOGGING|NOLOGGING** clause specifies the logging attribute of the PDB. The logging attribute controls whether certain DML operations are logged in the redo log file or not.
- STANDBYS=ALL|NONE** specifies whether the new PDB is included in standby CDBs.

## Synchronization

- If the PDB is in `MOUNTED` or `READ-ONLY` mode
- If customer-created common users or roles are created / modified / dropped
- The PDB needs to be resynchronized with the root.
  - Open the PDB in `READ-WRITE` mode to synchronize the PDB with the target CDB.
  - If you open the PDB in `READ-ONLY` mode, an error is returned.
- A compatibility check is automatically performed:
  - Any violation is reported in the `PDB_PLUG_IN_VIOLATIONS` view.
  - If there is no violation, the PDB status is changed to `NORMAL`.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After a PDB is created by using seed PDB or plugging or cloning methods, you can view the status of the new PDB by querying the `STATUS` column of the `CDB_PDBS` view.

If common users and roles are created before the new PDB is opened, the PDB must be synchronized to retrieve the new common users and roles from the root. The synchronization is automatically performed when the PDB is opened in read/write mode.

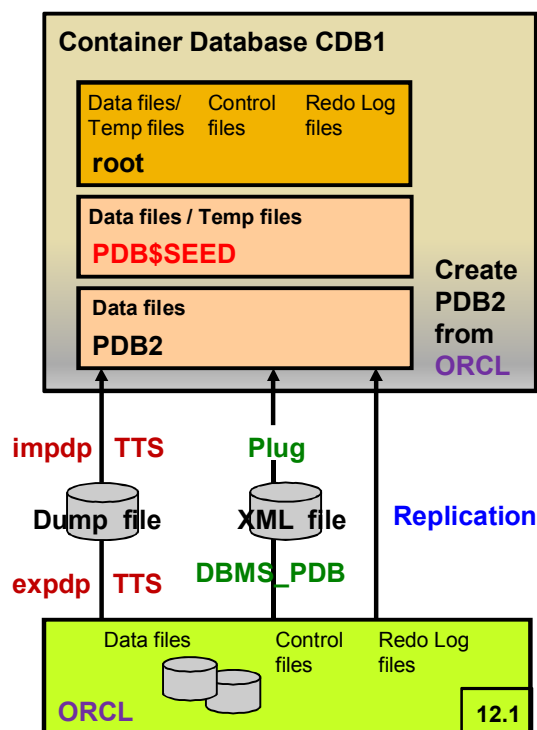
If you open the PDB in read-only mode, an error is returned.

When a new PDB is opened, Oracle Database checks the compatibility of the PDB with the CDB.

Each compatibility violation is either a warning or an error. If a compatibility violation is a warning, then the warning is recorded in the alert log, but the PDB is opened normally without displaying a warning message. If a compatibility violation is an error, then an error message is displayed when the PDB is opened, and the error is recorded in the alert log. You must correct the condition that caused each error. When there are errors, access to the PDB is limited to users with the `RESTRICTED SESSION` privilege so that the compatibility violations can be addressed. You can view descriptions of violations by querying the `PDB_PLUG_IN_VIOLATIONS` view.



## Method 2: Plug a Non-CDB into a CDB



Three possible methods:

- TTS or TDB or full export/import
- XML file definition with `DBMS_PDB`
- Replication

Entities are created in the new PDB:

- Tablespaces: SYSTEM, SYSAUX, TEMP
- A full catalog
- Common users: SYS, SYSTEM
- A local administrator (PDBA)
- A new default service

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are three possible methods to plug a non-CDB database into a CDB.

Whichever method is used, you have to get the non-CDB into a transactionally consistent state and open it in restricted mode.

- Either use transportable tablespace (TTS) or full conventional export/import or transportable database (TDB) provided that in the last one any user-defined object resides in a single user-defined tablespace.
- Or use `DBMS_PDB` package to construct an XML file describing the non-CDB data files to plug the non-CDB into the CDB as a PDB. This method presupposes that the non-CDB is an Oracle 12c database.
- Or use replication with GoldenGate

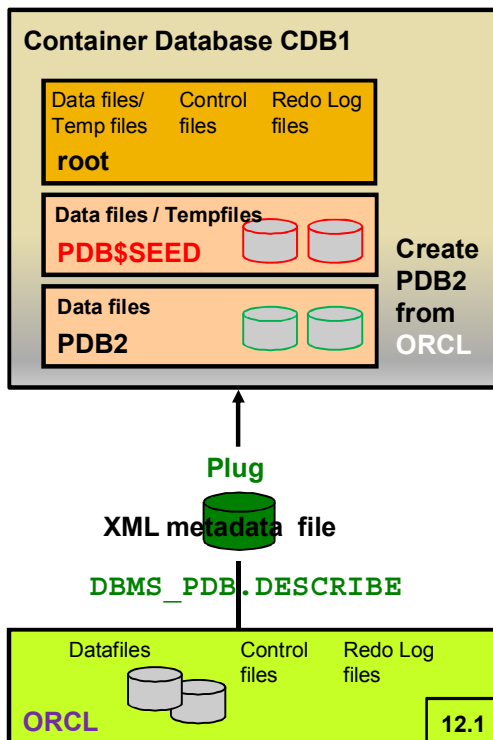
Using the `DBMS_PDB` package is the easiest option.

If the `DBMS_PDB` package is not used, using export/import is usually simpler than using GoldenGate replication, but export/import might require more downtime during the switch from the non-CDB to the PDB.

If you choose to use export/import, and you are moving a whole non-CDB into the CDB, transportable databases (TDB) is usually the best option. If you choose to export and import part of a non-CDB into a CDB, transportable tablespaces (TTS) is the best option.

To understand the steps of the first or third method, refer to the appendix of the course, Appendix B “Other PDB Creation Methods,” and to the following “Oracle Database 12c: Full Transportable Export/Import” white paper (<http://www.oracle.com/technetwork/database/enterprise-edition/full-transportable-wp-12c-1973971.pdf>).

## Plug a Non-CDB in to CDB Using DBMS\_PDB



1. Open **ORCL** in READ ONLY mode.

2. 

```
SQL> EXEC DBMS_PDB.DESCRIBE
      ('/tmp/ORCL.xml')
```

3. Connect to the target CDB1 CDB as a common user with CREATE PLUGGABLE DATABASE privilege.

4. Plug in the unplugged **ORCL** as PDB2.

```
SQL> CREATE PLUGGABLE DATABASE
      PDB2 USING '/tmp/ORCL.xml';
```

5. Run the noncdb\_to\_pdb.sql script.

```
SQL> CONNECT sys@PDB2 AS SYSDBA
SQL> @$ORACLE_HOME/rdbms/admin/noncdb_to_pdb
```

6. Open PDB2.

```
SQL> ALTER PLUGGABLE DATABASE
      PDB2 OPEN;
```

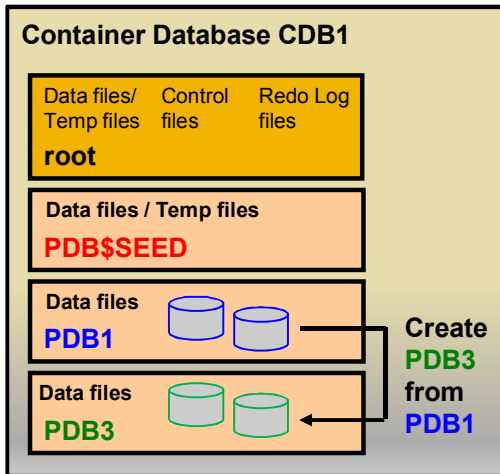
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The technique with DBMS\_PDB package creates an unplugged PDB from an Oracle Database 12c non-CDB. The unplugged PDB can then be plugged in to a CDB as a new PDB. Running the DBMS\_PDB.DESCRIBE procedure on the non-CDB generates an XML file that describes the future PDB. You can plug in the unplugged PDB in the same way that you can plug in any unplugged PDB, using the XML file and the non-CDB data files. The steps are the following:

1. Connect to non-CDB ORCL and first ensure that the non-CDB ORCL is in a transactionally consistent state and place it in read-only mode.
2. Execute the DBMS\_PDB.DESCRIBE procedure, providing the file name that will be generated. The XML file contains the list of data files to be plugged. The XML file and the data files described in the XML file comprise an unplugged PDB.
3. Connect to the target CDB to plug the unplugged ORCL as PDB2.
4. Before plugging the unplugged PDB, make sure it can be plugged into a CDB using the DBMS\_PDB.CHECK\_PLUG\_COMPATIBILITY procedure. Execute CREATE PLUGGABLE STATEMENT using the new clause USING 'XMLfile.xml'. The list of data files from ORCL is read from the XML file to locate and name the data files of PDB2.
5. Run the \$ORACLE\_HOME/rdbms/admin/noncdb\_to\_pdb.sql script to delete unnecessary metadata from PDB SYSTEM tablespace. This script must be run before the PDB can be opened for the first time. This script is required for plugging non-CDBs only.
6. Open PDB2 to verify that the application tables are in PDB2.

## Method 3: Clone Local PDBs



**PDB3 owns:**

- SYSTEM, SYSAUX tablespaces
- Full catalog
- A temporary tablespace
- SYS, SYSTEM common users
- Same local administrator name
- New service name

1. Set the DB\_CREATE\_FILE\_DEST or DB\_FILE\_NAME\_CONVERT instance parameter or use the CREATE\_FILE\_DEST clause. 12.1.0.2

2. Connect to the root.

3. Quiesce PDB1:

```
SQL> ALTER PLUGGABLE DATABASE
      pdb1 CLOSE;
SQL> ALTER PLUGGABLE DATABASE
      pdb1 OPEN READ ONLY;
```

4. Clone PDB3 from PDB1:

```
SQL> CREATE PLUGGABLE DATABASE
      pdb3 FROM pdb1;
```

5. Open PDB3 in read-write mode.

```
SQL> ALTER PLUGGABLE DATABASE
      pdb3 OPEN;
```

6. Reopen PDB1.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

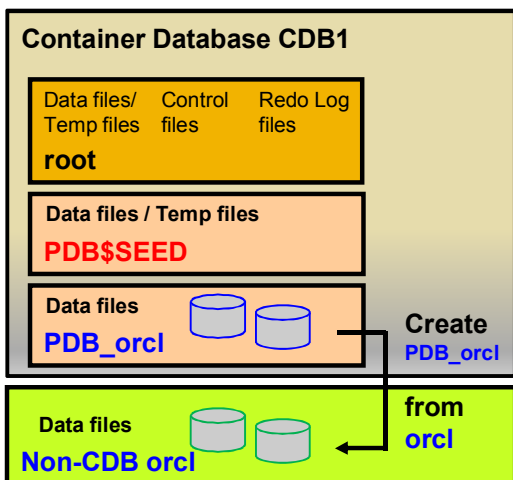
This technique copies and plugs a PDB from a CDB in to a CDB. The source PDB can be in a local CDB or in a remote CDB.

The steps to clone a PDB within the same CDB are the following:

1. Set the instance parameter DB\_CREATE\_FILE\_DEST= 'PDB3dir' (OMF) or PDB\_FILE\_NAME\_CONVERT= 'PDB1dir', 'PDB3dir' (non OMF). You can also use the CREATE\_FILE\_DEST clause.
2. Connect to the root of the CDB as a common user with CREATE PLUGGABLE DATABASE privilege.
3. Quiesce the source PDB used to clone, using the command ALTER PLUGGABLE DATABASE pdb1 READ ONLY after closing the PDB using the command ALTER PLUGGABLE DATABASE CLOSE.
4. Use the command CREATE PLUGGABLE DATABASE to clone the PDB pdb3 from pdb1.
5. Then open the new PDB pdb3 with the ALTER PLUGGABLE DATABASE command.
6. Finally, reopen the PDB pdb1.

If you do not use OMF, use the CREATE PLUGGABLE DATABASE command with the FILE\_NAME\_CONVERT=('pdb1dir', 'pdb3dir') clause to define the directory of the source files to copy from PDB1 and the target directory for the new files of PDB3.

## Method 3: Clone a Non-CDB or Remote PDB



**PDB\_orcl** owns:

- SYSTEM, SYSAUX tablespaces
- Full catalog
- A temporary tablespace
- SYS, SYSTEM common users
- New service name

1. Set **ORCL** in READ ONLY mode.
2. Connect to the CDB to create the database link:

```
SQL> CREATE DATABASE LINK
      link_orcl connect to system
      identified by ***
      using 'orcl';
```

3. Clone the non-CDB:

```
SQL> CREATE PLUGGABLE DATABASE
      pdb_orcl FROM NON$CDB@link_orcl
      CREATE_FILE_DEST = '.../PDB_orcl';
```

4. Run the noncdb\_to\_pdb.sql script.

```
SQL> CONNECT sys@pdb_orcl AS SYSDBA
SQL> @$ORACLE_HOME/rdbms/admin/noncdb_to_pdb
```

5. Open **PDB\_ORCL** in read-write mode.

```
SQL> ALTER PLUGGABLE DATABASE
      pdb_orcl OPEN;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

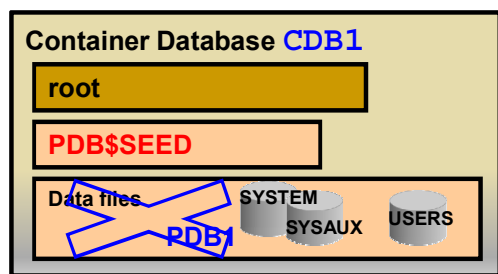
This technique copies a non-CDB or remote PDB and plugs the copy in to a CDB.

The steps to clone a non-CDB or remote PDB into a CDB are the following:

1. Set the non-CDB or remote PDB in READ ONLY mode.
2. Connect to the root of the target CDB as a common user with CREATE PLUGGABLE DATABASE privilege.
3. Create a database link that allows a connection to the remote non-CDB or PDB as a user with CREATE PLUGGABLE DATABASE privilege.
4. Use the CREATE PLUGGABLE DATABASE command to clone the non-CDB as described in the slide. If you clone a remote PDB, use the source PDB name in place of NON\$CDB. Ensure that the new PDB does not conflict with a name of any container within the CDB.
5. If the cloned source is a non-CDB, it is necessary to run the \$ORACLE\_HOME/rdbms/admin/noncdb\_to\_pdb.sql script.
6. Then open the new PDB with the ALTER PLUGGABLE DATABASE command.
7. Finally, you can re-open the non-CDB or remote PDB.

There are additional clone options like SNAPSHOT COPY. Refer to the *Oracle Database Administrator's Guide 12c Release 1 (12.1)*.

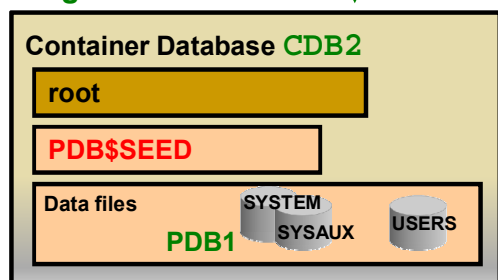
## Method 4: Plug Unplugged PDB in to CDB



Unplug PDB1



Plug PDB1



Unplug PDB1 from CDB1:

1. Connect to CDB1 as a common user.
2. Verify that PDB1 is closed.
3. 

```
SQL> ALTER PLUGGABLE DATABASE  
pdb1 UNPLUG INTO  
      'xmlfile1.xml';
```
4. Optionally, drop PDB1 from CDB1.

Plug PDB1 in to CDB2:

1. Connect to CDB2 as a common user.
2. Use DBMS\_PDB package to check the compatibility of PDB1 with CDB2.
3. 

```
SQL> CREATE PLUGGABLE DATABASE  
pdb1 USING 'xmlfile1.xml'  
      NOCOPY;
```
4. Open PDB1 in read-write mode.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can create a PDB in a CDB by the unplugging/plugging method.

Unplugging a PDB disassociates the PDB from a CDB. You unplug a PDB when you want to move the PDB to a different CDB, or when you no longer want the PDB to be available.

The first step is to unplug PDB1 from CDB1. The second step is to plug PDB1 in to CDB2.

To unplug PDB1 from CDB1, first connect to the root of CDB1, and check that the PDB is closed. Then use `ALTER PLUGGABLE DATABASE` with the `UNPLUG` clause to specify the database to unplug and the XML file to unplug it into. A PDB must be dropped from the CDB before it can be plugged back into the same CDB. If the PDB is plugged into another CDB, the PDB does not need to be dropped if the data files are copied.

Before plugging PDB1 into CDB2, you can optionally check whether the unplugged PDB is compatible with the CDB2 with the `DBMS_PDB.CHECK_PLUG_COMPATIBILITY` function.

To plug PDB1 into CDB2, connect to the CDB2 root as a common user and use `CREATE PLUGGABLE DATABASE pdb1 USING 'xmlfile1.xml'`. The last step is opening the PDB.

## Method 4: Flow

Several clauses can be used in conjunction:

Are new PDB files based on same files that were used to create existing PDB in CDB?

If not, `AS CLONE` clause is required and so, it ensures that Oracle Database generates unique PDB DBID, GUID, and other identifiers expected for the new PDB.

XML file accurately describes current locations of files?

If not, the `SOURCE_FILE_NAME_CONVERT` clause is required.

Are files in correct location?

If not, specify `COPY` to copy files to new location or `MOVE` to move them to another location.  
If yes, use `NOCOPY`. `COPY` as the default.

- `FILE_NAME_CONVERT` or `CREATE_FILE_DEST` clauses of `CREATE PLUGGABLE DATABASE` statement
- Instances `DB_CREATE_FILE_DEST` or `PDB_FILE_NAME_CONVERT` parameters

Do you want to specify storage limits for PDB?

If yes, specify the `STORAGE` clause.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Several clauses can be used in conjunction according to different assumptions:

First question: Are the files of the new PDB based on the same files that were used to create an existing PDB in CDB? If not, the `AS CLONE` clause is required and so, it ensures that Oracle Database generates unique PDB DBID, GUID, and other identifiers expected for the new PDB.

Second question: Does the XML file accurately describe the current locations of the files? If not, the `SOURCE_FILE_NAME_CONVERT` clause is required.

Third question: Are the files are in the correct location? If not, specify `COPY` to copy the files to a new location or `MOVE` to move them to another location. If yes, use `NOCOPY`. `COPY` is the default.

Use one of these techniques to specify the target location:

- In the `CREATE PLUGGABLE DATABASE` statement, include a `FILE_NAME_CONVERT` or `CREATE_FILE_DEST` clause that specifies the target locations based on the names of the source files.
- Or enable OMF to determine the target location by using the `DB_CREATE_FILE_DEST` initialization parameter.
- Or specify the target locations in the `PDB_FILE_NAME_CONVERT` initialization parameter.

If you use more than one of these techniques, the precedence order is:

- FILE\_NAME\_CONVERT or CREATE\_FILE\_DEST clause
- Oracle Managed Files by using the DB\_CREATE\_FILE\_DEST initialization parameter
- PDB\_FILE\_NAME\_CONVERT initialization parameter

Fourth question: Do you want to specify storage limits for the PDB? If yes, specify the STORAGE clause.

That's why according to the different combinations, you can use different syntaxes:

```
CREATE PLUGGABLE DATABASE PDB1AS CLONE USING
'/disk1/usr/salespdb.xml' COPY;
```

```
CREATE PLUGGABLE DATABASE PDB1USING '/disk1/usr/salespdb.xml'
SOURCE_FILE_NAME_CONVERT = ('/disk1/oracle/sales',
'/disk2/oracle/sales') NOCOPY STORAGE (MAXSIZE 500M
MAX_SHARED_TEMP_SIZE 100M);
```

```
CREATE PLUGGABLE DATABASE PDB1USING '/disk1/usr/salespdb.xml' COPY
FILE_NAME_CONVERT = ('/disk1/oracle/sales', '/disk2/oracle/sales');
```

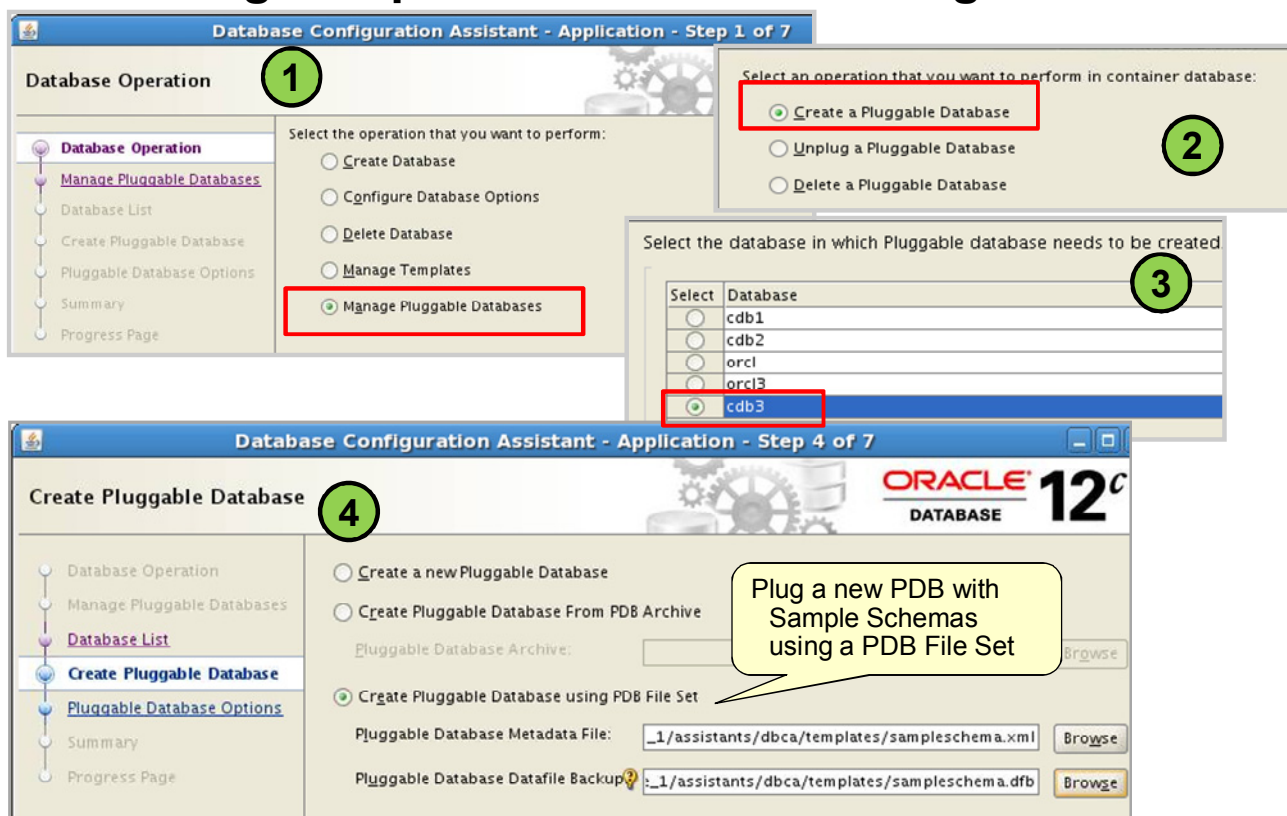
To know if a PDB is unplugged, display the STATUS column in CDB\_PDBS view: the value is UNPLUGGED:

```
SQL> select pdb_name , status from CDB_PDBS;
```

| PDB_NAME  | STATUS    |
|-----------|-----------|
| PDB1      | NORMAL    |
| PDB\$SEED | NORMAL    |
| PDB3      | UNPLUGGED |



# Plug Sample Schemas PDB: Using DBCA

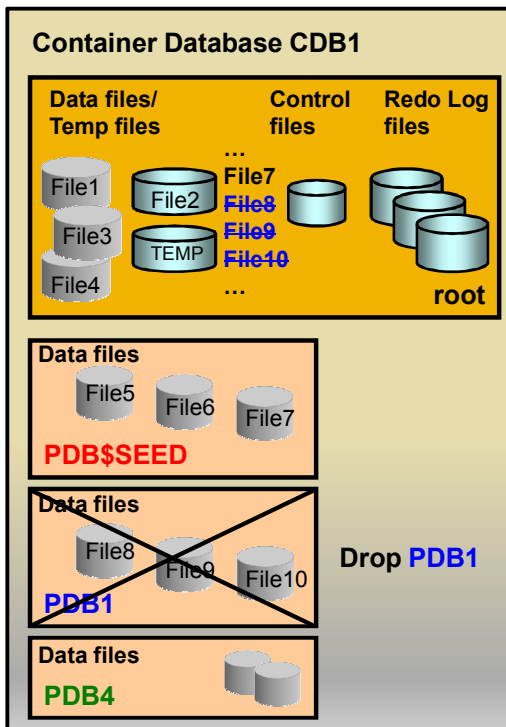


Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can plug a new PDB with the sample schemas using DBCA.

1. In DBCA, click the Manage Pluggable Databases.
2. Then choose Create a Pluggable Database.
3. Select the CDB in which you intend to create the new PDB.
4. Click Create Pluggable Database by using PDB File Set. Browse until you find both files:
  - Pluggable Database Metadata File:  
\$ORACLE\_HOME/assistants/dbca/templates/sampleschema.xml
  - Pluggable Database Datafile Backup:  
\$ORACLE\_HOME/assistants/dbca/templates/sampleschema.dfb
5. Define a name for the new PDB and a location for the data files.  
You can also define a PDB User to create a new administrator for the PDB.
6. Click Next and Finish.

## Dropping a PDB



```
SQL> ALTER PLUGGABLE DATABASE
      pdb1 CLOSE;
SQL> DROP PLUGGABLE DATABASE
      pdb1 [INCLUDING DATAFILES] ;
```

- Updates control files
- If INCLUDING DATAFILES :
  - Removes **PDB1** datafiles
- If KEEP DATAFILES (default):
  - Retain data files
  - Can be plugged in another or the same CDB
- Requires SYSDBA privilege
- Cannot drop seed PDB

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you no longer need the data in a PDB, you can drop the PDB.

You can also unplug it and then drop it specifying `KEEP DATAFILES` to retain the data files associated with the PDB after the PDB is dropped. `KEEP DATAFILES` is the default behavior.

Keeping data files may be useful in scenarios where an unplugged PDB is plugged in to another CDB.

When you drop a PDB specifying `INCLUDING DATAFILES`, all of its data files listed in the control file are deleted.

With or without the clause `INCLUDING DATAFILES`, the `DROP PLUGGABLE DATABASE` statement modifies the control files to eliminate all references to the dropped PDB.

PDB backups are not removed, but you can use RMAN to remove them. This operation requires the `SYSBACKUP` privilege.

To perform the operation, use either SQL\*Plus or SQL Developer or DBCA or Enterprise Manager Cloud Control.

You cannot drop the seed pluggable database.

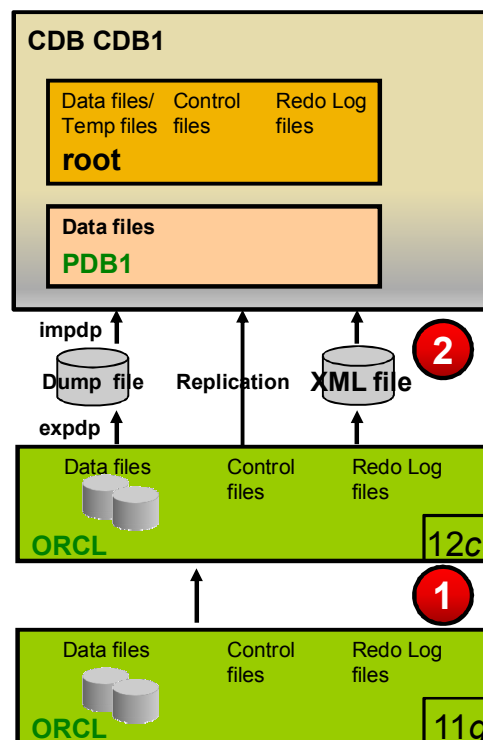
# Migrating pre-12.1 Databases to 12.1 CDB

There are two methods:

1. Upgrade an existing pre-12.1 database to 12c.
2. Plug-in non-CDB into a CDB.

Or

1. Pre-create a PDB in CDB.
2. Use 11g expdp / 12c impdp or replication between non-CDB and PDB.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are two methods to migrate a non-container 11g database to a 12c CDB.

The first method consists of two steps:

1. Upgrade the 11g database to 12c non-CDB.
2. Plug in the 12c non-CDB into a CDB: Use `DBMS_PDB.DESCRIBE` procedure to generate the XML file to plug the data files into the CDB as a new PDB.

The second method consists of two steps:

1. Pre-create a PDB in the CDB from the seed PDB. This operation establishes a Oracle Database 12c dictionary in the newly created PDB.
2. Use either export/import or replication to load the 11g data into the newly created PDB of the CDB.

## Quiz

Which of the following are true about cloning a PDB into the same CDB? Select all that apply.

- a. It is not possible. You can only clone a PDB into another CDB.
- b. You can clone only one PDB into the same CDB.
- c. Cloning a PDB can use the source files copy method to the target PDB files.
- d. Cloning a PDB can use the `NOCOPY` clause if the target PDB files will use the source files.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: c, d**

## Quiz

Which of the following are true about dropping a PDB?

- a. You can drop a PDB only if the PDB is closed.
- b. You can possibly drop the seed PDB, but you will not be able to create any other PDB within the CDB.
- c. You can drop a PDB and keep the data files to be reused by another PDB.
- d. When you drop a PDB, the data files and redo log files are automatically removed from the storage file system.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a, c**

## Summary

In this lesson, you should have learned how to:

- Configure and create a CDB
- Create a PDB from `PDB$SEED`
- Plug or clone a non-CDB into a CDB
- Clone of a local or remote PDB into a CDB
- Unplug and plug a PDB from one CDB to another a CDB
- Explore the instance
- Explore the structure of PDBs
- Drop a PDB
- Migrate a pre-12.1 non-CDB database to CDB

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 4-1: Creating a new CDB
- 4-2: Exploring CDB and PDB structures
- 4-3: Creating a PDB from the seed
- 4-4: Cloning a PDB within the same CDB
  - Using SQL\*Plus
  - Using SQL Developer
- 4-5: Cloning a non-CDB into a CDB
- 4-6: Merging CDBs into a single one

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# 5

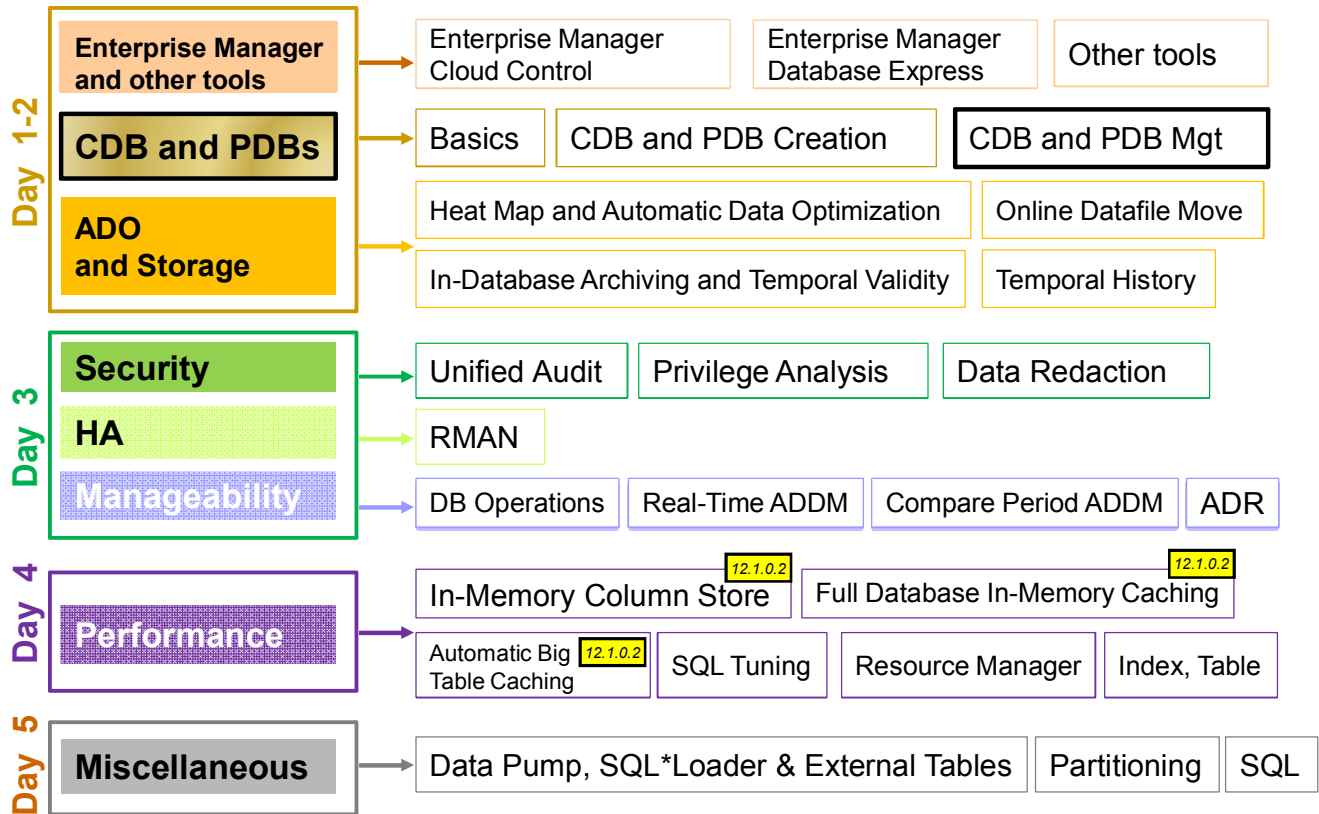
## Managing Multitenant Container Databases and Pluggable Databases

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Objectives

After completing this lesson, you should be able to:

- Establish connections to CDB / PDB
- Start up and shut down a CDB
- Open and close PDBs
- Preserve or discard the PDBs open mode 12.1.0.2
- Change the different modes and settings of PDBs
- Evaluate the impact of parameter value changes

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of the multitenant architecture and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*

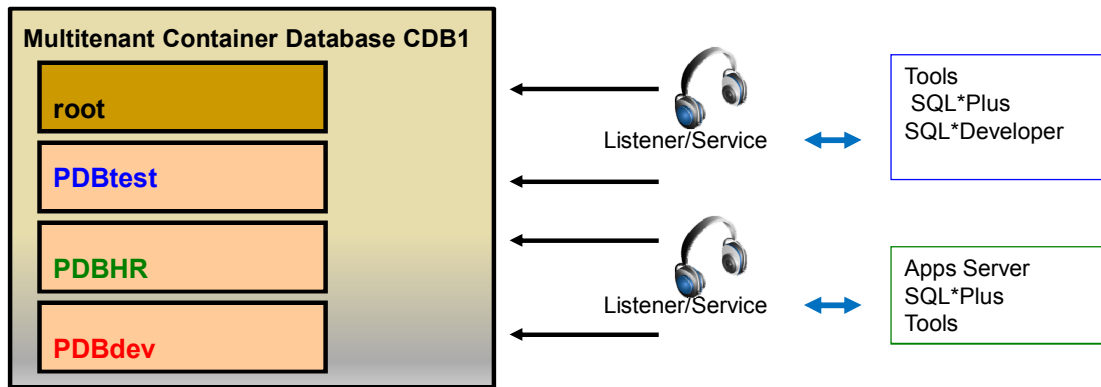
Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library:
  - *Managing Pluggable Databases (PDBs) Using SQL Developer*

Refer to MOS notes:

- *Oracle Multitenant Option - 12c: Frequently Asked Questions (Doc ID 1511619.1)*

# Connection



- Every PDB has a default service.

```
SQL> SELECT name, pdb FROM cdb_services;
```

- Service name has to be unique across CDBs.

```
SQL> CONNECT / AS SYSDBA
SQL> CONNECT sys@PDBtest AS SYSDBA
SQL> CONNECT local_user1@hostname1:1525/PDBHR
SQL> CONNECT common_user2@PDBdev
SQL> SHOW CON_NAME
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Default Services

Before performing any maintenance and management operation in PDBs, you examine how to connect to a CDB and to a particular PDB. Any container in a CDB owns a service name.

- The root container service name is the CDB name given at the CDB creation concatenated with domain name.
- Each new PDB is assigned a service name: the service name is the PDB name given at PDB creation concatenated with domain name. If you create or plug a PDBtest PDB, its service name is PDBtest concatenated with domain name. The container service names must be unique within a CDB, and even across CDBs that register with the same listener.
- You can find all service names maintained in a CDB and PDBs in CDB\_SERVICES or V\$SERVICES views. The PDB column shows the PDB to which the services are linked.

## Connecting to a CDB or a PDB

- To connect to the CDB, the root, use local OS authentication or the root service name. For example, if you set the ORACLE\_SID to the CDB instance name and use the command CONNECT / AS SYSDBA, you are connected to the root under common SYS user granted system privileges to manage and maintain all PDBs.

If you use the service name, you can use the EasyConnect syntax or the alias from the `tnsnames.ora`.

Using EasyConnect, you would enter the following connect string:

```
SQL> CONNECT username@hostname:portnumber/service_name
```

Using the `tnsnames.ora` file, you would enter the following connect string:

```
SQL> CONNECT username@net_service_name
```

- To connect to a desired PDB, use either Easyconnect or the alias from the `tnsnames.ora` file, for example as shown in the slide. In our examples, the net service name in the `tnsnames.ora` matches the service name.

## Creating Services

If your database is not being managed by Oracle Restart or Oracle Clusterware, you can create or modify services for each PDB by using the `DBMS_SERVICE` package. In this case, the PDB property is set to the current PDB where the operation is performed.

```
SQL> CONNECT system@salespdb
```

```
SQL> EXEC DBMS_SERVICE.CREATE_SERVICE('salesrep','salesrep')
```

```
SQL> EXEC DBMS_SERVICE.START_SERVICE('salesrep')
```

You can also create or modify services for each PDB using Server Control utility with the `srvctl add service` command specifying the PDB name by using an additional attribute `-pdb`:

```
$ srvctl add service -db mycdb -service salesrep -pdb
salespdb
```

This example adds the `salesrep` service for the PDB `salespdb` in the CDB with `DB_UNIQUE_NAME mycdb`.

## Configuring an Operating System User for a PDB (Oracle Database 12.1.0.2)

Instead of using the Oracle operating system user for a PDB, you can set a specific user account to be the operating system user for that PDB. If you do not set a specific user to be the operating system user for the PDB, by default the PDB uses the Oracle operating system user. For the root, you can use the Oracle operating system user when you must interact with the operating system.

For better security, Oracle recommends that you set a unique operating system user for each PDB in a multitenant environment. Doing so helps to ensure that operating system interactions are performed as a less powerful user than the Oracle operating system user, and helps to protect data that belongs to one PDB from being accessed by users who are connected to other PDBs.

To set the operating system user for a PDB:

1. Log in to the root as a user and execute the `DBMS_CREDENTIAL.CREATE_CREDENTIAL` procedure to create an Oracle credential for the fred operating system user:

```
SQL> EXEC DBMS_CREDENTIAL.CREATE_CREDENTIAL ( -  
        credential_name => 'CDB1_PDB1_FRED', username =>  
        'fred', -  
        password => 'password')
```

2. Connect to the PDB to set the `PDB_OS_CREDENTIAL` to the new credential:

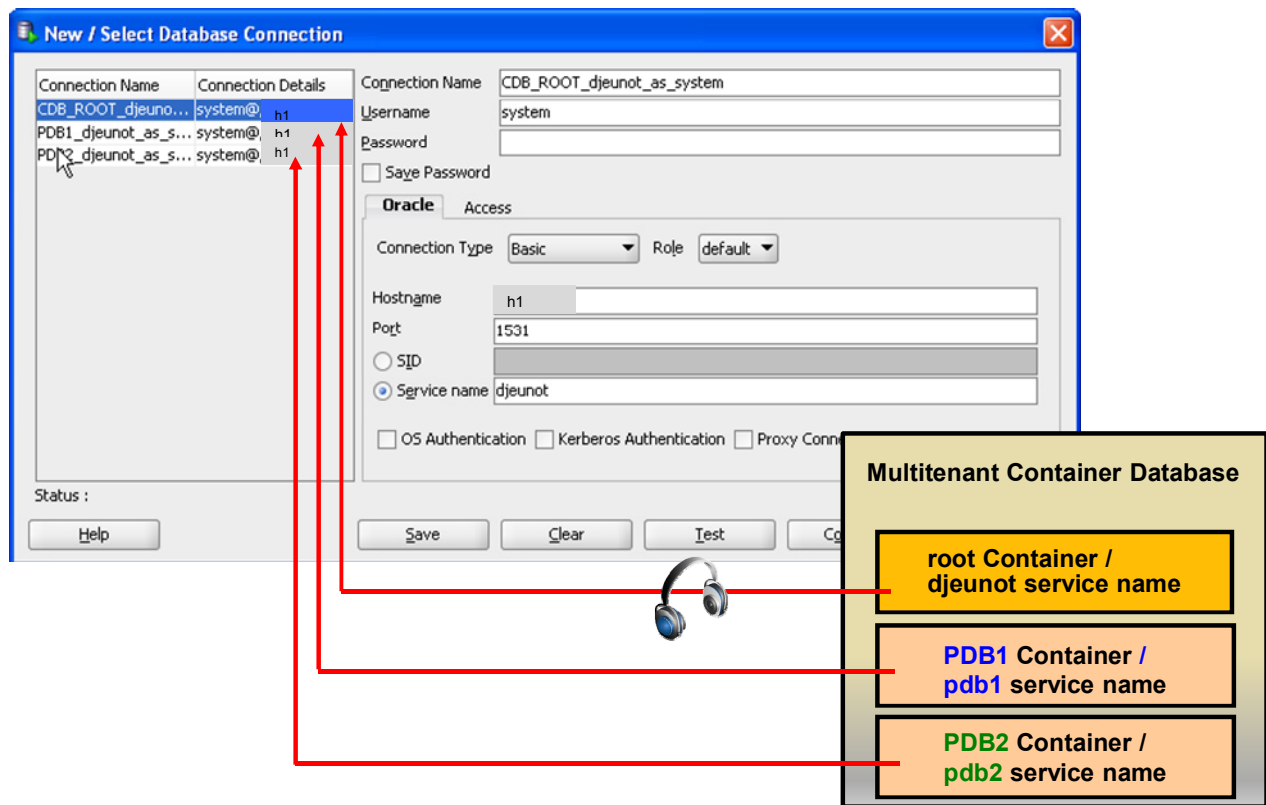
```
SQL> ALTER SYSTEM SET PDB_OS_CREDENTIAL=CDB1_PDB1_FRED  
SCOPE=spfile;
```

3. Restart the CDB instance.

4. Test the connection as fred:

```
[fred] $ sqlplus sys@pdb1 as sysdba
```

# Connection with SQL\*Developer



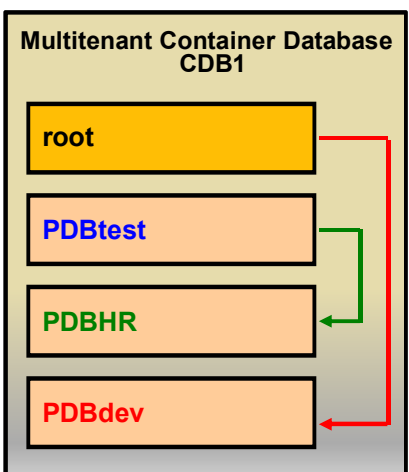
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can also use SQL Developer to connect to the CDB, to the root and to any of the PDBs. As shown in the slide, in the left pane, there are three configured connection names, one to connect to the root, another one to connect to `pdb1`, and a third one to connect to `pdb2`.

In the right pane, the connection name `CDB_root_djeunot_as_system` shows that the connection connects to the service name `djeunot` being the CDB name—hence the root—as user `SYSTEM`.

## Switching Connections



Two possible ways to switch connection between containers within a CDB:

- Reconnect:

```
SQL> CONNECT / AS SYSDBA
SQL> CONNECT local_user1@PDBdev
```

- Use ALTER SESSION statement:

```
SQL> CONNECT sys@PDBtest AS SYSDBA
SQL> ALTER SESSION SET CONTAINER=PDBHR;
SQL> SHOW CON_NAME
SQL> ALTER SESSION SET CONTAINER=CDB$ROOT;
```

- Using CONNECT allows connection under common or local user.
- Using ALTER SESSION SET CONTAINER allows connection under common user only granted new system privilege SET CONTAINER.
  - AFTER LOGON triggers do not fire.
  - Transactions are still pending after switching containers.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A CDB administrator can connect to any container in the CDB by using either CONNECT or ALTER SESSION SET CONTAINER to switch between containers. For example, the CDB administrator can connect to root in one session, and then in the same session switch to the PDBHR container. This requires to be a common user known in both containers, and a system privilege SET CONTAINER.

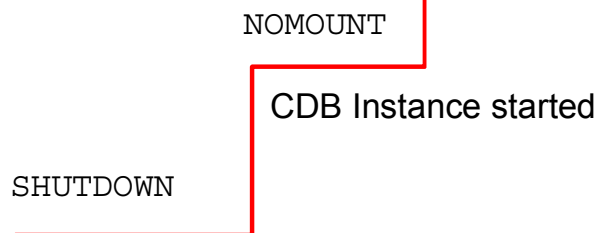
- Using the CONNECT command allows connections under common or local users.
- Using ALTER SESSION SET CONTAINER allows connections under a common user only granted the new system privilege SET CONTAINER.
  - Be aware that AFTER LOGON trigger do not fire.
  - Transactions that are not committed nor rolled back in the original container are still in a pending state when switching to another container and when switching back to the original container.

## Starting Up a CDB Instance

```
SQL> CONNECT sys@CDB1 AS SYSDBA
SQL> STARTUP NOMOUNT
```

```
SQL> SELECT name, open_mode
FROM v$pdb;

no rows selected
```

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a non-RAC environment, a CDB runs with a single instance. The instance is started exactly the same way as for non-CDB databases, using a `STARTUP NOMOUNT` statement. You need to be connected to the root of the CDB as `SYSDBA` to start the instance up.

Use `V$PDBS` view to view the open mode of PDBs.



## Mounting a CDB

```
SQL> CONNECT sys@CDB1 AS SYSDBA
SQL> STARTUP MOUNT
```

Or

```
SQL> ALTER DATABASE cdb1 MOUNT;
```

```
SQL> SELECT name,open_mode
2 FROM v$pdb;
```

| NAME      | OPEN_MODE |
|-----------|-----------|
| PDB\$SEED | MOUNTED   |
| PDB1      | MOUNTED   |
| PDB2      | MOUNTED   |

MOUNT

- CDB control files opened for the instance
- Root mounted
- PDBs mounted

NOMOUNT

Instance  
started

SHUTDOWN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A CDB may be started up with the `MOUNT` option. The command used to mount a CDB is the same as for non-CDB databases, using a `STARTUP MOUNT` statement. Again, you need to be connected to the root of the CDB as `SYSDBA` to perform this operation.

When a CDB is mounted, the root is mounted, which means that the control files are opened, as well as the PDBs.

Use the `open_mode` column from the `V$PDBS` view to verify that all PDBs are mounted.

## Opening a CDB

```
SQL> STARTUP
```

Or

```
SQL> ALTER DATABASE cdb1 OPEN;
```

```
SQL> SELECT name,open_mode
2 FROM v$pdb;
```

| NAME      | OPEN_MODE |
|-----------|-----------|
| PDB\$SEED | READ ONLY |
| PDB1      | MOUNTED   |
| PDB2      | MOUNTED   |

OPEN

- Root opened
- PDBs **still mounted**,  
**except seed in RO**

MOUNT

- CDB control files opened for the instance
- Root mounted
- PDBs mounted

NOMOUNT

Instance started

SHUTDOWN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When a CDB is opened, the root is opened, which means that all redo log files and root data files are opened while all PDBs are still only mounted. Only connections to the root allow operations. Separate statements need to be issued to open PDBs, unless triggers automatically open PDBs after `STARTUP DATABASE`.

Use the `open_mode` column from the `V$PDBS` view to verify that all PDBs are still mounted, except the seed being in read-only mode. This allows creating new PDBs from it.

## Opening a PDB

```
SQL> CONNECT sys@CDB1 AS SYSDBA
SQL> ALTER PLUGGABLE DATABASE pdb1 OPEN;
```

Or

```
SQL> ALTER PLUGGABLE DATABASE ALL OPEN;
```

```
SQL> SELECT name,open_mode
2 FROM v$pdb;
```

| NAME      | OPEN_MODE  |
|-----------|------------|
| -----     | -----      |
| PDB\$SEED | READ ONLY  |
| PDB1      | READ WRITE |
| PDB2      | READ WRITE |

PDB OPEN

PDBs **opened RW**,  
except seed in RO

OPEN

- Root opened
- PDBs still mounted,  
**except seed in RO**

MOUNT

- CDB control files opened for the instance
- Root mounted
- PDBs mounted

NOMOUNT

Instance  
started

SHUTDOWN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To open a PDB or some of them or all of them, connect to the root as SYSOPER or SYSDBA and issue an ALTER PLUGGABLE DATABASE OPEN statement, specifying the PDB or PDBs names or ALL EXCEPT or ALL.

This operation opens the data files of the PDBs opened and provides availability to users.

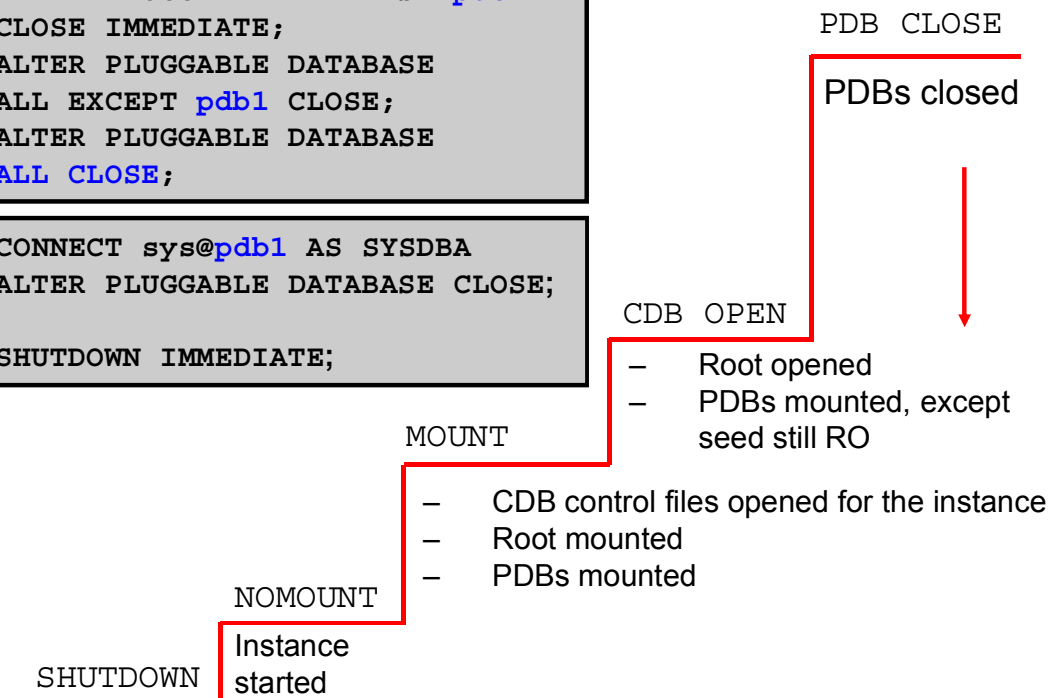
Use the open\_mode column from the V\$PDBS view to verify that all PDBs are all in READ WRITE open mode, except the seed being still in READ ONLY open mode.

You can also open a PDB while connected as SYSDBA within the PDB. In this case, it is not necessary to name the PDB to open.

## Closing a PDB

```
SQL> CONNECT / AS SYSDBA
SQL> ALTER PLUGGABLE DATABASE pdb1
      CLOSE IMMEDIATE;
SQL> ALTER PLUGGABLE DATABASE
      ALL EXCEPT pdb1 CLOSE;
SQL> ALTER PLUGGABLE DATABASE
      ALL CLOSE;
```

```
SQL> CONNECT sys@pdb1 AS SYSDBA
SQL> ALTER PLUGGABLE DATABASE CLOSE;
Or
SQL> SHUTDOWN IMMEDIATE;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To close a PDB or some of them or all of them, connect to the root as SYSOPER or SYSDBA and issue an `ALTER PLUGGABLE DATABASE CLOSE` statement, specifying the PDB or PDBs names or `ALL EXCEPT` or `ALL`. If you use the `CLOSE IMMEDIATE` clause, the transactions in the selected PDBs are rolled back and the sessions disconnected. If you omit the `IMMEDIATE` clause, the statement hangs until all sessions are disconnected.

This operation closes the data files of the closed PDBs and denies availability to users. Though all PDBs are closed, it is still possible to perform operations from the root, such as dropping PDBs or creating new PDBs from the seed.

The statement `SHUTDOWN IMMEDIATE` when connected to a PDB is equivalent to `ALTER PLUGGABLE DATABASE CLOSE`. It closes the PDB.

**Note:** Though `SHUTDOWN IMMEDIATE` issues the traditional message `ORACLE instance shut down`, this does not mean that the instance is down. Understand that the PDB is closed.

If the PDB is already closed, then the message explains the situation clearly with:

```
SQL> shutdown immediate
```

```
ORA-65020: Pluggable database already closed
```

## Shutting Down a CDB Instance

```
SQL> CONNECT sys@CDB1 AS SYSDBA
SQL> SHUTDOWN IMMEDIATE
```

- All PDBs closed (no new specific message)
- CDB closed
- CDB dismounted
- Instance shut down

```
SQL> CONNECT sys@PDB1 AS SYSDBA
SQL> SHUTDOWN IMMEDIATE
```

- PDB closed

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

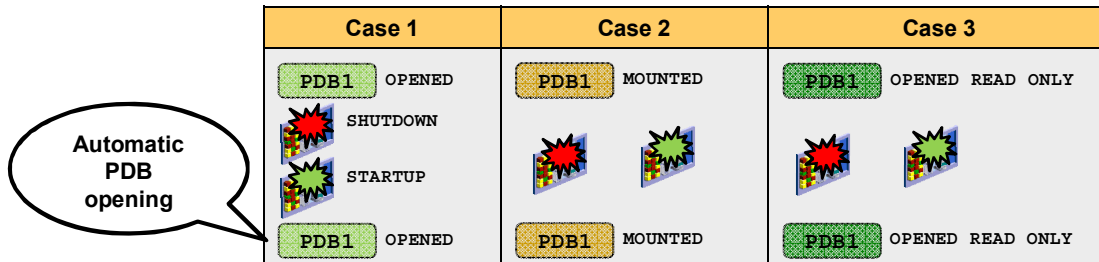
When a CDB is shut down, the data files of the root container and all PDBs are closed, then all the control files are closed and in the last step the instance is shut down.

When a PDB is shut down, this means that the data files of the PDB are closed.

# Automatic PDB Opening 12.1.0.2

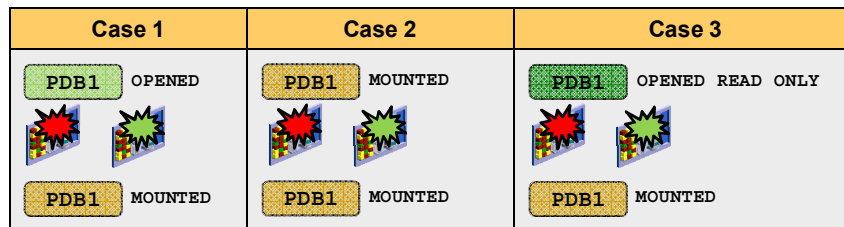
- Automatically keep PDBs state after CDB STARTUP:

```
SQL> ALTER PLUGGABLE DATABASE pdb1 SAVE STATE;
```



- Automatically discard PDBs state after CDB STARTUP:

```
SQL> ALTER PLUGGABLE DATABASE pdb1 DISCARD STATE;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After restarting a CDB instance, the PDBs are by default kept in mounted mode. If you want the PDBs to automatically open whenever the CDB restarts, use the `SAVE STATE` clause of the `ALTER PLUGGABLE DATABASE` command to preserve a PDB's open mode across CDB restart. The `SAVE STATE` clause saves the last open state of the PDB. So, the PDB will open after the CDB restart only if the PDB was in the open state when the `SAVE STATE` clause was used to save the last state. To revert back to the default behavior, use the `DISCARD STATE` clause.

## New Event Triggers

`AFTER CLONE` trigger fires in the copy of a PDB after it has been cloned. `BEFORE UNPLUG` trigger fires in the PDB before any unplug operation starts. `ON PLUGGABLE DATABASE` must be specified when creating any of the two triggers. Such triggers are deleted after they fire. Any failure in any of the two triggers will cause the clone operation to fail.

Another database event is introduced in Oracle Database 12c PS1. The `BEFORE/AFTER SET CONTAINER ON PLUGGABLE DATABASE` fires before or after an `ALTER SESSION SET CONTAINER` statement is executed.

## Changing PDB Open Mode

After closing a PDB, open in three possible modes:

- Restricted read-write:

```
SQL> ALTER PLUGGABLE DATABASE pdb1 OPEN RESTRICTED;
```

```
SQL> SELECT name, open_mode, restricted FROM v$pdb;
```

| NAME  | OPEN_MODE  | RES |
|-------|------------|-----|
| ----- | -----      | --- |
| PDB1  | READ WRITE | YES |
| PDB2  | READ WRITE | NO  |

- Read-only:

```
SQL> ALTER PLUGGABLE DATABASE ALL OPEN READ ONLY;
```

- Read-write:

```
SQL> ALTER PLUGGABLE DATABASE ALL CLOSE;
```

```
SQL> ALTER PLUGGABLE DATABASE ALL OPEN;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can change the mode of each PDB to perform specific administration operations.

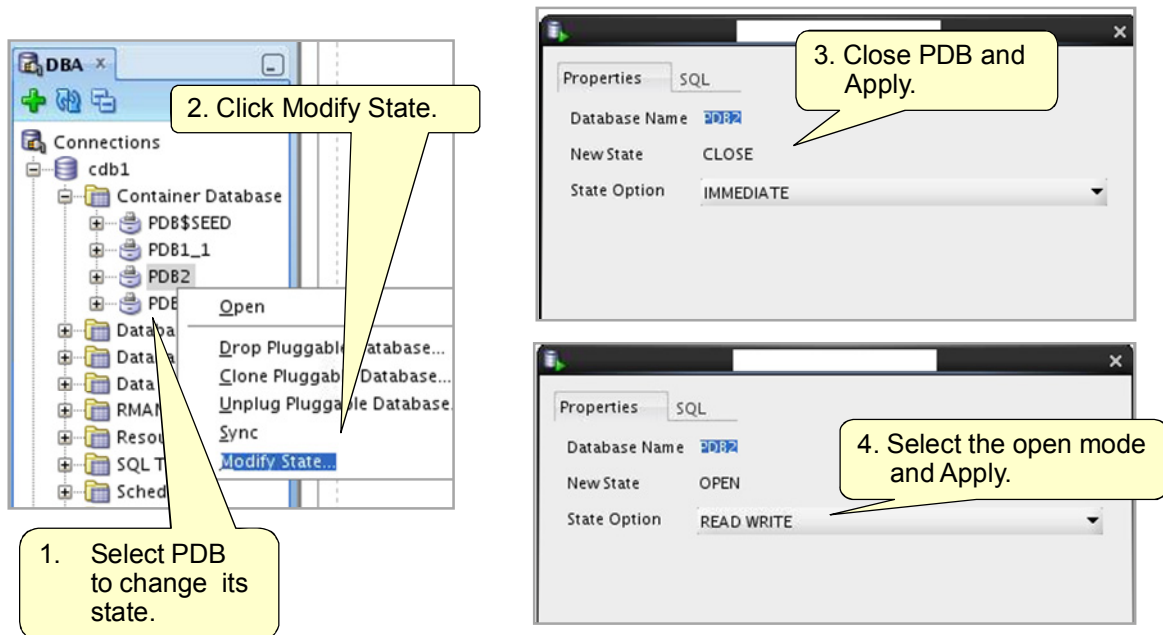
The first example opens the PDB in the `RESTRICTED READ WRITE` mode. This allows only users with `RESTRICTED SESSION` privilege to connect. This allows the local administrator of the PDB to manage file movement, backups, and to prevent sessions from accessing the data.

Use the `V$PDBS` view to verify that the PDB is in `RESTRICTED READ WRITE` open mode.

The second example opens all the PDBs in a `READ ONLY` mode. Any session connected to the PDB can perform read-only transactions only.

To change the open mode, first close the PDB. You can apply the same open mode to all PDBs or to some of them.

## Changing PDB Mode: With SQL Developer



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the source PDB is still opened, you have to open the PDB in READ ONLY mode. So, first close the PDB and then open the PDB in READ ONLY mode. Then you can reiterate the clone operation.



## Modifying PDB Settings

- Bring a PDB data file online.
- Change the PDB default tablespace.
- Change the PDB default temporary tablespace.
- Set the PDB storage limit.
- Change the global name.

```
SQL> CONNECT sys@pdb1 AS SYSDBA
SQL> ALTER PLUGGABLE DATABASE DATAFILE '/u03/pdb1_01.dbf'
      ONLINE;
```

```
SQL> ALTER PLUGGABLE DATABASE DEFAULT TABLESPACE pdb1_tbs;
```

```
SQL> ALTER PLUGGABLE DATABASE DEFAULT TEMPORARY TABLESPACE
      temp_tbs;
```

```
SQL> ALTER PLUGGABLE DATABASE STORAGE (MAXSIZE 2G);
```

```
SQL> ALTER PLUGGABLE DATABASE RENAME GLOBAL_NAME TO pdbAPP1;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can modify settings of each PDB without necessarily changing the mode of the PDB. You have to be connected in the PDB to perform the settings changes.

The first example uses a `DATAFILE` clause to bring the data file online.

The second example sets the default permanent tablespace to `pdb1_tbs` for the PDB.

The third example sets the default temporary tablespace to `temp_tbs` for the PDB.

The fourth example sets the storage limit for all tablespaces that belong to the PDB to two gigabytes.

The fifth example changes the global database name of the PDB to `pdbAPP1`. The new global database name for this PDB must be different from that of any container in the CDB and this operation can be done only in restricted mode.

## Instance Parameter Change Impact

- A single SPFILE per CDB
- PDB value changes are:
  - Loaded in memory after PDB close
  - Stored in dictionary after CDB shutdown
  - Only for parameters `ISPDB_MODIFIABLE=TRUE`

```
SQL> CONNECT sys@pdb1 AS SYSDBA
Connected.
SQL> ALTER SYSTEM SET ddl_lock_timeout=10;
System altered.
SQL> show parameter ddl_lock_timeout
```

| NAME             | TYPE    | VALUE |
|------------------|---------|-------|
| -----            | -----   | ----- |
| ddl_lock_timeout | boolean | 10    |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There is a single SPFILE per CDB to store parameters. Values of parameters are associated with the root, apply to the root, and serve as default values for all other containers.

You can set different values in PDBs for the parameters where the column `ISPDB_MODIFIABLE` in `V$PARAMETER` is `TRUE`. These are set in the scope of a PDB, then they are remembered properly across PDB close/open and across bouncing the CDB instance. They also travel with clone and unplug/plugin operations. Other initialization parameters can be set for the root only.

```
SQL> select PDB_UID, NAME, VALUE$ from pdb_spfile$;
```

| PDB_UID    | NAME                             | VALUE\$ |
|------------|----------------------------------|---------|
| -----      | -----                            | -----   |
| 3100074415 | optimizer_use_sql_plan_baselines | FALSE   |
| 2862146267 | optimizer_use_sql_plan_baselines | FALSE   |
| 2862146267 | ddl_lock_timeout                 | 10      |

## Instance Parameter Change Impact: Example

```
SQL> CONNECT sys@pdb2 AS SYSDBA

SQL> ALTER SYSTEM SET ddl_lock_timeout=20 scope=BOTH;

SQL> ALTER PLUGGABLE DATABASE CLOSE;
SQL> ALTER PLUGGABLE DATABASE OPEN;
```

```
SQL> CONNECT / AS SYSDBA
SQL> select VALUE, ISPDB_MODIFIABLE, CON_ID
       from V$SYSTEM_PARAMETER
       where name ='ddl_lock_timeout';
```

| VALUE | ISPDB | CON_ID |
|-------|-------|--------|
| 0     | TRUE  | 0      |
| 10    | TRUE  | 3      |
| 20    | TRUE  | 4      |



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In this example, a different value of the DDL\_LOCK\_TIMEOUT parameter is set in pdb2. The value changes are remembered after the PDB close and open. The new column CON\_ID in view V\$SYSTEM\_PARAMETER shows the DDL\_LOCK\_TIMEOUT value in each container, the root, pdb1 and pdb2.

## Quiz

When you `STARTUP` a CDB, is the sequence of operations performed automatically? True or False?

- a. The instance is started.
- b. Control files are opened.
- c. The root container is opened (redo logs and root data files).
- d. Seed pluggable database is in `READ ONLY` mode.
- e. Other PDBs are still in `MOUNTED` mode.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Answer: False

All PDBs might not be kept mounted. If PDBs were opened and their state saved before the CDB restart, these PDBs open automatically.

## Quiz

When you `STARTUP` a CDB, all PDBs can be opened in read-write mode.

- a. False
- b. True

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

The seed PDB will always remain in READ ONLY open mode.

## Summary

In this lesson, you should have learned how to:

- Establish connections to CDB / PDB
- Start up and shut down a CDB
- Open and close PDBs
- Preserve or discard the PDBs open mode **12.1.0.2**
- Change the different modes and settings of PDBs
- Evaluate the impact of parameter value changes

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 5-1: Starting up and shutting down a CDB
- 5-2: Opening and closing PDBs
- 5-3: Changing PDB Open Modes
- 5-4: Changing Instance Parameters (*Optional*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# 6

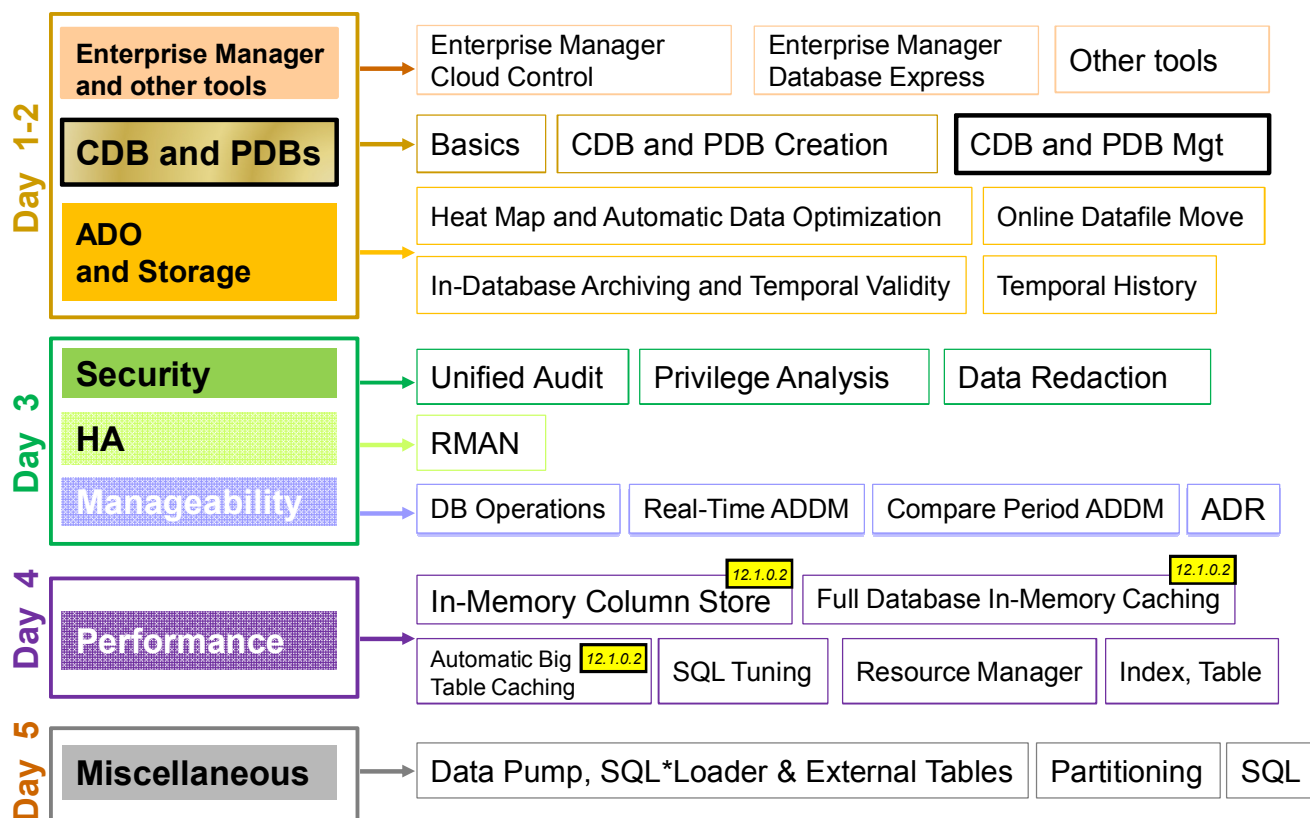
## Managing Tablespaces and Users in CDB and PDBs

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Objectives

After completing this lesson, you should be able to manage:

- Permanent tablespaces in CDB and PDBs
- Temporary tablespaces in CDB and PDBs
- Common and local users
- Common and local roles
- Common and local privileges

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of Oracle PDB's new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*
- *Oracle Database Security Guide 12c Release 1 (12.1)*

Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library:
  - *CDB and PDBs Administration – Manage Common and Local Users*
  - *CDB and PDBs Administration – Manage Common and Local Roles and Privileges*
- MOS note: *Oracle Multitenant Option – 12c: Frequently Asked Questions (Doc ID 1511619.1)*

## Tablespaces in PDBs

- A tablespace in a PDB can contain objects associated with exactly one PDB.
- In `CREATE DATABASE`:
  - `USER_DATA TABLESPACE` replaces automatic creation of `USERS` tablespace by DBCA.
- There is only one active `UNDO` tablespace per CDB instance.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a non-CDB, all the tablespaces belong to one database. In the CDB, one set of tablespaces belong to the root container, and each PDB has its own set of tablespaces.

Common objects are created and their data stored in a tablespace in the root container. The common object is visible in the PDBs through links.

There are new clauses in the `CREATE DATABASE` command. The `USER_DATA TABLESPACE` allows you to specify a default tablespace other than `USERS` when using DBCA to create a database. This tablespace will also be used for XDB options.

The `UNDO` tablespace is common to all PDBs, that is, there is only one active `UNDO` tablespace per CDB instance.

## Creating Permanent Tablespaces in a CDB

- Create a permanent tablespace in a PDB:

```
SQL> CONNECT system@PDB1
SQL> CREATE TABLESPACE tbs_PDB1_users DATAFILE
      '/u1/app/oracle/oradata/cdb/pdb1/users01.dbf'
      SIZE 100M;
```

- Create a permanent tablespace in the root container:

```
SQL> CONNECT system@cdb1
SQL> CREATE TABLESPACE tbs_CDB_users DATAFILE
      '/u1/app/oracle/oradata/cdb/cdb_users01.dbf'
      SIZE 100M;
```

The Oracle logo, consisting of the word "ORACLE" in a bold, sans-serif font, with the letter "O" stylized to include a vertical line through it.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

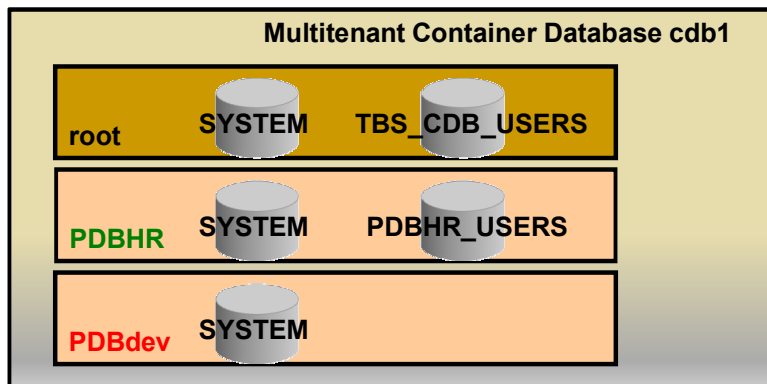
The `CREATE TABLESPACE` command should be familiar. The change in its behavior in a CDB is that the tablespace is created in the container where the command is executed.

Separating the data files into different directories by PDB can help determine which files belong to which PDB, though it is not necessary.

There should not be any application data in the root. Any application data should be in the PDBs. Although you can perform the same tablespace operations in the root and in PDBs, nevertheless all objects should be stored in PDBs.

**Note:** Oracle Database 12.1.0.2 introduces a new clause of the `CREATE PLUGGABLE DATABASE` statement, the `USER_TABLESPACES` clause specifies which tablespaces are available in the new PDB. For example, when you move a non-CDB to a PDB, you can list one or more application tablespaces to include.

## Assigning Default Tablespaces



- In the CDB:

```
SQL> CONNECT system@cdb1
SQL> ALTER DATABASE
      DEFAULT TABLESPACE tbs_cdb_users;
```

- In the PDB:

```
SQL> CONNECT pdb1_admin@pdbhr
SQL> ALTER PLUGGABLE DATABASE
      DEFAULT TABLESPACE pdbhr_users;
```

ORACLE

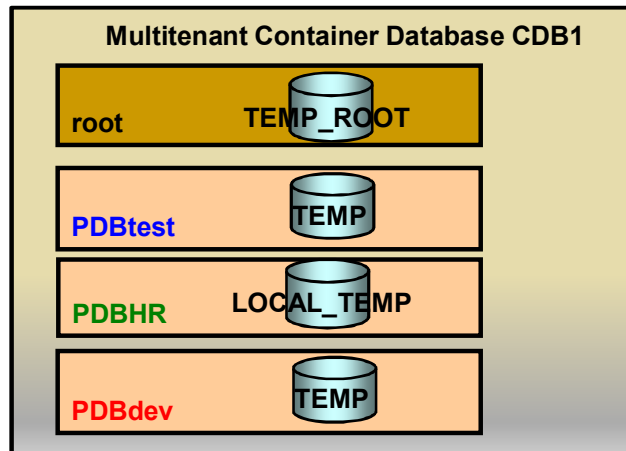
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The default tablespace for a database is a database property. To change the default tablespace for a CDB root container, you must connect to the root container as a user with the proper privileges, and issue the `ALTER DATABASE` command. This operation does not change the default permanent tablespace of PDBs.

To change the default tablespace for a PDB you must connect to the PDB as a user with the proper permissions and issue the `ALTER PLUGGABLE DATABASE` command. When connected to the PDB, the `ALTER DATABASE` and `ALTER PLUGGABLE DATABASE` commands perform the same modifications to the PDB. The `ALTER DATABASE` command is allowed for backward compatibility.

## Creating Local Temporary Tablespaces

- Only one default temporary tablespace or tablespace group is allowed per CDB or PDB.
- Each PDB has its own temporary tablespaces or tablespace groups.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The root or a PDB can have only one default temporary tablespace or tablespace group. As with a non-CDB, each container can hold other temporary tablespaces to which users can be assigned.

Each PDB can have temporary tablespaces for use by local or common users in the PDB. These temporary tablespaces will be transported with the PDB when it is unplugged.

## Assigning Default Temporary Tablespaces

- In the CDB:

```
SQL> CONNECT system@cdb1
SQL> ALTER DATABASE
      DEFAULT TEMPORARY TABLESPACE temp_root;
```

- In the PDB:

```
SQL> CONNECT pdb1_admin@pdbhr
SQL> ALTER PLUGGABLE DATABASE
      DEFAULT TEMPORARY TABLESPACE local_temp;
```

or

```
SQL> CONNECT pdb1_admin@pdbtest
SQL> ALTER DATABASE
      DEFAULT TEMPORARY TABLESPACE temp;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The default temporary tablespace for the root is set at the root container level. There may be multiple temporary tablespaces, but only one can be the default.

A default temporary tablespace (or tablespace group) can be set for each PDB. A PDB may have multiple temporary tablespaces, but only one default per PDB.

When you create a user you can specify a temporary tablespace to be used by that user. If a temporary tablespace is not specified, the default tablespace for the PDB is used.

The amount of space a PDB can use in the shared temporary tablespace can be limited:

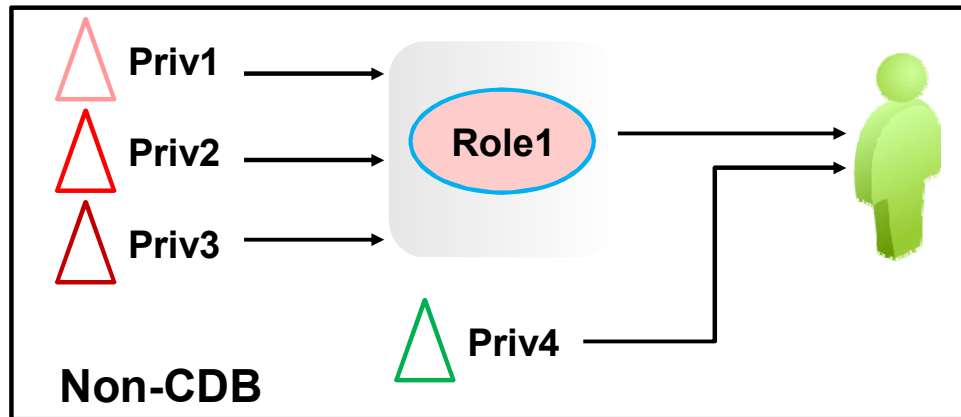
```
ALTER PLUGGABLE DATABASE STORAGE (MAX_SHARED_TEMP_SIZE 500M);
```

In this example, if the value used by sessions that are connected to the PDB is greater than 500M, then no additional storage in the shared temporary tablespace will be available for sessions connected to the PDB until the amount of storage used by them becomes smaller than 500M.

When you unplug a PDB from a CDB, its temporary tablespaces are also unplugged.

## Users, Roles, and Privileges

- Each user can exercise granted privileges in the context of a single database.
- A role is a collection of privileges.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a non-CDB, a user can perform operations authorized by system privileges, on objects allowed by object privileges. These privileges apply only to the database to which the user is connected.

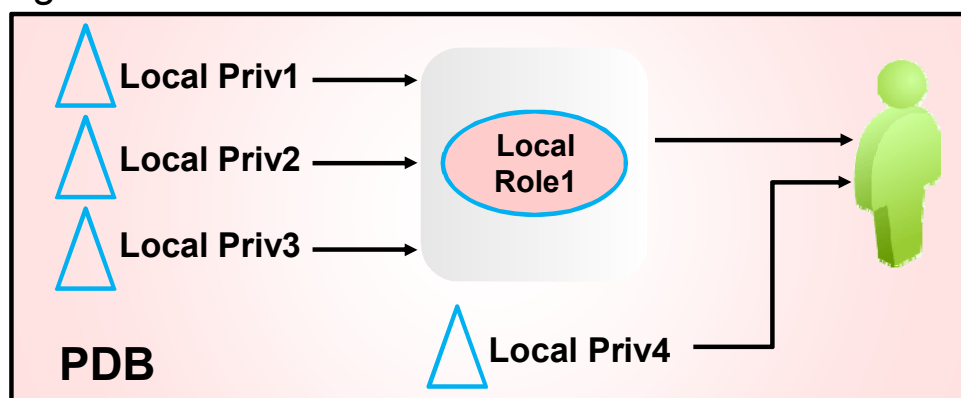
A role is a set of privileges. The privileges can be exercised in the context of a specific database. Privileges can be granted either directly or through roles.

In the past, the phrase “in the context of a single database” has been implied. When you consider that there was only one database per instance, this seems a trivial assertion, but this is key to understanding local users, local privileges, and local roles in PDBs.



## Local Users, Roles, and Privileges

- Each **local** user can exercise granted privileges in the context of a single **PDB**.
- A **local** role is a collection of privileges that are assigned at user login to a specific **PDB**.
- A **local** privilege is one that is granted in the context of a single **PDB**.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A local user exists in one and only one PDB. Even if multiple PDBs have local users with the same name and credentials, each local user is distinct.

In a PDB, the local users behave in exactly the same manner as users defined in the context of a non-CDB. A local user can perform DDL operations authorized by system privileges and DML and other operations on objects allowed by object privileges. The privileges apply only to the PDB in which the local user is defined. The differences between the rules for a non-CDB and a PDB are highlighted. As you can see, they are almost identical.

The commands to create local users and roles in a PDB are the same as for a non-CDB.

## Creating a Local User

- A local user with proper privileges can create another local user.
- The syntax is the same as in non-CDB.

```
SQL> CONNECT system@pdb1
SQL> CREATE USER george IDENTIFIED BY x;
```

- You cannot create local users in the root.
- A local user connects only to the PDB where it exists.

```
SQL> CONNECT george@pdb1
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

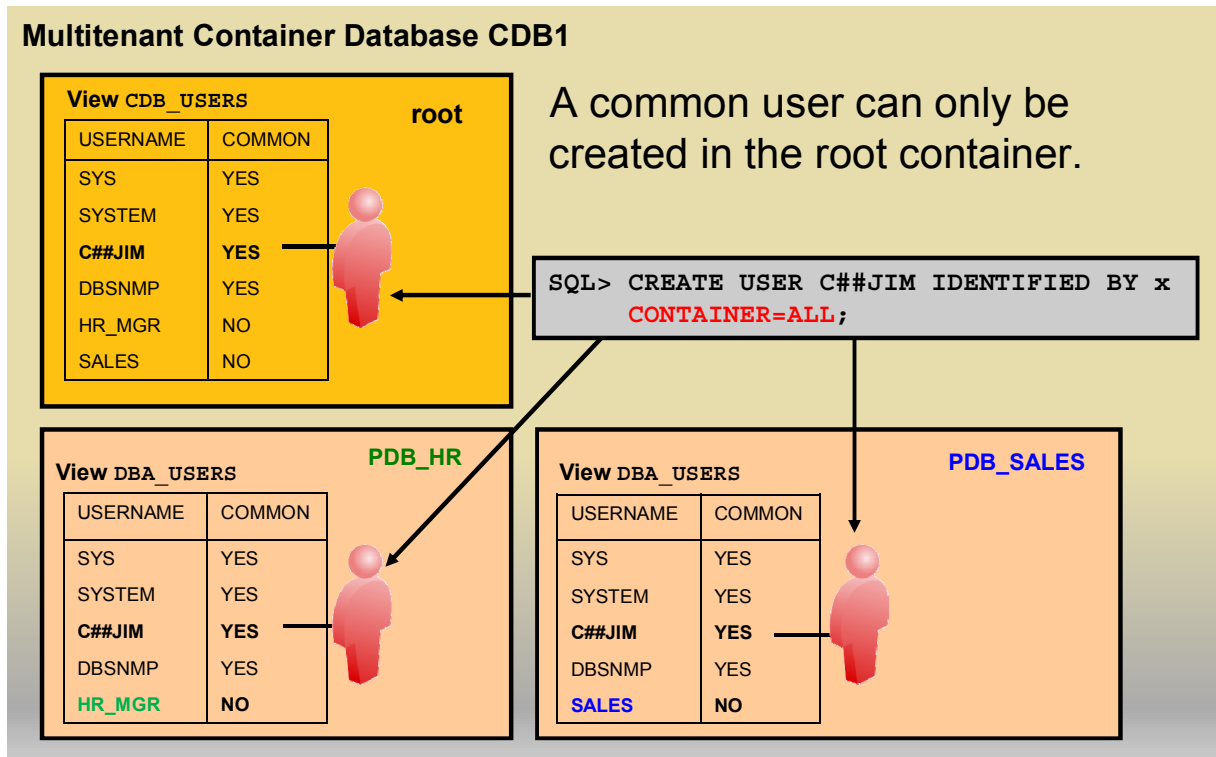
A local user can be created by another local user with the proper privilege, typically one with the DBA role in the PDB. The CREATE USER privilege is required.

The syntax is the same as in a traditional or non-CDB. This is in keeping with the application transparency goal for PDBs. The syntax for commands in a PDB are identical to commands in a non-CDB.

**Note:** The DROP USER and ALTER USER commands are identical, as the commands in a non-CDB.

You cannot create local users in the root.

# Common Users



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A common user is a user that has the same username and authentication credentials across multiple PDBs, unlike the local user which exists in one and only one PDB.

A common user cannot have the same name as any local user across all of the PDBs. A common user can be created in the root container, and only in the root container: a common user is a database user that has the same identity in the root and in every existing and future PDB.

A local user cannot create a common user.

A common user with the proper privileges can create a common user by using the CONTAINER=ALL clause. To create a common user, the common user must have the SET CONTAINER privilege for all the PDBs. A common user can connect to a specific PDB and with the proper privilege in the PDB create a local user with the CONTAINER = CURRENT clause.

**Note:** If a PDB is closed, the common and local users of the PDB are not visible because the metadata is retrieved from the PDB SYSTEM tablespace.

## Creating Users

- A common user can create common users and local users.
- The **CONTAINER** clause determines the type of user created.
- Create a **common** user in the root container:

```
SQL> CREATE USER C##_GEORGE IDENTIFIED BY x
      CONTAINER=ALL;
```

- Create a **local** user in a PDB:

```
SQL> CREATE USER L_FRED IDENTIFIED BY y
      CONTAINER=CURRENT;
```

- A common user exists in each container:

```
SQL> CONNECT c##jim@cdb1 → jim connection in root
SQL> CONNECT c##jim@pdb1 → jim connection in pdb1
SQL> CONNECT c##jim@pdb2 → jim connection in pdb2
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Only a common user with the `CREATE USER` and the `SET CONTAINER` common privileges can create a common user. A common privilege is a privilege granted across all the PDBs. A common user with the `CREATE USER` and `SET CONTAINER` privileges in the PDB can create a local user in that PDB.

The type of user created is determined by the `CONTAINER` clause.

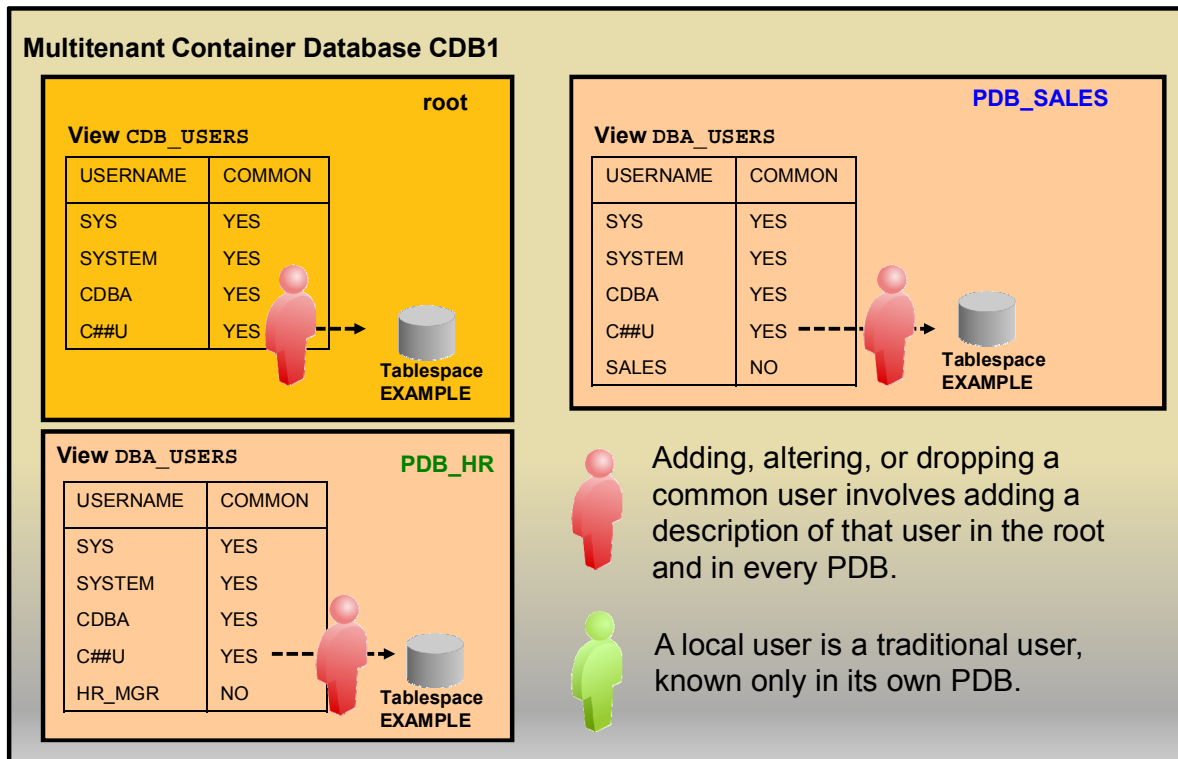
- The first command shown in the slide creates a common user when issued in the root container. To create a common user, you must be connected to the root container. The name of a common user must begin with `C##` characters.
- The second command creates a local user when the common user is connected to a specific PDB.

If the container clause is omitted, the default depends on the context. If the common user is connected to the root container, the default is `CONTAINER=ALL`. If the common user is connected to a PDB, the default is `CONTAINER=CURRENT`.

When creating a common user, any tablespace, tablespace group or profile specified in the `CREATE` command must exist in every PDB. If none of these are specified, the default `TABLESPACE`, `TEMPORARY TABLESPACE`, and `PROFILE` for the PDB will be used.

**Note:** The `DROP USER` and `ALTER USER` commands are the same as in a non-CDB.

# Common and Local Schemas/Users



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Adding, altering, or dropping a common user involves adding, changing, or dropping a description of that user in the root and in every PDB. After changing the container, common users can or cannot perform operations depending on the type of privileges granted in the container (PDB).

A PDB that was not `OPEN READ WRITE` or `RESTRICTED` when a common user was created attempts to add the description of that common user as a part of the synchronization operation. If during the process, a new common username conflicts with a local username defined in a given PDB or a tablespace, a tablespace group or a profile which was specified in the `CREATE USER` statement does not exist in that PDB, an error is reported.

Altering or dropping a common user involves modifying or removing a description of that user in the root container and in every PDB that is `OPEN READ WRITE` or `RESTRICTED`, effectively replaying the statement in each container.

## Common and Local Privileges

- A privilege granted across all containers is a common privilege.
- A privilege granted in the context of a single PDB is a local privilege.
- Local users can exercise privileges only locally in the context of the PDB.
- Common users can exercise privileges only in the context of the PDB to which they are connected.
- Common users connected to the root container can exercise cross-container privileges, such as creating a common user.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

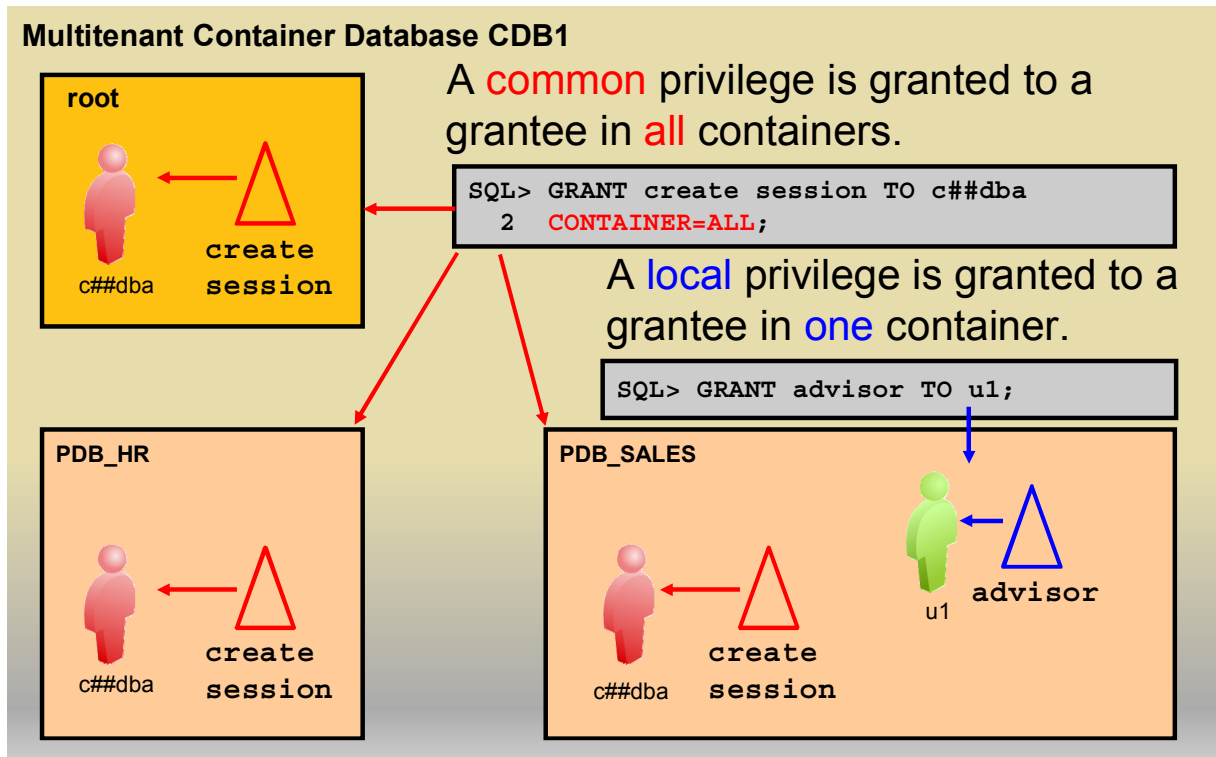
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A privilege becomes common or local based on the way it is granted. A privilege granted across all containers is a common privilege. A privilege granted in a specific PDB is local.

Common and local users can exercise common and local privileges that have been granted in the context of the PDB to which they are connected. A common user can be granted both common and local privileges. That means the privileges granted to a common user can be different in every PDB.

Only common users can connect to the root container. Even in the root container, privileges can be granted as local or common. Cross-container operations such as creating a common user can only be performed while connected to the root container and by common users with common privileges for those operations.

# Granting and Revoking Privileges



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A privilege granted across all containers is a common privilege. In the example in the slide, the `CREATE SESSION` privilege is granted commonly to the `c##dba` user by using the `CONTAINER=ALL` clause. The grant operation is replicated in all containers. Consequently, the same user `c##dba` is granted the same privilege in each container.

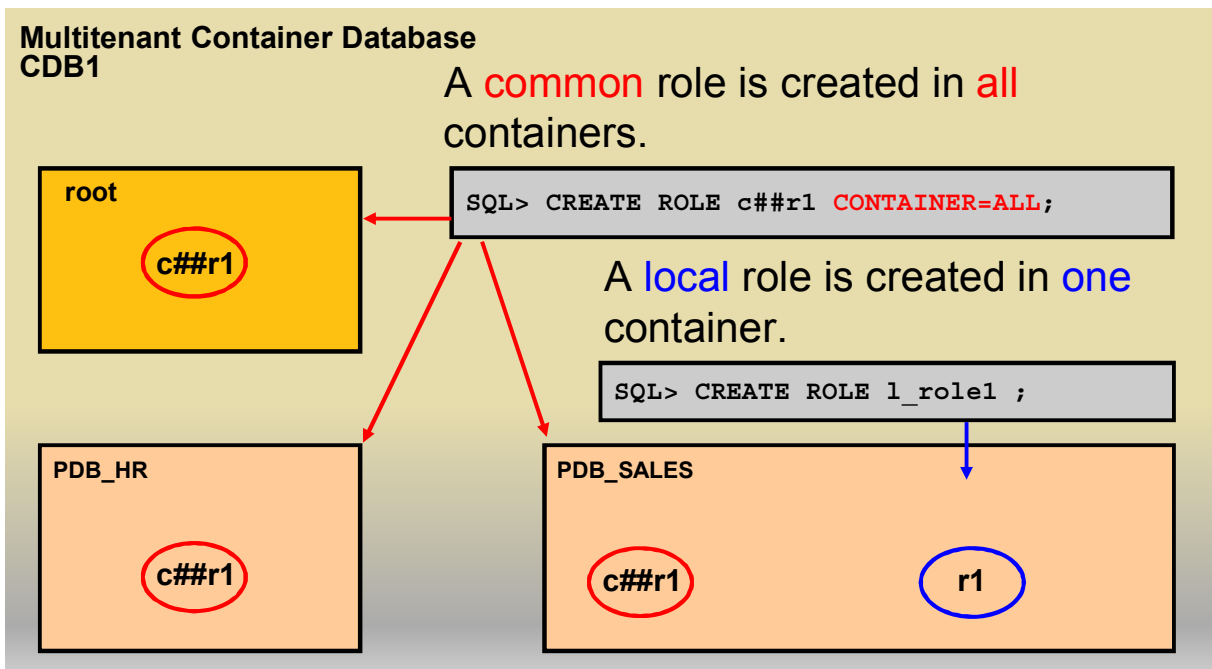
A privilege granted in a specific PDB is local. In the example in the slide, the `ADVISOR` privilege is granted locally to the `u1` user. The grant operation is not replicated in all containers. Even if the user were a common user, the grant operation would not have been replicated. Consequently, the user `u1` is granted the privilege in the `PDB_SALES` container only.

The syntax and behavior of `GRANT` and `REVOKE` are mostly unchanged. The syntax has been extended with the `CONTAINER` clause so that users with the proper privileges can grant privileges commonly or locally.

When a user grants a privilege with the `CONTAINER=ALL` clause, the privilege becomes a common privilege. The user must have the `SET CONTAINER` privilege for all the PDBs because he grants the privilege to the same common user in each container.

When a user grants a privilege with the `CONTAINER=CURRENT` clause, the privilege becomes a local privilege.

# Creating Common and Local Roles



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A role created across all containers is a common role. In the example in the slide, the c##r1 role is created commonly by using the CONTAINER=ALL clause. The CREATE operation is replicated in all containers. Consequently, the same role c##r1 is created in each container.

A role created in a specific PDB is local. In the example in the slide, the l\_role1 role is created locally. The CREATE operation is not replicated in all containers. Consequently, the l\_role1 role is created in the PDB\_SALES container only.

The syntax and privileges used are the same as in a non-CDB for a local user to create a local role.

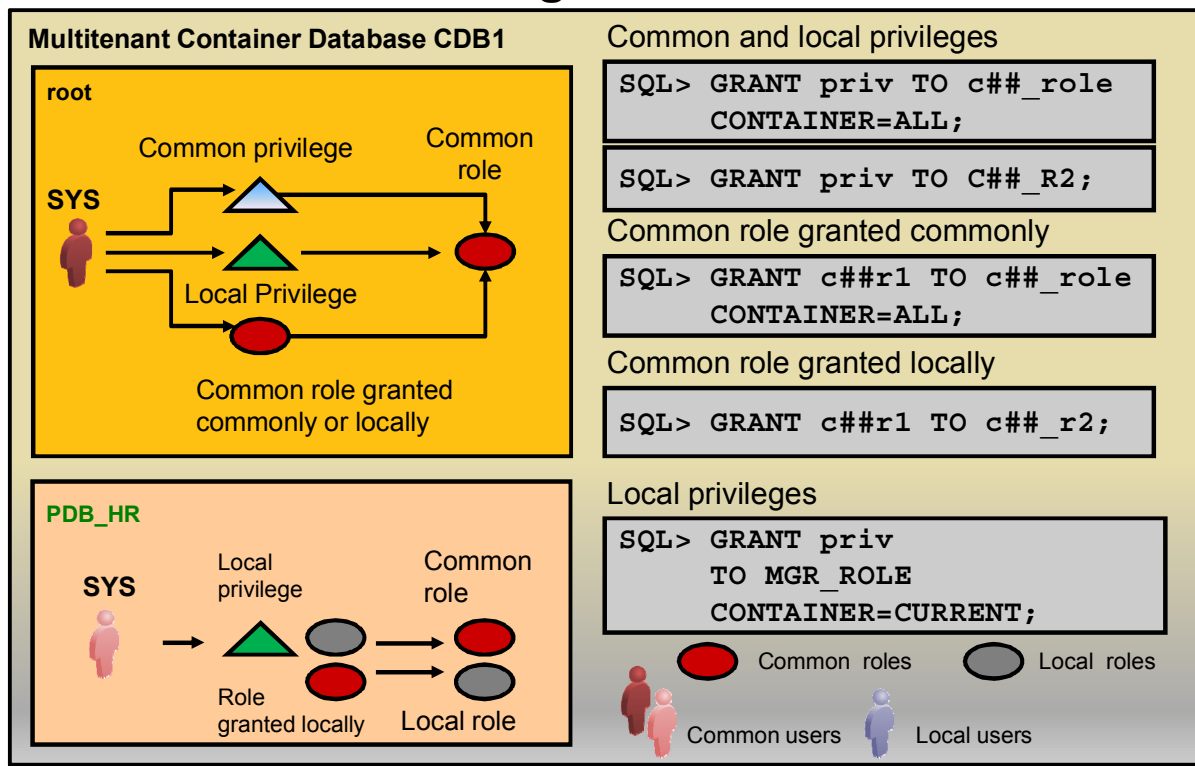
A user can create common roles if the user has the CREATE ROLE privilege, and SET CONTAINER for all PDBs. A common user can create a local role if the user has the CREATE ROLE and SET CONTAINER privilege for that PDB. The CONTAINER clause determines whether the role is common or local. A common role must begin with C## characters.

Any role can be granted to any role or user. It does not matter whether the user or role is defined to be local or common. Consider that a role is a container for privileges. When a role is granted to a user, the privileges in the role are limited to the context of the PDB.

For example, the common role C##R1 is created with a common privilege CREATE SESSION. When the C##R1 role is granted to a common user, C##TEST, that user can connect to any PDB. But when C##R1 role is granted to a local user, lu\_PDB1, defined in PDB\_HR, that local user can only connect to PDB\_HR.



# Granting Common or Local Privileges/Roles to Roles



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To grant a role, the grantor must have the GRANT ANY ROLE privilege or have been granted the role WITH ADMIN OPTION.

Privileges assigned to a common role are common privileges as long as CONTAINER=ALL is specified and therefore are applied in all containers. If CONTAINER=ALL is not specified, the privilege assigned to a common role is local. The first command shown in the slide shows a privilege granted commonly to a common role, and therefore usable in each of the PDBs. The second command shown in the slide shows a privilege granted locally to a common role, and therefore usable only in the PDB where the command is executed.

Common roles can be created in the root container only.

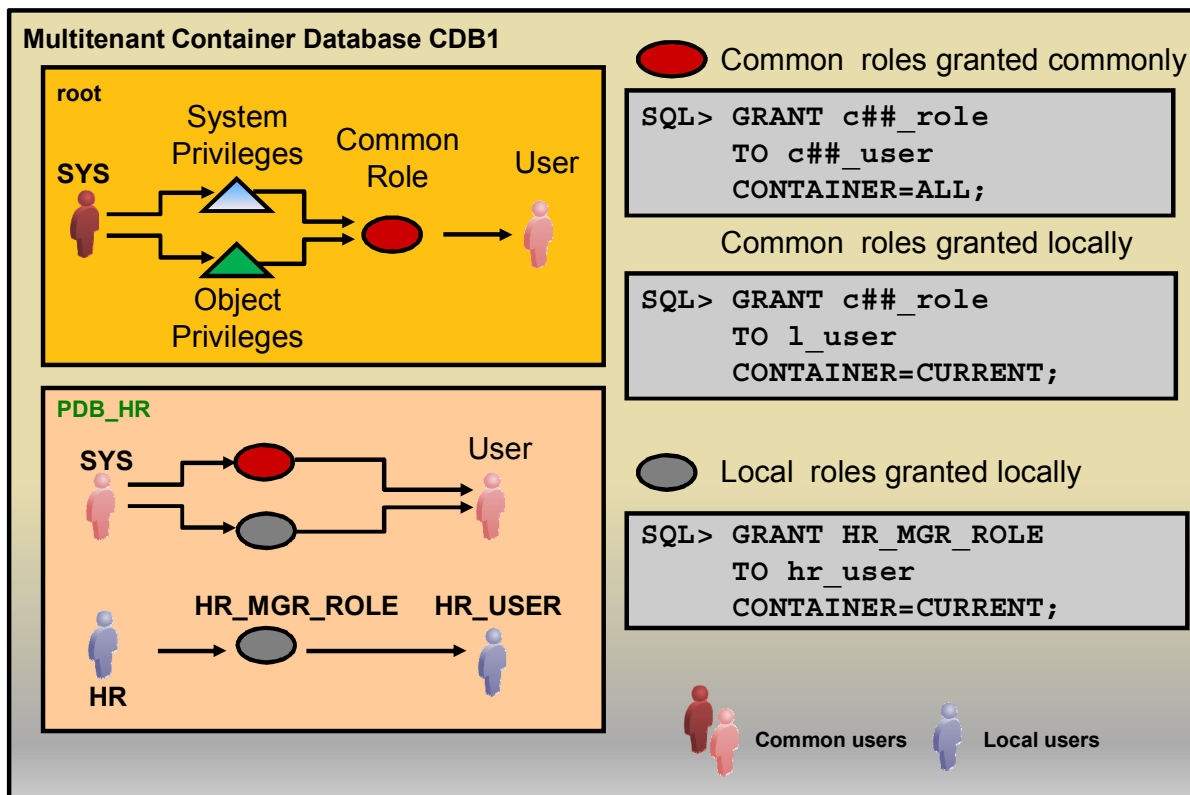
Example: A common role, C##\_VIEWER, granted commonly the SELECT ANY TABLE privilege is granted to a common user C##\_HR\_CLERK. The SELECT ANY TABLE privilege can be used by C##\_HR\_CLERK while connected to any PDB.

Privileges assigned to local roles can only be local privileges. A local role can be created in a specific PDB and cannot be created in the root container. The fifth command executed in the PDB\_HR container shows a privilege being granted locally to the MGR\_ROLE role.

Common roles may be granted to any user or role commonly or locally in the root. Local and common roles can be granted to any user or role locally only in any PDB.

A local user can be assigned a common role, and the privileges assigned by that role can only be exercised in the PDB where the local user is defined.

# Granting Common and Local Roles to Users



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Common roles may be granted to any user or role in any PDB. Local roles may be granted to any user or role that exists in the same PDB. In the first example the common role, C##\_ROLE, is granted to the common user C##\_USER. This grants the role in all the containers.

A local role can be granted to a common user and a common role to a local user. A privilege granted directly or through a role is exercised in the context of a single PDB. Example: a common role C##\_VIEWER, containing SELECT ANY TABLE privilege is granted to a common user C##\_HR\_CLERK. The SELECT ANY TABLE privilege can be used by C##\_HR\_CLERK while connected to any PDB. A local role LR\_HR in the PDB\_HR PDB has the INSERT INTO HR.EMPLOYEES privilege and is granted to C##\_HR\_CLERK. C##\_HR\_CLERK can use the INSERT INTO HR.EMPLOYEES only while connected to PDB\_HR.

A local user can be assigned a common role, and as before the privileges assigned by that role can only be exercised in the PDB where the local user is defined. If the local role LR\_HR is granted to the C##\_USER, the C##\_USER would be able to exercise the INSERT INTO HR.EMPLOYEES privilege only while connected to PDB\_HR.

The syntax and behavior of GRANT and REVOKE are mostly unchanged. The syntax has been extended with the CONTAINER clause so that common and local roles can be granted or revoked commonly or locally. A common role can be granted or revoked locally to or from a user or role with the CONTAINER=CURRENT clause or commonly with the CONTAINER=ALL clause.

## Creating Shared and Non-Shared Objects

- Non-shared tables can be created by common and local users.
- Shared tables CANNOT be created by users.
- Shared tables can be created by Oracle-supplied users.

The Oracle logo, consisting of the word "ORACLE" in white, uppercase letters on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Non-shared tables and other non-shared database objects may be created in a PDB by either common or local users. Shared tables can be created by Oracle-supplied common users. Common users created by a user who is granted the `CREATE USER` privilege cannot create shared tables or objects.

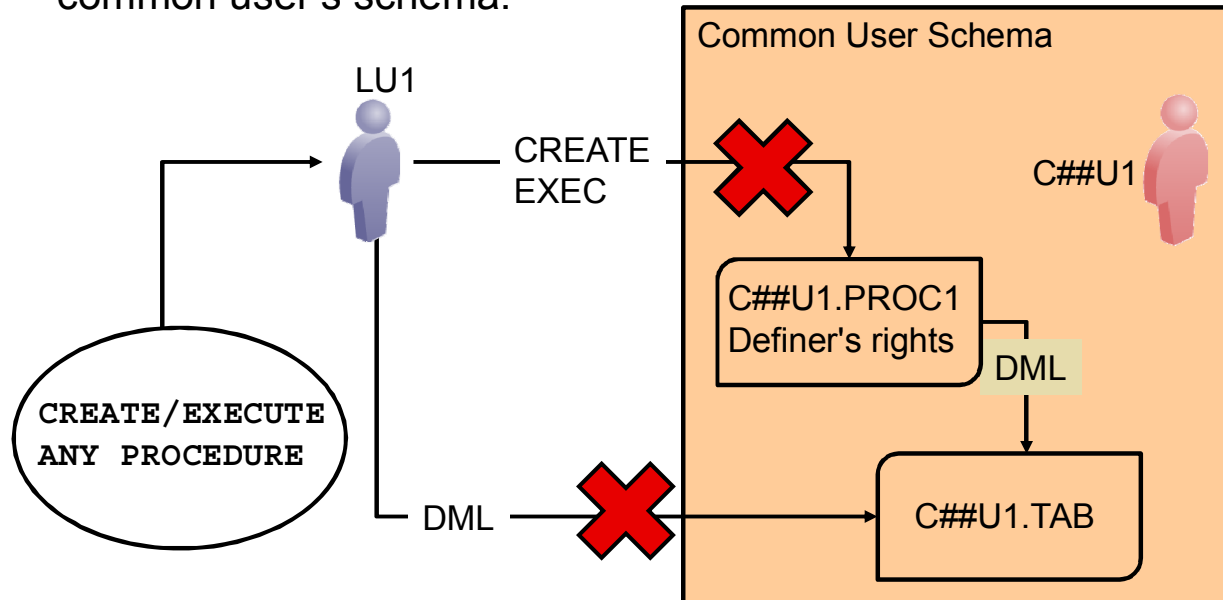
There are two types of shared objects, Object Link and Metadata Link.

- Object Links point to objects in the root and are used to share data across containers. An example of these Object Link objects is AWR objects.
- Metadata Links share metadata with objects in the root but have private copies of data. An example of these Metadata Link objects is `DBA_xxx` views.

You can find the type of sharing in the `SHARING` column of `DBA_OBJECTS`.

## Restriction on Definer's Rights

Local users **cannot** exercise local system privileges on a common user's schema.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A user cannot exercise local system privileges on a common user's schema.

An example: Suppose a local user does not have privileges to perform DML on a table in a common users schema, but does have CREATE ANY PROCEDURE and EXECUTE ANY PROCEDURE privileges. The local user is not allowed to create a definer's rights PL/SQL procedure that can issue statements as the common user against the common user's schema and then proceed to execute that procedure to get around restrictions imposed on local users.

## Quiz

You can create roles in PDBs. Which statement is true about common and local roles?

- a. You can create local roles in the root.
- b. You can create common roles only when connected in the root.
- c. You can create common roles in PDBs.
- d. You can create common and local roles in seed.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

## Quiz

Choose all that apply to make a true statement. In a PDB, a common user and a local user can:

- a. Have the same name
- b. Be granted the same privileges
- c. Be granted both local and common roles
- d. Open and close the PDB if granted the appropriate privilege

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b, c, d**

## Summary

In this lesson, you should have learned how to manage:

- Any type of tablespaces in CDB and PDBs
- Common and local users
- Common and local roles
- Common and local privileges

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 6-1: Managing tablespaces
- 6-2: Managing common and local users
- 6-3: Managing common and local roles
- 6-4: Managing common and local privileges

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



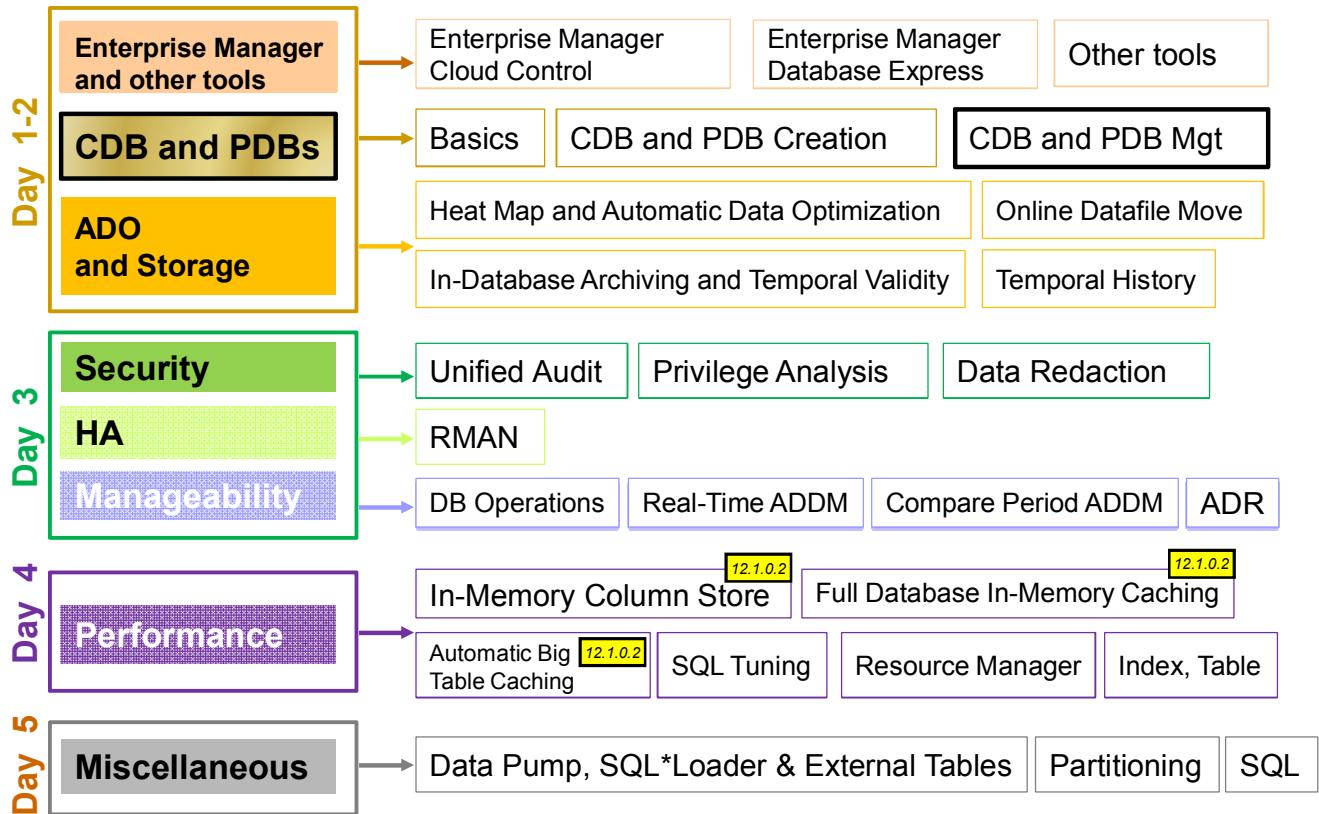


# **Backup, Recovery, and Flashback CDBs and PDBs**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Objectives

After completing this lesson, you should be able to:

- Perform CDB and PDB backups
- Recover CDB from essential file loss
- Recover PDB from PDB data file loss
- Perform flashback database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of Oracle PDB new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*
- *Oracle Database Backup and Recovery User's Guide 12c Release 1 (12.1)*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series* demonstrations:
  - *Oracle Database 12c: Recovering a Pluggable Database*
- *Oracle By Example (OBE)*:
  - *Performing Point-In-Time Recovery for a Pluggable Database*
- *MOS note: Oracle Multitenant Option – 12c: Frequently Asked Questions (Doc ID 1511619.1)*

# Goals

- Back up CDB and applications independently:
  - ARCHIVELOG mode at CDB level
  - CDB and PDB backups
  - Hot backup at CDB and PDB levels
- Recover CDB or PDBs at different levels:
  - Instance failure: CDB level
  - Complete media recovery:
    - CDB or PDB temp file
    - Control file/redo log file/root data file: CDB mounted
    - PDB data file
  - Incomplete media recovery: CDB mounted or PDB closed
  - Flashback database: CDB mounted

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Any database, non-CDB or CDB, needs to be backed up in case of potential recovery required.

The ARCHIVELOG mode can be set only at the CDB level and not at the PDB level due to redo log files sharing.

In Oracle Database 11g, you can perform whole and partial non-CDB backup and on another level, hot tablespace or data file backups.

In Oracle Database 12c, you can still perform the same types of backups. The new level for backing up data is the PDB level.

The granularity of media recovery is very flexible and can be done for the entire CDB, for a PDB, for a tablespace, for a data file, or even for a block.

You can perform different types of recovery when a failure occurs:

- What to do if an instance failure occurs?
- What to do if a PDB temp file is missing?
- How to proceed if a control file is missing or corrupted?
- What to do if a root data file is missing or corrupted?
- How to proceed if a PDB data file is missing or corrupted?
- Do you use flashback database if a local schema is dropped or is there another type of flashback available at the PDB level?

## New Syntax and Clauses in RMAN

```
$ export ORACLE_SID=cdb1
$ rman TARGET /
```

```
$ rman TARGET sys@sales_pdb
```

- DATABASE keyword operates on all PDBs and root, or a PDB:

```
RMAN> BACKUP DATABASE;
RMAN> RECOVER DATABASE;
```

- PLUGGABLE DATABASE clause operates on individual PDBs:

```
RMAN> BACKUP PLUGGABLE DATABASE hr_pdb, sales_pdb;
RMAN> RECOVER PLUGGABLE DATABASE hr_pdb;
```

- Backup, restore, recover the root using CDB\$ROOT keyword:

```
RMAN> BACKUP PLUGGABLE DATABASE "CDB$ROOT";
```

- Qualify the tablespace of a PDB with the PDB name:

```
RMAN> BACKUP TABLESPACE sales_pdb:tbs2;
RMAN> RESTORE TABLESPACE sysaux;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use Recovery Manager (RMAN) or Enterprise Manager to back up and recover entire CDBs, partial CDBs, individual whole PDBs, or partial PDBs such as tablespace/datafile of specific PDBs.

The CDB or PDBs are the possible targets for RMAN. An individual PDB is a valid RMAN TARGET database. Connect to the root or a PDB as a user with SYSDBA or SYSBACKUP privilege.

The traditional syntax such BACKUP DATABASE, RESTORE DATABASE, RECOVER DATABASE operates on the root and all of its PDBs, and, therefore, on the whole CDB, or on a single PDB depending on the connection.

A new syntax is introduced to allow back up, restore, recover single or several PDBs. To back up, restore, or recover the root, CDB\$ROOT must be used.

RMAN TABLESPACE syntax can be qualified with the PDB name so that the user can specify the name of the TABLESPACE as it is known to PDB during backup, restore, recover commands. If no PDB qualifier is used, the root is used by default. To list the tablespaces and their associated PDB, use the REPORT SCHEMA syntax.

The same new clauses can be applied to DUPLICATE, SWITCH, REPORT, CONVERT, CHANGE, LIST, DELETE.

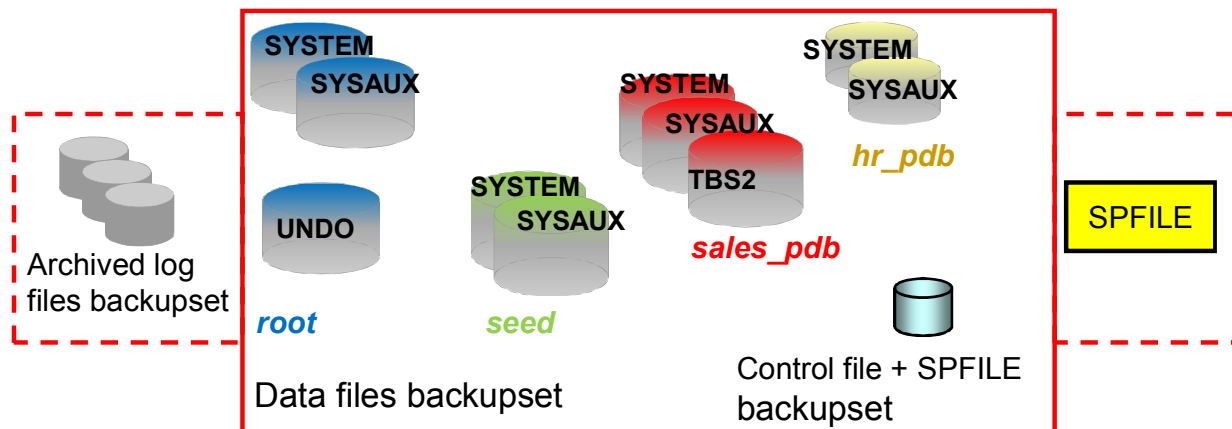
The root needs to be available for recovery, backup, and RMAN backups.

# CDB Backup: Whole CDB Backup

Back up all the PDB's data files and root files.

```

RMAN> CONNECT TARGET /
RMAN> BACKUP DATABASE PLUS ARCHIVELOG;
  
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

RMAN can back up the entire CDB and contained or individual PDBs. In addition, individual tablespaces or data files can be backed up from a specific PDB.

A whole database backup of a CDB can be, in a similar way to non-CDBs, either backup sets or image copies of the entire set of data files, namely the root data files and all PDBs data files, and the control file. The server parameter file (SPFILE) and the control files are automatically backed up. You can optionally include the archived redo log files.

Using RMAN to make an image copy of all the CDB files simply requires mounting or opening the CDB, starting RMAN, connecting to the root with `SYSDBA` or `SYSBACKUP` privilege, and entering the `BACKUP` command shown in the slide. Optionally, you can supply the `DELETE INPUT` option when backing up archive log files.

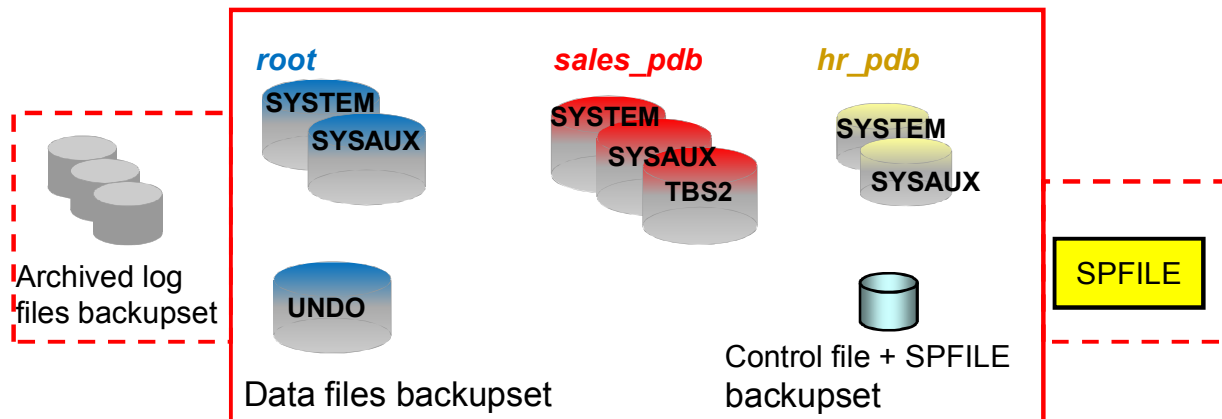
You can also create a backup (either a backup set or image copies) of previous image copies of all data files and control files in the CDB by using the following command:

```
RMAN> BACKUP COPY OF DATABASE;
```

## CDB Backup: Partial CDB Backup

Back up the root and/or individual PDBs.

```
RMAN> CONNECT TARGET /
RMAN> BACKUP PLUGGABLE DATABASE "CDB$ROOT", sales_pdb;
RMAN> BACKUP PLUGGABLE DATABASE hr_pdb PLUS ARCHIVELOG;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A partial CDB backup backs up the entire set of data files of the root, all data files of defined PDBs, and the control file and SPFILE.

The `BACKUP PLUGGABLE DATABASE "CDB$ROOT", sales_pdb` command backs up all data files of the root container, namely the **SYSTEM**, **SYSAUX** and **UNDO** data files, and then all data files of the **sales\_pdb** PDB, namely the **SYSTEM**, **SYSAUX**, and **TBS2** data files.

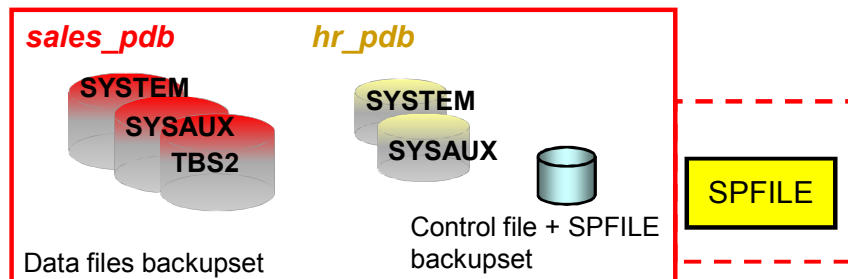


## PDB Backup: Whole PDB Backup

Backup whole PDBs with RMAN:

```
RMAN> CONNECT TARGET /
RMAN> BACKUP PLUGGABLE DATABASE sales_pdb;
```

```
RMAN> BACKUP PLUGGABLE DATABASE sales_pdb, hr_pdb;
```



```
RMAN> CONNECT TARGET sys@sales_pdb
RMAN> BACKUP DATABASE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A whole PDB backup backs up the entire set of data files of an individual PDB, and the control file and SPFILE.

In the first example, the `BACKUP PLUGGABLE DATABASE` command backs up all data files of the `sales_pdb` PDB, namely the `SYSTEM`, `SYSAUX`, and `TBS2` data files.

In the second example, the same command backs up all data files of two named PDBs. You could also connect to the PDB target as displayed in the third example:

```
$ rman TARGET sys@sales_pdb
```

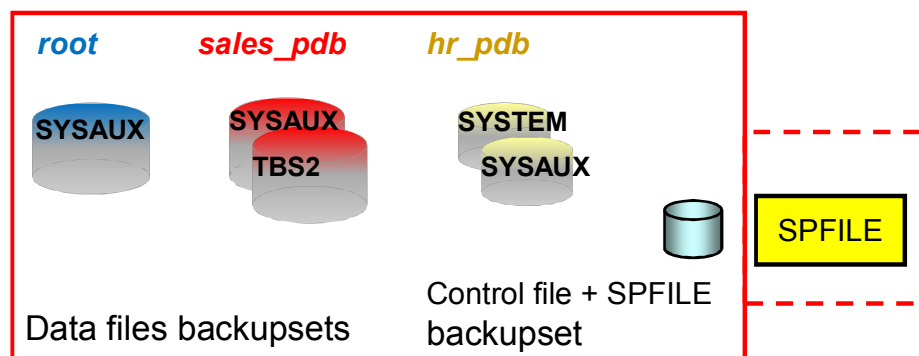
In this case, use the `BACKUP DATABASE` command and not the `BACKUP PLUGGABLE DATABASE` command.

## PDB Backup: Partial PDB Backup

```

RMAN> CONNECT TARGET /
RMAN> REPORT SCHEMA;
RMAN> BACKUP TABLESPACE sales_pdb:tbs2;
RMAN> BACKUP TABLESPACE hr_pdb:system,
        sales_pdb:sysaux;
RMAN> BACKUP TABLESPACE sysaux, hr_pdb:sysaux;

```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A partial PDB backup backs up the data files named tablespaces of individual PDBs, and the control file and SPFILE.

The example uses the `BACKUP TABLESPACE` command to back up all data files of the **TBS2** tablespace of **sales\_pdb** PDB. To find the names of tablespaces within PDBs, use the `REPORT SCHEMA` command.

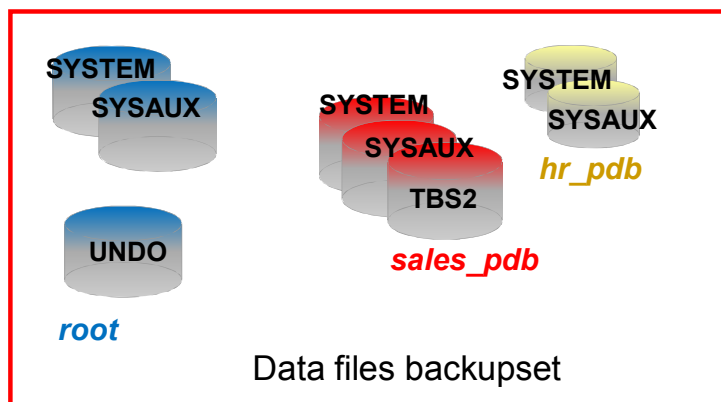
The second backup uses the same command to back up all data files of the **SYSTEM** tablespace of **hr\_pdb** PDB and all data files of the **SYSAUX** tablespace of **sales\_pdb** PDB.

The third backup uses the same command to back up all data files of the **SYSAUX** tablespace of the **root** and all data files of the **SYSAUX** tablespace of **hr\_pdb** PDB.

## PDB Backup: User-Managed Hot PDB Backup

Perform a user-managed hot PDB backup.

```
SQL> CONNECT system@sales_pdb
SQL> ALTER DATABASE BEGIN BACKUP;
SQL> !cp sales_pdb/datafiles /backup_dir_sales_pdb
SQL> ALTER DATABASE END BACKUP;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After setting the whole PDB in backup mode, all data files of the PDB are placed in **ACTIVE** mode. This mode allows you to back up the data files with an O/S command.

## Practice

- 7-1: Cold backup of a CDB
- 7-2: Whole CDB backup with RMAN
- 7-3: PDB backup with RMAN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Recovery

- Instance recovery: **CDB level only**
- Automatic missing temp file recreation at **CDB open**
- Complete media recovery after file loss or corruption
  - **CDB:** Same as for non-CDB
    - Redo log files, control files
    - Root data files
  - **PDB** data files
  - **Tablespace:** PDB or CDB data files
- Incomplete media recovery after file loss or corruption
  - CDB: The whole CDB back in time
  - PDB: A whole PDB back in time
  - TSPITR for **any tablespace** except SYSTEM, UNDO, SYSAUX
- Block recovery: No change
- **Flashback database: CDB mounted**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a CDB, the granularity of media recovery is very flexible and can be done for the entire CDB, for a PDB, for a tablespace, for a data file, or even for a block.

Crash and instance recovery is supported for a CDB as a whole, because there is one instance for the root and all its PDBs. When the instance crashes, the only possible instance recovery is a CDB instance recovery.

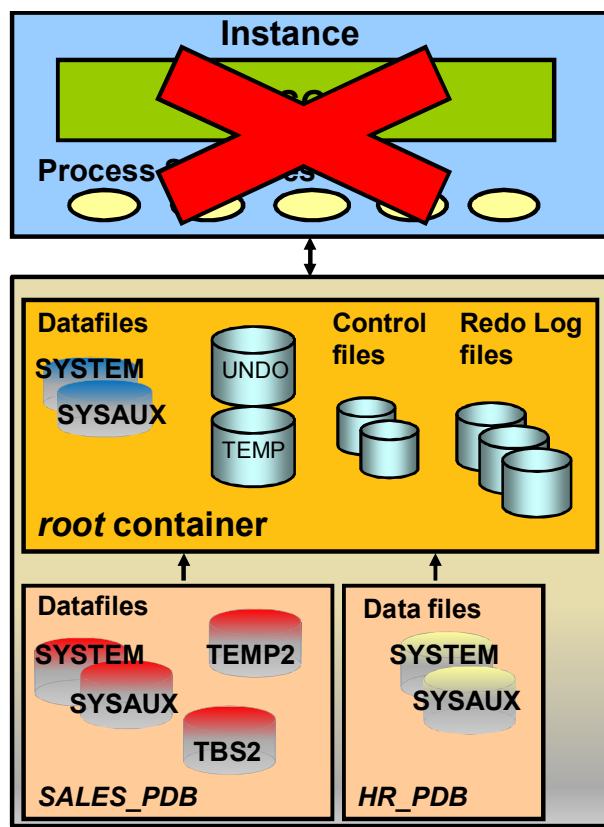
If redo log files or control files, data files from the root `SYSTEM` or `UNDO` tablespace are lost, because these files belong to the root, the only possible media recovery is at CDB level.

There exists a new level of media recovery supported, individual PDB media recovery after datafile loss or corruption of a PDB.

There exists a new level of incomplete or PITR recovery, PDB PITR that will look like a tablespace recovery.

Correcting human error applying to the root and PDBs requires flashback at the CDB level only. For example, if you dropped a common or local schema, you must use flashback database applying to all containers, the root, and the PDBs.

## Instance Failure



PDB instance recovery is **impossible**.

After instance failure:

- Connect to the root
- Open the root
- Open PDBs:
  - Automatic if saved state configured

```
SQL> STARTUP;
```

– Manually:

```
SQL> STARTUP;
SQL> ALTER PLUGGABLE DATABASE
      ALL OPEN;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Crash/instance recovery is supported for a CDB as a whole, because there is one instance for the root and all its PDBs.

Redo log files required to perform the instance recovery are stored in the unique set of redo log files of the CDB. There is a single shared redo stream for root and all the PDBs used to perform instance recovery. The instance recovery is performed while opening the root.

There is no way to perform a PDB instance recovery.

For an instance to open a data file, the system change number (SCN) contained in the data file's header must match the current SCN that is stored in the control files.

If the numbers do not match, the instance applies redo data from the online redo logs, sequentially “redoing” transactions until the data files are up-to-date. After all data files have been synchronized with the control files, the root is opened and by default all PDBs are still in mount state.

When redo logs are applied, all transactions are applied to bring the CDB up to the state as of the time of failure. This usually includes transactions that are in progress but have not yet been committed. After the root has been opened, the uncommitted transactions are rolled back on root data files. After the PDBs are opened, the uncommitted transactions are rolled back on PDBs data files. When opening a CDB, by default all PDBs remain in mounted state. Either last open state was previously saved to automatically open the PDBs, or you have to execute the `ALTER PLUGGABLE DATABASE ALL OPEN` command to open them all.

## NOARCHIVELOG Mode

If the database is in NOARCHIVELOG mode, and a data file is lost, perform the following tasks:

- Shut down the instance if it is not already down.
- Restore the entire CDB including all data files and control files.
- Start up the instance and open the CDB and all PDBs.

Users must re-enter all changes made since the last backup.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The loss of any data file from a CDB in NOARCHIVELOG mode requires complete restoration of the CDB, including control files and all data files of the root and all PDBs. If you have incremental backups, you need to perform the restore and recover operations on the CDB.

With the database in NOARCHIVELOG mode, recovery is possible only up to the time of the last backup. So users must reenter all changes made since that backup.

For this type of recovery, proceed with the following steps:

- Shut down the instance if it is not already down.
- Restore the entire CDB including all data and control files from the last backup.
- Start up the instance and open the root and PDBs.

In the next slides, you consider that the CDB is in ARCHIVELOG mode.

## Media Failure: CDB Temporary File Recovery

SQL statements that require temporary space may fail if one of the temp files is missing: similar to non-CDB.

```
SQL> CONNECT / AS SYSDBA
SQL> select * from DBA_OBJECTS
            order by 1,2,3,4,5,6,7,8,9,10,11,12,13;
select * from DBA_OBJECTS order by
1,2,3,4,5,6,7,8,9,10,11,12,13
            *
ERROR at line 1:
ORA-01565: error in identifying file
'/u01/app/oracle/oradata/CDB1/temp01.dbf'
ORA-27037: unable to obtain file status
Linux Error: 2: No such file or directory
```

- Automatic re-creation of temporary files at CDB opening
- Manual re-creation also possible

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you create a user, you can specify a temporary tablespace to be used by the user. If a temporary tablespace is not specified, the default tablespace for the PDB is used. If a default tablespace is not specified for the PDB, the temporary tablespace for the CDB is used.

If a temp file belonging to the CDB temporary tablespace is lost or damaged, and the user issuing the statement uses it, an error during the execution of SQL statements that require this temporary space occurs.

The SQL statement shown in the slide has a long list of columns to order by, which results in the need for temporary space from the root temporary tablespace. The missing file error is encountered when this statement requiring a sort is executed.

The CDB instance can start up with a missing temporary file. If any of the temporary files do not exist when the CDB instance is started, they are created automatically and the CDB opens normally. When this happens, a message like the following appears in the alert log during startup:

- Re-creating the temp file /u01/app/oracle/oradata/CDB1/temp01.dbf

You can decide a manual recreation instead, while connected to root:

```
SQL> ALTER TABLESPACE temp ADD TEMPFILE
2  '/u01/app/oracle/oradata/CDB1/temp02.dbf' SIZE 20M;
SQL> ALTER TABLESPACE temp DROP TEMPFILE
3  '/u01/app/oracle/oradata/CDB1/temp01.dbf';
```



## Media Failure: **PDB Temporary File Recovery**

SQL statements that require temporary space may fail if one of the temp files is missing.

```
SQL> CONNECT local_user@HR_PDB
SQL> select * from my_table order by
1,2,3,4,5,6,7,8,9,10,11,12,13;
select * from my_table order by
1,2,3,4,5,6,7,8,9,10,11,12,13
*
ERROR at line 1:
ORA-01565: error in identifying file
'/u01/app/oracle/oradata/CDB1/HR_PDB/temp2_01.dbf'
ORA-27037: unable to obtain file status
Linux Error: 2: No such file or directory
```

- Automatic re-creation of temporary files at **PDB opening**
- Manual re-creation also possible

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If a temp file belonging to a PDB temporary tablespace is lost or damaged, and the user issuing the statement uses it, an error during the execution of SQL statements that require that temporary space for sorting occurs.

The SQL statement shown in the slide has a long list of columns to order by, which results in the need for temporary space from the PDB temporary tablespace. The missing file error is encountered when this statement requiring a sort is executed.

The PDB can open with a missing temporary file. If any of the temporary files do not exist when the PDB is opened, they are automatically re-created. They are also automatically re-created at CDB startup.

You can perform a manual re-creation instead, while connected to the PDB:

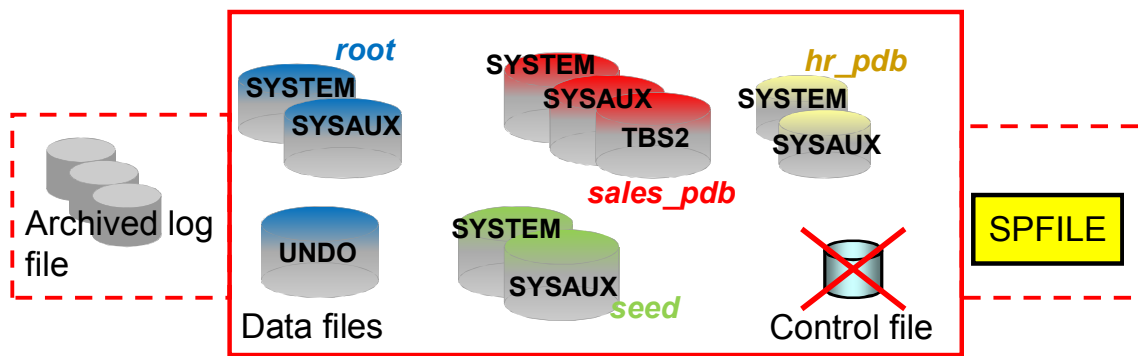
```
SQL> ALTER TABLESPACE temp ADD TEMPFILE
2 '/u01/app/oracle/oradata/CDB1/HR_PDB/temp2_02.dbf'
3 SIZE 20M;
SQL> ALTER TABLESPACE temp DROP TEMPFILE
2 '/u01/app/oracle/oradata/CDB1/HR_PDB/temp2_01.dbf';
```

## Media Failure: Control File Loss

Similar to non-CDBs: CDB mounted

```

RMAN> CONNECT TARGET /
RMAN> STARTUP NOMOUNT;
RMAN> RESTORE CONTROLFILE FROM AUTOBACKUP;
RMAN> ALTER DATABASE MOUNT;
RMAN> RECOVER DATABASE;
RMAN> ALTER DATABASE OPEN RESETLOGS;
RMAN> ALTER PLUGGABLE DATABASE ALL OPEN;
  
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If a control file is missing or corrupted, because control files belong to the CDB, the instance soon crashes and a whole CDB media recovery is required.

- First start the CDB instance.
- Then restore the control file from a backup.
- Mount the CDB.
- Then recover and open the CDB in resetlogs.

Two possibilities:

```

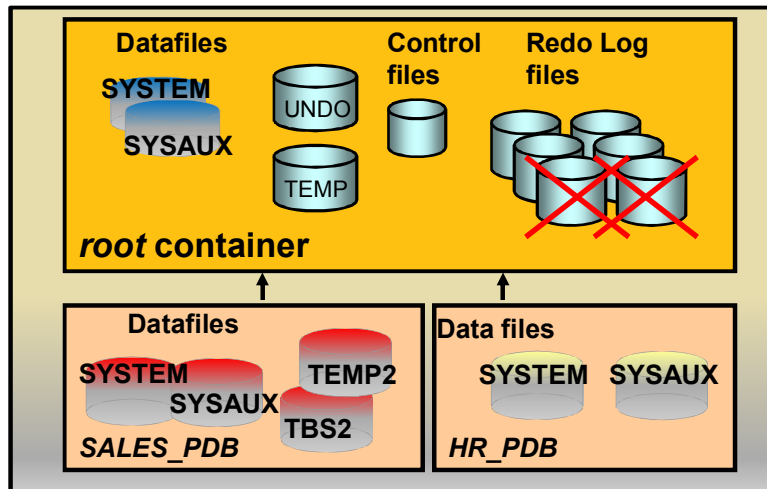
RMAN> ALTER DATABASE OPEN RESETLOGS;
RMAN> ALTER PLUGGABLE DATABASE ALL OPEN;
  
```

Or

```

SQL> ALTER DATABASE OPEN RESETLOGS;
  
```

## Media Failure: Redo Log File Loss



Similar to non-CDB procedures:

- Connect to the root container.
- Examine the `STATUS` of the lost file: ACTIVE, INACTIVE, CURRENT.
- Proceed as with non-CDBs.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

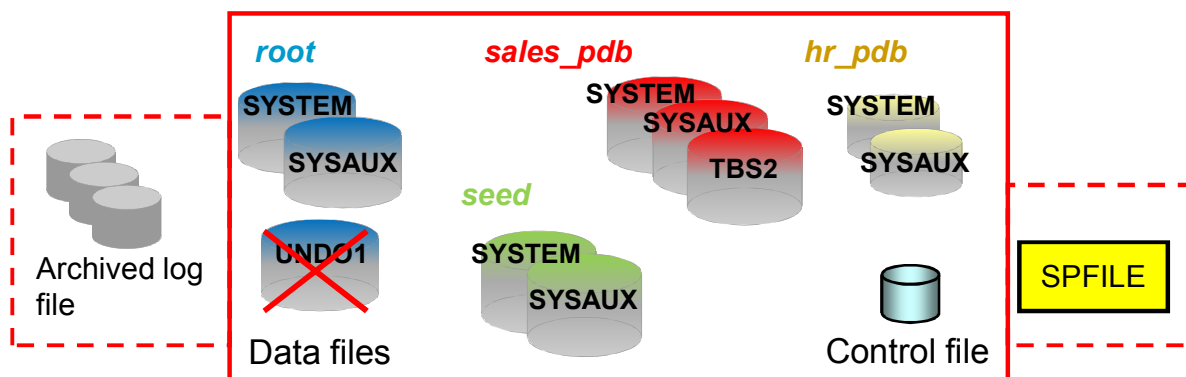
Missing or corrupted redo log files, because there is only one redo stream for the CDB instance (or one redo stream for each instance of a RAC CDB), require a whole CDB media recovery as explained in the slide.

Depending on whether a whole redo log group or only a redo log member is missing, follow the same procedures as those for non-CDBs.

## Media Failure: **Root SYSTEM or UNDO Data File**

Similar to non-CDBs: CDB mounted

```
RMAN> STARTUP MOUNT;
RMAN> RESTORE TABLESPACE undo1;
RMAN> RECOVER TABLESPACE undo1;
RMAN> ALTER DATABASE OPEN;
RMAN> ALTER PLUGGABLE DATABASE ALL OPEN;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the missing or corrupted data file belongs to the root container **SYSTEM** or **UNDO** tablespace, the CDB instance requires a shutdown and a media recovery. In a RAC environment, you would shut all CDB instances down.

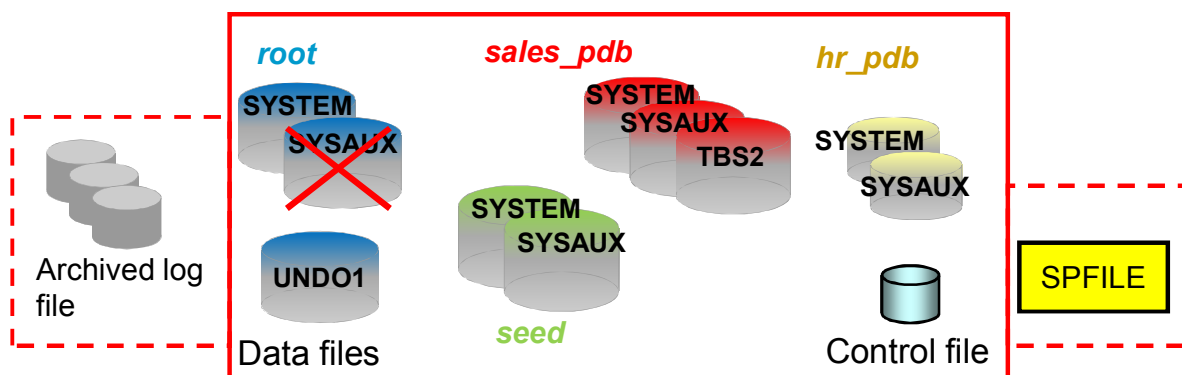
This means that all PDBs will be closed.

The CDB must be mounted before restoring and recovering the missing root data file. After the root data file is recovered, open the CDB and all PDBs.

## Media Failure: **Root SYSAUX Data File**

Similar to non-CDBs: tablespace OFFLINE

```
RMAN> ALTER TABLESPACE sysaux OFFLINE IMMEDIATE;
RMAN> RESTORE TABLESPACE sysaux;
RMAN> RECOVER TABLESPACE sysaux;
RMAN> ALTER TABLESPACE sysaux ONLINE;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the data file missing or corrupted is a data file of the root tablespaces other than SYSTEM or UNDO, you offline the tablespace, and then perform a tablespace media recovery.

This means that no PDB is closed.

The CDB remains opened while restoring and recovering the missing root data file.

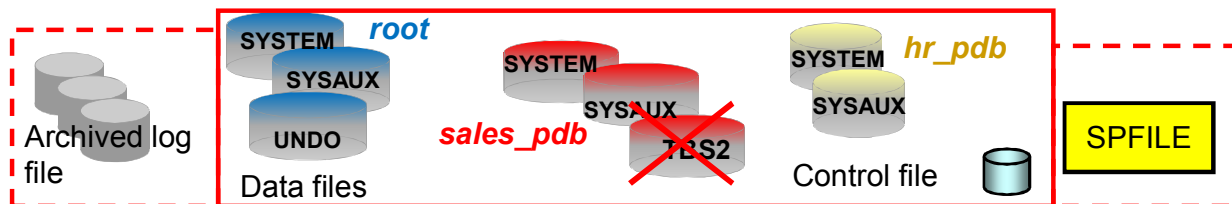
After the root data file is recovered, bring the tablespace online.

## Media Failure: PDB Data File

Similar to non-CDBs: Perform the recovery within the PDB

- Connect to the PDB.
- Put the tablespace OFFLINE.
- Other PDBs are not impacted.

```
SQL> CONNECT system@sales_pdb
SQL> ALTER TABLESPACE tbs2 OFFLINE IMMEDIATE;
RMAN> CONNECT TARGET /
RMAN> RESTORE TABLESPACE sales_pdb:tbs2;
RMAN> RECOVER TABLESPACE sales_pdb:tbs2;
SQL> ALTER TABLESPACE tbs2 ONLINE;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If the data file missing or corrupted belongs to a PDB, the PDB needs not be closed. The data file with the error is taken **OFFLINE IMMEDIATE**. Media recovery for that data file is required and performed in a similar way to media recovery of a set of tablespaces. During that recovery time, users can work with other PDB tablespaces and within other PDBs.

# Media Failure: PITR

- PDB PITR

```

RMAN> ALTER PLUGGABLE DATABASE PDB1 CLOSE;
RMAN> RUN {
    SET UNTIL SCN = 1851648 ;
    RESTORE pluggable DATABASE pdb1;
    RECOVER pluggable DATABASE pdb1
        AUXILIARY DESTINATION='/u01/app/oracle/oradata';
    ALTER PLUGGABLE DATABASE pdb1 OPEN RESETLOGS;
}

```

- PDB Tablespace PITR

```

RMAN> RECOVER TABLESPACE PDB1:TEST_TBS
        UNTIL SCN 832972
        AUXILIARY DESTINATION '/tmp/CDB1/reco';
RMAN> ALTER TABLESPACE PDB1:TEST_TBS ONLINE;

```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you need to recover a PDB database to a point in time in the past beyond flashback retention, then in this case, flashback is not possible, therefore a point-in-time recovery is necessary.

Recovering a PDB to a point-in-time does not affect all parts of the CDB: the whole CDB and all other PDBs are opened. After recovering a PDB to a specified point-in-time, when you open the PDB by using the `RESETLOGS` option, a new incarnation of the PDB is created. The PDB `RESETLOGS` does not perform a `RESETLOGS` for the CDB.

- A PDB record in the control file is updated.
- Each redo log record carries PDB id in the redo header. This is how recovery knows which redo applies to which PDB. Redo logs are shared by all PDBs; redo from each PDB is written to a single set of redo logs.

Conceptually a PDB resetlogs is similar to a database resetlogs.

After recovery, the old backup of the PDB remains valid and can be used if a media failure occurs. After restoring/recovering a PDB to a past point in time, one cannot open the PDB read only. PDB read-write open through resetlogs is required.

A PDB incarnation is a subincarnation of the CDB. For example, if the CDB is incarnation 5, and a PDB is incarnation 3, then the fully specified incarnation number of the PDB is (5, 3). The initial incarnation of a PDB is 0. To view the incarnation of a PDB, query the `V$PDB_INCARNATION` view.

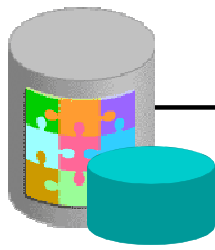
If you do not use a fast recovery area, you must specify the temporary location of the auxiliary set files by using the `AUXILIARY DESTINATION` clause (example in the slide).

Each PDB has its own set of tablespaces. TSPITR can be used to recover a tablespace to an earlier point in time. Perform the TSPITR as described in the second example in the slide, specifying the full tablespace name including the PDB name.

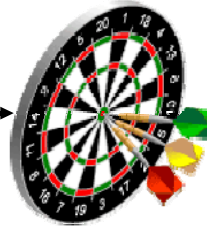
Recovering a tablespace of a PDB to a point-in-time does not affect all parts of the CDB: the whole CDB is still opened and therefore all PDBs are opened. After recovering a tablespace back in time, put the tablespace back online.



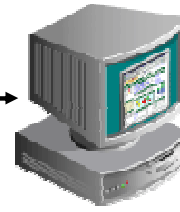
## Flashback CDB



1. Configure the FRA.



2. Set the retention target.



3. Enable Flashback Database.

```
SQL> SHUTDOWN IMMEDIATE
SQL> STARTUP MOUNT
SQL> ALTER DATABASE ARCHIVELOG;
SQL> ALTER SYSTEM SET
      DB_FLASHBACK_RETENTION_TARGET=2880 SCOPE=BOTH;
SQL> ALTER DATABASE FLASHBACK ON;
SQL> ALTER DATABASE OPEN;
```

If the CDB is in ARCHIVELOG mode, there is no need to restart it.

- No flashback of root without flashing back the whole CDB.
- No flashback to a point earlier than the point at which PDBPITR was performed.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can configure Flashback Database for the CDB as you would do for any non-CDB:

1. Configure the Fast Recovery Area.
2. Set the retention target with the `DB_FLASHBACK_RETENTION_TARGET` initialization parameter.
3. Enable Flashback Database with the following command:

```
SQL> ALTER DATABASE FLASHBACK ON;
```

Before you can issue the command to enable Flashback Database, the database must be configured for archiving.

You can disable Flashback Database with the `ALTER DATABASE FLASHBACK OFF` command. As a result, all existing Flashback Database logs are deleted automatically.

Restrictions:

- You cannot flash back the root alone without flashing back the entire CDB.
- Flashback Database operations on a CDB may not be permitted if point-in-time recovery has been performed on any of its PDBs. When point-in-time recovery is performed on a PDB, you cannot directly rewind the CDB to a point that is earlier than the point at which DBPITR for the PDB was performed.

## Flashback CDB

A common user is dropped.

### 1. Flashback CDB: CDB mounted in exclusive mode

```
SQL> STARTUP MOUNT
SQL> FLASHBACK DATABASE TO SCN 53943;
```

### 2. To review changes: Open CDB and PDBs in READ ONLY

```
SQL> ALTER DATABASE OPEN READ ONLY;
SQL> ALTER PLUGGABLE DATABASE ALL OPEN READ ONLY;
```

### 3. To finalize: Flash back again if necessary and open CDB with RESETLOGS.

```
RMAN> SHUTDOWN IMMEDIATE
RMAN> STARTUP MOUNT
RMAN> FLASHBACK DATABASE TO SCN 10;
RMAN> ALTER DATABASE OPEN RESETLOGS;
RMAN> ALTER PLUGGABLE DATABASE ALL OPEN;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A common schema has been accidentally dropped in the root. You have to flash back the CDB to the time before the common user was dropped impacting all PDBs.

You can use the `RMAN FLASHBACK DATABASE` command to execute the Flashback Database operation. You can use `SEQUENCE` and `THREAD` to specify a redo log sequence number and thread as a lower limit.

Alternatively, you can use the `SQL FLASHBACK DATABASE` command to return the database to a past time or SCN. If you use the `TO SCN` clause, you must provide a number. If you specify `TO TIMESTAMP`, you must provide a time stamp value. You can also specify a restore point name.

#### Note

- The CDB must be mounted in exclusive mode to issue the `FLASHBACK DATABASE` command and opened read-only to review changes. The CDB must be opened read/write with the `RESETLOGS` option when finished.
- When the CDB is opened in `READ ONLY` mode, the PDBs are still mounted. Open PDBs in `READ ONLY` mode, too, to review changes.

## Special Situations

- Creating a control file backup script:

```
SQL> ALTER DATABASE BACKUP CONTROLFILE TO TRACE;
```

```
CREATE CONTROLFILE ...
  datafile sales_pdb file1
  datafile sales_pdb file1
  ...
  datafile hr_pdb file1 ...;
```

- PDB close incompatible with CDB hot backup:

```
SQL> ALTER DATABASE BEGIN BACKUP;
SQL> ALTER PLUGGABLE DATABASE CLOSE;
ALTER PLUGGABLE DATABASE pdb1_1 close
*
ERROR at line 1:
ORA-01149: cannot shutdown - file 10 has online backup set
ORA-01110: data file 10: '/D1/oradata/cdb1/pdb1_1/users01.dbf'
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Besides the control file backup with RMAN auto-backup configuration, you can also use the SQL statement that generates a script including the `CREATE CONTROLFILE` statement with all data files of the CDB, including the root and PDBs.

The statement is still the same as for non-CDBs and requires a connection to the root:

```
SQL> ALTER DATABASE BACKUP CONTROLFILE TO TRACE;
```

While performing a hot CDB backup, you cannot close any PDB.

## Quiz

Which statements are true about PDB and CDB backups?

- a. You can connect to a specific target PDB in RMAN.
- b. You can back up a whole PDB only if connected to the target PDB in RMAN.
- c. You may back up specific PDB tablespaces with a tablespace backup.
- d. You may back up the root container with a container-specific backup.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a, c, d**

## Quiz

You can recover a single PDB, even if the CDB is in mounted state.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Summary

In this lesson, you should have learned how to:

- Perform CDB and PDB backups
- Recover CDB from essential file loss
- Recover PDB from PDB data file loss
- Perform flashback database

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 7-4: Recovery from PDB data file loss

Optional practices:

- 7-5: SQL PDB Hot backup
- 7-6: SQL control file backup
- 7-7: Recovery from all control files loss
- 7-8: Recovery from redo log member loss
- 7-9: Recovery from SYSTEM root data file loss
- 7-10: Recovery from a non-essential root data file loss
- 7-11: Point-in-time recovery of PDB tablespaces
- 7-12: CDB flashback from common user DROP

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# **Module**

## **Automatic Data Optimization and Storage Enhancements**

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



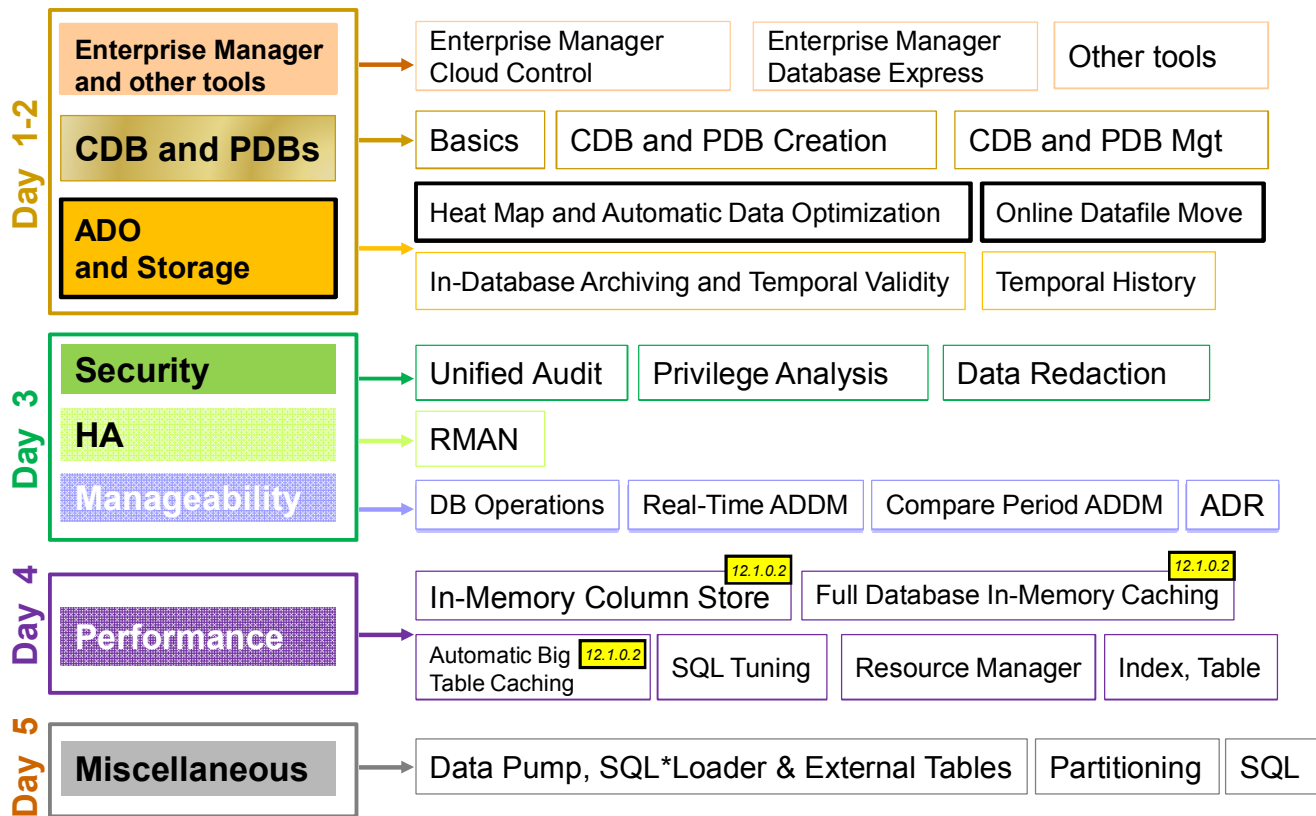
# 8

## Heat Map, Automatic Data Optimization, and Online Data File and Partition Move

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Heat Map and Automatic Data Optimization:** Heat Map and Automatic Data Optimization (ADO) can be used to implement your Information Lifecycle Management (ILM) strategy, along with Partitioning, Advanced Compression, and Hybrid Columnar Compression.

**Storage:** The Oracle Database 12c **Online Move data file** feature provides the capability to move an online data file from one kind of storage system to another while the database is opened and accessing the file. The Oracle Database 12c **Online Move partition** feature provides the capability to move and compress a partition while the partition is being accessed by DML statements.

# Objectives

After completing this lesson, you should be able to:

- Describe activity tracking with Heat Map
- Use views to display Heat Map statistics
- Describe Automatic Data Optimization
- Create ADO policies
- Use views and procedures to monitor ADO
- Move a data file online
- Move a partition online

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

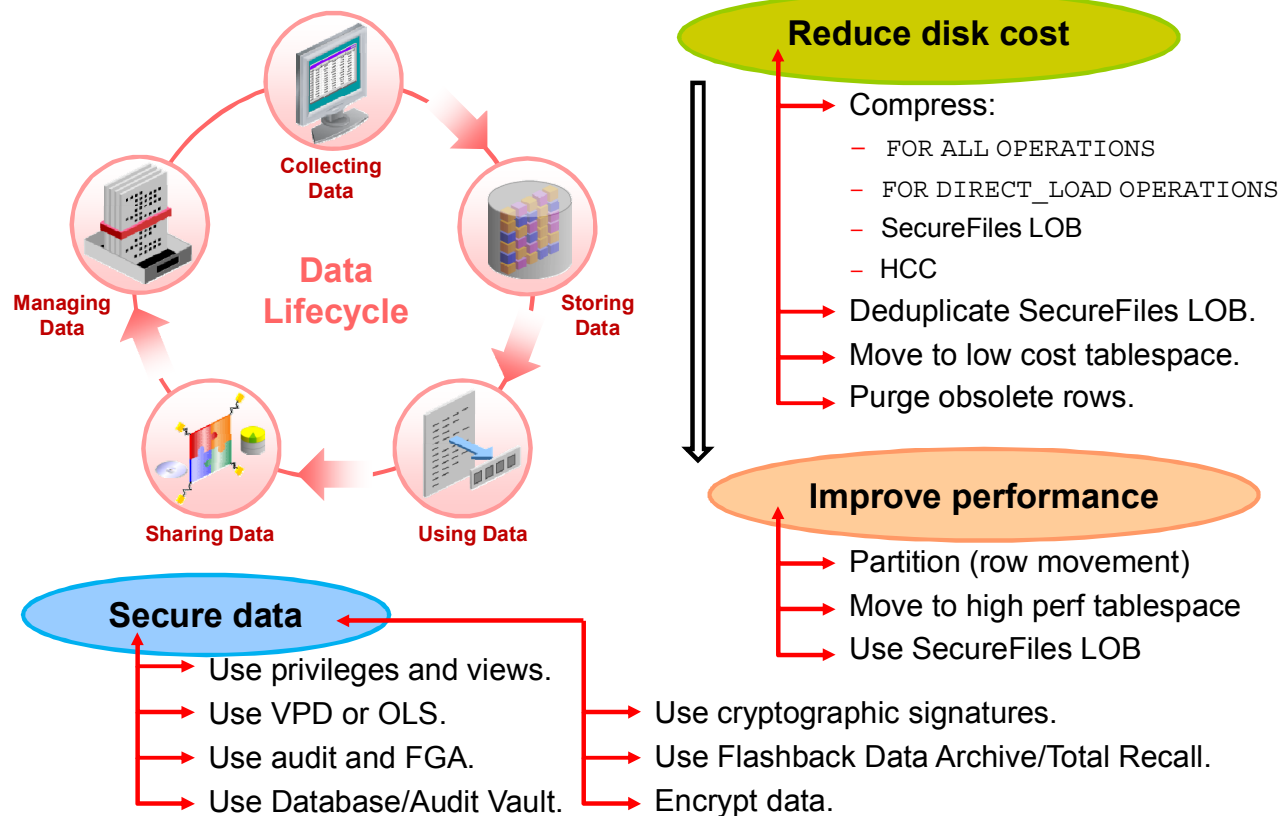
**Note:** For a complete understanding of Heat Map and Automatic Data Optimization new features and usage, and Online Move data file and partitions features, refer to the following guides in the Oracle documentation:

- *Oracle Database VLDB and Partitioning Guide 12c Release 1 (12.1)*
- *Oracle Database Administrator's Guide 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – “DBMS\_ILM”, “DBMS\_ILM\_ADMIN”, and “DBMS\_HEAT\_MAP” chapters*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series* demonstrations:
  - *Online data file Move*
- *Oracle By Example (OBE):*
  - *Setting Up Compression Tiering for Automatic Data Optimization*
  - *Setting Up Storage Tiering Automatic Data Optimization*

# ILM Challenges and Solutions



ORACLE

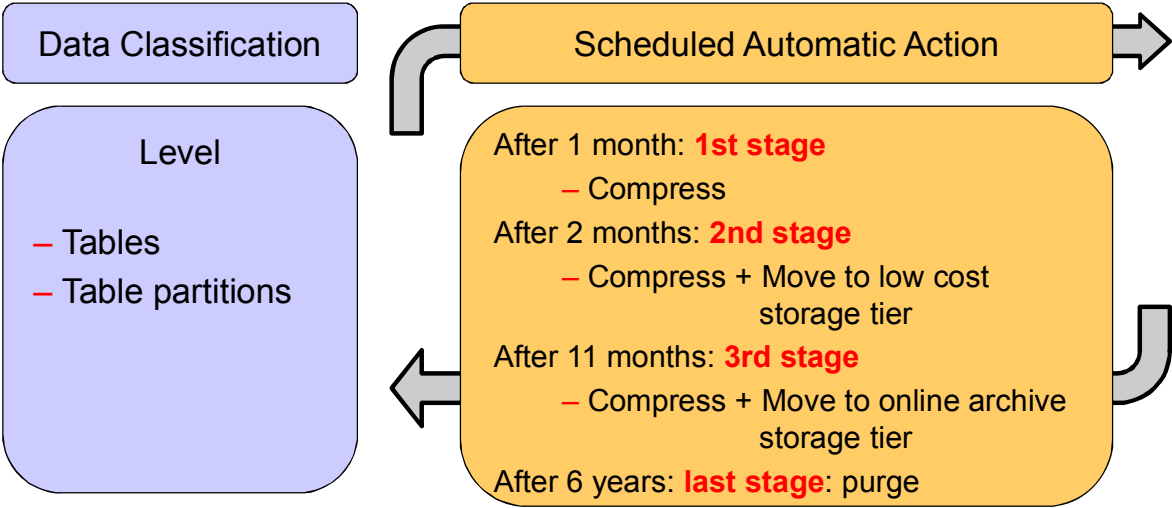
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

What is Information Lifecycle Management, also referred to as ILM? It is a strategy for managing business data over its lifetime to reduce storage costs, improve data access within the database, and adapt to regulatory requirements. ILM is the practice of applying policies for the effective management of information throughout its useful life. ILM includes every phase of a "row." It consists of the policies, processes, practices, and tools that are used to align the business value of information with the most appropriate and cost-effective IT infrastructure from the time information is conceived through its final disposition.

Customers of Oracle Database are interested in leveraging compression and storage tiering for satisfying their ILM needs. While for many customers compression may be sufficient, a subset of customers wants to optimize dormant data further by moving it to high-density, low-cost Oracle Storage or to other lower-cost storage.

To utilize storage tiering or compression tiering for dormant data (say, sales orders more than one year old), online transaction processing (OLTP) applications need to implement Partitioning for the larger tables. Oracle supports Partitioning and provides high-level documentation on how to use it, but it is left to the customer to implement and validate Partitioning.

# ILM Components



Lifecycle definition:

| Stage:      | Current Orders                     | Previous Orders  | Old Orders | Very Old Orders | End of Life for Orders |
|-------------|------------------------------------|------------------|------------|-----------------|------------------------|
| Tier:       | High Performance                   | High Performance | Low Cost   | Online Archive  |                        |
| Attributes: | None                               | Compress         | Compress   | Compress        |                        |
| Retention:  | Current Month                      | 2 Months         | 11 Months  | 6 Years         | Purged                 |
| Time:       | Most Recent Data → → → Oldest Data |                  |            |                 |                        |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

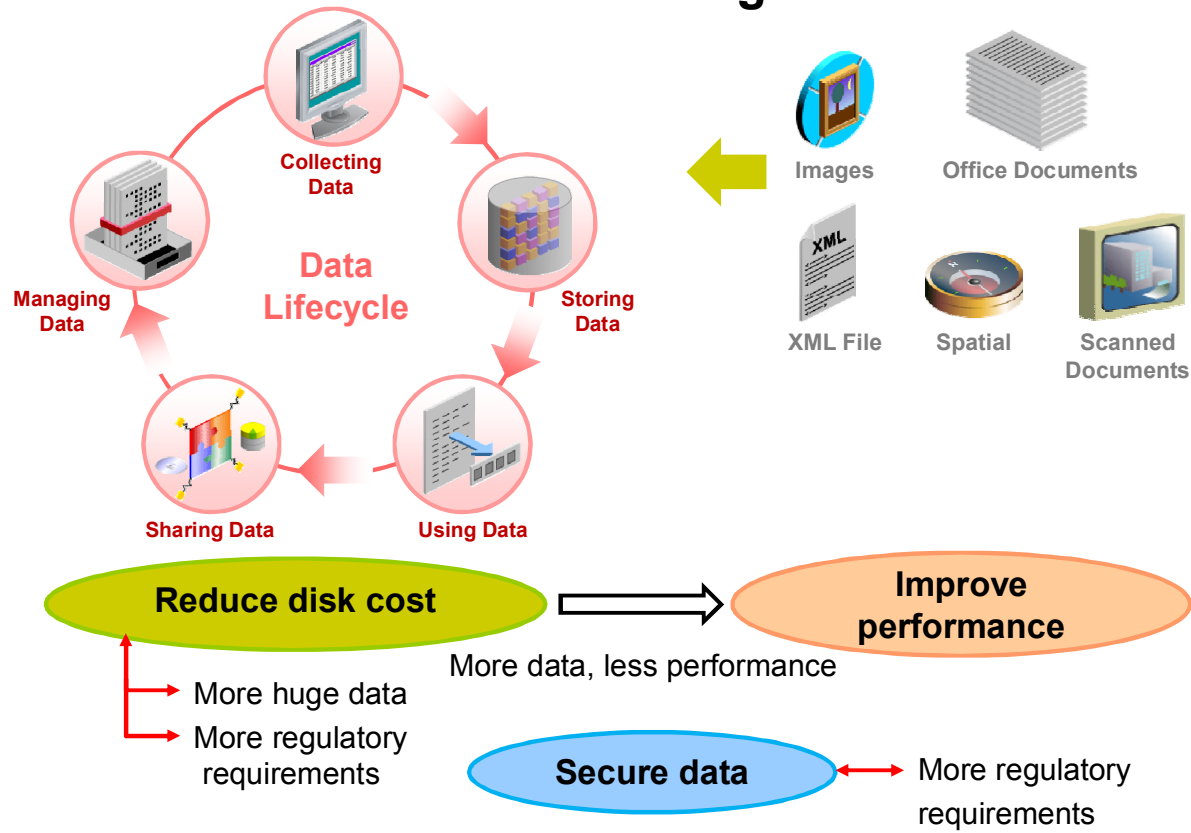
How does a lifecycle definition work in Oracle Database 11g ILM? Consider a typical ILM flow implemented by a major enterprise: data is first bulk-loaded into a table. At this stage, the database administrator (DBA) compresses only active partitions, including the most active one: OLTP Table Compression is optimized for data that is being concurrently modified.

After a month, activity has subsided, although significant OLTP transactions are still carried out. At this stage, data in the former most active partition stays in OLTP Table compressed format, and a new partition is automatically created.

Two months down the line, the data is rarely modified and accessed rarely as well. At this stage, the partition can be moved to a lower-cost storage tier and also to a higher compression level, with Hybrid Columnar Compression (HCC) on Exadata or Oracle Storage.

After 11 months, data is considered dormant because it is no longer accessed or updated, and is moved to another lower-cost storage tier, and the highest compression level with HCC, and marked as read-only data.

## ILM Challenges



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Challenges for ILM in Oracle Database 12c are still the same as in Oracle Database 11g:

- Reduce storage costs
- Improve performance
- Secure data

In addition, the amount of data continues to grow, with unstructured data consuming huge volumes of bytes. A number of companies are compelled to retain data for extended periods of time to meet compliance requirements. As a result, more data is being kept in online mode for longer periods of time to keep it available, which increases storage costs. Managing data growth continues to be *the* top priority for database users. Performance of applications often decreases as a result of data growth.

In Oracle Database 11g, life cycle event scanning and subsequent actions need to be performed manually. In Oracle Database 12c, new solutions allow the user to set policies that define application-specified rules on classified data. The rules enforce data flows automatically, with minimal manual intervention.

## Solutions

Reduce disk cost:

- Manage storage more efficiently with different tiers of storage.
- Use compression levels.

Improve performance:

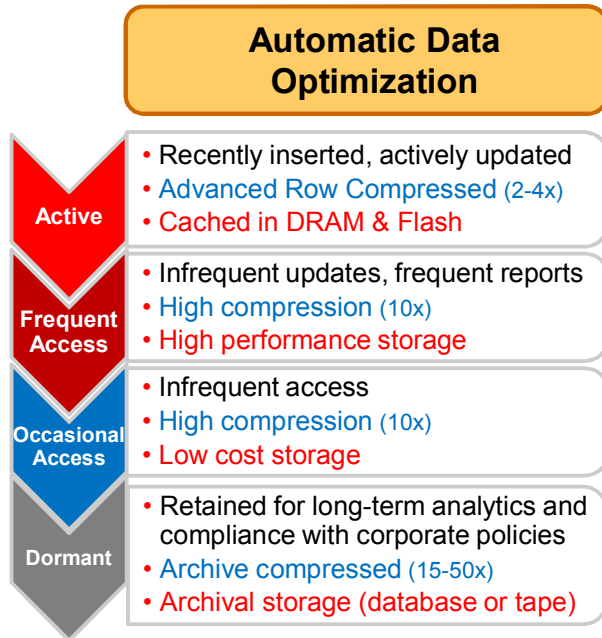
- Compress at different levels

### Automatic Data Optimization:

- Policies automatically:
  - Compress according to user-defined rules
  - Move data due to space pressure

### In-Database Archiving:

- Row-archival
- Temporal validity



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Disk Cost and Performance Decrease

As data grows, the typical reaction is to buy and install more disk storage. A smarter approach would be to implement a tiered storage strategy, and put information life cycle strategies in place to manage data better and more cost effectively.

The business requirements are not the same for all data in the database. Data goes through various stages in its life cycle. It starts out as active data, when the data is queried and modified heavily. After a period of time, this data becomes less active. It is a time period when it is queried often (for example, for report generation), but it is rarely modified. In the third phase, the data becomes more or less dormant. It is neither queried nor updated, but it needs to be kept for compliance and regulatory purposes. So it goes to an archiving storage that can still be within the database, and not necessarily outside the database.

It is often necessary to move segments from one tablespace to another. For example, segments can be moved to another tablespace due to space pressure on the tablespace they reside in, thus achieving storage tiering. The ILM infrastructure must manage storage tiering for table segments.

It is also often necessary to move a segment to a new compression level or to move a group of rows from one compression level to another without moving them to a different segment. For example, rows that are no longer likely to be modified can be compressed to row store advanced compression level within the same segment.

## What Is Heat Map and Automatic Data Optimization (ADO)?

Oracle Database 12c includes activity tracking with Heat Map providing the ability to track and mark data as it goes through life cycle changes:

- Data accesses at segment-level
- Data modifications at block and segment levels

Block-level and segment-level statistics collected in memory are stored in tables in the `SYSAUX` tablespace.

ADO allows you to create policies that use Heat Map statistics to compress and move data only when necessary. ADO automatically evaluates and executes policies that perform compression and storage tiering actions.

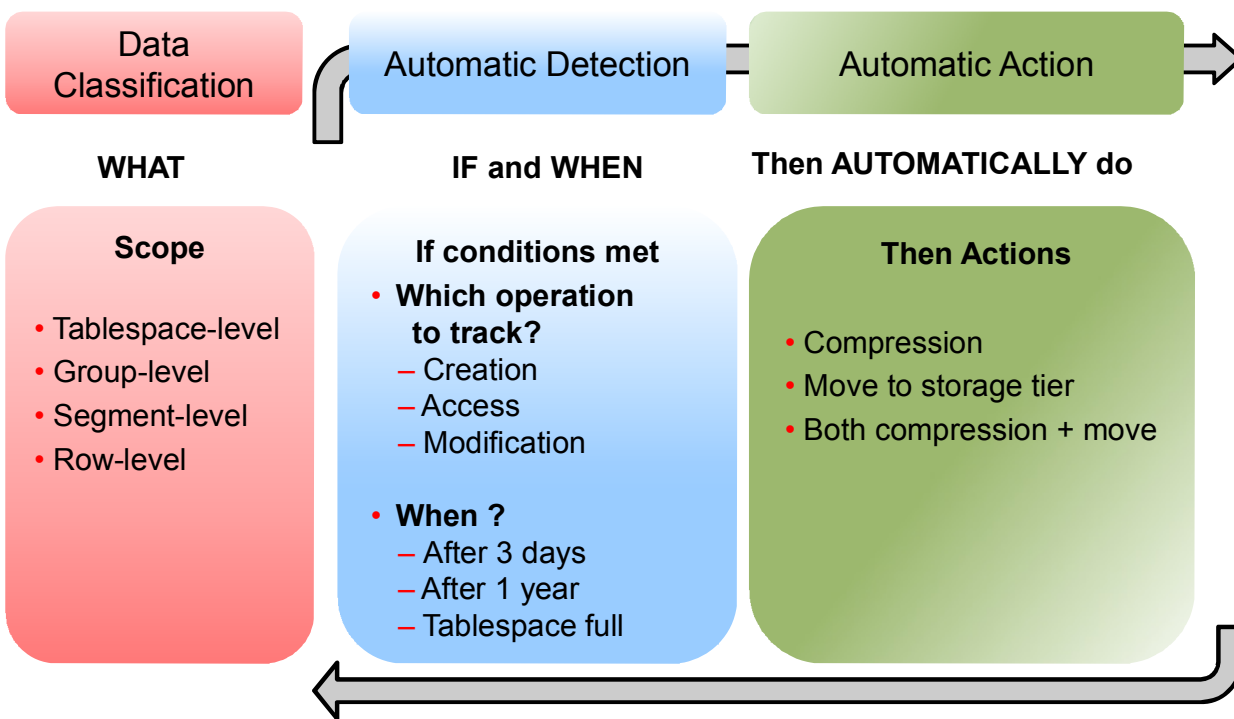
## How Is In-Database Archiving Enhanced in Oracle Database 12c?

- In-Database Archiving solves problems regarding archived and active data. It gives users the ability to keep both active operational data and dormant archived data in the same database tables. While achieving operational efficiency of archiving, applications can easily access only data that is in an operationally active state although archived data is kept in the same table.
- Another new capability, Temporal Validity, creates a valid time dimension for each row of a table, and is another way of indicating operational relevance. Each valid time dimension consists of two date-time columns specified in the table definition. If the data is no longer valid, or not yet valid, this data can easily be hidden from regular queries.



## Components

### Heat Map and Automatic Data Optimization



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Consider the same flow as defined previously: data is first bulk-loaded into a table. At this stage, it might be required to have no compression to meet storage and performance service level agreements (SLAs). Then the data is heavily updated and modified for some time, so it is best to leave the data uncompressed. The question is how long this period will last until the number of access and modifications would decrease and you should think about compression and even data movement to another storage tier. How long will the next period last until there will be no more data access and you should think about higher compression and even data movement to an archive storage tier?

- ADO provides the ability to declare policies on different scope levels in the database: tablespace, object, and row-levels, automatically enforced by the ADO infrastructure. An ADO policy is a procedure that executes the registered operations for the requested object to get them to the required state transparently without any user intervention.
- Policies automate data movement between different tiers of storage within the database. They also compress at different levels for each tier, and mainly control when the data movement takes place.

For example, when partition rows were not updated for three days, automatic compression based on policy is performed. This capability obviates the need for the application to make explicit schema changes.

ADO policies allow you to specify the conditions under which row, partition, table access or modification need to be satisfied before a compression or a move action should be performed.

When ADO policies are set at tablespace level as a default behavior, the table, partition, and subpartitions, including new interval partitions stored on that tablespace, are automatically compressed and moved to a lower-cost tablespace based on the policies.

ADO allows the user to set policies that enforce data flows automatically, with minimal intervention.

## What Is Automatic Data Optimization?

- ADO policies automatically compress data when it qualifies.
- ADO policies automatically move segments when necessary.
- ADO is dependent on Heat Map, and will not work unless Heat Map is enabled.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered on a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

ADO policies automatically execute actions under predefined conditions:

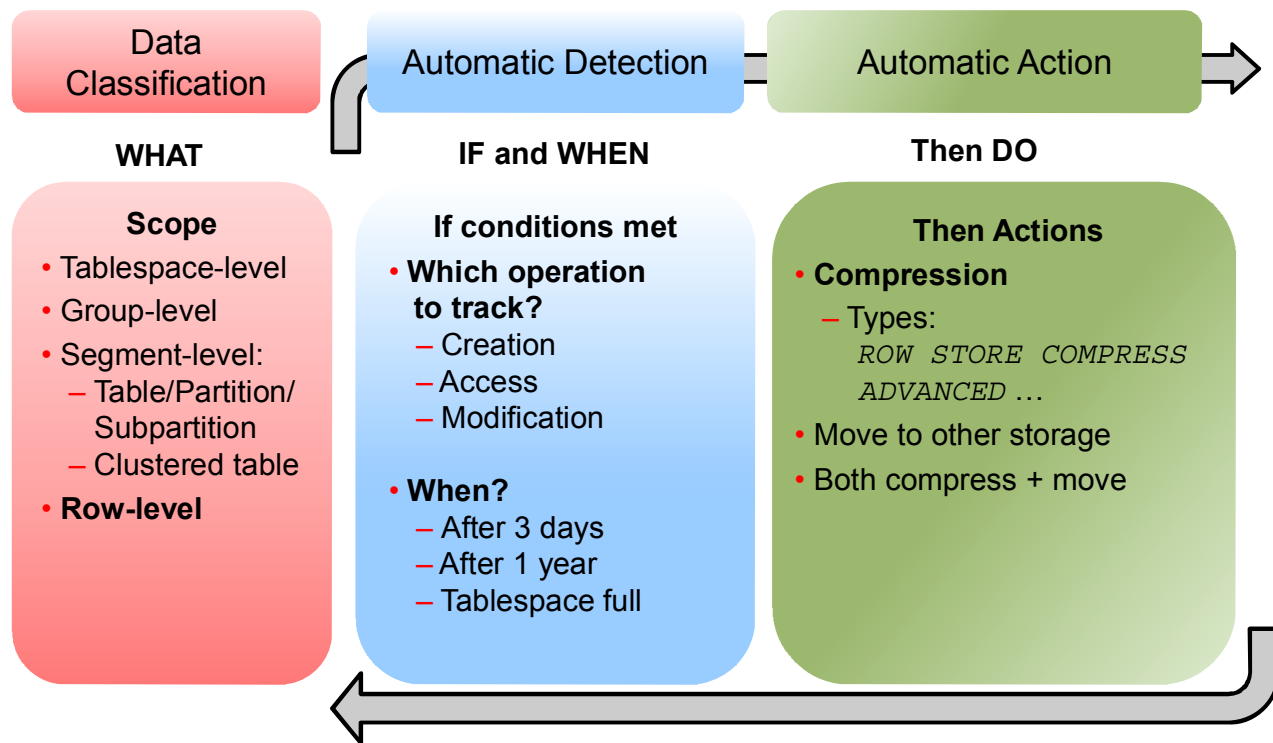
- Compress data only when it qualifies.
- Move segments to other storage tiers under space pressure.

ADO can execute compression and data movement only if Heat Map is enabled. After being enabled, Heat Map automatically collects statistics to execute ADO actions after the statistics evaluation is completed.

Automatic Data Optimization requires the Advanced Compression option.

# Data Classification Levels

## Activity Tracking and Automatic Data Optimization



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, the lowest level to determine data classes is the segment, table, or partition.

In Oracle Database 12c, you can classify each row of a table based on its operational relevance by using the new row-level data classification.

- Modifications tracked at row-level

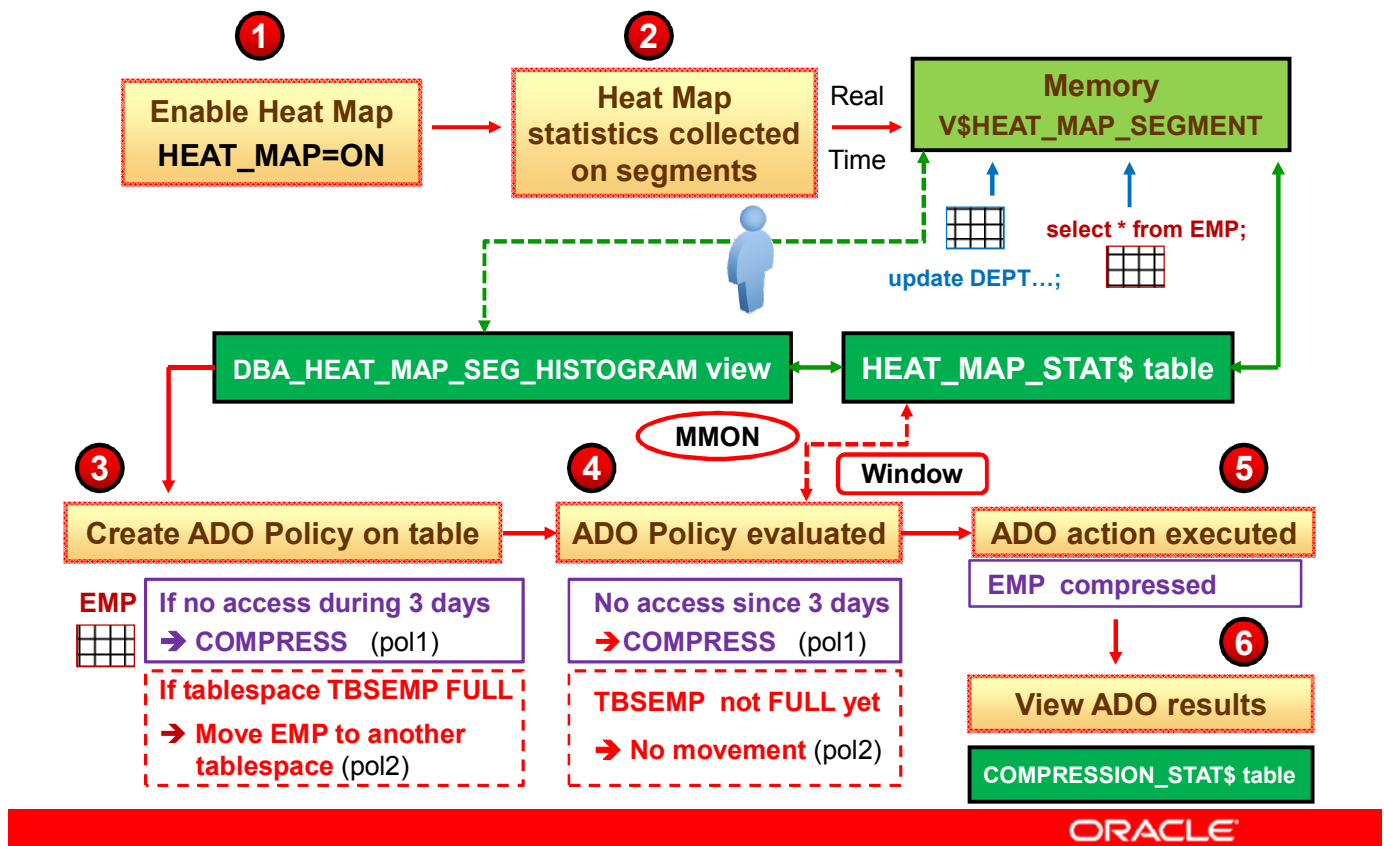
Row-level statistics are managed in-line as part of the work done when a process modifies data.

At segment-level, access and modifications are tracked:

- Last access time:
  - Fetch-by-rowid tracking: Collect ROWIDs on access path of index range scan/fetch by ROWID.
  - Full-table-scan tracking
- Last modification time

Segment-level activity tracking information is automatically flushed for persistence by scheduled `DBMS_SCHEDULER` jobs hourly. This means that segment-level statistics are guaranteed to checkpoint to on-disk statistics tables.

# Heat Map and ADO



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The slide shows how to set up the different steps between Heat Map and ADO to automate the movement of a segment to another tablespace and/or the compression of blocks or a segment depending on certain conditions defined in ADO policies.

1. The first operation for the DBA is to enable Heat Map, tracking the activity on blocks and segments. Heat Map activates system-generated statistics collection, such as segment access and row and segment modification.
2. Real-time statistics are collected in memory (`V$HEAT_MAP_SEGMENT` view) and regularly flushed by scheduled `DBMS_SCHEDULER` jobs to the persistent table `HEAT_MAP_STAT$`. The persistent data is visible by using the `DBA_HEAT_MAP_SEG_HISTOGRAM` view.
3. The next operation for the DBA is to create ADO policies on segments or groups of segments or as default ADO behavior on tablespaces.
4. The next step for the DBA is to schedule when ADO policy evaluation must happen if the default scheduling does not match the business requirements. ADO policy evaluation relies on Heat Map statistics. MMON evaluates row-level policies periodically and start jobs to compress whichever blocks qualify. Segment-level policies are evaluated and executed only during the maintenance window.
5. The DBA can then view ADO execution results by using the `DBA_ILMEVALUATIONDETAILS` and `DBA_ILMRESULTS` views.
6. Finally, the DBA can verify if the segment moved and is therefore stored on the tablespace defined in the ADO policy and or if blocks or the segment got compressed viewing the `COMPRESSION_STAT$` table.

## Enabling Heat Map Segment-Level Statistics

### 1. Enable activity tracking or heat map:

```
SQL> ALTER SYSTEM SET heat_map = ON;
```

- View **segment-level** statistics by using views:

- DBA\_HEAT\_MAP\_SEG\_HISTOGRAM
- DBA\_HEAT\_MAP\_SEGMENT
- V\$HEAT\_MAP\_SEGMENT

```
SQL> SELECT object_name, subobject_name, track_time,
           segment_write WRI, full_scan FTS, lookup_scan LKP
FROM DBA_HEAT_MAP_SEG_HISTOGRAM;
```

| OBJECT_NAME    | SUBOBJECT_NAME | TRACK_TIME | WRI | FTS | LKP |
|----------------|----------------|------------|-----|-----|-----|
| INTERVAL_SALES | P1             | 02-JAN-12  | YES | YES | NO  |
| INTERVAL_SALES | P2             | 28-MAR-12  | NO  | YES | NO  |
| INTERVAL_SALES | P3             | 28-MAR-12  | NO  | NO  | YES |
| I_EMPNO        |                | 07-dec-12  | YES | NO  | YES |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The first step is to enable activity tracking by using a new initialization parameter `HEAT_MAP`. The setting collects system-generated statistics at the instance level. Turning the `HEAT_MAP` parameter `ON` causes DML and access of all segments, other than the ones in the `SYSTEM` and `SYSAUX` tablespaces, to be tracked in memory and then flushed to on-disk statistics tables in the `SYSAUX` tablespace.

After you enabled Heat Map, statistics are collected for different activities, data read, and data write. ADO uses these statistics to trigger actions after evaluation.

To access these statistics, use the `DBA_HEAT_MAP_SEG_HISTOGRAM` view that contains daily segment heat map activity information about tables, table partitions, indexes and LOB segments. After rows are inserted into a table, updated, and selected, statistics are visible in the `DBA_HEAT_MAP_SEG_HISTOGRAM` and `DBA_HEAT_MAP_SEGMENT` views.

The views rely on the persistent `SYS.HEAT_MAP_STAT$` table and the `V$HEAT_MAP_SEGMENT` view that shows real time segment activity information from memory.

The columns of `DBA_HEAT_MAP_SEG_HISTOGRAM` view are the following:

- `TRACK_TIME` column: Displays the system time when the segment access was tracked
- `SEGMENT_WRITE` column: Indicates whether the segment has write access
- `FULL_SCAN`: Indicates whether segment has full scan
- `LOOKUP_SCAN`: Indicates whether the segment has lookup scan

## DBA\_HEAT\_MAP\_SEGMENT View

View the latest segment activity tracking time:

```
SQL> SELECT object_name,
           segment_write_time WRITE_T, segment_read_time READ_T,
           full_scan FTS_T, lookup_scan LKP_T
FROM DBA_HEAT_MAP_SEGMENT;
```

| OBJECT_NAME | WRITE_T   | READ_T | FTS_T     | LKP_T     |
|-------------|-----------|--------|-----------|-----------|
| EMP         |           |        | 07-dec-12 |           |
| T1          | 07-dec-12 |        | 08-dec-12 |           |
| EMPLOYEE    | 07-dec-12 |        | 08-dec-12 | 08-dec-12 |
| I_EMPNO     | 07-dec-12 |        |           | 08-dec-12 |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The DBA\_HEAT\_MAP\_SEGMENT view shows the latest segment activity tracking time. It shows the last activity time of each segment.

The columns of DBA\_HEAT\_MAP\_SEGMENT view are the following:

- SEGMENT\_WRITE\_TIME column: Indicates the latest time stamp on which the segment has write access
- SEGMENT\_READ\_TIME column: Indicates the latest time stamp on which the segment has read access
- FULL\_SCAN column: Indicates the latest time stamp on which the segment has full scan
- LOOKUP\_SCAN column: Indicates the latest time stamp on which the segment has index scan

## Block-Level Statistics

View [block-level](#) statistics using two new PL/SQL table functions after rows are modified:

- `DBMS_HEAT_MAP.BLOCK_HEAT_MAP`
- `DBMS_HEAT_MAP.EXTENT_HEAT_MAP`

```
SQL> SELECT segment_name, tablespace_name, block_id,
        writetime
        FROM table(dbms_heat_map.block_heat_map
        ('SCOTT', 'EMPLOYEE', NULL, 8, 'ASC'));
```

| SEGMENT_ | TABLESPACE_NAME | BLOCK_ID | WRITETIME |
|----------|-----------------|----------|-----------|
| -----    | -----           | -----    | -----     |
| EMPLOYEE | LOW_COST_STORE  | 196      | 12-DEC-12 |
| EMPLOYEE | LOW_COST_STORE  | 197      | 12-DEC-12 |
| EMPLOYEE | LOW_COST_STORE  | 198      | 12-DEC-12 |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `DBMS_HEAT_MAP.BLOCK_HEAT_MAP` function returns the block-level heatmap for a table segment. The statistic shows the latest modification time of the block. The input parameters are the following:

- Owner: Owner of the segment
- Segment\_name: Table name of a non-partitioned table or partition of partitioned table.
- Partition\_name: Defaults to NULL. For a partitioned table, specify the partition or subpartition segment name.
- Sort\_columnid: ID of the column to sort the output on. Valid values 1..9.
- Sort\_order: Defaults to NULL. Possible values: `ASC`, `DESC`

The output parameters are the following:

- Owner: Owner of the segment
- Segment\_name: Segment name of the non-partitioned table
- Partition\_name: Partition or subpartition name
- Tablespace\_name: Tablespace containing the segment
- File\_id: Absolute file number of the block in the segment
- Relative\_fno: Relative file number of the block in the segment
- Block\_id: Block number of the block
- Writetime: Last modification time of the block



## Extent-Level Statistics

After rows are modified, view aggregates at **extent**:

- Minimum modification time
- Maximum modification time

```
SQL> SELECT segment_name, block_id, blocks,
           max_writetime
       FROM table(dbms_heat_map.extent_heat_map
                ('SCOTT', 'EMPLOYEE'));
```

| SEGMENT_ | BLOCK_ID | BLOCKS | MAX_WRITETIME      |
|----------|----------|--------|--------------------|
| -----    | -----    | -----  | -----              |
| EMPLOYEE | 136      | 8      | 12-DEC-12 12:58:46 |
| EMPLOYEE | 144      | 8      | 12-DEC-12 12:58:46 |
| EMPLOYEE | 256      | 128    | 12-DEC-12 12:58:47 |
| EMPLOYEE | 384      | 128    | 12-DEC-12 12:58:47 |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `DBMS_HEAT_MAP.EXTENT_HEAT_MAP` function returns the extent-level heatmap for a specific table segment. Aggregates at extent level including minimum modification time and maximum modification time are returned. The input parameters are the owner of the segment, the segment\_name and the partition\_name.

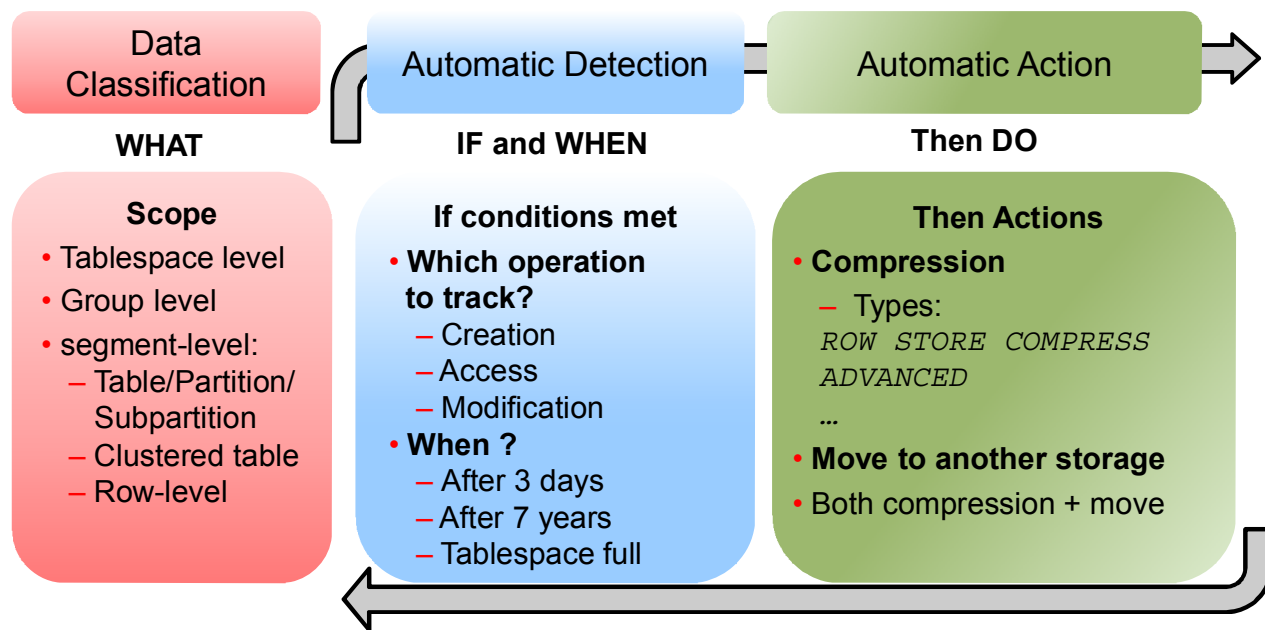
The output parameters are the following:

- Owner: Owner of the segment
- Segment\_name: Segment name of the non-partitioned table
- Partition\_name: Partition or subpartition name
- Tablespace\_name: Tablespace containing the segment
- File\_id: Absolute file number of the block in the segment
- Relative\_fno: Relative file number of the block in the segment
- Block\_id: Block number of the block
- Blocks: Number of blocks in the extent
- Bytes: Number of bytes in the extent
- Min\_writetime: Minimum of last modification time of the block
- Max\_writetime: Maximum of last modification time of the block
- Avg\_writetime: Average of last modification time of the block

## Defining Automatic Detection Conditions

### 2. Create ADO policy:

- Define **WHICH** condition fires action.
- Define **WHEN** the policy takes effect.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The second step is to create the ADO policy to define the automatic detection:

- Define WHICH condition would fire a compress or move action:
  - No access
  - No data modification (updates/deletes/inserts on row or DDL ALTER statement on a segment)
  - Object/row creation
  - Tablespace fullness
- Define WHEN the policy takes effect:
  - After *n* days or months or years of no data access
  - After no more modification
  - After object creation
  - When the tablespace containing the object meets the tablespace fullness threshold

## Defining Automatic Actions

### 2. Create ADO policy to define action and level of execution.

|                              | No modification | Creation | No Access | Tablespace fullness | Custom policy |
|------------------------------|-----------------|----------|-----------|---------------------|---------------|
| Compression (Row)            | ✓               |          |           |                     |               |
| Compression (Segment)        | ✓               | ✓        | ✓         |                     | ✓             |
| Compression (Group)          | ✓               | ✓        | ✓         |                     |               |
| Compression (Tablespace)     | ✓               | ✓        | ✓         |                     |               |
| Storage Tiering (Segment)    |                 |          |           | ✓                   | ✓             |
| Storage Tiering (Tablespace) |                 |          |           | ✓                   |               |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Still in the second step, define the action and the level of automatic execution:

- The action executed (see possible actions in the first column of the table in the slide):
  - Compression and which type of compression
  - Data movement to other storage tier
  - Both when defining two policies on the same segment
- The possible levels of execution:
  - ROW:** Row-level ADO policies can only be created based on modification time.
  - SEGMENT:** Segment-level ADO policies can apply to tables or partitions.
  - GROUP:** Group-level ADO policies indicate that the table's SecureFiles LOBs are compressed as well. Global indexes are maintained. An ADO policy can be specified for a table with the GROUP keyword that is part of the POLICY clause for compression. If the table becomes eligible for an ADO action at any time, the same ADO action would be performed on all the SecureFiles LOBs of the table as well. For example, if the ADO action is compression, the dependent objects like SecureFiles LOBs would be compressed at compression levels corresponding to a default mapping between heap segment compression levels and those of SecureFiles LOBs. Similar semantics hold for the GROUP keyword for ADO policies on table partitions.
  - TABLESPACE:** A DEFAULT ADO policy defined on a tablespace applies to all segments that will be created in the tablespace.

# Compression Scopes and Types

## Compression Scope

- Tablespace
- Group
- Segment
- Row

## Compression Type

- ROW STORE COMPRESS [BASIC]
- ROW STORE COMPRESS ADVANCED
- COLUMN STORE COMPRESS FOR QUERY LOW / HIGH
- COLUMN STORE COMPRESS FOR ARCHIVE LOW / HIGH

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The first possible action is compression. A few reminders about compression: Compression can occur while data is being inserted, updated, or bulk-loaded into a table.

- ROW STORE COMPRESS BASIC or ADVANCED is used for rows inserted without using direct-path insert and updated rows, using the Advanced Compression option (ACO). ROW STORE COMPRESS ADVANCED is the new syntax used with the “Advanced Row Compression” feature, new name for the old OLTP Table Compression feature part of ACO. ROW STORE COMPRESS ADVANCED on the heap table maps to LOW for SecureFiles LOB segments when the GROUP keyword is used.
- COLUMN STORE COMPRESS FOR QUERY LOW or HIGH provides a higher level of compression than ROW STORE compression. It works well when load performance is critical, frequent queries are run against the table, and no normal DML is expected. Column Store is a feature commonly referred to as Columnar Compression or just Columnar or HCC. COLUMN STORE COMPRESS FOR QUERY LOW/QUERY HIGH on a heap table maps to MEDIUM for SecureFiles LOB segments.
- COLUMN STORE COMPRESS FOR ARCHIVE LOW/ARCHIVE HIGH compression provides the highest level of compression and works well for infrequently accessed data, mostly for read-only data. It enables HCC. COLUMN STORE COMPRESS FOR ARCHIVE LOW/ARCHIVE HIGH on a heap table maps to MEDIUM for SecureFiles LOB segments.

## Creating Compression Policies Tablespace and Group

- **TABLESPACE**-level compression policy:

```
SQL> ALTER TABLESPACE tbs1 DEFAULT ILM ADD POLICY
      ROW STORE COMPRESS ADVANCED
      SEGMENT AFTER 30 DAYS OF NO ACCESS;
```

- **GROUP**-level compression policy:

```
SQL> ALTER TABLE tab1 ILM ADD POLICY
      ROW STORE COMPRESS ADVANCED
      GROUP AFTER 90 DAYS OF NO MODIFICATION;
```

```
SQL> ALTER TABLE tab2 MODIFY PARTITION p1 ILM ADD POLICY
      COLUMN STORE COMPRESS FOR ARCHIVE HIGH
      GROUP AFTER 6 MONTHS OF CREATION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

All the examples shown in the slide assume that Heat Map is enabled.

The first example sets a default tablespace ADO policy so that segments stored in the tablespace inherit the automatic compression with level ROW STORE COMPRESS ADVANCED after 30 days of no access.

In the second and third examples, because the GROUP keyword is specified, dependent SecureFiles LOBs are compressed as well. Global indexes are maintained.

The second example will automatically compress the table and SecureFiles LOBS after 90 days of no modification, with the ROW STORE COMPRESS ADVANCED compression level.

The third example will automatically compress the partition P1 and SecureFiles LOBS after six months of creation, with the COLUMN STORE COMPRESS FOR ARCHIVE HIGH compression level.

# Creating Compression Policies

## Segment and Row

- **SEGMENT-level** compression policy:

```
SQL> ALTER TABLE tab4 ILM ADD POLICY
      COLUMN STORE COMPRESS FOR QUERY HIGH
      SEGMENT AFTER 90 DAYS OF NO MODIFICATION;
```

```
SQL> ALTER TABLE tab5 ILM ADD POLICY
      COLUMN STORE COMPRESS FOR ARCHIVE HIGH
      SEGMENT AFTER 6 MONTHS OF NO ACCESS;
```

```
SQL> ALTER TABLE tab6 ILM ADD POLICY
      ROW STORE COMPRESS
      SEGMENT AFTER 2 YEARS OF CREATION;
```

- **ROW-level** compression policy:

```
SQL> ALTER TABLE tab1 ILM ADD POLICY
      ROW STORE COMPRESS ADVANCED
      ROW AFTER 30 DAYS OF NO MODIFICATION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The first example will automatically compress the table as soon as there is no modification on the segment within 90 days, with the COLUMN STORE COMPRESS FOR QUERY HIGH compression level.

The second example will automatically compress the table as soon as there is no access to the segment within six months, with COLUMN STORE COMPRESS FOR ARCHIVE HIGH compression level.

The third example will automatically compress the table two years after the table creation, with the ROW STORE COMPRESS BASIC compression level.

The fourth example will automatically compress any block in which all the rows have not been modified in the last 30 days. The only possible compression level at ROW-level is ROW STORE COMPRESS ADVANCED.

**Note:** There is no conflict if you have Advanced Row Compression set on a table, and also have ADO policies defined to compress the segment or blocks on policy conditions.

When rows are inserted into the table, they are compressed with Advanced Row Compression.

If your ADO policy says to compress for one of the HCC choices at the segment-level, that compression is executed as expected when the partition/table qualifies.

If your ADO policy says to perform a Row Compress at the row (block) level, that compression does not execute because the rows in each block are already compressed—unless you have done things to cause the rows to be uncompressed, such as an `ALTER TABLE ... MOVE PARTITION` to uncompress all the data in a partition.

# Creating Storage Tiering Policy

## Movement Level

– Segment-level

## Storage Tiering

– Tablespace

```
SQL> ALTER TABLE tab6 MODIFY PARTITION p1 ILM ADD POLICY
      TIER TO low_cost_storage [ON my_cond];
```

Source tablespace fullness thresholds:

```
SQL> SELECT * FROM dba_ilmparameters;
```

| NAME             | VALUE |
|------------------|-------|
| -----            | ----- |
| TBS PERCENT USED | 85    |
| TBS PERCENT FREE | 25    |

```
SQL> EXEC
      DBMS_ILM_ADMIN.CUSTOMIZE_ILM(DBMS_ILM_ADMIN.TBS_PERCENT_FREE,15)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The second possible action is data movement to another storage tier, whether it is a lower cost storage tier, or a higher-performance storage tier with other compression capabilities such as HCC. The only possible scope for data movement is SEGMENT.

The example shown in the slide automatically moves the partition P1 of the table TAB6 to the LOW\_COST\_STORAGE tablespace when the source tablespace that contains the partition has reached the fullness threshold.

The tiering fullness threshold of the source tablespace depends on two parameters:

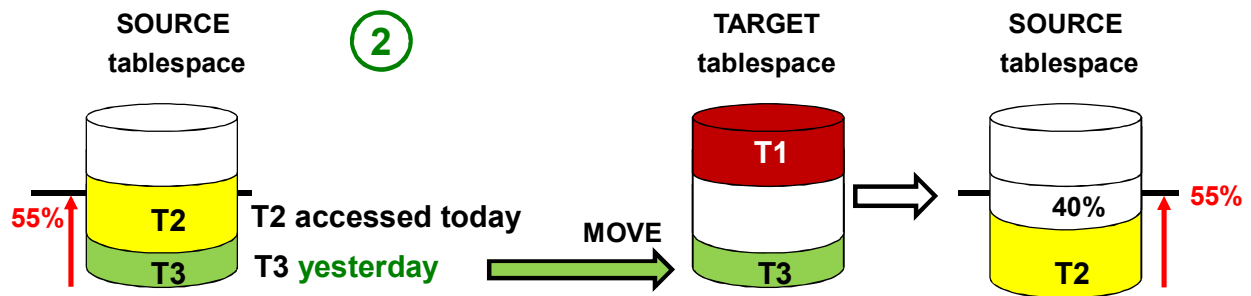
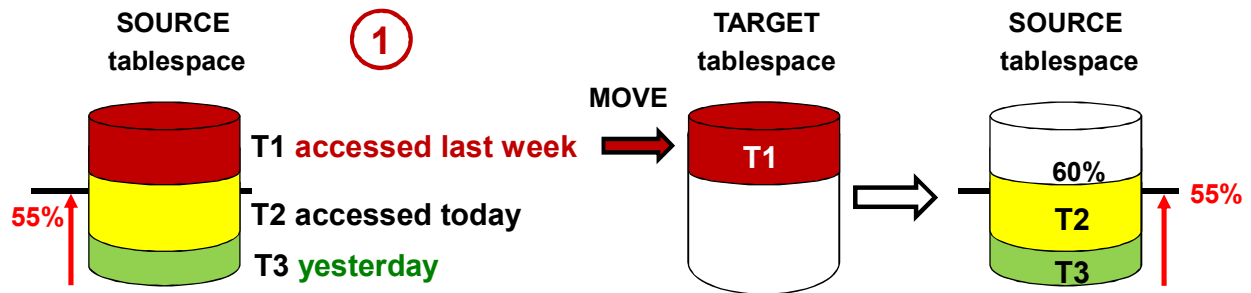
- Objects with tiering policy will be moved if the tablespace they reside in becomes TBS PERCENT USED full (defaulted to 85).
- Objects will be moved to the target tablespace until the source tablespace becomes TBS PERCENT FREE free (defaulted to 25).
- Both values can be controlled via the DBMS\_ILM\_ADMIN.CUSTOMIZE\_ILM procedure and displayed in the DBA\_ILMPARAMETERS view.

A segment-level policy is executed only once in the lifetime of the policy. After the first execution, the policy is disabled. This applies only to tiering policies.

Note that it is possible to add a custom condition to tiering policies, allowing you to trigger movement of data based on conditions other than how full the tablespace is.



## Storage Tiering: **Priority**



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

TBS PERCENT USED and TBS PERCENT FREE defaulted to 85 and 25, respectively. This means that, whenever the source tablespace's usage percent goes beyond 85%, any tiering policy specified on its objects will be executed, and all objects will be moved to the target tablespace (because the source tablespace should be 25% free).

Now if the first parameter is set to 55%, such as in our example, which table would move first if several tables of the same tablespace are under the tiering policy? The least recently accessed table will move first.

Therefore, in the slide, the T1 table, accessed last week, moves the first to the target tablespace defined in the tiering policy. The space used in the source tablespace is still above the TBS PERCENT USED value of 55%, and therefore another table needs to move to the target tablespace. T3 accessed yesterday is elected to move to the target tablespace. Now that T1 and T3 freed enough space in the source tablespace to get a TBS PERCENT USED value below 55% and a TBS PERCENT FREE value above 45%, table T2 can remain in the source tablespace.

## Storage Tiering: **READ ONLY**

READ ONLY tier storage:

```
SQL> ALTER TABLE tab7 ILM ADD POLICY  
      TIER TO tablespace_tbs READ ONLY  
      SEGMENT ON CUSTOM_ILM_RULES;
```

```
SQL> ALTER TABLE sales MODIFY PARTITION HY_2010 ILM ADD POLICY  
      TIER TO tablespace_tbs READ ONLY  
      SEGMENT AFTER 90 DAYS OF CREATION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When a segment is moved to another tablespace, another option is to set the target tablespace to **READ ONLY** after the object is moved. This is very beneficial during backup, because Oracle Recovery Manager (RMAN) has the ability to skip such tablespaces. Using this clause implies that you either specify the **AFTER** clause or the **ON** clause.

## Policy Relying on Function

Customized action:

- Create a function returning TRUE/FALSE:

```
SQL> CREATE FUNCTION CUSTOM_ILM_RULES
      (objn IN NUMBER)
      RETURN BOOLEAN ...
      /
```

- Use the function in the policy declaration:

```
SQL> ALTER TABLE EMP      ILM ADD POLICY
                               ROW STORE COMPRESS ADVANCED
                               SEGMENT ON CUSTOM_ILM_RULES;
```

```
SQL> ALTER TABLE sales MODIFY PARTITION before_jan_2005
                               ILM ADD POLICY
                               TIER TO history ON CUSTOM_ILM_RULES;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If users need more flexibility to tailor an ADO policy, they can do so with a custom ADO policy. Custom ADO policies utilize a user-provided function to evaluate the policy.

The PL/SQL function encapsulates any business logic required to make a decision about ADO on an object, returning a BOOLEAN value or a number.

The ADO policy will be either enforced or ignored according to the BOOLEAN returned. If the value returned is TRUE, the policy will be executed. If the value returned is FALSE, no action will be taken until the next time the policy is evaluated for execution.

The syntax for custom and non-custom policies cannot be mixed.

Customized policies can be used only with segment-level policies.

Take the second example: When the BEFORE\_JAN\_2005 table partition contains fewer rows than a defined threshold (a certain elapsed time after its creation), you decide to move the segment to an archive storage tier. The rules are described in the function.

## Multiple SEGMENT Policies on a Segment

New data loaded: **cycle starts**  
 After 1 day: **Advanced Row compression**  
 After 1 month: **Query High Compression**  
 After 1 year: **Move to low cost tablespace**  
 After 2 years: **cycle ends: purge**

```
SQL> ALTER TABLE t1 ILM ADD POLICY ROW STORE COMPRESS ADVANCED
      SEGMENT AFTER 1 DAY OF NO MODIFICATION;
```

```
SQL> ALTER TABLE t1 ILM ADD POLICY
      COLUMN STORE COMPRESS FOR QUERY HIGH
      SEGMENT AFTER 1 MONTH OF NO MODIFICATION;
```

```
SQL> ALTER TABLE t1 ILM ADD POLICY
      TIER TO history ON t1_check_after_one_year;
```

```
SQL> ALTER TABLE t1 ILM ADD POLICY COLUMN STORE COMPRESS FOR
      ARCHIVE HIGH SEGMENT AFTER 1 YEAR OF NO ACCESS;
```

```
SQL> DROP TABLE t1 PURGE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Several policies can be applied on the same segment as long as they do not have conflicting conditions. Otherwise, the presence of another policy on the same object can disallow certain policies from being applied on the same object.

Conflicts appear if all policies on the same (object, action) are not on the same statistic. For example, all compression policies have to be applied on the same statistic, either access time, modification time, or creation time.

It would not make sense to have an Advanced Row compression policy to compress after 3 days of no modification, and also have a QUERY HIGH compression policy to compress after 3 days of no modification. What would make sense is to have an Advanced Row compression policy to compress after 1 day of creation, then have a QUERY HIGH compression policy to compress after 30 days of no modification, then have a ARCHIVE HIGH compression policy to compress after 180 days of no access.

Greater compression levels have to be specified at greater elapsed times.

In this example, the life cycle starts with data load and ends with the table purge operation.

- After one day, ADO evaluates if there were any modification at the table level. If there were none, it proceeds with Advanced Row compression, the lowest level of compression.

- After one month, ADO evaluates if there were any other modifications still at the table level. If there were none, it proceeds with the `QUERY HIGH` compression, a higher level of compression than the Advanced Row compression and with a greater elapsed time.
- After one year, ADO evaluates if there were any modifications. Setting the tablespace fullness threshold to 0, it necessarily moves the segment to the `HISTORY` tablespace.

After a segment has been moved to a particular compression level, all segment-level policies at a lower compression level are disabled. That is why a segment can move to greater compression levels only via the ILM framework. Row-level policies are not affected by this rule.

Also, a compression policy would be executed only if it moves the object to a higher compression level than the dictionary compression level of the object at the time of policy evaluation.

## Only One Single ROW Policy on a Segment

- **Compression type (Row Store Compress Advanced ) and scope (row):**  
only 1 per object

```
SQL> ALTER TABLE t2 ILM ADD POLICY ROW STORE COMPRESS ADVANCED
      ROW AFTER 2 DAYS OF NO MODIFICATION;
```

```
SQL> ALTER TABLE t2 ILM ADD POLICY
      ROW STORE COMPRESS ADVANCED
      ROW AFTER 6 MONTHS OF NO MODIFICATION;
```

- **Segment-level policies do not override row-level policies**

```
SQL> ALTER TABLE t2 ILM ADD POLICY COLUMN STORE COMPRESS FOR
      QUERY HIGH SEGMENT AFTER 1 YEAR OF NO MODIFICATION;
```

```
SQL> ALTER TABLE t2 ILM ADD POLICY
      COLUMN STORE COMPRESS FOR ARCHIVE HIGH
      SEGMENT AFTER 6 YEARS OF NO ACCESS;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- Only one (action, compression type, scope) is allowed on an object. This rule makes sense only for compression policies, where the action is `COMPRESSION`, the type is the compression type (`ROW STORE COMPRESS`), and the scope is either `ROW` or `SEGMENT`. The second statement fails at policy creation because a `COMPRESSION` policy is already configured on the same scope (`ROW`). The same error is issued if the scope is `SEGMENT` with the same compression type.
- `SEGMENT`-level policies do not necessarily override the row-level policies. The third statement succeeds because it configures a `COMPRESSION` policy with a different compression type on the same statistic (`NO MODIFICATION`). The fourth example fails because it configures a `COMPRESSION` policy with a different compression type but on a different statistic (`NO ACCESS`). A segment tiering policy could be set.

# Policy Inheritance

The child-level policy overrides the parent-level policy with the same action.

The table-level policy:

- Overrides the tablespace-level policy
- Is inherited at the partition-level

Inheritance is additive for different actions in policies.

```
SQL> select POLICY_NAME "POLICY", OBJECT_NAME,
           SUBOBJECT_NAME "SUBOBJECT", OBJECT_TYPE, INHERITED_FROM
FROM DBA_ILMOBJECTS;
```

| POLICY | OBJECT_NAME | SUBOBJECT  | OBJECT_TYPE     | INHERITED_FROM       |
|--------|-------------|------------|-----------------|----------------------|
| P281   | T1          |            | TABLE           | TABLESPACE           |
| P341   | SALES       |            | TABLE           | POLICY NOT INHERITED |
| P341   | SALES       | SALES_1994 | TABLE PARTITION | TABLE                |
| P341   | SALES       | SALES_1996 | TABLE PARTITION | TABLE                |
| P350   | T3          |            | TABLE           | POLICY NOT INHERITED |
| P360   | SALES       | SALES_1995 | TABLE PARTITION | POLICY NOT INHERITED |
| P610   | T4          |            | TABLE           | POLICY NOT INHERITED |
| P281   | T5          |            | TABLE           | TABLESPACE           |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you implement ADO policies on a segment, the following inheritance rules apply:

- For example, if there is a compression policy at the tablespace-level and a compression policy is directly specified on an object, the tablespace-level policy would be overridden.
- The nearest ancestor overrides the others if the action is the same. For example, if POLICY1 is specified at the tablespace level, and POLICY2 at the table level, POLICY2 (which is the nearest ancestor) will be inherited at the partition level, if the action specified by both policies is the same.
- Inheritance is additive for different actions in policies.

The DBA\_ILMOBJECTS view shows all policies inherited by an object. They also have a field to indicate the level from which the policies were inherited.

In the example, T1 and T5 tables inherited the P281 policy from the tablespace they reside in, the SALES\_1995 table partition did not inherit the P360 policy from the SALES table. The policy was directly applied to the table partition, whereas the SALES\_1994 and SALES\_1996 table partitions inherited the P341 policy from the SALES table.

## Displaying Policies

### DBA\_ILMPOLICIES/DBA\_ILMDATAMOVEMENTPOLICIES

```
SQL> SELECT * FROM DBA_ILMPOLICIES;
```

| POLICY_NAME | POLICY_TYPE   | TABLESPACE | ENABLED |
|-------------|---------------|------------|---------|
| P281        | DATA MOVEMENT |            | YES     |
| P381        | DATA MOVEMENT |            | YES     |
| P400        | DATA MOVEMENT | ITB        | YES     |

```
SQL> SELECT policy_name, action_type,
           compression_level, tier_tablespace TBS
FROM DBA_ILMDATAMOVEMENTPOLICIES;
```

| POLICY_NAME | ACTION_TYPE | COMPRESSION_LEVEL | TBS |
|-------------|-------------|-------------------|-----|
| P281        | COMPRESSION | QUERY HIGH        |     |
| P381        | COMPRESSION | ADVANCED          |     |
| P400        | STORAGE     |                   | ITB |

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

After ADO policy implementation, use views to:

- Verify the configured policies
- Monitor ADO to verify that data movement and compression took place automatically when desired

DBA\_ILMPOLICIES provides the list of all policies and their respective ENABLED status.

An interesting view is DBA\_ILMDATAMOVEMENTPOLICIES.

In the example, two compression policies are created with different compression levels, and a third TIER TO policy to move a segment to the ITB tablespace under certain conditions. The conditions are stored in the CONDITION\_TYPE and CONDITION\_DAYS columns.



## Displaying Policies

### DBA\_ILMDATAMOVEMENTPOLICIES

```
SQL> SELECT policy_name POL, action_type, scope,
           condition_type, condition_days
FROM DBA_ILMDATAMOVEMENTPOLICIES;
```

| POL  | ACTION_TYPE | SCOPE   | CONDITION_TYPE         | CONDITION_DAYS |
|------|-------------|---------|------------------------|----------------|
| P341 | COMPRESSION | GROUP   | LAST MODIFICATION TIME | 90             |
| P342 | COMPRESSION | ROW     | LAST MODIFICATION TIME | 30             |
| P361 | COMPRESSION | GROUP   | LAST ACCESS TIME       | 180            |
| P381 | COMPRESSION | SEGMENT | LAST MODIFICATION TIME | 90             |
| P401 | COMPRESSION | SEGMENT | LAST ACCESS TIME       | 180            |
| P402 | STORAGE     | SEGMENT |                        | 0              |
| P461 | COMPRESSION | SEGMENT | USER DEFINED           | 0              |
| P462 | STORAGE     | SEGMENT | USER DEFINED           | 0              |
| P482 | COMPRESSION | SEGMENT | LAST MODIFICATION TIME | 30             |
| P681 | COMPRESSION | GROUP   | CREATION TIME          | 180            |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the example, a row-level compression policy P342 is created to automatically compress the blocks of a table after 30 days of no modification:

```
SQL> alter table employee ILM ADD POLICY ROW STORE COMPRESS ADVANCED
           ROW after 30 days of no modification;
```

CONDITION\_TYPE represents the statistic type.

## Preparing Evaluation and Execution

| Segment-level policy  | Row-level policy                   |
|-----------------------|------------------------------------|
| In maintenance window | Regularly by MMON every 15 minutes |

- Change the interval for ADO policies evaluation.

```
SQL> EXEC DBMS_ILM_ADMIN.CUSTOMIZE_ILM(-
                                     DBMS_ILM_ADMIN.EXECUTION_INTERVAL,1)
```

- Two ways for ADO execution with **DBMS\_ILM** package:
  - Immediate ADO task execution on a set of objects.

```
SQL> VAR v_taskid NUMBER
SQL> EXEC DBMS_ILM.EXECUTE_ILM (owner =>'SCOTT', object_name
=>'EMPLOYEE', task_id =>:v_taskid)
```

- Customize the set of objects before ADO task execution.

- **Stop** execution.

```
SQL> EXEC DBMS_ILM.STOP_ILM(245321)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

It is sometimes necessary to move data as quickly as possible from one tier to another or compress data as soon as possible, and it is not possible to wait until the next maintenance window.

The **DBMS\_ILM** package supports immediate evaluation and immediate/background scheduling of execution of ADO-related tasks. The **DBMS\_ILM** package supports the following two ways for scheduling ADO-related tasks execution.

- A database user schedules immediate ADO execution on a set of objects. Although you can create a schedule to trigger the ADO actions evaluation more often than it does with the maintenance window, you can start the evaluation now. The **EXECUTE\_ILM** procedure provides that ability, regardless of any previously scheduled ADO execution.
- A database user can first view the results of evaluation of ILM policies on a set of objects before ADO-related tasks execution and customize the list of objects candidate for ADO execution.

It is also possible to stop a particular execution.

## Customizing Evaluation and Execution

- Use the DBMS\_ILM package and procedures to:
  - Customize the set of objects to evaluate
  - Schedule execution of ADO tasks



```
SQL> EXEC DBMS_ILM.PREVIEW_ILM (TASK_ID => :v_taskid, -
                                ILM_SCOPE => DBMS_ILM.SCOPE_SCHEMA)
```

```
SQL> EXEC DBMS_ILM.EXECUTE_ILM_TASK (TASK_ID => 26382, -
    EXECUTION_MODE      => DBMS_ILM.SCHEDULE_IMMEDIATE, -
    EXECUTION_SCHEDULE => DBMS_ILM.ILM_EXECUTION_OFFLINE)
```

```
SQL> SELECT task_id, state FROM dba_ilmtasks;

TASK_ID STATE
-----
26482  ACTIVE
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the DBMS\_ILM.PREVIEW\_ILM procedure to view the results of evaluation of ADO policies on a set of objects. The scope of evaluation can be one of the two:

- A whole schema: Attribute ILM\_SCOPE => DBMS\_ILM.SCOPE\_SCHEMA. The evaluation selects all ADO policies in the current schema. (This is the default.)
- The whole database: Attribute ILM\_SCOPE => DBMS\_ILM.SCOPE\_DATABASE. The evaluation selects all ADO policies in the database.

The task created is in INACTIVE state. The user can still add or delete objects to this set and review the results of ADO policy evaluation again. The user repeats this step until he/she finalizes the set of objects for ADO execution.

Then use the EXECUTE\_ILM\_TASK procedure to perform immediate scheduling of ADO execution on this set of objects. The state of the task moves to ACTIVE state when it starts executing. Once the execution of the task is finished, the state becomes COMPLETE.

The execution of the task with the EXECUTE\_ILM\_TASK procedure launches a job executed according to the attributes set in the procedure:

- EXECUTION\_MODE: ILM\_EXECUTION\_ONLINE value executes the task online and ILM\_EXECUTION\_OFFLINE value executes the task offline.
- EXECUTION\_SCHEDULE: Identifies when the ILM task should be executed. The only possible value in Oracle Database 12.1 is DBMS\_ILM.SCHEDULE\_IMMEDIATE.

# Monitoring Evaluation and Execution

- DBA\_ILMTASKS
- DBA\_ILMEVALUATIONDETAILS
- DBA\_ILMRESULTS

| SQL> SELECT * FROM dba_ilmtasks; |            |          |               |            |                 |
|----------------------------------|------------|----------|---------------|------------|-----------------|
| TASK_ID                          | TASK_OWNER | STATE    | CREATION_TIME | START_TIME | COMPLETION_TIME |
| 18566                            | SCOTT      | INACTIVE | 12-DEC-12     |            |                 |
| 18542                            | SCOTT      | ACTIVE   | 11-DEC-12     | 12-DEC-12  |                 |

| SQL> SELECT task_id, policy_name, object_name,<br>selected_for_execution, job_name<br>FROM dba_ilmevaluationdetails; |             |             |                            |            |
|----------------------------------------------------------------------------------------------------------------------|-------------|-------------|----------------------------|------------|
| TASK_ID                                                                                                              | POLICY_NAME | OBJECT_NAME | SELECTED_FOR_EXECUTION     | JOB_NAME   |
| 18762                                                                                                                | P281        | EMPLOYEE    | POLICY DISABLED            |            |
| 18542                                                                                                                | P281        | EMPLOYEE    | SELECTED FOR EXECUTION     | ILMJOB5002 |
| 18862                                                                                                                | P301        | EMPLOYEE    | PRECONDITION NOT SATISFIED |            |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

View the results of the ADO tasks and jobs executions in the following views:

- DBA\_ILMTASKS contain new information about ADO tasks every time a user invokes the EXECUTE\_ILM or EXECUTE\_ILM\_TASK procedures. A task ID is returned to track this particular invocation. A task ID is also generated to track periodic internal ADO executions by the database. This view contains information on all ADO executions. You can see that all SYS evaluation tasks are regularly executed every 15 minutes.
- DBA\_ILMEVALUATIONDETAILS contain details on the tasks execution after the ADO evaluation. The SELECTED\_FOR\_EXECUTION column informs whether the policy has been selected for execution on the object on which the ADO policy exists. A "SELECTED FOR EXECUTION" value triggers an ADO job whereas a "PRECONDITION NOT SATISFIED" value does not. This column can take one of the following values: POLICY DISABLED, POLICY OVERRULED, INHERITED POLICY OVERRULED, JOB ALREADY EXISTS, NO OPERATION SINCE LAST ILM ACTION, TARGET COMPRESSION NOT HIGHER THAN CURRENT, STATISTICS NOT AVAILABLE

The first row of the first example shows that an ADO task was in `INACTIVE` state, probably due to an ADO preview not yet finalized by the user. The second row shows a task being in `ACTIVE` state with a creation date and a starting date, but not yet completed with a completion date.

The first row of the second example shows that an ADO policy exists on the `EMPLOYEE` table but is disabled. The second row shows that the same ADO policy had been evaluated positively and generated a job to execute ADO actions. The third row shows that another ADO policy on the same table has not positively passed the evaluation, the conditions under which ADO actions should be triggered are not satisfied.

- `DBA_ILMRESULTS` provides information about ADO jobs manual or automatic executions.

```
SQL> SELECT task_id, cast(job_name as varchar2(20))
        job_name, job_state, completion_time
        FROM DBA_ILMRESULTS;
```

| TASK_ID | JOB_NAME | JOB_STATE              | COMPLETION_TIME |
|---------|----------|------------------------|-----------------|
| 666     | ILMJOB59 | COMPLETED SUCCESSFULLY | 28/09/2011      |

A segment level policy is designed for one-time action only. Once the policy executes successfully, it is disabled and is not evaluated again. The user can explicitly enable this policy again if reevaluation is desired.

A segment level policy is disabled if it is successfully executed. If a segment level policy is scheduled and does not complete successfully, it would be reevaluated later. However, if it is not completed successfully after being scheduled thrice, the policy would be disabled.

A row-level policy is never disabled. Every new job for executing a row-level policy looks for cold rows not compressed previously. However, after completion (successful or otherwise) it is not disabled and would always be re-evaluated later.

## ADO DDL

- Enable all policies on an object:

```
SQL> ALTER TABLE tab1 ILM ENABLE_ALL;
```

```
SQL> ALTER TABLE tab2 MODIFY PARTITION p1 ILM ENABLE_ALL;
```

- Disable all policies on an object:

```
SQL> ALTER TABLE tab1 ILM DISABLE_ALL;
```

- Delete all policies on an object:

```
SQL> ALTER TABLE tab1 ILM DELETE_ALL;
```

```
SQL> ALTER TABLE tab1 MODIFY PARTITION p1 ILM DELETE_ALL;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enabling all ADO policies on a table or partition is done in the first and second examples.

Disabling all ADO policies on a table is done in the third example. Disabling all ADO policies on a table partition would use the `MODIFY PARTITION` and `DISABLE_ALL` clauses.

Deleting all ADO policies on a table or partition is done in the fourth and fifth examples.

## Turning ADO Off and On

- Turn off ADO without disabling or deleting all policies:

```
SQL> exec DBMS_ILM_ADMIN.DISABLE_ILM
```

- Turn ADO back on for all policies:

```
SQL> exec DBMS_ILM_ADMIN.ENABLE_ILM
```

- View ADO parameters:

```
SQL> select * from DBA_ILMPARAMETERS where name='ENABLED';
```

| NAME    | VALUE |
|---------|-------|
| -----   | ----- |
| ENABLED | 1     |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can turn off background ADO by using the `DBMS_ILM_ADMIN.DISABLE_ILM` procedure, and you can turn it on by using the `DBMS_ILM_ADMIN.ENABLE_ILM` procedure. The `ENABLED` attribute is displayed in the `DBA_ILMPARAMETERS` view. The value set to 1 means that ADO is enabled, 2 means that ADO is disabled. Disabling ADO does NOT delete ADO policies or disable them.

You need the `SYSDBA` privilege to use this package.

## Stop Activity Tracking and Clean Up Heat Map Statistics

- Stop activity tracking in session:

```
SQL> ALTER SESSION SET HEAT_MAP = OFF;
```

- Stop activity tracking at instance level:

```
SQL> ALTER SYSTEM SET HEAT_MAP = OFF;
```

- Clean up heat map statistics:

```
SQL> exec DBMS_ILM_ADMIN.CLEAR_HEAT_MAP_ALL;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can stop the activity tracking by setting the `HEAT_MAP` parameter to `OFF` at the session or instance scope.

Disabling Heat Map for a specific session which has Heat Map enabled at instance level allows the subsequent SQL statements for that session not to be tracked by Heat Map.

To clear all heat map statistics in the `SYS.HEAT_MAP_STAT$` table, use the `DBMS_ILM_ADMIN.CLEAR_HEAT_MAP_ALL` procedure.



## Specific Situations of Activity Tracking

- Create the `NEW_EMP` table from a table with ADO policies:

```
SQL> CREATE TABLE new_emp AS SELECT * FROM emp;
```

- ➔ The `NEW_EMP` table does not inherit the ADO EMP policies.

- Row-level ADO policy job fails before end.
- Segment-level ADO job fails before end.



- COMPRESS attribute: Compatible with ADO compression policy ?



```
SQL> CREATE TABLE tabcomp (c number) ROW STORE COMPRESS;
SQL> ALTER TABLE tabcomp ILM ADD POLICY
      ROW STORE COMPRESS ADVANCED
      ROW AFTER 6 DAYS OF NO MODIFICATION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- You create an `EMP` table with ADO policies and then a `NEW_EMP` table selecting the structure and rows from the source `EMP` table. Note that the `NEW_EMP` table does not inherit the ADO policies of the source table.
- It is possible for ADO jobs to fail before completing their task.
  - In the case of row-level compression ADO policies, each block is compressed in a transaction. If the transaction is in progress when the job is killed, it will be rolled back. If some blocks were already compressed, those changes will not be affected.
  - The case of segment-level ADO jobs is more complex. For example, a job involving `ALTER TABLE ... MOVE` can fail after moving the segment, but before rebuilding the indexes on the segment. In such cases, jobs will be spawned to rebuild all indexes that were usable before the `ALTER TABLE ... MOVE` operation started.

- There is no conflict if you have both Advanced Row compression set on a table, and ADO policies causing data to be compressed based on conditions. When rows are inserted into the table, they are compressed with Advanced Row compression. If the ADO policy says to compress for one of the HCC choices at the segment-level, the compression executes as expected when the partition/table qualifies. If the ADO policy says to compress at the row (block) level, the compression does not execute because the rows in each block are already compressed due to the `ROW STORE COMPRESS` attribute set on the table—unless you have done operations to cause the rows to be uncompressed, such as `ALTER TABLE ... MOVE PARTITION` to uncompress all the data in a partition.

## Quiz

Which of the following is a limitation in ADO compression?

- a. You cannot compress at block level.
- b. You cannot compress at segment level.
- c. You cannot compress upon your own conditions.
- d. None of them

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: d**

## Quiz

You can stop activity tracking at the segment level in your session.

- a. True
- b. False

ORACLE

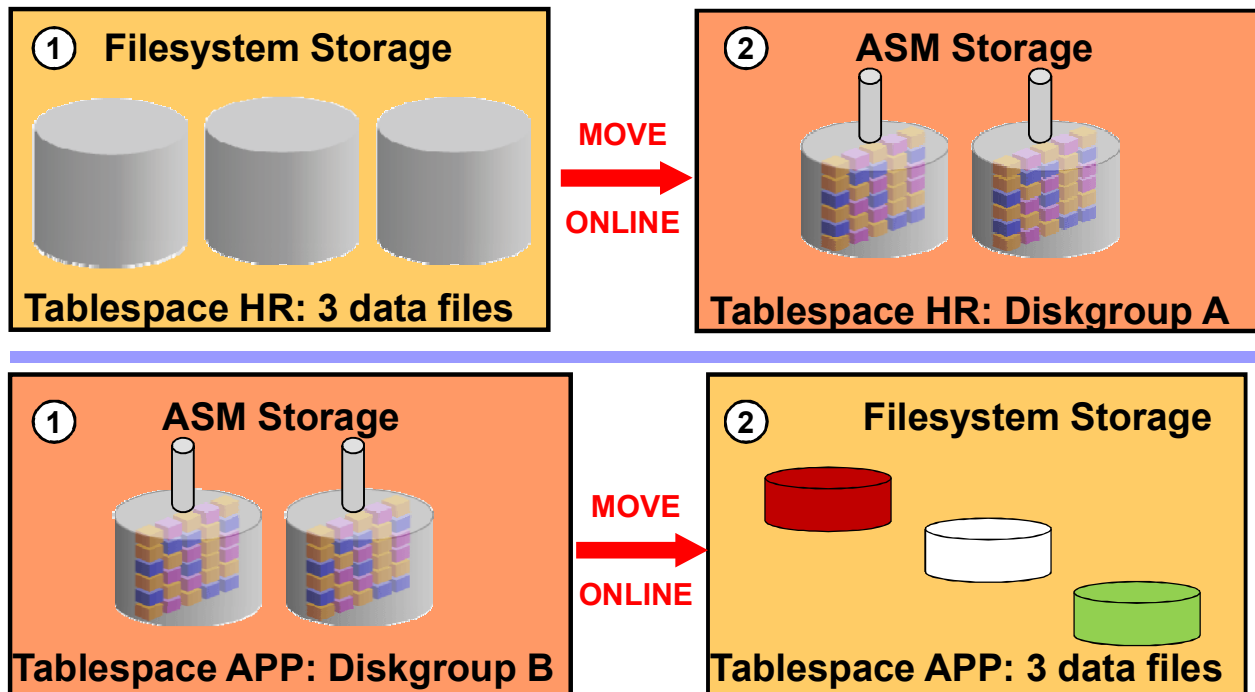
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

The heat map can be stopped at instance or session scope, not at a lower level than these.

## Online Move Data File

Move a **data file** to another kind of storage system **ONLINE**.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle Database 12c Online Move data file feature provides the capability to move an online data file from one kind of storage system to another while the database is open and accessing the file.

This implies that queries and DML and DDL operations can be performed while the data file is being moved:

- You can create tables and indexes.
- You can rebuild indexes online.
- You can select tables and partitions data.
- If objects are compressed while the data file is moved, the compression remains the same.

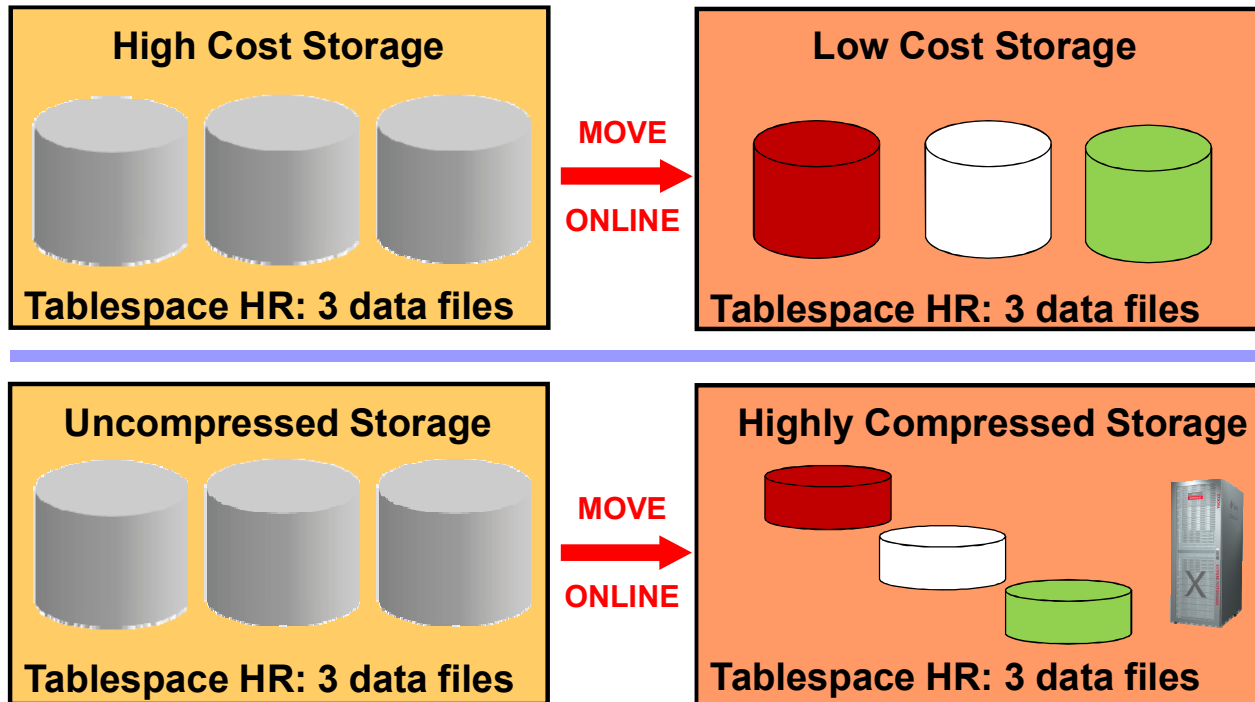
This means that many maintenance operations can be used to move:

- Data files from one kind of storage system to another
- A database into ASM without going down

ADO can benefit from this new enhancement. It allows easy, transparent ADO actions, such as data movement to other storage tier while tablespaces or data files move to another storage tier. Associated segments consequently can move to another storage tier.

# Compression

Move **data file** online to lower-cost storage:



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This means that many maintenance operations can be used to:

- Move tablespaces or data files from a high-cost storage to a lower-cost storage and meanwhile let ADO data movement actions execute.
- Move uncompressed or BASIC/OLTP compressed data to another storage system with a higher compression level, such as HCC. It can be `COMPRESSION FOR QUERY` for data warehouse performance queries or `FOR ARCHIVE` for archiving dormant data. The ADO compression action can benefit from this new enhancement.

## REUSE and KEEP

No need for database SHUTDOWN / data file OFFLINE

```
SQL> alter database rename file '/u01/t1.f' to '/u02/t1.f';
...
ORA-01511: error in renaming log/data files
ORA-01121: cannot rename database file 5 - file is in use or
recovery
```

11g

```
SQL> ALTER DATABASE MOVE datafile '/disk1/myexample01.dbf'
TO '/disk2/myexample01.dbf' REUSE;
```

12.1

```
SQL> ALTER DATABASE MOVE datafile '/disk1/myexample01.dbf'
TO '+DiskGroup2' KEEP;
```

```
SQL> ALTER DATABASE MOVE datafile 5 TO '+DiskGroup3';
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The advantage is that while you move a data file to another location, disk, or storage system, you do not have to shut down the database or move the data file offline.

While the different operations displayed in the slide are executing, ADO actions, such as compression of segments or data movement, can take place.

The Oracle Database 11g example shows that the file has to be moved offline first.

The Oracle Database 12c example shows how to move a data file from disk1 to disk2.

The **TO** clause can be omitted only when an Oracle-managed file is used. In this case, the **DB\_CREATE\_FILE\_DEST** parameter should be set to indicate the new location.

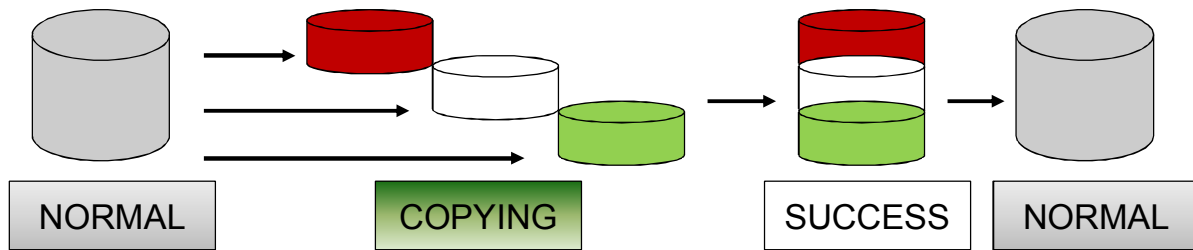
If the **REUSE** clause is specified, the new file is created even if it already exists. The **REUSE** clause can be used if a previous move command failed and the user reissues the same move command. If the **KEEP** clause is specified, the old file will be kept after the move operation. The **KEEP** clause is not allowed if the source file is an Oracle-managed file.

By default, neither **REUSE** nor **KEEP** are used.

## States

Progress in `V$SESSION_LONGOPS`:

- One row for each file moved
- Number of blocks moved so far



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The state transition of a successful online move operation is usually `NORMAL` to `COPYING` to `SUCCESS` and finally to `NORMAL`.

Moving a data file might be a long operation.

Use the `V$SESSION_LONGOPS` view to display ongoing online move operations. Each ongoing operation has one row.



## Compatibilities

|                             |                                                                                                                                                                                                              |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOT compatible with:</b> | <ul style="list-style-type: none"> <li>• Data file OFFLINE</li> <li>• Concurrent FLASHBACK DATABASE operation</li> <li>• Concurrent media recovery</li> <li>• data file RESIZE (shrink) operation</li> </ul> |
| <b>Compatible with:</b>     | <ul style="list-style-type: none"> <li>• Block media recovery</li> <li>• Tablespace made READ ONLY or READ WRITE</li> <li>• data file RESIZE (extension)</li> <li>• Online backup</li> </ul>                 |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

An Online Move data file operation is not compatible when:

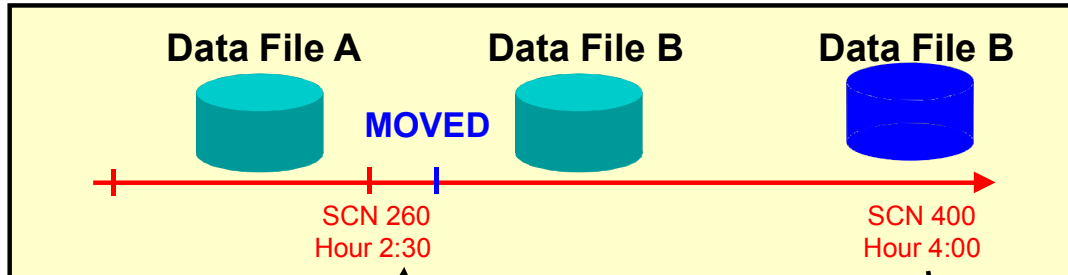
- The data file is an OFFLINE data file
- A concurrent flashback database operation is executing
- A media recovery is completing
- A file shrink operation or tablespace offline/drop operation involving the same file is performing

But it is compatible with:

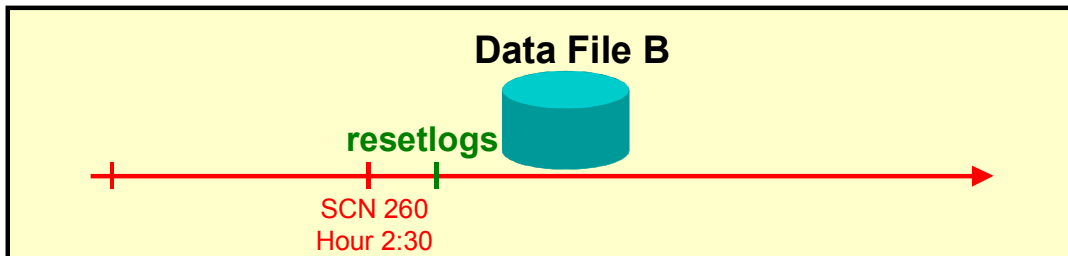
- Block media recovery
- ALTER TABLESPACE READ ONLY or READ WRITE operations
- Data file extension operation
- Tablespace/database online backup mode involving the same file

# Flashback Database

Flashback database to SCN 260:



After flashback database:



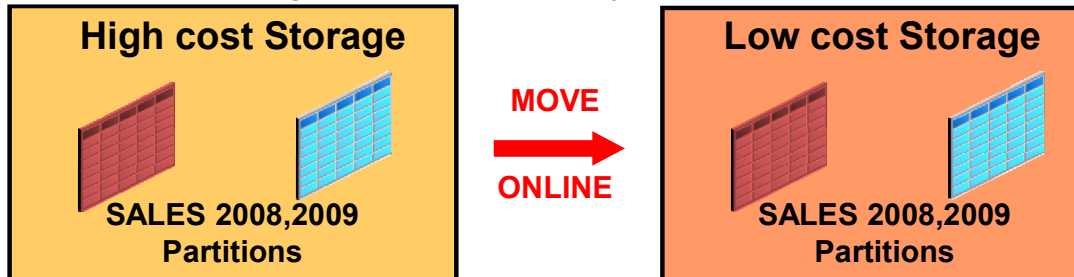
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

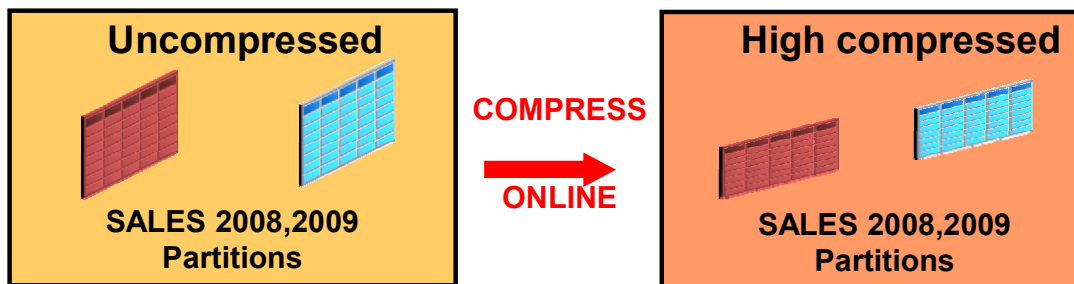
If a flashback database is executed to a time when the file was not yet moved, the flashback database operation will not change the file name to the original name, although it will bring back the old contents of the file.

# Online Move Partition

- **Move / split / merge partitions** and subpartitions **ONLINE** to low cost storage once infrequently accessed or updated



- **Compress partitions** and subpartitions **ONLINE**



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In 11g, if you use conventional DML on a table partition compressed with basic compression or Hybrid Columnar Compression, then all inserted and updated rows are stored uncompressed or in a less-compressed format. To "pack" the compressed table partition so that these rows are compressed, use an `ALTER TABLE MOVE PARTITION` statement. This prevents any updates and loads until it completes.

12c Online Partition maintenance enhancements provide the capability to move table partitions or subpartitions online without preventing concurrent DML operations.

This can be used to:

- Move partitions and subpartitions from one kind of storage to another
- Move time-based partitions and subpartitions to low cost storage once they become infrequently accessed (for example, according to ADO policies configured)
- Compress time-based partitions and subpartitions according to ADO policies configured

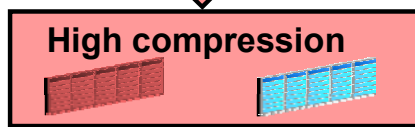
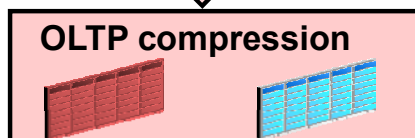
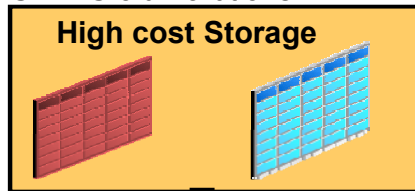
`ALTER TABLE MOVE (SUB) PARTITION` is made online to allow ADO policies to automatically move partitions or change partition compression properties in the background.

ILM can benefit from this new enhancement to move and compress table partitions according to ADO policies configured on the table or the partition.

When a partition meets the policy criterion set, the partition can be moved or compressed online under the scene.

## Online Move Partition: **Benefits**

SALES old Partitions



- **DML** allowed, not DDL
- **Move, split, or merge partitions ONLINE** to low cost storage
- **Global and local indexes** maintained

```
SQL1> UPDATE ORDERS
      SET SAL_ORD_P1=SAL*2
      WHERE DATE_P1='2001';
```

```
SQL2> ALTER TABLE ORDERS
      MOVE PARTITION ORD_P1
      TABLESPACE lowtbs
      UPDATE INDEXES ONLINE;
```

- **Compress partitions ONLINE**

```
SQL> ALTER TABLE ORDERS
      MOVE PARTITION ORD_P1
      ROW STORE COMPRESS
      UPDATE INDEXES ONLINE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

An ALTER TABLE MOVE PARTITION by using the ONLINE option allows DML operations to run uninterrupted on the partition that is being moved. The operation does not take an X DML lock on the (sub)partition being moved, but it prevents concurrent DDLs on the affected partitions, until the operation is completely done. The ALTER TABLE MOVE PARTITION ONLINE completes after all DMLs that started before the DDL statement complete.

In addition, global and local indexes are maintained during the move partition by using the UPDATE INDEXES clause. A manual index rebuild is no longer required.

Online operation in the presence of domain indexes is not supported, when UPDATE INDEXES is specified. Online operation on IOTs is not supported.

## Online Move Partition: **Compress**

MOVE is also used for compression.

- ROW STORE COMPRESS [BASIC]

```
SQL> ALTER TABLE ORDERS MOVE PARTITION ORD_P1
      ROW STORE COMPRESS UPDATE INDEXES ONLINE;
```

- ROW STORE COMPRESS ADVANCED

```
SQL> ALTER TABLE ORDERS MOVE PARTITION ORD_P1
      ROW STORE COMPRESS ADVANCED UPDATE INDEXES ONLINE;
```

- COLUMN STORE COMPRESS

```
SQL> ALTER TABLE ORDERS MOVE PARTITION ORD_P1
      COLUMN STORE COMPRESS FOR QUERY HIGH
      UPDATE INDEXES ONLINE;
```

```
SQL> ALTER TABLE ORDERS MOVE PARTITION ORD_P1
      COLUMN STORE COMPRESS FOR ARCHIVE HIGH
      UPDATE INDEXES ONLINE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

MOVE is typically the vehicle for compressing existing segments; therefore, Row Store or Column Store compression types are supported. These are the new compression types of the Advanced Row Compression.

Row Store or Column Store are attributes of a table and partition, like the organization clause.

ROW STORE and COMPRESS are the new attributes to compress a table or a partition (ROW STORE versus COLUMN STORE or HCC).

ADVANCED and BASIC are types of compression, used uniformly for tables, LOBs and partitions to convey that it is part of ACO.

The first example shows how to compress with the BASIC level an uncompressed partition ORD\_P1 online. Global and local indexes are maintained during the partition compression.

The second example shows how to compress with the OLTP level a partition ORD\_P1 online. Global and local indexes are maintained during the partition compression.

The third and fourth examples show how to compress with HCC level a partition ORD\_P1 online. Global and local indexes are maintained during the partition compression.

## Quiz

Which of the following is true regarding Online Move Datafile?

- a. Moving a data file online prevents DML operations on segments stored in the data file.
- b. Flashing back a database to a point in time before a data file had been renamed online retains the new name.
- c. Moving a data file online can be performed if the data file is in OFFLINE mode.
- d. Moving a data file online cannot be performed if the data file is under BEGIN BACKUP.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

## Summary

In this lesson, you should have learned how to:

- Describe the new Oracle Database 12c Automatic Data Optimization features
- Describe activity tracking with Heat Map
- Use views to display Heat Map statistics
- Describe Automatic Data Optimization
- Create ADO policies
- Use views and procedures to monitor Automatic Data Optimization
- Move a data file online
- Move a partition online

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 8-1: Enabling Heat Map
- 8-2: Creating an ADO TIER TO policy
- 8-3: Creating an ADO COMPRESS policy
- 8-4: Cleaning up ADO policies and Heat Map Statistics
- 8-5: Moving data files online

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.





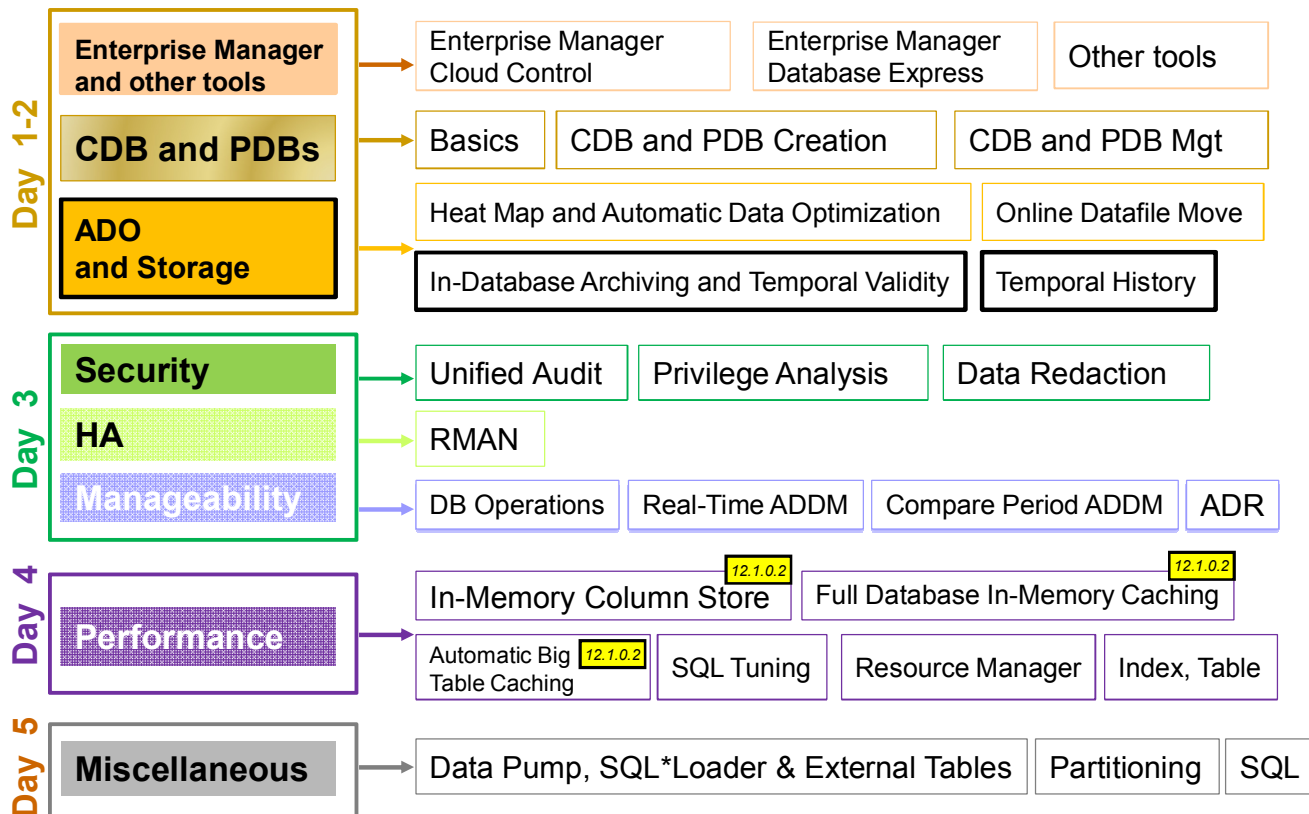
# In-Database Archiving and Temporal



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**In-Database Archiving:** In-Database Archiving solves problems regarding non-active data. It gives users the ability to keep both active operational data and non-active data in the same database tables. While achieving operational efficiency of archiving, applications can easily be limited to only access data that is in an operationally active state although archived data is kept in the same table.

**Temporal Validity:** Temporal Validity is a new temporal database capability of Oracle Database 12c. Temporal Validity provides support for valid-time (user-defined time) semantics. This new capability allows you to create a valid time dimension for each row of a table, and is another way of indicating operational relevance of rows.

The Total Recall database option, also known as Flashback Data Archive (FDA), is now called **Temporal History** and is no longer an option. In Oracle Database 12c, it is enhanced with new capabilities.

# Objectives

After completing this lesson, you should be able to:

- Distinguish In-Database Archiving from Temporal Validity
- Enable row-archival for visibility of active data only
- Describe the advantages of temporal validity versus temporal history
- Set temporal validity at table level
- Use a new SQL temporal data type
- Use temporal validity to reflect business validity
- Understand session-level or flashback point-in-time query
- Describe Temporal History (FDA) enhancements

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Note:** For a complete understanding of Oracle Information Lifecycle Management new features and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database VLDB and Partitioning Guide 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – “DBMS\_FLASHBACK\_ARCHIVE” chapter*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle By Example (OBE):*
  - *Using In-Database Row Archiving*
  - *Implementing Temporal Validity*

## Archiving Challenges

Challenges in archiving include:

- Keeping archived data instantly available in the database
- Writing programs to reload data from tape into database
- Backing up huge amounts of data, including old data
- Removing old data

The Oracle logo, consisting of the word "ORACLE" in white capital letters on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There usually comes a point during the life cycle of the data when data is no longer being regularly accessed and is considered eligible for archiving. Traditionally, the data would have been removed from the database and stored on tape, because it can store vast quantities of information for a very low cost. This solution has major disadvantages such as:

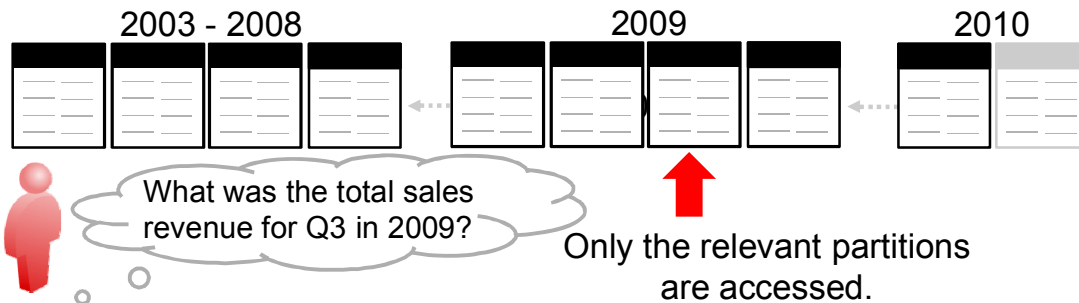
- The data is not instantly available
- Developers have to write programs to reload the data into the database from the tape archive. This could prove to be expensive and time-consuming

The historical data is commonly not kept in the database because backing up huge amounts of old data has a major impact on backups.

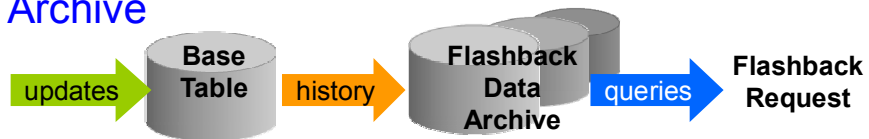
Today in Oracle Database 11g, it is no longer necessary to archive that data to tape, instead it can remain in the database. This is called In-Database Archiving, a mechanism that stores operational and archival data in the same database as opposed to out-of-database archiving of old/historical data.

# Archiving Solutions

## Partitioning



## Flashback Data Archive



## ILM Online Archive at SEGMENT level

- Storage tiering
- OLTP / HCC compression for Archive

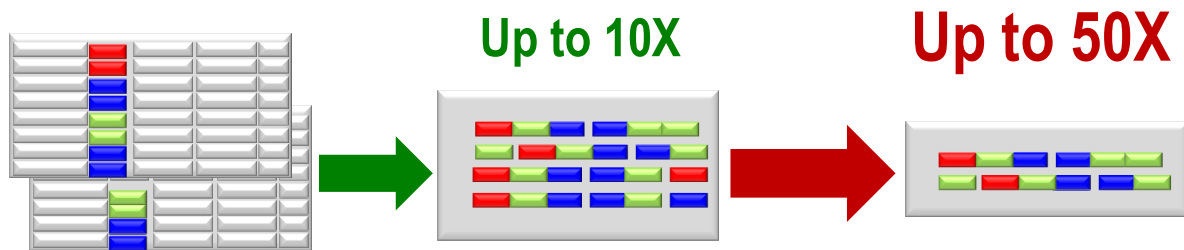
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 11g In-Database Archiving can be achieved with three different solutions:

- **Partitioning** data to distinguish active data from those in archived state. ILM can therefore compress or move historical partitions to other low cost storage tiers.
- **Flashback Data Archive** (FDA – Total Recall option in Oracle Database 11g) that provides the ability to automatically track and archive transactional data changes to specified database objects. An FDA consists of multiple tablespaces storing historic data from all transactions against tracked tables for the time duration specified using the `RETENTION` parameter in partitioned compressed system tables. Historical data can be queried using the Flashback Query `AS OF` clause. Archived historic data that has aged beyond the retention period is automatically purged.
- **ILM Online Archive**, a storage tier, where all the data that is seldom accessed or modified are stored. The storage tier is likely to be extremely large and to store the maximum quantity of data. Various techniques can compress the data. This tier can be located in the database, stored on low-cost storage devices. The data is still online and available, for a cost that is only slightly higher than storing this information on tape, without the disadvantages that come with archiving data to tape. If the Online Archive storage tier is identified as read-only, then subsequent backups are not required after the initial database backup.

## In-Database Archiving: HCC



- **Query mode** for data warehousing tables
  - Typical 10x compression ratios
- **Archival mode** for old data
  - Typical 15-50x compression ratios
- Data stored by column and then compressed

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle has implemented deeper table compression features, called Hybrid Columnar Compression, or HCC. It is designed for “In-Database archiving” providing 15-times to 50-times compression, the goal being to keep as much data as possible online in Oracle Database. This type of compression requires Exadata, ZFS, or Pillar Axiom 600 storage and has two main levels.

- **Warehouse Compression:** Is ideal for compressing data that are used for queries in a data-warehousing environment in which tables are being queried when compression ratio is achieved, meaning a 10-times reduction in the amount of I/O needed to complete a scan of a table, with a corresponding improvement in query performance
- **Archive Compression:** Is designed for “In-Database Archiving” keeping as much data as possible online in Oracle Databases using a deeper compression ratio, and achieving up to 50 times compression ratios. It is best suited for data that is non-active, typically stored offline. But now, Archive Compression delivers an online archive.

## How Does It Work?

Typical database compression algorithms compress repeating values found within rows of data stored in a database. So for example, all the order dates within an order row will be compressed. However, as the rows are stored on disk in row format, there is a lot of non-relevant information stored between each occurrence of the next value of an order date: between each order date, additional values are found for order ID, customer ID, product IDs, and so on.

Hybrid Columnar Compression uses a different technique to store the column values. Instead of storing in a row format, the data is effectively stored by column: for instance, all the order dates will be stored together, then all the order IDs, then all the customer IDs, and so on. This means that within a unit of compression, a much higher rate of repeating values is found, and a greater compression ratio can be achieved. Each compression unit can span multiple data blocks. The values for a particular column may or may not span multiple blocks. The compression is no longer bound to one data block.



# Archiving Challenges and Solutions

## Challenges:

- Keep non-active and active data within the same table.
- Access to active data only.
- Decrease archived storage amount.
- Keep performance access to archived data.

## Solutions:

- **In-Database Archiving**
  - Active
  - Not active
- **Temporal Validity**
  - Define valid-time dimension.
  - Filter on valid-time columns to access active data only.

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Until now, archived data is not distinguishable from active data at `SEGMENT` level. Oracle Database 12c provides two capabilities to distinguish active rows from those in non-active state within the same table:

- In-Database Archiving capability with row-archival management
- Temporal Validity: a new feature of Temporal

**Note:** The Total Recall option in Oracle Database 11g (the feature name is Flashback Data Archive known as FDA) is now called “Temporal” in Oracle Database 12c.

Temporal includes:

- The existing feature Flashback Data Archive that is now called “Temporal History”
- A new feature called “Temporal Validity”

**In-Database Archiving**, enhanced in Oracle Database 12c with row-archival management, indicates as a row property the activeness of row data. Distinct lifecycle states for rows in a table can be defined as active and not active.

**Temporal Validity** provides the ability to add a valid-time dimension consisting of two date-time columns specified in the table definition. Using a valid-time dimension for each row of a table is a way of indicating operational relevance. If the data is no longer valid, this data can be archived and hidden in regular queries. Processing a small active data set on a table that grows in size becomes more efficient. Data visibility relies on an explicit or implicit predicate on the valid-time dimension. While achieving operational efficiency of archiving, the applications easily access only data that is in an operationally active state although archived data is kept in the same table.

## In-Database Archiving

- Enable row-archival: `ORA_ARCHIVE_STATE` column added.

```
SQL> CREATE TABLE emp
      (EMPNO NUMBER(7), FULLNAME VARCHAR2(40),
       JOB VARCHAR2(9), MGR NUMBER(7))
      ROW ARCHIVAL;
```

- New rows: `ORA_ARCHIVE_STATE` value **0**
- Update `ORA_ARCHIVE_STATE` to **1** for row **archiving**.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To distinguish active rows from those in non-active state within the same table, and then compress the latter ones, use the new In-Database Archiving capability.

The row-archival attribute indicates as a row property the activeness of row data.

Distinct states for a row in a table can be defined: active and non-active.

To set the value of the activeness of a row data, `ROW ARCHIVAL` needs to be enabled at the segment level. This adds a new column `ORA_ARCHIVE_STATE` that will contain the state of each row.

By default, the row once inserted is active, and the `ORA_ARCHIVE_STATE` value is '0'.

After a certain elapsed time, the row can be less frequently accessed and rarely updated, but still considered as active. Then the row will be rarely accessed and no more updated, it is retained for records retention and/or compliance purposes: it is considered in a non-active state. The value '1' – or any value other than '0' – in the `ORA_ARCHIVE_STATE` column would reflect the non-active state.

## ORA\_ARCHIVE\_STATE column

- View the ORA\_ARCHIVE\_STATE column.

```
SQL> SELECT ORA_ARCHIVE_STATE, fullname FROM emp;
```

```
ORA_ARCHIVE_STATE  FULLNAME
```

```
-----
0 JEAN             ← Active
1 ADAM             ← Not active
1 TOM              ← Not active
1 JIM              ← Not active
```

- Set rows in archive state.

```
SQL> UPDATE emp SET ORA_ARCHIVE_STATE = 1
      WHERE empno < 100;
```

- Set rows back in active state.

```
SQL> UPDATE emp SET ORA_ARCHIVE_STATE = 0;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Data in a non-active state can be compressed and be subject to storage compression.

As a hidden column, ORA\_ARCHIVE\_STATE is visible only when explicitly specified in select-lists of queries.

The administrator has to update the value of the ORA\_ARCHIVE\_STATE column to one to set the row in archive state.

# Session Visibility Control

- Session visibility control: By default, active rows only

```
SQL> select ORA_ARCHIVE_STATE, fullname from emp;

ORA_ARCHIVE_STATE FULLNAME
-----
0 JEAN                ← Active
```

- Enable application visibility of all rows.

```
SQL> alter session set ROW ARCHIVAL VISIBILITY = ALL;

SQL> select ORA_ARCHIVE_STATE, fullname from emp;

ORA_ARCHIVE_STATE FULLNAME
-----
0 JEAN                ← Active
1 ADAM                ← Not active
1 JIM                 ← Not active
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

By default, sessions have the visibility of active rows only.

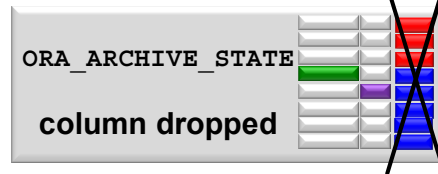
The application visibility of active data can be enabled at the session-level. Rows that were moved to the non-active lifecycle state are filtered by the system, and only active rows are visible.

The ALTER SESSION statement is extended with a new ROW ARCHIVAL VISIBILITY clause. By default, the value is ALL. The ACTIVE value allows viewing only active data, and not non-active data, in the session.

## Disable Row-Archival

- Disable row-archival: `ORA_ARCHIVE_STATE` column removed

```
SQL> ALTER TABLE emp NO ROW ARCHIVAL;
```



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you disable the row-archival attribute on a table because there is no need anymore to distinguish active and non-active rows, the `ORA_ARCHIVE_STATE` column is automatically dropped.

## Quiz

Which statement is true about row-archival?

- a. The `ORA_ARCHIVE_STATE` column always exists at table creation.
- b. Oracle automatically sets appropriate values in the `ORA_ARCHIVE_STATE` column when data ages.
- c. By default, the `ORA_ARCHIVE_STATE` column is set to 0 when a new row is inserted.
- d. The `ORA_ARCHIVE_STATE` column is part of the Temporal Validity feature.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

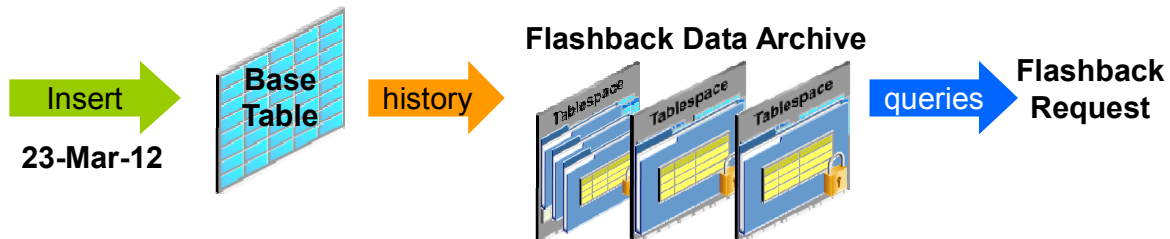
**Answer: c**

# Temporal Validity Versus Temporal History

- Temporal Validity

| Emp IDc | Job       | Hire Date        |
|---------|-----------|------------------|
| 100     | Clerk     | 22-Apr-11        |
| 200     | Developer | 12-Dec-11        |
| 400     | Salesman  | <b>22-Mar-12</b> |

- Temporal History



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

With the In-Database Archiving, you can distinguish active rows from non-active rows within the same table, and then compress the non-active rows. You now look at another way to achieve the same goal with Temporal Validity.

Applications often annotate the validity of a fact recorded in a table with dates or timestamps that are relevant to the underlying business (for example, the hire date of an employee in HR applications, or the effective date of coverage in the insurance industry). This temporal attribute is called temporal validity, and is usually controlled by the user or application who defines the valid-time dimension at table creation.

Temporal validity dates or timestamps are different than the dates or timestamps indicating when the fact was recorded in the database. The date or timestamp when the fact was recorded in the database are attributes of Temporal History (also known as Flashback Data Archive) and are system-managed. The Oracle Database 11g Total Recall feature (or FDA) implements the transaction time of temporal history.

In the slide, employee 400 was hired on Mar 22, but the row was entered in the `HR.EMP` table on Mar 23. Mar 22 is the valid time temporal start date and Mar 23 is the transaction time temporal start date.

By using the valid time temporal implicit filter on the valid-time dimension, queries can show rows that are currently valid or that will be valid in the future. The query is able to hide rows whose facts are not currently valid.

Bi-temporal queries can use both valid time temporal and transaction time temporal dates.



## PERIOD FOR Clause Concept

- Keep both active and non-active data in the same table.
- Define one valid-time dimension at table creation.
  - Explicitly define the two date-time columns.

```
SQL> CREATE TABLE emp
      ( empno number, salary number, deptid number,
        name VARCHAR2(100),
        user_time_start DATE, user_time_end DATE,
        PERIOD FOR user_time (user_time_start,user_time_end));
```

- Or implicit automatic valid-time columns are created.

```
SQL> CREATE TABLE emp2
      ( empno number, salary number, deptid number,
        name VARCHAR2(100),
        PERIOD FOR user_time);
```

- Insert rows explicitly naming the valid-time columns.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A valid-time dimension, represented by the new `PERIOD FOR` clause, consists of two date-time columns that can be specified in the table definition, as shown in the first example or that are automatically created, as shown in the second example.

To hide valid-time dimension columns, just specify a `PERIOD FOR` clause name without any date columns. Oracle creates two hidden columns by using the name of the valid-time dimension as a prefix for the names of the two columns. The valid-time dimension name is used to drop the dimension if required. As shown in the second example, you defined the `user_time` as the name of the valid-time dimension and `user_time` is used as the prefix for the two date columns automatically created, `USER_TIME_START` and `USER_TIME_END`.

To insert rows into a table with a valid-time dimension, you would name the two valid-time date columns as follows:

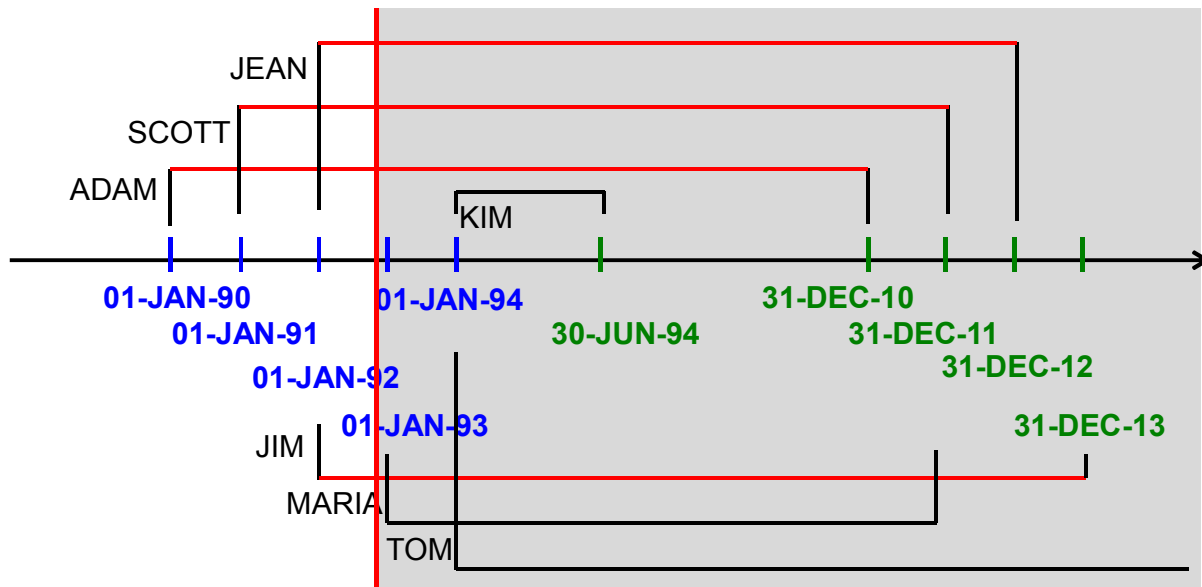
```
SQL> INSERT INTO emp2 (empno, salary, deptid, name,
                      user_time_start, user_time_end)
      VALUES (1,1000,20, 'John', SYSDATE, NULL);

SQL> select EMPNO, user_time_start, user_time_end from emp2;
EMPNO USER_TIME_START                USER_TIME_END
```

```
-----
1 17-AUG-12 09.58.03.000000 AM +00:00
```

## Filtering on Valid-Time Columns: Example 1

Filter on valid-time columns to access active data only.



```
SQL> select * from hr.emp as of PERIOD FOR user_time
      to_date('01-DEC-1992', 'dd-mon-yyyy') ;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

How do you filter on valid-time columns? Use the `SELECT` statement with the `PERIOD FOR` clause or use the `DBMS_FLASHBACK_ARCHIVE` procedure.

- There is one set of data that is “valid” based on its valid-start and valid-end times and the query time (AS OF or undecorated).
- On the other hand there is the other set of rows where the query time falls outside the valid-start and valid-end times.

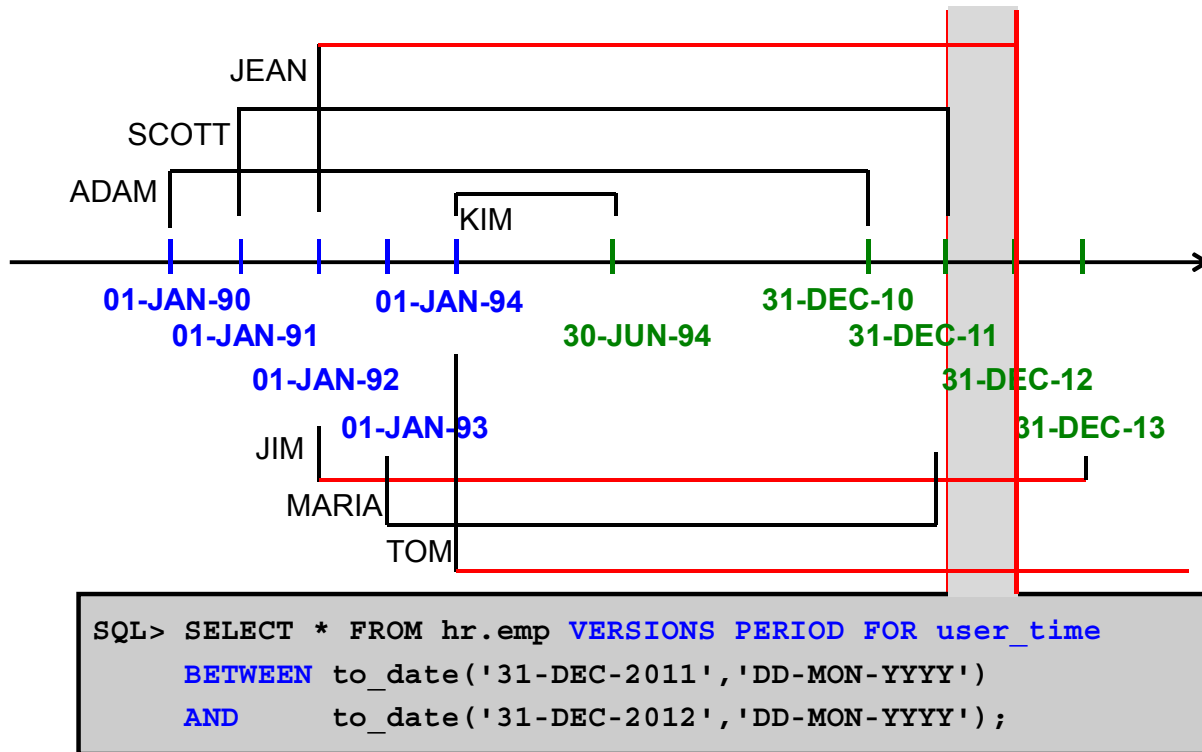
Both sets of rows data reside in the same table. However, by controlling the visibility of data to the valid rows, you can limit what queries and DMLs affect. Until now, you could do as-of and versions queries for transaction time. Now you can do as-of and versions queries for valid time.

For each new employee that you inserted in the table, you included the hire dates, valid-time start dates, and valid end dates. The dates represent the activeness of each row. These dates are entered by the application and correspond to valid dates. The time that the rows were inserted and committed in the table corresponds to the transaction date.

You can filter the active employees by using the following new `PERIOD FOR` clause. The query displays all active employees who were valid at the explicit date of '01-DEC-1992', which is the date that belongs to the valid period; that is, between `USER_TIME_START` and `USER_TIME_END`.

## Filtering on Valid-Time Columns: Example 2

Use versions queries for valid time.



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can filter the active employees by using the following new `VERSIONS PERIOD FOR BETWEEN` clause:

```
SQL> select * from hr.emp VERSIONS PERIOD FOR user_time
      BETWEEN to_date('31-DEC-2011', 'dd-mon-yyyy')
      AND to_date('31-DEC-2012', 'dd-mon-yyyy') ;
```

The query displays all employees whose `VALID_TIME_START` is less than or equal to '31-DEC-2011' and `VALID_TIME_END` greater than or equal to '31-DEC-2012'.

Queries that mix valid-time and transaction-time dimensions are called bi-temporal queries.

This example shows rows as of the specified transaction time, which are valid now.

```
SQL> select * from hr.emp
      as of period for user_time to_date('31-DEC-1992', 'dd-mon-yyyy')
      as of timestamp to_date ('30-mar-2012','dd-mon-yyyy');
```

## DBMS\_FLASHBACK\_ARCHIVE

- Visibility control applies to queries and DML.
- Full visibility applies to DDL.
- Visibility is set with DBMS\_FLASHBACK\_ARCHIVE:
  - Set the visibility to data valid as of the given time.

```
SQL> exec DBMS_FLASHBACK_ARCHIVE.ENABLE_AT_VALID_TIME('ASOF', -
to_timestamp('29-SEP-10 05.44.01 PM'))
```

- Set the visibility to data currently valid within the valid time period at the session level.

```
SQL> exec
DBMS_FLASHBACK_ARCHIVE.ENABLE_AT_VALID_TIME('CURRENT')
```

- Set the visibility to data to the full table level.

```
SQL> exec DBMS_FLASHBACK_ARCHIVE.ENABLE_AT_VALID_TIME('ALL')
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Users can modify visibility within a session via new DBMS\_FLASHBACK\_ARCHIVE package. Visibility control applies to all SQL SELECT and DML statements.

DDLs will default to getting full visibility to the table data. For example, CTAS, online redefinition, and ALTER TABLE MOVE operations will have full visibility of the table data. Hidden columns are also visible to the DDL operations, resulting in the preservation of those columns and their data.

The first example sets the valid time visibility to '29-SEP-2010', showing only rows overlapping the given date.

The second example sets the visibility to data currently valid within the period time at the session level.

The third example sets the visibility to the full table, which is the default temporal table visibility.

## Quiz

Choose the correct statement regarding the Temporal Validity feature.

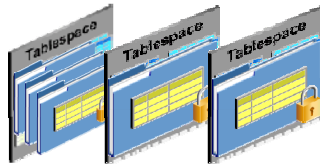
- a. Temporal Validity and temporal history cannot be used in the same query.
- b. Temporal Validity visibility control can be set at session level.
- c. Temporal Validity columns are automatically updated by Oracle.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

# Temporal History Enhancements: FDA Optimization



|                                                                                                                                                              |  |                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div style="background-color: yellow; padding: 2px; display: inline-block;"><b>Pre-12c</b></div> History or archive tables:<br>- Compressed and deduplicated |  | <div style="background-color: red; color: white; padding: 2px; display: inline-block;"><b>12c</b></div> History or archive tables:<br>- <b><u>Not</u></b> compressed or deduplicated |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
SQL> CREATE FLASHBACK ARCHIVE fla1 TABLESPACE tbs1
      RETENTION 5 YEAR;
```

**12c**

 History or archive tables: Compressed or deduplicated

```
SQL> CREATE FLASHBACK ARCHIVE fla1 TABLESPACE tbs1
      OPTIMIZE DATA RETENTION 5 YEAR;
```

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Flashback Data Archive in Oracle Database 11g

In Oracle Database 11g, FDA history tables are compressed, deduplicated, and internally partitioned.

## Temporal History in Oracle Database 12c

In Oracle Database 12c, FDA history tables are no more compressed or deduplicated at the FDA creation. Creating a table with the `OPTIMIZE DATA` clause, table and LOB compression and LOB deduplication is automatically turned on, using any of the following features: Advanced Row table compression, Advanced LOB Compression, Advanced LOB Deduplication, segment-level and row-level ADO compression. ADO is enabled to allow new data to be archived as uncompressed and over time is compressed in the background.

FDA history tables already compressed and deduplicated in 11g or before upgrade to 12c will continue to get compression and deduplication.

To stop optimization on FDA history tables, execute the following statement:

```
SQL> ALTER FLASHBACK ARCHIVE fla1 NO OPTIMIZE DATA;
```

**Note:** With Oracle Database 12.1.0.2, Flashback Data Archive is compatible with pluggable databases.

## Temporal History Enhancements: User Context Metadata

The user context of transactions on FDA enabled tables is:

- Collected by an attribute: NONE (default), TYPICAL, ALL
  - Set with `DBMS_FLASHBACK_ARCHIVE.SET_CONTEXT_LEVEL`
  - If TYPICAL, collects database user id, global user id, client identifier, service name, module name, or host name
- Obtained from USERENV by using `GET_SYS_CONTEXT`

```
SQL> EXEC DBMS_FLASHBACK_ARCHIVE.SET_CONTEXT_LEVEL('TYPICAL')
PL/SQL procedure successfully completed
```

```
SQL> SELECT DBMS_FLASHBACK_ARCHIVE.GET_SYS_CONTEXT
           (VERSIONS_XID, 'USERENV', 'SESSION_USER'),
           VERSIONS_XID,VERSIONS_STARTTIME,VERSIONS_ENDTIME,
           employee_id, salary
FROM hr.employees VERSIONS BETWEEN SCN MINVALUE
                        AND MAXVALUE ;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, Flashback Data Archive does not collect the user context data of transactions executed on FDA-enabled tables. Oracle Database 12c now allows the retrieval of this information.

- The user context of a transaction executed on a Flashback Data Archive–enabled table is collected and retrievable. The parameters of the USERENV namespace describe the current session. The user context is obtained from `DBMS_FLASHBACK_ARCHIVE.GET_SYS_CONTEXT`.
- The user context collection is controlled by a parameter, set by `DBMS_FLASHBACK_ARCHIVE.SET_CONTEXT_LEVEL` whose values can be NONE, TYPICAL, or ALL. By default no user context is collected. Some examples of information, which are collected in the TYPICAL case, are database user ID, global user ID, client identifier, service name, module name, or host name.
- Rows in these tables are purged when the commit time is older than the retention of the flashback archive with the longest retention.
- Each row of the user context can only be read by the DBA or the owner of the transaction.

## Summary

In this lesson, you should have learned how to:

- Distinguish In-Database Archiving from Temporal Validity
- Enable In-Database Archiving for visibility of active data only
- Describe the advantages of temporal validity versus temporal history
- Set temporal validity at table level
- Use a new SQL temporal data type
- Use temporal validity to reflect business validity
- Understand session-level or flashback point-in-time query
- Describe Temporal History (FDA) enhancements

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



## Practice

- 9-1: Using In-Database Archiving with Row-archival
- 9-2: Using Temporal Validity to automatically view valid-time rows only
- 9-3: Using user context information in the history table of FDA-enabled tables (*optional*)
- 9-4: Cleaning up FDA

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# **Module**

# **Security**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

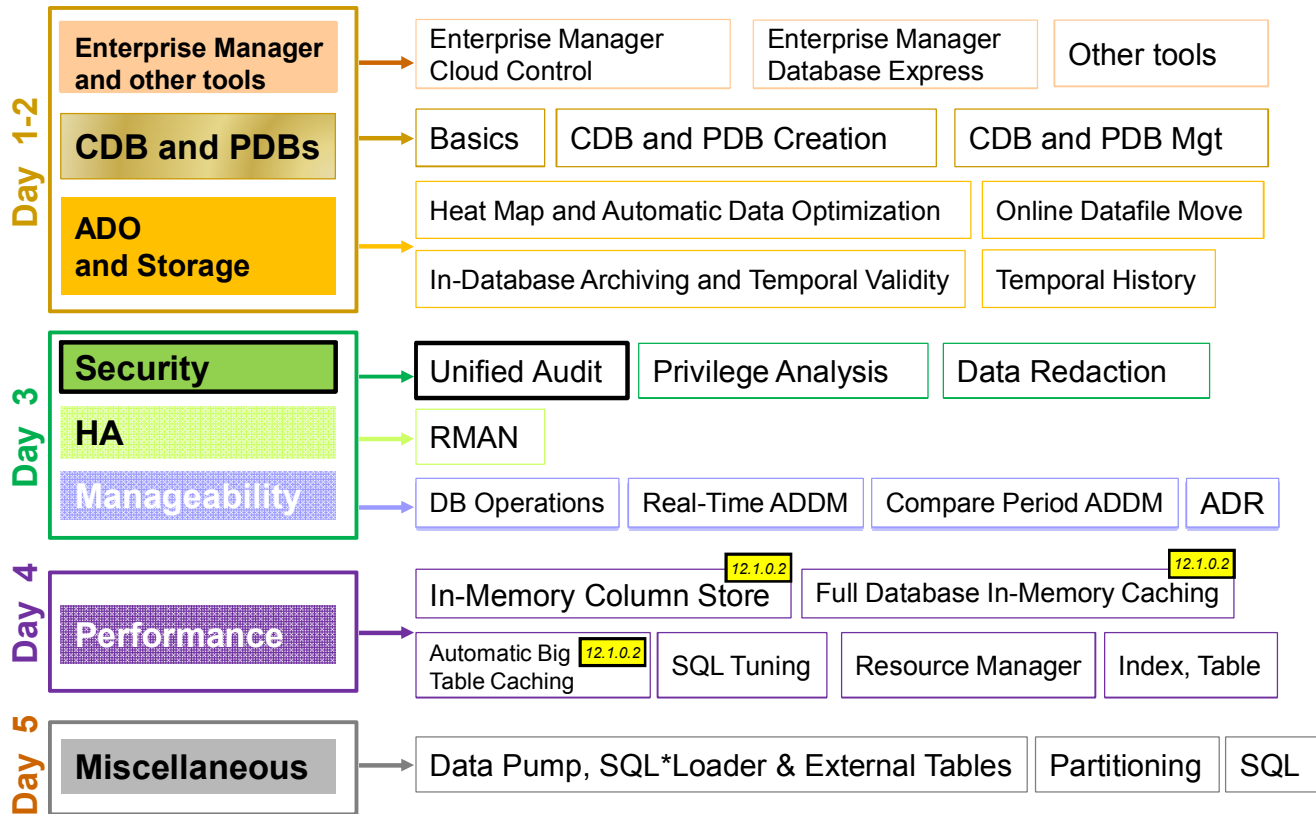
# 10

## Auditing

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Unified Audit:** The unified audit facility serves to consolidate all audit trails into a single unified audit trail table, improve audit performance, secure audit data for all RDBMS and other components like RMAN and Data Pump, and simplify audit configuration.

# Objectives

After completing this lesson, you should be able to:

- Understand the implementation and benefits of using the unified audit trail
- Enable unified auditing
- Configure the unified auditing
- Create and enable audit policies
- Add application context data to audit records
- View data in the unified audit trail
- Clean up the unified audit trail

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of Oracle Unified Auditing new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Security Guide 12c Release 1 (12.1)*

Refer to other sources of information:

- *Oracle Database 12c New Features Demo Series* demonstrations under Oracle Learning Library:
  - *Unified Auditing*
- *Oracle By Example (OBE)*:
  - *Auditing With Unified Auditing*

## Types of Auditing

Monitor activity:

| Type of Auditing               | Activity Audited                                                                                                                                          |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mandatory auditing             | <ul style="list-style-type: none"> <li>STARTUP, SHUTDOWN</li> <li>SYSDBA, SYSOPER, SYSASM connections</li> </ul>                                          |
| Standard database auditing     | AUDIT_TRAIL parameter                                                                                                                                     |
| Privileged users auditing      | AUDIT_SYS_OPERATIONS = TRUE                                                                                                                               |
| Fine-grained auditing (FGA)    | DBMS_FGA.ADD_POLICY                                                                                                                                       |
| Oracle Database Vault auditing | Any violation against any component: <ul style="list-style-type: none"> <li>Realm, Command Rule, Secure Application Role</li> <li>Rule, Factor</li> </ul> |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Compliance, privacy, and increasingly sophisticated security threats have resulted in an almost mandatory requirement for turning on native auditing. This is especially true for monitoring privileged users, but also for actions that may be out of the normal operations that one would expect to see in a production environment. In fact, privileged users have increasingly been the target of external hackers. As a result, auditing privileged user activity and application changes such as DDL operations are especially important. Database link creation, create table, and truncate table are just a few of the DDL operations that should be audited. Starting with Oracle Database 11g, auditing for these types of operations is configured by default.

Oracle Database 11g provides three different types of auditing. The administrator can audit all actions that take place within the database. It is important to remember that auditing should be focused so that only events that are of interest are captured. Properly focused auditing has minimal impact on system performance and provide highly relevant data for reporting and alerting purposes.

- Standard database auditing captures several pieces of information about an audited event, including that the event occurred, when it occurred, the user who caused the audited event, and which client machine was being used when the event happened.
- Audit privileged user operations with the `AUDIT_SYS_OPERATIONS` parameter.
- Independently of standard database auditing, use fine-grained auditing to audit SQL data manipulation language statements occurring only when specific columns are impacted under certain conditions.

# Audit Trail Implementation

Store audit records:

| Audit Trail                                                  | Activity Audited                                                                                               |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Send audit records to DB/OS files                            | <b>AUDIT_TRAIL</b> = DB / OS / XML<br>DB, EXTENDED<br>XML, EXTENDED<br><b>AUDIT_FILE_DEST</b> = directory_path |
| Move SYS.AUD\$, SYS.FGA_LOG\$ tables to a defined tablespace | Use <b>DBMS_AUDIT_MGMT</b> package<br><b>SET_AUDIT_TRAIL_LOCATION</b> procedure                                |
| DVSYSAUDIT_TRAIL\$                                           | Oracle Database Vault audit                                                                                    |

Secure audit data:

- syslog audit: **AUDIT\_SYSLOG\_LEVEL=local1.info**
- Oracle Audit Vault

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The **AUDIT\_TRAIL** parameter defines where audit records will be written. This parameter is set to **DB** by default. If **AUDIT\_TRAIL** is set to a value of **NONE**, standard database auditing is disabled. The **AUDIT\_TRAIL** parameter is a static parameter and cannot be modified without restarting the database instance.

The possible parameter values that enable auditing are:

- **DB**: Enables database auditing and directs all audit records to the database audit trail (the **SYS.AUD\$** table)
- **DB, EXTENDED**: In addition to action of the **DB** value, captures the SQL executed along with any bind variables (**SQLTEXT** and **SQLBIND** columns) of the audit table
- **XML**: Enables database auditing and writes all audit records to the XML format OS files
- **XML, EXTENDED**: Similar behavior as **DB, EXTENDED**
- **OS**: Enables database auditing and directs audit records to the operating system files.

For all **AUDIT\_TRAIL** options that write to the OS files, the default directory is determined by **AUDIT\_FILE\_DEST**.

Oracle Database Vault audit data is stored in the **DVSYSAUDIT\_TRAIL\$** table.



Compliance regulations and laws require businesses to secure business and personal data for customers, employees, and partners. Businesses must demonstrate compliance with the regulations by auditing users, activities, and associated data.

- `AUDIT_TRAIL` can be set to `OS`, and the audit files are created in the `AUDIT_FILE_DEST` directory and are owned by the oracle user.
- If you also set the `AUDIT_SYSLOG_LEVEL` parameter to values, the audit files are owned by `root`, and they are visible only to `root`. The records are sent to the `syslog` process.
- A more secure solution is Oracle Audit Vault, which is able to:
  - Transparently collect and consolidate audit data from several Oracle databases
  - Help detect and prevent insider threats by alerting you to suspicious activity
  - Help organizations simplify compliance reporting with built-in and custom reports

# Oracle Database 12c Auditing

- **Simplicity:**
  - Audit options grouped into a simple audit policy
  - Simpler action-based audit configuration
  - Condition-based audit configuration
  - Users exempted from being audited
- **Consolidation:** One single unified audit trail
- **Security:**
  - Read-only audit trail table
  - Any operation related to audit configuration audited
  - Any SYS user actions audited
  - Separation of audit administration duties with audit roles
- **Performance:**
  - Negligible overhead

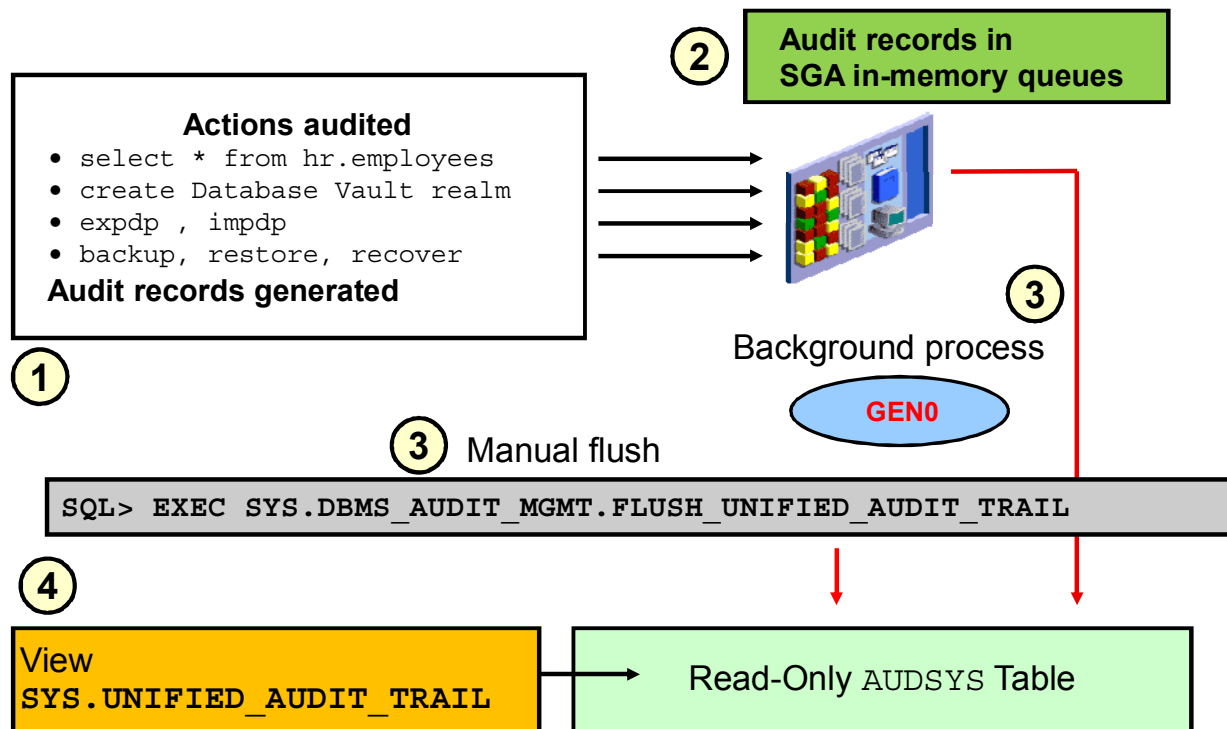
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The unified auditing facility needs to address the following challenges:

- **Simplicity:**
  - Grouping audit options into a simple audit policy
  - Allowing simpler action-based audit configurations
  - Setting condition-based audit configurations
  - Exempting users from being audited
- **Consolidation:** Merging all audit trails into a single unified audit trail table
- **Security:**
  - Relying on a read-only audit trail table
  - Auditing any operation related to audit configuration
  - Auditing any SYS user auditable action
  - Separating audit administration duties with audit administration roles, AUDIT\_ADMIN and AUDIT\_VIEWER
- **Performance:**
  - Oracle Database 12c auditing provides the ability to use System Global Area (SGA) queues for accumulating audit records, enabling auditing to be turned on with negligible overhead.

# Security and Performance: Audit Architecture



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

How does the new audit architecture work?

Named queues inside the SGA have a corresponding persistent storage in tables owned by AUDSYS.

A background process flushes the contents to the table every three seconds.

Each client has two SGA queues so that a client can continue to write to the second queue while the first queue is being persisted to the table.

You can see the audit trail records in the UNIFIED\_AUDIT\_TRAIL view.

The SYS.DBMS\_AUDIT\_MGMT.FLUSH\_UNIFIED\_AUDIT\_TRAIL procedure is only available in case you need to explicitly flush the queues to disk before the 3 seconds delay.

## Consolidation

### Basic information + specific components columns in SYS.UNIFIED\_AUDIT\_TRAIL view:

|                                                       |                                                                                                                                                |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| FGA event                                             | <ul style="list-style-type: none"> <li>• FGA_POLICY_NAME</li> </ul>                                                                            |
| Data Pump Export / Import                             | <ul style="list-style-type: none"> <li>• DP_TEXT_PARAMETERS1</li> <li>• DP_BOOLEAN_PARAMETERS1</li> </ul>                                      |
| RMAN backup / recovery                                | <ul style="list-style-type: none"> <li>• RMAN_OPERATION</li> <li>• RMAN_OBJECT_TYPE</li> <li>• RMAN_DEVICE_TYPE</li> <li>• RMAN_XXX</li> </ul> |
| OLS operations                                        | <ul style="list-style-type: none"> <li>• OLS_POLICY_NAME</li> <li>• OLS_XXX</li> </ul>                                                         |
| Database Vault violations<br>or configuration changes | <ul style="list-style-type: none"> <li>• DV_XXX</li> </ul>                                                                                     |
| Real Application Security                             | <ul style="list-style-type: none"> <li>• XS_XXX</li> </ul>                                                                                     |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Database components can be audited like:

- FGA component, which is registered with the framework. The specific FGA information is stored in the FGA\_POLICY\_NAME column.
- Data Pump when export or import operations are executed. The specific Data Pump information is stored in two new columns: DP\_TEXT\_PARAMETERS1 and DP\_BOOLEAN\_PARAMETERS1.
- Oracle RMAN when backup, restore, or recovery operations are executed. The specific RMAN information is stored in several new columns: RMAN\_OPERATION would contain either 'Backup', 'Restore', or 'Recover'. RMAN operations are by default audited.
- OLS when applying an OLS policy on a table, for example. The specific OLS information is stored in the OLS\_XXX columns.
- Oracle Database Vault when configuration changes or violations occur against Oracle Database Vault components, such as realms, command rules, and rulesets factors. The specific Database Vault information is stored in the DV\_XXX columns.
- RAS operations. The specific RAS information is stored in the XS\_XXX columns.

## Data Pump Audit Policy

1. Create the audit policy for the component and actions.

```
SQL> CREATE AUDIT POLICY dp_pol1 ACTIONS  
      COMPONENT=datapump export;
```

2. Enable the audit policy.

```
SQL> audit policy DP_POL1;
```

3. Perform an export operation.

```
$ expdp hr/oracle_4U dumpfile=HR_tables tables=EMPLOYEES,JOBS  
      directory=DATA_PUMP_DIR
```

4. Flush the audit data to disk.

```
SQL> EXEC SYS.DBMS_AUDIT_MGMT.FLUSH_UNIFIED_AUDIT_TRAIL
```

5. View the Data Pump–specific audit information.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you want the Data Pump Export or Import operations to be audited, you must perform these steps:

1. Create an audit policy for export or import or for both. An audit policy is a named group of audit settings that enable you to audit a particular aspect of user behavior in the database.
2. Next, enable the audit policy that was created to audit export operations.
3. Perform an export operation. At each export operation, the specific Data Pump information is then stored in two new columns of the `UNIFIED_AUDIT_TRAIL` view: `DP_TEXT_PARAMETERS1` and `DP_BOOLEAN_PARAMETERS1`.
4. Flush the audit information to disk by using the `DBMS_AUDIT_MGMT.FLUSH_UNIFIED_AUDIT_TRAIL` procedure.
5. Display the Data Pump audit information in the `UNIFIED_AUDIT_TRAIL` view.

# Unified Audit Implementation

## → Mixed auditing mode:

Allows smooth migration of existing databases to use the unified auditing features

## → Unified auditing mode:

Before populating the UNIFIED\_AUDIT\_TRAIL view:

- Enable unified auditing.
  1. Shut down all processes and database instances.
  2. `cd $ORACLE_HOME/rdbms/lib`  
`make -f ins_rdbms.mk uniaud_on ioracle`  
`ORACLE_HOME=$ORACLE_HOME`
  3. Start all Oracle processes of all instances.
- Define a tablespace for the read-only audit table.

The AUDIT\_XXX instance parameters are no longer observed.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Mixed Auditing Mode

Mixed auditing mode allows both the new audit engine and the traditional audit engine to work simultaneously.

- On upgrade, none of the existing audit settings are affected and the database continues to support the traditional audit facilities. It is recommended you migrate to the unified audit facility to take advantage of the new audit architecture and performance improvements. Before you decide to switch to the unified auditing mode, you can use the mixed mode by creating a policy with `CREATE AUDIT POLICY` command and then enabling it with `AUDIT` command. If you do not wish to create a new policy, you can simply enable one of the predefined policies—`ORA_SECURECONFIG`, `ORA_ACCOUNT_MGMT`, or `ORA_DATABASE_PARAMETER`. Either of this puts the database in mixed auditing mode. The old audit syntax continues to work and the old audit destinations continues to be written to. However, new audit policies can be created and audit data also writes to the new unified audit trail.
- When a database is created, mixed auditing mode is used by default through the predefined enabled policy `ORA_SECURECONFIG`. But unified auditing mode is not yet enabled.

## Unified Auditing Mode

To enable unified auditing, relink Oracle with the `uniaud_on` option after shutting down all Oracle processes and database instances in the `ORACLE_HOME`. By default, the new unified audit data is written to the `SYSAUX` tablespace. If you do not want to use the `SYSAUX` tablespace, create a new tablespace to host the new read-only audit trail table, which is owned by the new `AUDSYS` schema. Note that the `AUDSYS` user is locked by default. After enabling unified audit mode, none of the old audit instance parameters are honored: `AUDIT_TRAIL`, `AUDIT_FILE_DEST`, `AUDIT_SYS_OPERATIONS`, `AUDIT_SYSLOG_LEVEL`. There is no possibility to write audit records to OS or syslog.

## Quiz

To audit RMAN backup, restore, and recovery operations, you must create new audit policies.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Answer: b

Unlike other Oracle Database components, you do not create a unified audit policy for Oracle RMAN events. The `UNIFIED_AUDIT_TRAIL` view automatically captures commonly audited Oracle RMAN events.



## Quiz

Select the schema owner of the unified audit trail.

- a. SYS
- b. AUDSYS
- c. AUDIT\_ADMIN
- d. LBACSYS
- e. DVSYS

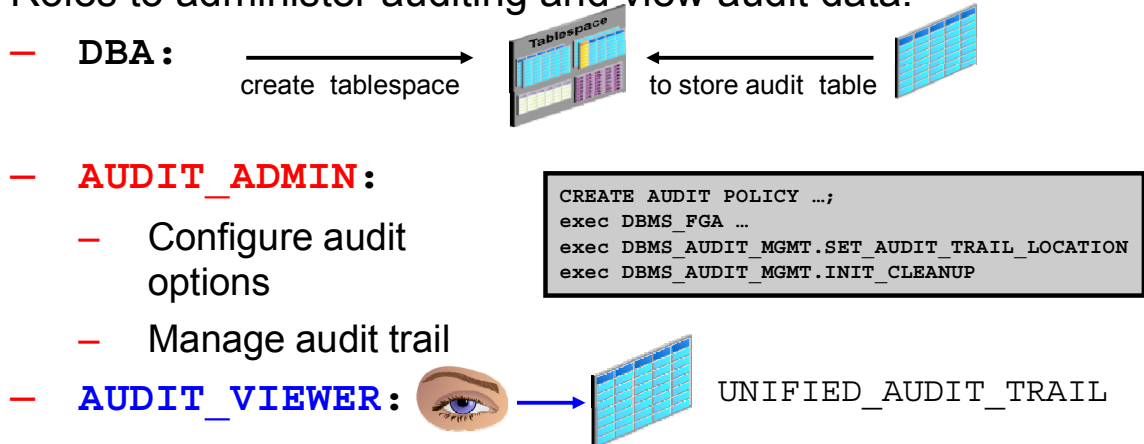
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

# Security

- Read-only audit trail table in AUDSYS schema
- Any audit-related activity mandatorily audited
- All top-level statements mandatorily audited (STARTUP, SHUTDOWN, ALTER DATABASE, and ALTER SYSTEM)
- Roles to administer auditing and view audit data:



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Compliance regulations and laws require businesses to secure audit data by auditing users, activities, and associated data.

The SYS user, connected as SYSDBA, SYSOPER, SYSBACKUP, SYSASM, SYSKM, or SYSDG, is automatically audited:

- During any top-level statement execution, such as STARTUP, SHUTDOWN, ALTER DATABASE, and ALTER SYSTEM, until the database opens.
- When system-wide audit policies are configured for the SYS user.
- The mandatory and SYS auditing records are written to \$ORACLE\_BASE/audit/<dbname> when the database is not yet open. The DBMS\_AUDIT\_MGMT.LOAD\_UNIFIED\_AUDIT\_FILES procedure allows you to load them into the database when the database is open. The procedure deletes the external audit files after they are successfully loaded in the database.

The audit trail table owned by the AUDSYS schema is in read-only mode, and can be updated only during recovery or upgrade operations by using the SYSDBA privilege.

Any audit-related activity, such as creation, modification, deletion, enabling of audit policies, and execution of DBMS\_AUDIT\_MGMT and DBMS\_FGA packages, are audited by mandate.

The SYSDBA privilege is no longer required to configure audit settings and management of the audit trail. To enable separation of duties, the AUDIT\_ADMIN role is required to perform these functions. The AUDIT\_VIEWER role allows for viewing and analyzing the audit data.

## Simplicity: Audit Policy

- ➔ • No more `AUDIT_XXX` instance parameters
- ➔ • To start auditing, perform the following steps:
  1. Create audit policies used to group audit options:
 

**CREATE AUDIT POLICY** command:

    - Audit options: System-wide or object-specific or role
    - Optional **WHEN** condition **EVALUATE PER STATEMENT**
    - Optional event-handler alert
    - **CONTAINER** = `CURRENT` | `ALL`
  2. Enable/disable audit policies: **AUDIT** and **NOAUDIT**
    - All users by default
    - **BY** username
    - **EXCEPT** username
- ➔ • View audited data in the `UNIFIED_AUDIT_TRAIL` view

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In a unified audit database, the following actions are audited by mandate: `CREATE AUDIT POLICY`, `ALTER AUDIT POLICY`, `DROP AUDIT POLICY`, `AUDIT`, `NOAUDIT`, `EXECUTE` of `DBMS_FGA`, and `EXECUTE` of `DBMS_AUDIT_MGMT`.

Auditing actions based on system or object privileges or roles are performed, provided that you create and enable audit policies. What are audit policies?

- A named group of audit options that enable you to audit a particular aspect of user behavior in the database by using system, object, or role privileges
- Created with the `CREATE AUDIT POLICY` command requiring the `AUDIT_ADMIN` role
- Enforced conditionally and evaluated during the statement execution or once during the session or at the instance level
- Supplied with an event-handler procedure alerting that the audit occurred
- Applied in a multitenant container database (CDB) either in the root or a pluggable database (PDB)
- Not active until explicitly enabled with the `AUDIT` command.  
Be aware that existing standard non-policy audit options configured with the `AUDIT` and `NOAUDIT` commands are not effective.
- Applied with the exception of certain user lists

## Step 1: Creating the Audit Policy

- Create audit policies based on system-wide audit options:  
System privileges or actions or roles or all of them

```
SQL> CREATE AUDIT POLICY audit_mixed_pol4
      PRIVILEGES DROP ANY TABLE ROLES emp_role;
```

- Create audit policies based on object-specific options.

```
SQL> CREATE AUDIT POLICY audit_objpriv_pol7
      ACTIONS EXECUTE, GRANT ON hr.raise_salary_proc;
```

- Create audit policies based on condition and evaluation  
PER SESSION OR STATEMENT OR INSTANCE

```
SQL> CREATE AUDIT POLICY audit_mixed_pol5
      ACTIONS RENAME ON hr.employees, ALTER ON hr.jobs,
      WHEN 'SYS_CONTEXT (''USERENV'', ''SESSION_USER'')='''JIM'''
      EVALUATE PER SESSION;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To create an audit policy, you must first define system-wide or object-specific audit options.

### System-Wide Options

- The **privilege audit option** audits all events, which exercise the specified system privilege, as in the first example in the slide.
- The **action audit option** indicates which RDBMS action should be audited, such as the ALTER TRIGGER action in the second example.
- The **role audit option** audits the use of all system or object privileges granted directly to the role MGR\_ROLE, as in the third example.

You can configure privilege, action, and role audit options in the same audit policy, as shown in the first example.

Find the list of auditable system-wide options in the SYS.AUDITABLE\_SYSTEM\_ACTIONS table.

### Object-Wide Options

The second example creates an audit policy to audit an execute action on any procedural object, and any grant on the HR.RAISE\_SALARY\_PROC procedure. The object-level audit options are dynamic. That is, changes in these options become applicable for current and subsequent user sessions.

## Conditions and Evaluation

The audit policies can evaluate the condition per statement, once per session or once per instance. The third example in the slide creates an audit policy to audit any rename action on the `HR.EMPLOYEES` table, and any alter action on the `HR.JOBS` table, provided the user executing the audited statement is JIM. The condition is evaluated only once in the session.

## Step 2: Enabling/Disabling the Audit Policy

- Enable audit policies:

- Apply to all users.

```
SQL> AUDIT POLICY audit_syspriv_pol1;
```

- Apply only to some users.

```
SQL> AUDIT POLICY audit_pol2 BY scott, oe;
```

```
SQL> AUDIT POLICY audit_pol3 BY sys;
```

- Exclude some users.

```
SQL> AUDIT POLICY audit_pol4 EXCEPT jim, george;
```

- Audit the recording based on failed or succeeded actions.

```
SQL> AUDIT POLICY audit_syspriv_pol1 WHENEVER SUCCESSFUL ;
```

```
SQL> AUDIT POLICY audit_objpriv_pol2 WHENEVER NOT SUCCESSFUL ;
```

```
SQL> AUDIT POLICY auditpol5 BY joe WHENEVER SUCCESSFUL ;
```

- Disable audit policies by using the NOAUDIT command.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Audit Policy Enabling

The next step is to enable the audit policy by using the `AUDIT` statement.

All users are audited by default, except if you define a list of those audited.

- Apply the audit policy to one or more users by using the `BY` clause.  
The audit Administrator audits `SYS`-user-specific actions the same way as other user actions.
- Exclude some users by using the `EXCEPT` clause.  
You cannot have a `BY` list and an `EXCEPT` list in the same policy enablement statement.

Audit records are generated whether the user's actions failed or succeeded. If you want to audit actions only when the user's actions failed, use the `WHENEVER NOT SUCCESSFUL` clause. If you want to audit actions only when the user's actions succeeded, use the `WHENEVER SUCCESSFUL` clause. When you omit the `WHENEVER` clause, the statement is audited whether the action is successful or not, and the `RETURN_CODE` column displays whether the action succeeded or not.

### Audit Policy Disabling

To disable an audit policy, use the `NOAUDIT` command.

## Viewing the Audit Policy

- View the audit policy.

```
SQL> SELECT POLICY_NAME, AUDIT_OPTION, CONDITION_EVAL_OPT
       FROM   AUDIT_UNIFIED_POLICIES;
```

| POLICY_NAME | AUDIT_OPTION   | CONDITION_EVAL_OPT |
|-------------|----------------|--------------------|
| POL1        | DELETE         | INSTANCE           |
| POL2        | TRUNCATE TABLE | NONE               |
| POL3        | RENAME         | SESSION            |
| POL4        | ALL ACTIONS    | STATEMENT          |

- View the enabled audit policy.

```
SQL> SELECT POLICY_NAME, ENABLED_OPT, USER_NAME, SUCCESS, FAILURE
       FROM   AUDIT_UNIFIED_ENABLED_POLICIES;
```

| POLICY_NAME | ENABLED_ | USER_NAME | SUC | FAI |
|-------------|----------|-----------|-----|-----|
| POL2        | EXCEPT   | SYSTEM    | NO  | YES |
| POL4        | BY       | SYS       | YES | YES |
| POL6        | BY       | ALL USERS | YES | NO  |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Verify the list of audit policies created in the AUDIT\_UNIFIED\_POLICIES view.

- The CONDITION\_EVAL\_OPT column defines when the condition, if any, is evaluated.

Verify the list of enabled audit policies in the AUDIT\_UNIFIED\_ENABLED\_POLICIES view.

- The ENABLED\_OPT column defines if a list of audited users is defined with the BY value or an exception list of excluded users is defined with the EXCEPT value. The audited or excluded users are listed in the USER\_NAME column.
- The SUCCESS and FAILURE columns define if the policy generates audit records only when the user's actions succeed or fail.

## Using Predefined Audit Policies

```
SQL> select POLICY_NAME, AUDIT_OPTION
       from   AUDIT_UNIFIED_POLICIES
       where  policy_name in ('ORA_ACCOUNT_MGMT',
                             'ORA_DATABASE_PARAMETER',
                             'ORA_SECURECONFIG');
```

| POLICY_NAME            | AUDIT_OPTION                  |
|------------------------|-------------------------------|
| ORA_ACCOUNT_MGMT       | ALTER ROLE ...                |
| ORA_DATABASE_PARAMETER | ALTER DATABASE                |
| ORA_DATABASE_PARAMETER | ALTER SYSTEM ...              |
| ORA_SECURECONFIG       | ADMINISTER KEY MANAGEMENT ... |

- The ORA\_SECURECONFIG audit policy audits privileges that the Oracle Database 11g default audit setting audits.
- The ORA\_SECURECONFIG audit policy is enabled by default.
- ORA\_SECURECONFIG records all login events for all users.
- Exclude from ORA\_SECURECONFIG specific accounts.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In the Oracle Database 12c, there are three main predefined audit policies for operations, which are commonly audited and recommended under Oracle's Audit Best Practices. The \$ORACLE\_HOME/rdbms/admin/secconf.sql script creates the three predefined audit policies and enables the ORA\_SECURECONFIG audit policy only.

- The ORA\_ACCOUNT\_MGMT audit policy audits events for the user account and privilege management.
- The ORA\_DATABASE\_PARAMETER audit policy audits the changes made to the database parameters settings.
- The ORA\_SECURECONFIG audit policy audits the same list of privileges that Oracle Database 11g default audit setting does. In Oracle Database 11g, the \$ORACLE\_HOME/rdbms/admin/secconf.sql script sets the 11g default audits by using the AUDIT command to audit the same list of privileges as the Oracle Database 12c does by creating and enabling the ORA\_SECURECONFIG audit policy.

The users must enable or disable such precreated audit policies, based on their requirements.

Only ORA\_SECURECONFIG is enabled by default. Check in the AUDIT\_UNIFIED\_ENABLED\_POLICIES view. The ORA\_SECURECONFIG policy records all login events. This generates too many records in the audit storage. Exclude specific accounts like SYSMAN, DBSNMP accounts:

```
SQL> audit policy ora_secureconfig except SYSMAN;
```



## Including Application Context Data

| Context Namespace | Attribute / Value                  |
|-------------------|------------------------------------|
| CONTEXT_HR        | MyJob – Developer<br>MyLoc – Paris |
| CONTEXT_SALES     | MyRole – Manager                   |

- Use the `AUDIT CONTEXT ATTRIBUTES` command.

```
SQL> AUDIT CONTEXT NAMESPACE context_HR
      ATTRIBUTES MyJob, MyLoc
      CONTEXT NAMESPACE context_SALES
      ATTRIBUTES MyRole BY jim;
```

```
SQL> select APPLICATION_CONTEXTS from UNIFIED_AUDIT_TRAIL
      where APPLICATION_CONTEXTS is not null;
APPLICATION_CONTEXTS
-----
(context_HR,MyJob=Manager); (context_HR,MyLoc=Paris)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Capturing Application Context Information in the Unified Audit Trail

Applications set various contexts before executing actions on the database server, storing information like Module Name and Client\_ID. These values are typically stored in application contexts. An application context is an attribute name and attribute value pairs in a namespace, and each pair can be appended in audit data.

The audit administrator can configure application contexts and their attributes that must be captured in audit data by using the `AUDIT CONTEXT NAMESPACE ATTRIBUTES` command.

To display the audited application context data, select from the `UNIFIED_AUDIT_TRAIL` view.

To list the application contexts audited, use the `AUDIT_UNIFIED_CONTEXTS` view:

```
SQL> select * from audit_unified_contexts;
NAMESPACE          ATTRIBUTE          USER_NAME
-----
CONTEXT_HR          MYJOB             ALL_USERS
CONTEXT_HR          MYJOB             ALL_USERS
CONTEXT_SALES MYROLE             JIM
```

To disable an application context audit setting, execute the `NOAUDIT` statement.

## Dropping the Audit Policy

1. Disable the audit policy.

```
SQL> NOAUDIT POLICY audit_syspriv_pol1;
```

2. Drop the audit policy.

```
SQL> DROP AUDIT POLICY audit_syspriv_pol1;
```

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To drop a unified audit policy, first disable it, and then run the `DROP AUDIT POLICY` statement to remove it. If a unified system-related audit policy is already enabled for a session, the effect of dropping a system-related audit policy is not seen by this existing session. Until that time, the unified system-related audit policy's settings remain in effect. However, the effect is immediate for object-related unified audit policies.

In a CDB environment, you can drop a common audit policy only from `root` and a local audit policy only from the PDB to which it applies.

## Audit Cleanup

Clean up records in `SYS.UNIFIED_AUDIT_TRAIL`:

- Schedule an automatic purge job.

```
DBMS_AUDIT_MGMT.CREATE_PURGE_JOB
(AUDIT_TRAIL_TYPE=> DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
 AUDIT_TRAIL_PURGE_INTERVAL => 12,
 AUDIT_TRAIL_PURGE_NAME => 'Audit_Trail_PJ',
 USE_LAST_ARCH_TIMESTAMP => TRUE,
 CONTAINER => DBMS_AUDIT_MGMT.CONTAINER_CURRENT);
```

- Manually purge the audit records.

```
DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
  AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To perform the audit trail purge tasks, you use the `DBMS_AUDIT_MGMT` package. You must have the `AUDIT_ADMIN` role to use the package. By mandate, Oracle Database audits all executions of the `DBMS_AUDIT_MGMT` package procedures.

The `DBMS_AUDIT_MGMT.CREATE_PURGE_JOB` procedure owns a new attribute `CONTAINER`. Setting `CONTAINER` to `CONTAINER_CURRENT` deletes audit trail records for the current pluggable database. Setting `CONTAINER` to `CONTAINER_ALL` deletes audit trail records for all pluggable databases creating a job in the root and the invocation of this job will invoke cleanup in all the PDBs.

Specifies whether the last archived time stamp should be used for deciding on the records that should be deleted.

Setting `USE_LAST_ARCH_TIMESTAMP` to `TRUE` indicates that only audit records created before the last archive time stamp should be deleted. A value of `FALSE` indicates that all audit records should be deleted. The default value is `TRUE`. Oracle recommends using this value, as this helps guard against inadvertent deletion of records.

You can automate the cleanup process by creating and scheduling a cleanup purge job, or you can manually run a cleanup purge job. If you manually run cleanup purge jobs, use the `DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL` procedure with a new type value of `DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED`.

## Quiz

Select the view that you use to view the enabled audit policies.

- a. DBA\_UNIFIED\_AUDIT\_POLICIES
- b. UNIFIED\_AUDIT\_ENABLED\_POLICIES
- c. UNIFIED\_AUDIT\_POLICIES
- d. AUDIT\_UNIFIED\_ENABLED\_POLICIES
- e. UNIFIED\_AUDIT\_TRAIL

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: d**

## Summary

In this lesson, you should have learned how to:

- Implement and use the unified audit trail
- Enable unified auditing
- Configure the unified auditing
- Create and enable audit policies
- Add application context data to audit records
- View data in the unified audit trail
- Clean up the unified audit trail

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 10-1: Enabling unified auditing
- 10-2: Auditing RMAN operations
- 10-3: Auditing SYS and end-users (*optional*)
- 10-4: Excluding DBSNMP login events (*optional*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# 11

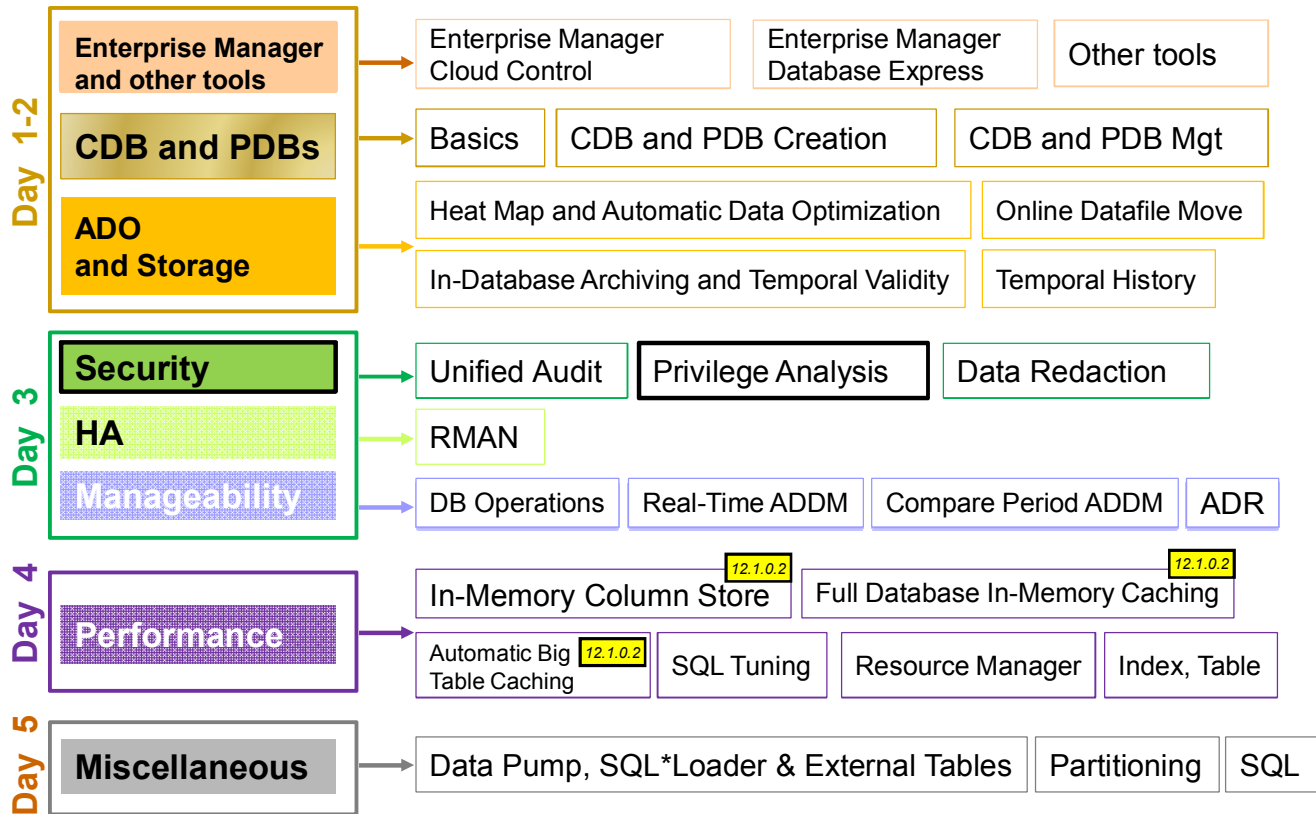
## Privileges

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Privilege Analysis** identifies unused privileges to be revoked.

**Other privileges enhancements** introduce new administrative system privileges, which are more task-oriented to allow separation of duty.

# Objectives

After completing this lesson, you should be able to:

- Implement separation of duty for database administration tasks
- Describe new administrative privileges
- Describe OS groups for administrative privileges
- Create the password file to support new privileges
- Explain the new system privilege `PURGE DBA_RECYCLEBIN`
- Use database privilege analysis
- Create and enable privilege analysis
- Control privileges for invoker's rights procedures
- Manage the invoker's rights behavior for views

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Note

For a complete understanding of the Privilege Analysis new feature and the new administrative privileges and other privileges usage, refer to the following guide in the Oracle documentation:

- *Oracle Database Security Guide 12c Release 1 (12.1)*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – “DBMS\_PRIVILEGE\_CAPTURE” chapter*

Refer to other sources of information available under Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series* demonstrations:
  - *Administrative Privileges*
  - *Privileges Capture*
  - *INHERIT PRIVILEGES Privilege and Invokers' Rights Procedures*
  - *INHERIT PRIVILEGES Privilege and Current\_User Views*
- *Oracle By Example (OBE):*
  - *Determining Least Privilege Access Using Privilege Analysis*

## Major Challenges

- **Separation of duty:** Administration tasks rely on SYSDBA.
  - New task-specific privileges for standard administrative tasks
    - Oracle RMAN administrator: backup and recovery
    - Data Guard administrator
    - Key management administrator: TDE keystore management
  - Security administrators set with Oracle Database Vault
    - DV owner and DV account manager
  - No change in SYSDBA privilege
  - Administrative privileges mandatorily audited
  - New system privilege: PURGE DBA\_RECYCLEBIN
- **Security:**
  - Identify unused privileges to revoke: Privilege Analysis
  - Check new privilege during invoker's rights procedures call
  - Check new privilege during invoker's rights views call

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

As one of the fundamental security requirements, the principle of separation of duty dictates that completion of any critical task has an associated role. In Oracle Database 11g, the administration of Oracle Database depends heavily on the SYSDBA administrative privilege.

For better separation of duty, Oracle Database now provides task-specific privileges to handle standard administration duties for Oracle Recovery Manager (Oracle RMAN), Oracle Data Guard, and transparent data encryption (TDE). The new privileges are based on the least privilege principle, in which a user is granted only the absolutely necessary privileges required to perform a specific function, and no more. This feature alleviates the need to unnecessarily grant the SYSDBA administrative privilege for many tasks. SYSDBA is necessary for critical situations, such as installation, upgrade, and urgent recovery.

The separation of duty improves database security by maintaining the privacy and the compliance requirements for data.

To achieve the least privilege principle, you must identify and revoke unnecessary privileges. Privilege Analysis helps identify used and unused privileges. The invoker's rights procedure calls let the invoker use all granted privileges during the procedure call. A new privilege gives invoking users control over who can access their privileges when they run an invoker's rights procedure. The same principle will apply to BEQUEATH CURRENT\_USER views covered later in the lesson.

## New Administrative Privileges

| Administrative Privilege | Username                                                                                           | Tasks                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| SYSDBA, SYSOPER          | SYS / PUBLIC                                                                                       | Same operations as in 11g                                             |
| SYSASM                   | SYS                                                                                                | Specific to ASM instances only                                        |
| <b>SYSBACKUP</b>         | <b>SYSBACKUP</b>  | Perform RMAN backup and recovery operations from RMAN or through SQL. |
| <b>SYSDG</b>             | <b>SYSDG</b>      | Perform Data Guard operations with Data Guard Broker or DGMGRL.       |
| <b>SYSKM</b>             | <b>SYSKM</b>      | Manage transparent data encryption keystore operations.               |

- Distinction between administrative privileges (**SYSBACKUP**) and predefined users (**SYSBACKUP**) 
- **SYSBACKUP**, **SYSDG**, **SYSKM** users cannot be dropped.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

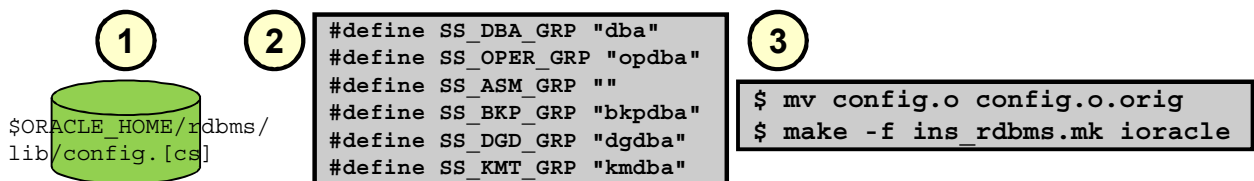
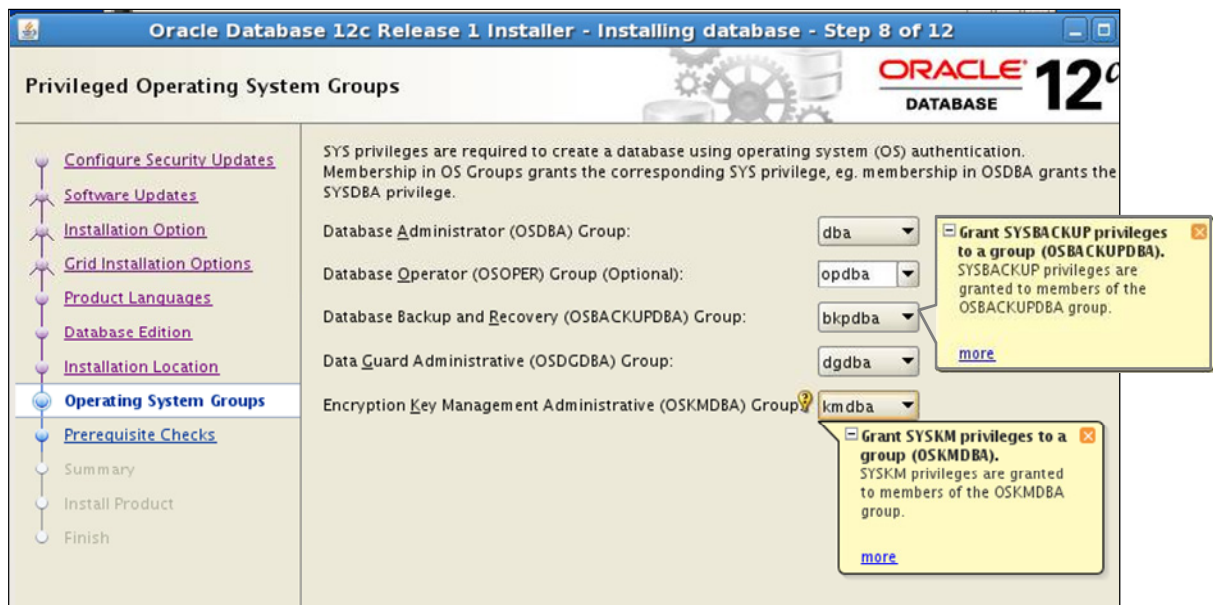
Oracle Database 12c introduces new administrative privileges that are more task-specific and least privileged to support specific administrative tasks. The SYSDBA administrative privilege is not altered for backward compatibility. However, the connection with the SYSDBA administrative privilege should be limited to major operations such as installation and upgrade.

- The SYSBACKUP administrative privilege allows you to perform Oracle RMAN backup and recovery operations from Oracle RMAN or through SQL, STARTUP and SHUTDOWN and other operations.
- The SYSDG administrative privilege allows you to perform Data Guard operations with Data Guard Broker or the DGMGRL command-line interface, and STARTUP and SHUTDOWN and other operations.
- The SYSKM administrative privilege allows you to manage transparent data encryption keystore operations.

These privileges enable you to connect to the database even if the database is not open. After connecting with one of these privileges, you are connected under a predefined user whose name is the privilege name. Each privilege is tied to a specific user.

```
SQL> connect / as SYSBACKUP
SQL> show user
USER is "SYSBACKUP"
```

# OS Authentication and OS Groups



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In OS authentication, the groups are created and assigned specific names as part of the database installation process, provided that OS UNIX or Windows groups are created and accounts are assigned to the appropriate operating system-defined groups.

Membership in a UNIX or Windows group affects your connection to the database as follows:

- If you are a member of the OSBACKUP group, and you specify AS SYSBACKUP when you connect to the database, you connect to the database with the SYSBACKUP administrative privilege under the SYSBACKUP user.
- If you are a member of the OSDG group, and you specify AS SYSDG when you connect to the database, you connect to the database with the SYSDG administrative privilege under the SYSDG user.
- If you are a member of the OSKM group, and you specify AS SYSKM when you connect to the database, you connect to the database with the SYSKM administrative privilege under the SYSKM user.

Without being a member of these OS groups, users will not be able to connect as administrative users using the OS authentication. That is, “CONNECT / AS SYSDBA” will fail. However, the users can still connect using other authentication mechanisms (for example, network, password, or directory based authentication). To change the OS group names (as shown in step 1 and 2 on the slide), ensure that you are using the groups defined in the \$ORACLE\_HOME/rdbms/lib/config.[cs] file. Next, shut down all databases, and then relink the Oracle executable as shown in step 3 on the slide.

### **Windows User Groups**

The Windows user groups are ORA\_%HOMENAME%\_SYSBACKUP, ORA\_%HOMENAME%\_SYSDG, and ORA\_%HOMENAME%\_SYSKM. These user groups cannot be changed.

# Password Authentication for SYSBACKUP

JOHN entry is not SYSBACKUP.

- Create a new password file with format 12:

```
$ orapwd file=orapwDBSEC password=x entries=10 format=12
```

- Grant the SYSBACKUP privilege to the user.

```
SQL> GRANT SYSBACKUP to JOHN;
```

```
SQL> SELECT USERNAME, SYSDBA, SYSOPER, SYSBACKUP FROM V$PWFIL_USERS;
```

| USERNAME | SYSDBA | SYSOP | SYSBA |
|----------|--------|-------|-------|
| -----    | -----  | ----- | ----- |
| SYS      | TRUE   | TRUE  | FALSE |
| JOHN     | FALSE  | FALSE | TRUE  |

- Connect AS SYSBACKUP.

```
$ rman target '"/JOHN@DBSEC AS SYSBACKUP"'
RMAN>
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The password authentication still requires the `REMOTE_LOGIN_PASSWORDFILE` instance parameter set to `EXCLUSIVE` and a password file.

New parameters during the creation of the password file define new formats and supported new administrative privileges.

- **FORMAT:** If 12, the default, the password file is created in a version12.x format and supports SYSBACKUP, SYSDG, and SYSKM administrative privileges. If `legacy`, the password file is created in legacy format, which is the format prior to Oracle Database 12c. You cannot set `FORMAT` to legacy when you specify the SYSBACKUP or the SYSDG argument.
- **SYSBACKUP:** If it is set to `y`, it creates a SYSBACKUP user entry in the password file, and you are prompted for the SYSBACKUP user password. The SYSBACKUP user password is stored in the created password file.
- **SYSDG:** If it is set to `y`, it creates a SYSDG entry in the password file, and you are prompted for the SYSDG password. The SYSDG password is stored in the created password file.
- **INPUT\_FILE:** Name of the input password file. ORAPWD migrates the entries in the input file to a new password file. This argument can convert a password file from legacy format to 12 format.

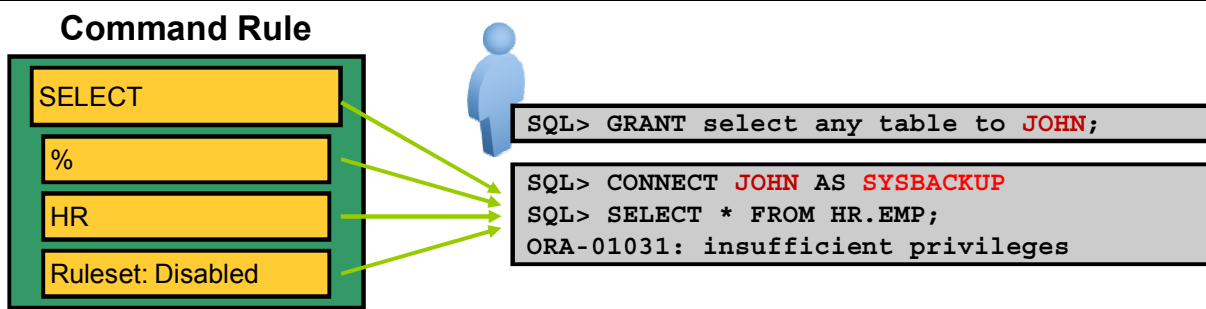
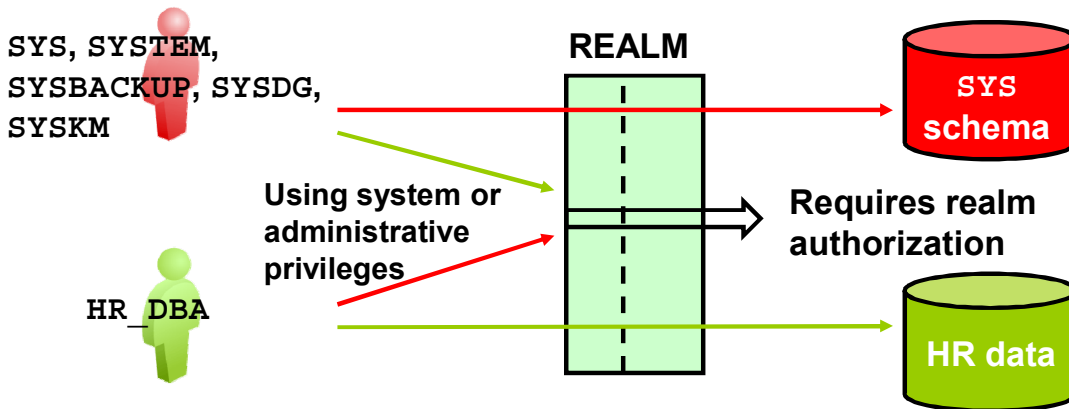
It is recommended not to use the `SYSDATABASE` user to connect as `SYSDATABASE`. The DBAs that are responsible for backup/recovery would all share the same password. This is undesirable first in terms of accountability. Also, it would be impossible to take away this privilege from particular DBAs later (unless changing the password). It is better to create a database account designated for each DBA and grant the `SYSDATABASE` privilege if necessary. In this way, no password is shared, and `SYSDATABASE` can be revoked from any DBA without affecting other DBAs.

If you are working in a multitenant container database, each PDB of the CDB can create a local user and grant the user the `SYSDATABASE` privilege. The user can connect to the PDB and perform the backups for the PDB and only for the PDB he can connect to.

```
$ rman TARGET '"tim/password@pdb1 AS SYSDATABASE"'
```



# Oracle Database Vault Data Protection and Administration Privileged Users



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## SYSBACKUP, SYSDG, SYSKM Privileges and Oracle Database Vault Protection

Oracle Database Vault provides two major mechanisms for enforcing separation of duty:

- Realms are a functional grouping of schemas, schema objects, or roles. They prevent users with system privileges from accessing the protected objects. For instance, users granted the `SELECT ANY TABLE` system privilege are not allowed to select a realm-secured table unless they are authorized for the realm.
- Command rules control the execution of a particular SQL command at run time. Specifically, a command that is protected by a command rule executes successfully only if the associated ruleset evaluates to `TRUE`.

Users with new administrative privileges are not exempt from the enforcement of realms or command rules.

To authorize users with the `SYSBACKUP` administrative privilege to access protected objects in a particular realm, the user needs to be authorized for the realm.

The example in the slide defines a command rule that prevents any `SELECT` on `HR` objects with the `Disabled` ruleset, including users with `SYSBACKUP` administrative privilege. This command rule applies even if the user was granted the `SELECT ANY TABLE` system privilege.

## Quiz

Which user is connected when using the `CONNECT TARGET /` statement in Recovery Manager?

- a. SYSBACKUP
- b. SYS

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Answer: b

The `AS SYSBACKUP` clause is required if you want to be connected as `SYSBACKUP` user.

## Quiz

The password file now has two formats to handle new administrative privileges.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## New System Privilege: PURGE DBA\_RECYCLEBIN

Purging all recycled objects:

- Requires the SYSDBA privilege

```
SQL> CONNECT / AS SYSDBA
SQL> purge dba_recyclebin ← SQL Command
```

Pre-12c

- Does not require the SYSDBA privilege
- Uses a new system privilege: PURGE DBA\_RECYCLEBIN

```
SQL> CONNECT system
SQL> grant PURGE DBA_RECYCLEBIN to dba_junior;
```

```
SQL> CONNECT dba_junior
SQL> purge dba_recyclebin;
DBA Recyclebin purged.
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

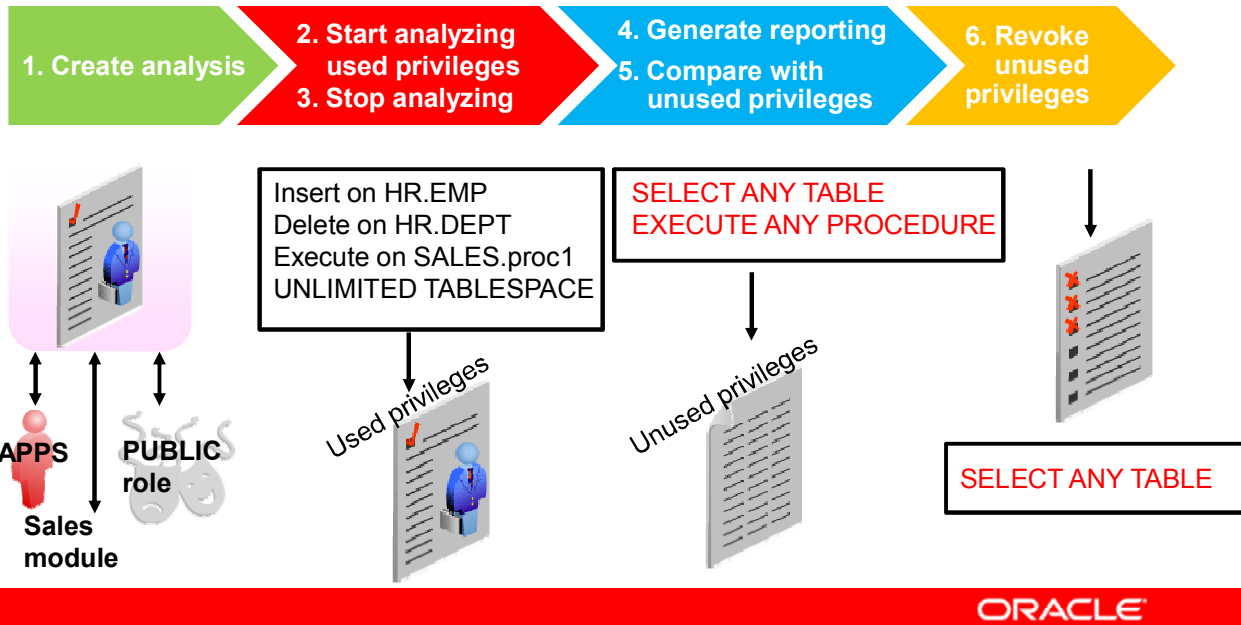
In Oracle Database 11g, the DBA uses the PURGE command to purge all objects. This operation requires the SYSDBA system privilege, which is highly undesirable in terms of separation of duty and least privilege.

In Oracle Database 12c, the same operation does not require the SYSDBA system privilege. It requires only the new PURGE DBA\_RECYCLEBIN system privilege.

# Privilege Analysis

Increase database security: Revoke unused privileges

- Analyze used privileges to revoke unnecessary privileges.
- Use new package: DBMS\_PRIVILEGE\_CAPTURE



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Excessive Privileges Challenge

In addition to concern about the SYSDBA privilege usage, another major concern is that existing database and application users have excessive privileges. Excessive privileges violate the principle of least privilege.

To achieve the least privilege principle, unused privileges need to be identified.

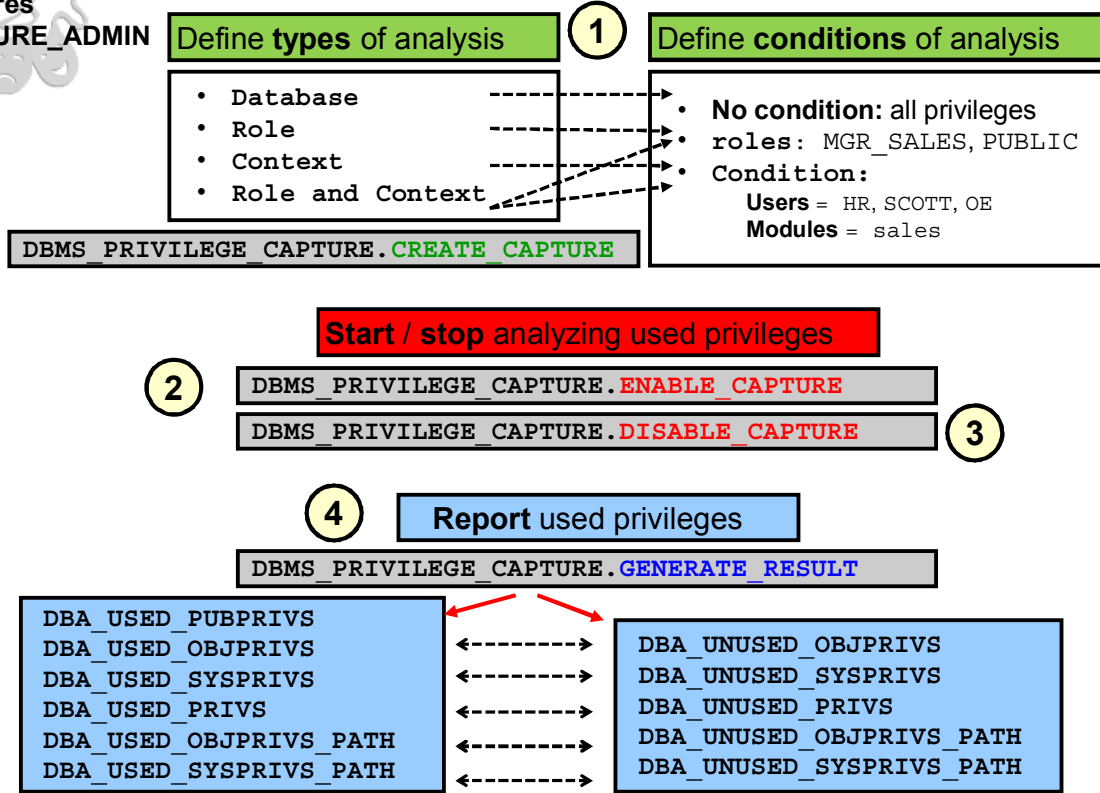
## Privilege Analysis

Oracle Database 12c offers a new package to analyze used privileges. Privilege Analysis requires Oracle Database Vault, which is a licensed option.

- You can use a privilege analysis policy to identify object and system privileges used to run an application module or to execute certain SQL statements or privileges used by defined roles.
- You can generate reports of used and unused privileges during the analysis period.
- The report helps the security officer revoke unnecessary privileges by comparing the used and unused granted privileges lists.

# Privilege Analysis Flow

Requires  
**CAPTURE\_ADMIN**  
role



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Privilege Analysis Types and Conditions

When creating an analysis, you first define the targeted objects to be analyzed in used privileges. You do that by setting the type of analysis:

- **Database analysis:** If no condition is given, it analyzes the used privileges (except privileges used by administrative users) within the whole database.
- **Role analysis:** If roles are defined, it analyzes the privileges exercised through any given role. For example, if you create a privilege analysis policy to analyze on PUBLIC, the privileges that are directly and indirectly granted to PUBLIC are analyzed when they are used.
- **Context-specific analysis:** If the contexts are defined, it analyzes the privileges that are used through a given application module or specified contexts.

Different conditions can be combined with “AND” and/or “OR” Boolean operators.

Because the created policy is not enabled by default, your next step is to enable the policy to start analyzing used privileges. After a certain time, you stop analyzing. Be aware that the collected information is stored in the SYSAUX tablespace. If the captures run over a longer period, the SYSAUX tablespace increases.

## Reporting Used Privileges

Your third step is to generate a report. Reporting includes two types of results:

- Used privileges visible in `DBA_USED_xxx` and `DBA_USED_xxx_PATH` views
- Unused privileges visible in `DBA_UNUSED_xxx` and `DBA_UNUSED_xxx_PATH` views

## Used Privileges Results

Compare with  
granted unused  
privileges.

- View SYSTEM and OBJECT used privileges:

```
SQL> select USERNAME, SYS_PRIV, OBJ_PRIV, OBJECT_OWNER, OBJECT_NAME
       from DBA_USED_PRIVS ;
```

| USERNAME | SYS_PRIV       | OBJ_PRIV | OBJECT_OWNER | OBJECT_NAME |
|----------|----------------|----------|--------------|-------------|
| JIM      |                | SELECT   | HR           | EMPLOYEES   |
| JIM      |                | DELETE   | HR           | EMPLOYEES   |
| TOM      | CREATE SESSION |          |              |             |
| TOM      |                | SELECT   | SH           | SALES       |
| OE       |                | UPDATE   | HR           | DEPARTMENTS |

- View the path for OBJECT used privileges:

```
SQL> select USERNAME, OBJ_PRIV, OBJECT_NAME, PATH
       from DBA_USED_OBJPRIVS_PATH where username in ('TOM','JIM')
```

| USERNAME | OBJ_PRIV | OBJECT_NAME | PATH                             |
|----------|----------|-------------|----------------------------------|
| OE       | UPDATE   | DEPARTMENTS | GRANT_PATH('OE')                 |
| JIM      | DELETE   | EMPLOYEES   | GRANT_PATH('JIM', 'HR_MGR')      |
| JIM      | SELECT   | EMPLOYEES   | GRANT_PATH('JIM', 'HR_MGR')      |
| TOM      | SELECT   | SALES       | GRANT_PATH('TOM', 'SALES_CLERK') |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

When you generate the analysis results, SYSTEM and OBJECT used privileges populate the views like DBA\_USED\_SYSPRIVS and DBA\_USED\_OBJPRIVS.

You can view all SYSTEM and OBJECT used privileges in the DBA\_USED\_PRIVS view.

If you need to know how the privileges were granted to the users, display the PATH column from DBA\_USED\_OBJPRIVS\_PATH and DBA\_USED\_SYSPRIVS\_PATH.

The second example reveals the following information:

- OE updated rows from the DEPARTMENTS table because he is directly granted the OBJECT privilege UPDATE.
- JIM deleted rows from the EMPLOYEES table because he is granted the OBJECT privilege DELETE through the HR\_MGR role.
- JIM selected rows from the EMPLOYEES table because he is granted the OBJECT privilege SELECT through the HR\_MGR role.
- TOM selected rows from the SALES table because he is granted the OBJECT privilege SELECT through the SALES\_CLERK role.



# Compare Used and Unused Privileges

Compare with  
granted unused  
privileges.

- View SYSTEM and OBJECT **used** privileges:

```
SQL> select USERNAME, SYS_PRIV, OBJ_PRIV, OBJECT_OWNER, OBJECT_NAME
       from DBA_USED_PRIVS where username='JIM';
```

| USERNAME | SYS_PRIV | OBJ_PRIV | OBJECT_OWNER | OBJECT_NAME |
|----------|----------|----------|--------------|-------------|
| JIM      |          | SELECT   | HR           | EMPLOYEES   |
| JIM      |          | DELETE   | HR           | EMPLOYEES   |

- View SYSTEM and OBJECT **unused** privileges:

```
SQL> select USERNAME, OBJ_PRIV, OBJECT_NAME, PATH
       from DBA_UNUSED_PRIVS where username='JIM';
```

| USERNAME | OBJ_PRIV | OBJECT_NAME | PATH                       |
|----------|----------|-------------|----------------------------|
| JIM      | INSERT   | EMPLOYEES   | GRANT_PATH('JIM','HR_MGR') |
| JIM      | UPDATE   | EMPLOYEES   | GRANT_PATH('JIM','HR_MGR') |

Revoke  
unused  
privileges.

- Decide if unused privileges need to be revoked.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can view all unused SYSTEM and OBJECT privileges granted in the DBA\_UNUSED\_PRIVS view.

If you compare the list of used and unused privileges, you can identify the privileges that are granted but are not used, and you can decide whether to revoke the unused privileges.

For the example in the slide, JIM used the SELECT and DELETE privileges on the HR.EMPLOYEES table, and he did not use INSERT or UPDATE on the same table. The INSERT and UPDATE privileges are granted through the HR\_MGR role.

## Listing Captures

```
SQL> select NAME, TYPE, ENABLED, ROLES, CONTEXT
       from   DBA_PRIV_CAPTURES;
```

| NAME               | TYPE             | ENABLED | ROLES                  | CONTEXT                                                                                     |
|--------------------|------------------|---------|------------------------|---------------------------------------------------------------------------------------------|
| All_privs          | DATABASE         | N       |                        |                                                                                             |
| ORA\$DEPENDENCY    | DATABASE         | N       |                        |                                                                                             |
| Public_privs       | ROLE             | N       | ROLE_ID_LIST(1)        |                                                                                             |
| HR_SH_privs        | ROLE             | Y       | ROLE_ID_LIST(112, 113) |                                                                                             |
| Privs_HR_OE_logged | CONTEXT          | N       |                        | SYS_CONTEXT('USERENV','SESSION_USER')='HR' OR<br>SYS_CONTEXT('USERENV','SESSION_USER')='OE' |
| HR_Sales_role      | ROLE_AND_CONTEXT | N       | ROLE_ID_LIST(113)      | SYS_CONTEXT('USERENV','SESSION_USER')='HR'                                                  |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To view the list of created analyses, use the DBA\_PRIV\_CAPTURES dictionary view. The TYPE column can hold four values: DATABASE, ROLE, CONTEXT, and ROLE\_AND\_CONTEXT. The ENABLED column is set to Y when the analysis is analyzing used privileges. The ROLES column holds the list of roles, and the CONTEXT column holds the condition. The roles and the condition are defined in the creation of the analysis policy.

The ORA\$DEPENDENCY capture is a default capture that will be generated if a DATABASE wide capture is started and results are generated. It contains all the privileges used for compiling dependency objects.

If the database is a CDB, you create, start, stop, and generate reports in the PDB that you are connected to.

This means that an analysis collects information from the sessions of either the root or a PDB where you created and started the analysis policy. It does not collect information for all containers during an analysis.

# Dropping an Analysis

## 1. Disable the analysis.

```
SQL> exec dbms_privilege_capture.DROP_CAPTURE('Capture1')
BEGIN dbms_privilege_capture.DROP_CAPTURE('Capture1'); END;

*
ERROR at line 1:
ORA-47932: Privilege capture Capture1 is still enabled.
ORA-06512: at "SYS.DBMS_PRIVILEGE_CAPTURE", line 82
ORA-06512: at line 1

SQL> exec dbms_privilege_capture.DISABLE_CAPTURE('Capture1')
PL/SQL procedure successfully completed.
```

## 2. Drop the analysis.

```
SQL> exec dbms_privilege_capture.DROP_CAPTURE('Capture1')

PL/SQL procedure successfully completed.
```

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To drop an analysis, first disable the policy if it was already started. Dropping a policy also drops all used and unused privilege records that are associated with this privilege policy.

## Quiz

To revoke unnecessary and unused privileges granted, use the Privilege Analysis. Is the sequence in the proper order? True or False?

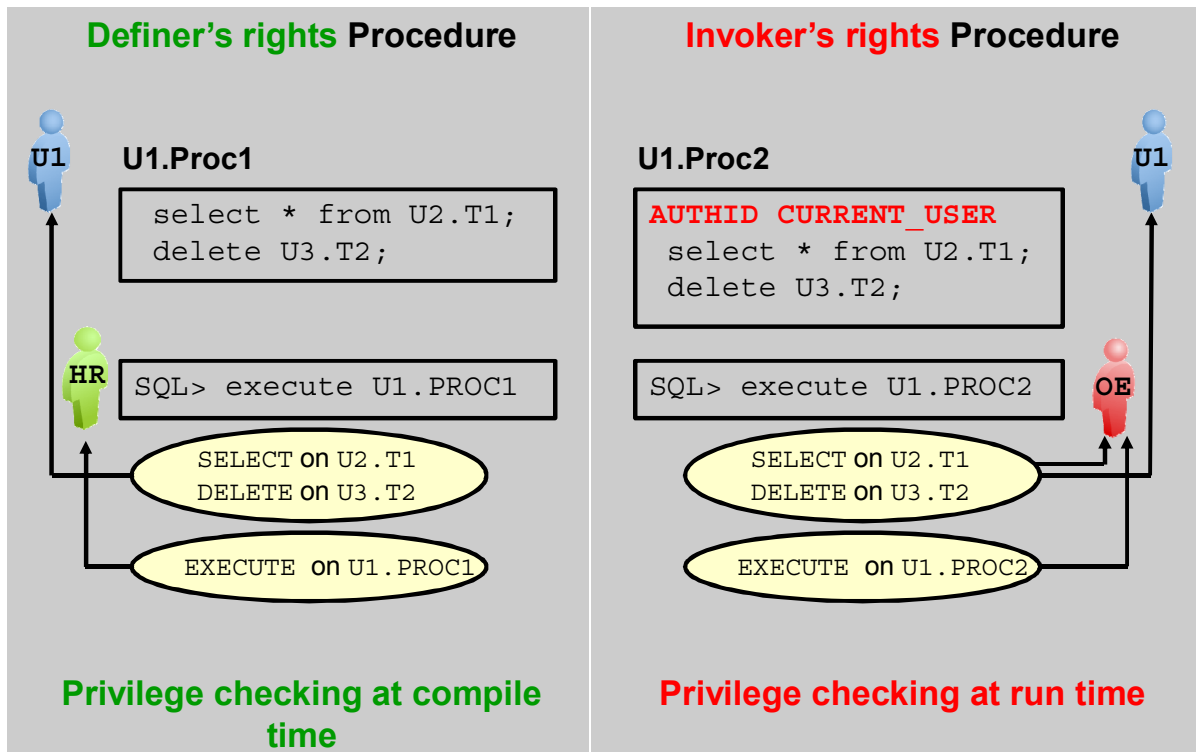
- a. Set up the analysis policy type (database, role, context).
- b. Start the analysis.
- c. Stop the analysis.
- d. Generate the results.
- e. View the results in `DBA_USED_PRIVS` and `DBA_UNUSED_PRIVS`.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: True**

# Privilege Checking During PL/SQL Calls



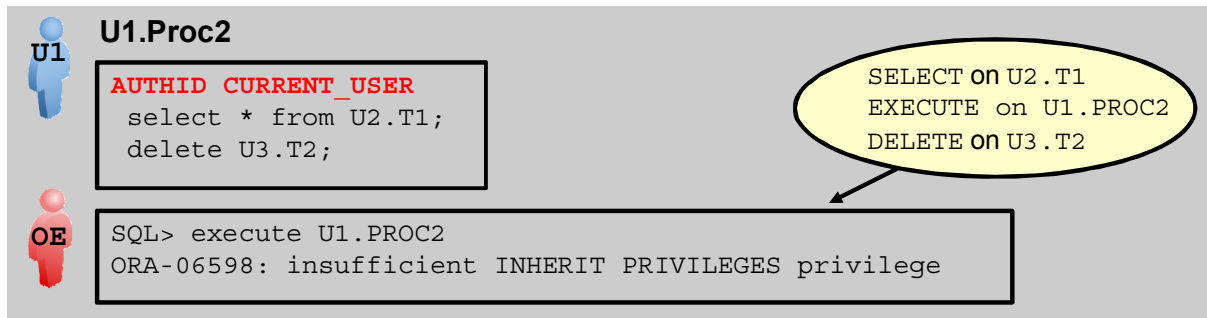
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In Oracle Database 11g, the AUTHID property of a stored PL/SQL unit affects the name resolution and privilege checking of SQL statements that the unit issues at runtime.

- A unit whose AUTHID value is DEFINER (the default) is called a definer's rights unit. The invocation of U1 . PROC1 by HR succeeds because the procedure runs with the privileges of its owner (U1).
- A unit whose AUTHID value is CURRENT\_USER is called an invoker's rights unit. The invocation of U1 . PROC2 by OE succeeds because the procedure runs with the privileges of the current user (OE), not the owner (U1). In this case, the privilege checking is performed at runtime.

# New Privilege Checking During PL/SQL Calls



- Additional required privilege checking at runtime:
  - Of a database user passing into an AUTHID CURRENT\_USER PL/SQL routine
  - Of a database user passing through an AUTHID CURRENT\_USER “callspec” over a C or Java routine
- INHERIT PRIVILEGES object privilege
- INHERIT ANY PRIVILEGES system privilege

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Risks

A low privilege user could own an invoker's rights procedure that could potentially perform unintended or malicious actions if it is executed by a high-privileged user.

An invoker's right procedure can perform inappropriate actions if it is invoked by another procedure that does not expect an invoker's rights procedure.

In Oracle Database 11g, the caller to a procedure had no control over who accessed the caller's privileges. Only the owner of the procedure controlled the right's inheritance.

## New Privilege Checking

Privilege checking in Oracle Database 12c implements a new restriction, not a new power. Existing cases that did not require a privilege check now require one. When a user runs an invoker's rights procedure, Oracle Database checks the procedure owner's privileges before initiating or running the code. The owner must have the INHERIT PRIVILEGES object privilege on the invoking user or the INHERIT ANY PRIVILEGES privilege. If this is not the case, the runtime system raises an error.

The session is temporarily switched into an environment that treats the entered routine as the definer's rights. It then checks that it has the INHERIT PRIVILEGES object privilege on the caller's active current user or that it has the INHERIT ANY PRIVILEGES system privilege. The session then reverts to its prior environment. This treatment of the routine as definer's rights mirrors the treatment of the routine during compilation.

## INHERIT (ANY) PRIVILEGES Privileges

- Invoking users can control who can access their privileges when they run an invoker's rights procedure.
- Invoking users grant the INHERIT PRIVILEGES object privilege only to **trusted users** (object is a **USER**).

```
SQL> CONNECT oe
SQL> grant INHERIT PRIVILEGES ON USER oe TO u1;
```

```
SQL> select PRIVILEGE, TYPE, TABLE_NAME, GRANTEE
       from DBA_TAB_PRIVS where grantee='U1';
```

| PRIVILEGE          | GRANTEE | TYPE        | TABLE_NAME | GRANTEE   |
|--------------------|---------|-------------|------------|-----------|
| INHERIT PRIVILEGES |         | <b>USER</b> | OE         | <b>U1</b> |

- A new user trusts ANY user invoking rights procedures by default.
- INHERIT ANY PRIVILEGES means on all users.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### INHERIT PRIVILEGES Object Privilege

The benefit of this privilege is that it gives invoking users control over who can access their privileges when they run an invokers' rights procedure.

```
SQL> CONNECT invoking_user
```

```
SQL> GRANT INHERIT PRIVILEGES ON USER inv_user TO proc_owner;
```

A new user trusts ANY user invoking rights procedures by default because PUBLIC is granted the INHERIT PRIVILEGES on PUBLIC, and therefore any new user.

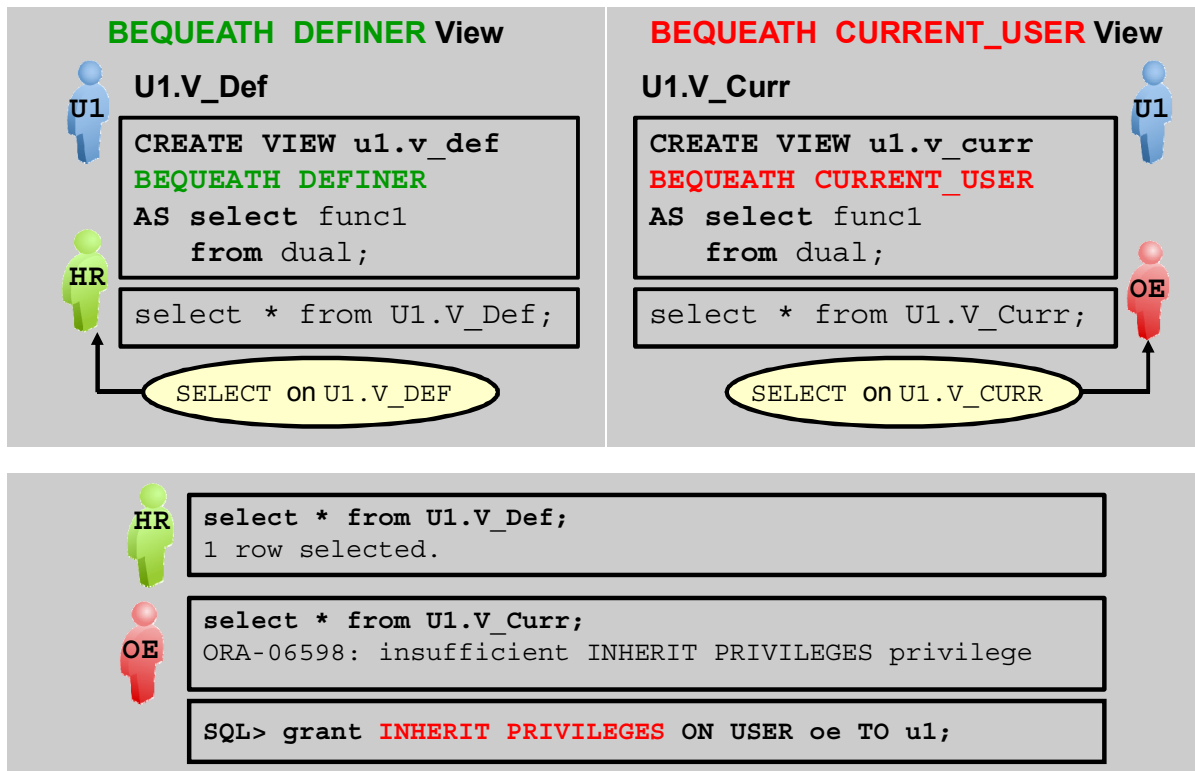
```
SQL> select PRIVILEGE, TYPE, TABLE_NAME, GRANTEE
       from DBA_TAB_PRIVS where type='USER' and table_name='X';
```

| PRIVILEGE          | TYPE | TABLE_NAME | GRANTEE |
|--------------------|------|------------|---------|
| INHERIT PRIVILEGES | USER | X          | PUBLIC  |

### INHERIT ANY PRIVILEGES System Privilege

A user being granted the INHERIT ANY PRIVILEGES system privilege inherits privileges on all users.

# Privilege Checking with New BEQUEATH Views



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## BEQUEATH Views

Oracle Database 12c introduces `BEQUEATH CURRENT_USER` views, which bring a security benefit to objects of this type similar to stored PL/SQL units. These views partially behave like invoker's rights rather than owner's rights. That is, when you call an `AUTHID CURRENT_USER` function or an invoker's rights PL/SQL or Java function, the current schema, current user, and currently enabled roles within the operation's execution can be inherited from the querying user's environment. `BEQUEATH CURRENT_USER` views are only a subset of the behavior of invoker's rights in this release.

The `BEQUEATH CURRENT_USER` views are in contrast to the `BEQUEATH DEFINER` behavior of existing views.

The `BEQUEATH` view type is displayed in a new `BEQUEATH` column in `DBA_VIEWS`.

## INHERIT PRIVILEGES and BEQUEATH CURRENT\_USER Views

The owner of a `BEQUEATH CURRENT_USER` view must have the `INHERIT PRIVILEGES` object privilege on the invoking user or the `INHERIT ANY PRIVILEGES` system privilege.



## Quiz

Select the statement that is true when you execute an invoker's rights procedure.

- a. The calling user must be granted the `INHERIT PRIVILEGES` object privilege on the user owner of the procedure.
- b. The calling user must grant the `INHERIT PRIVILEGES` object privilege on the user owner of the procedure.
- c. The owner of the procedure must grant the `INHERIT PRIVILEGES` object privilege on the calling user.
- d. The owner of the procedure must be granted the `INHERIT PRIVILEGES` object privilege on the calling user.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is centered within a solid red rectangular bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: d**

## Quiz

When you select from a `BEQUEATH CURRENT_USER` view, select the statement that is true when you call an `AUTHID CURRENT_USER` function.

- a. The current schema, user, and enabled roles are inherited from the querying user's environment.
- b. The current schema, user, and enabled roles are set to the owner of the view.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Summary

In this lesson, you should have learned how to:

- Use new administrative privileges like `SYSBACKUP`
- Create the password file to support new privileges
- Use new system privilege `PURGE DBA_RECYCLEBIN`
- Use database privilege analysis
- Create and enable privilege analysis
- View capture results to decide to revoke unused privileges
- Control privileges for invoker's rights procedures
- Use `INHERIT PRIVILEGES` privilege
- Create new `BEQUEATH` views
- Manage the invoker's rights behavior for `BEQUEATH` views

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice

- 11-1: Managing password file with SYSBACKUP entry
- 11-2: Capturing privileges
- 11-3: Capturing privileges through roles (*optional*)
- 11-4: Capturing privileges used in contexts (*optional*)
- 11-5: Using INHERIT PRIVILEGES object privilege on AUTHID CURRENT\_USER procedures (*optional*)
- 11-6: Using BEQUEATH CURRENT\_USER views (*optional*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



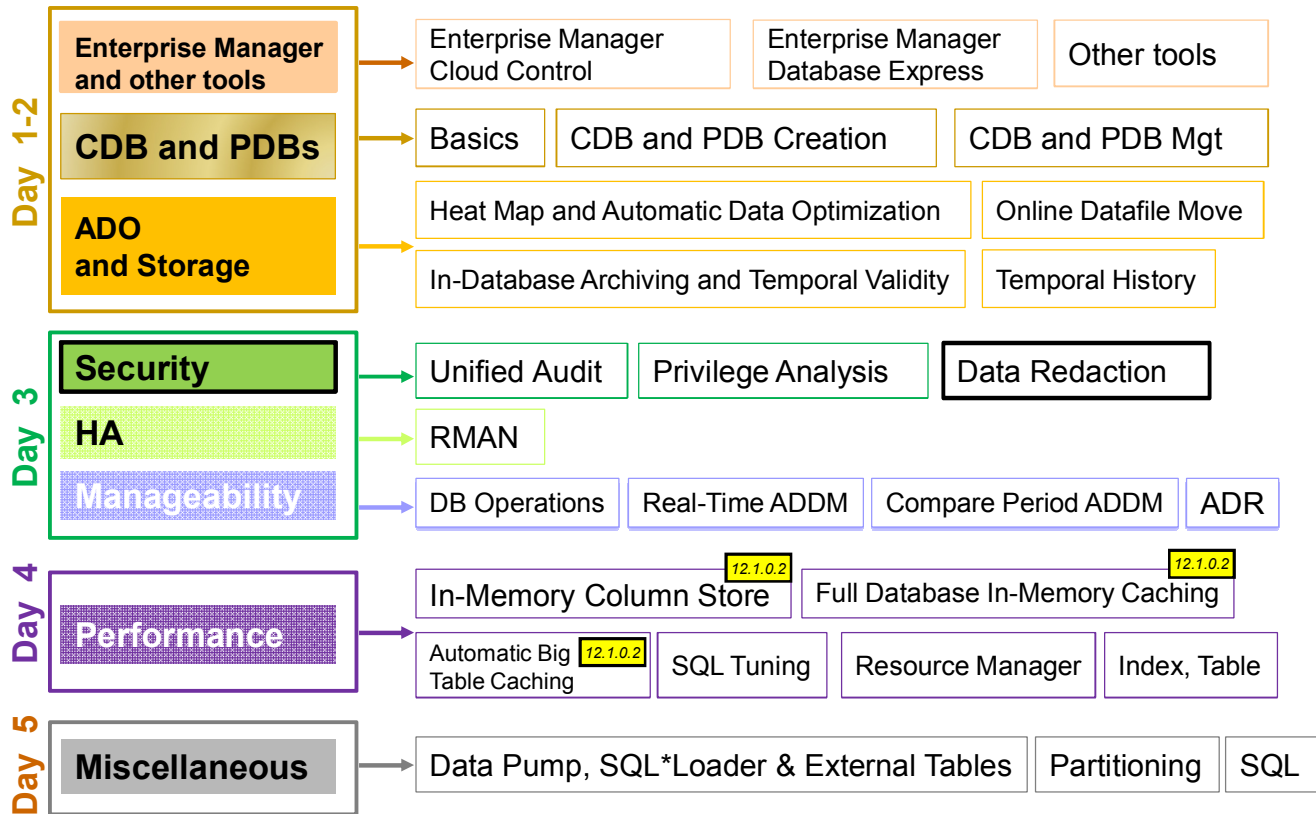
# 12

## Oracle Data Redaction

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Redaction is a new feature that shields sensitive data columns from application end users. This is done by modifying sensitive data columns contained in SQL query results on the fly right before the results are returned to applications. Data is neither updated nor stored in the blocks.

Data Redaction requires Oracle Advanced Security, which is a licensed option.

**Oracle Data Redaction** is totally different from **Oracle Data Masking**. Oracle Data Masking transforms data by using masking formats and stores new masked data in the blocks.

# Objectives

After completing this lesson, you should be able to:

- Describe Oracle Data Redaction
- Manage redaction policies
- View redaction policy information in the data dictionary

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of the Oracle Data Redaction new feature usage, refer to the following guide in the Oracle documentation:

- *Oracle Database Security Guide 12c Release 1 (12.1) – “Using Oracle Data Redaction to Protect Sensitive Data” chapter*
- *Oracle Database 2 Day course – “Limiting Access to Sensitive Data Using Oracle Data Redaction”*
- *Oracle Database PL/SQL Packages and Types Reference 12c Release 1 (12.1) – “DBMS\_REDACT” chapter*

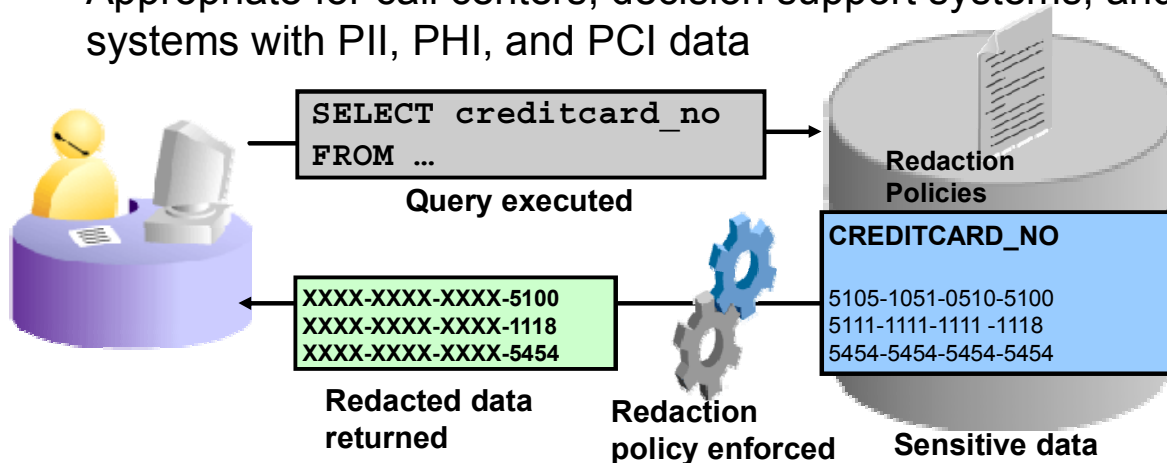
Refer to other sources of information available in Oracle Learning Library:

- *Oracle Database 12c New Features Demo Series demonstrations:*
  - *Data Redaction Basics*
- *Oracle By Example (OBE):*
  - *Protecting Data with Data Redaction*



## Oracle Data Redaction: Overview

- Dynamic redaction based on username, IP address, application context, and other factors
- Transparent, consistent enforcement in the database
- No measurable impact on production workloads
- Appropriate for call centers, decision support systems, and systems with PII, PHI, and PCI data



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Currently, many Oracle Database customers prevent the display of sensitive data to end users by performing redacting in each application. Oracle Data Redaction moves this functionality from the application to the database. This approach has several benefits over redacting data in the application tier, and it is useful in a variety of application scenarios.

Oracle Data Redaction is a transparent, flexible, and simple solution. It modifies sensitive data columns contained in SQL query results dynamically right before the results are returned to applications. The columns are redacted according to flexible policies that provide conditional redaction. The policies are managed directly within the database. For maximum transparency, redaction preserves the returned column data type and formatting, and it does not alter the underlying data blocks on disk or in cache. Oracle Data Redaction is designed to be fast so that it can be used on production systems. In addition, it is embedded in the database management system; so no separate installation is required.

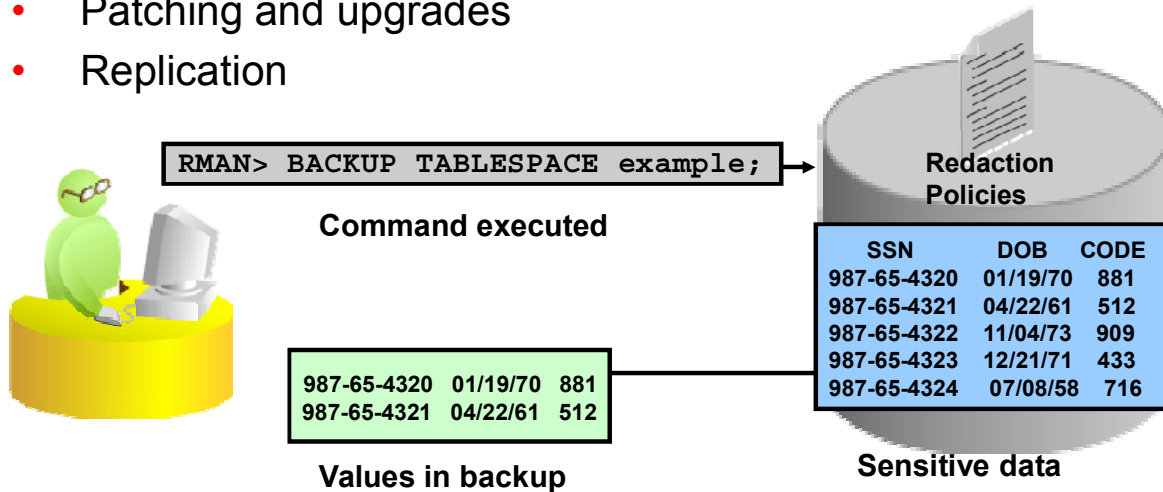
Oracle Data Redaction is useful in many scenarios. It can be applied to a range of packaged and custom applications to redact application screens, dashboards, and reports. It helps you to avoid making code changes in existing call center, human resources, sales, financial, and healthcare applications. These applications frequently manage payment card information (PCI), protected health information (PHI), or personally identifiable information (PII) information that is subject to regulation. Oracle Data Redaction is also useful for decision-support systems that aggregate large and diverse sets of data in a single repository for running analytics.

When an application issues a query, data is retrieved from the database and then the redaction policy is applied. Redaction takes place immediately preceding the return of selected data, and only at the top level of the `SELECT` list.

# Oracle Data Redaction and Operational Activities

Operational activities that are not subject to redaction:

- Backup and restore
- Import and export
- Patching and upgrades
- Replication



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Data retrieved during certain operational activities should not be redacted. Connections as the `SYS` user are always exempt from redaction policies. Utilities such as Data Pump perform recursive SQL as the `SYS` user, so redaction policies are not applied for export and import operations.

In this example, an RMAN command to back up the `EXAMPLE` tablespace is executed. The data is retrieved from the database, but is not redacted in the backup file.

**Note:** Data Redaction policies associated with tables and views are included in the export and import operation. Therefore, the policies are enabled and the data is redacted when users query the objects in the imported database.

## Available Redaction Methods

| Type               | Description                                                            |
|--------------------|------------------------------------------------------------------------|
| None               | No redaction is performed.                                             |
| Full               | Columns are redacted to constant values based on the column data type. |
| Partial            | User-specified positions are replaced by a user-specified character.   |
| Random             | Data type is preserved and different values are output each time.      |
| Regular Expression | A “match and replace” based on parameters is performed.                |

The Oracle logo, consisting of the word "ORACLE" in a bold, sans-serif font, with a red dot over the letter "A".

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Redaction supports four types of redaction: full, partial, random, and regular expression.

Support for a redaction type of “none” is provided so that you can create redaction policy definitions and evaluate their impact on the application functionality and performance.

# Oracle Data Redaction: Examples

| Data in the Database > Redacted Values |                                 |                                           |
|----------------------------------------|---------------------------------|-------------------------------------------|
| <b>Full Redaction</b>                  | 05/24/75<br>11 Rock Bluff Drive | → 01/01/01<br>→ xxxxxxxxx                 |
| <b>Partial Redaction</b>               | 068-35-2299<br>D1L86YZV8K       | → ***-**-2299<br>→ D1*****8K              |
| <b>Regular Expression Redaction</b>    | 94025-2450<br>Tom.Lee@acme.com  | → 94025-[hidden]<br>→ [redacted]@acme.com |
| <b>Random Redaction</b>                | 601111111111117<br>09/30/73     | → 422222222222226<br>→ 11/30/85           |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

This slide provides examples of data redaction based on the type of redaction that is configured. More detail about defining the redaction type is provided later in the lesson.

## What Is a Redaction Policy?

The redaction policy dictates:

- What to redact, as specified by:
  - *Schema name* (OBJECT\_SCHEMA)
  - *Object name* (OBJECT\_NAME)
  - *Column name* (COLUMN\_NAME)
- How to redact, as specified by:
  - *Function type* (FUNCTION\_TYPE)
  - *Function parameters* (FUNCTION\_PARAMETERS) or *regular expression parameters* (REGEXP\_\*)
- When to redact, as specified in a *policy expression* (EXPRESSION)

When you create the policy, you can provide only one “how to redact” specification.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To implement Oracle Data Redaction, you define a redaction policy that specifies what should be redacted, how the redaction should be applied, and when the redaction should take place.

- First, you identify a schema, table or view, and column to be redacted.
- Second, you specify the redaction method to use for the column, along with any required parameters. Only one column can be set when you first create the policy. However, you can later modify the policy to add more columns. If you do not provide a value for FUNCTION\_TYPE, full redaction is used for the specified column.
- Lastly, you specify the exact conditions that must be met for the redaction to be enforced. For example, you can choose to redact the columns by default and show actual data only for certain users or IP addresses. You can join several of these conditions together by using logical operators. This “when” condition is set once for the table or view, so it applies to all columns that are added to the policy.

Note that it is good practice to create a white list when you are defining security policies. A white list enables you to deny by default and add only authorized exceptions to the policy. In the redaction SQL expression, you can insert negative comparisons to build your white list. You can specify that redaction should take place for all users, IP addresses, and so forth that do not match values written into your expression. Black lists, which are the inverse of white lists, are also supported.

**Restrictions:**

- You cannot redact `SYS` or `SYSTEM` schema objects.
- You cannot redact virtual columns.
- You cannot redact columns of specific data types.
- You can apply VPD policies only to columns that have not been redacted.

## Managing Redaction Policies

- You use the procedures in the `DBMS_REDACT` package to manage redaction policies:
  - `ADD_POLICY`: Add a redaction policy to a table.
  - `DROP_POLICY`: Remove a redaction policy from a table.
  - `ALTER_POLICY`: Change a redaction policy.
  - `ENABLE_POLICY`: Enable a redaction policy after it is disabled.
  - `DISABLE_POLICY`: Disable a redaction policy.
- `EXECUTE` privilege on `DBMS_REDACT` is required to execute the procedures.
- Enterprise Manager Cloud Control 12c supports Oracle Data Redaction.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You use the `DBMS_REDACT` PL/SQL package and its procedures to define and manage redaction policies. You can also use Oracle Enterprise Manager Cloud Control 12c to define and manage redaction policies.

To use the procedures in the `DBMS_REDACT` package, you must be granted the `EXECUTE` privilege for the package. You do not need any privileges on the tables or views that you specify in the redaction policy.

**Best Practice:** To ensure that policies are not defined or altered inappropriately, restrict the `EXECUTE` privilege on the `DBMS_REDACT` package to those users who need to perform data redaction.



## Applying a Redaction Policy to a Table or View

Defining a redaction policy for the `SALARY` column in the `HR.EMPLOYEES` table:

```
DBMS_REDACT.ADD_POLICY
(policy_name      => 'EMPSAL_POLICY',
 object_schema   => 'HR',
 object_name      => 'EMPLOYEES',
 column_name     => 'SALARY',
 expression      =>
 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') != ''HR-VP'',
 function_type   => DBMS_REDACT.FULL);
```



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In this example, a redaction policy named `EMPSAL_POLICY` is defined on the `SALARY` column of the `HR.EMPLOYEES` table. This policy specifies that full redaction is used for the `SALARY` column values that are returned to all users other than `HR-VP`.

The `REDACTION_POLICIES` and `REDACTION_COLUMNS` views provide the list of redaction policies and redacted columns respectively.

The `REDACTION_VALUES_FOR_TYPE_FULL` view shows all of the current values for full redaction.

## Full Redaction: Examples

| Data definition   | Data in the database       | Redacted data          |
|-------------------|----------------------------|------------------------|
| SSN VARCHAR2 (11) | 987-65-4320<br>987-65-4321 | ' '                    |
| SSN NUMBER (9)    | 987654322<br>987654323     | 0<br>0                 |
| HIREDATE DATE     | 12-AUG-02<br>19-NOV-02     | 01-JAN-01<br>01-JAN-01 |

Change the default value of full redaction:

```
EXEC DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES
      (varchar_val => 'N');
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

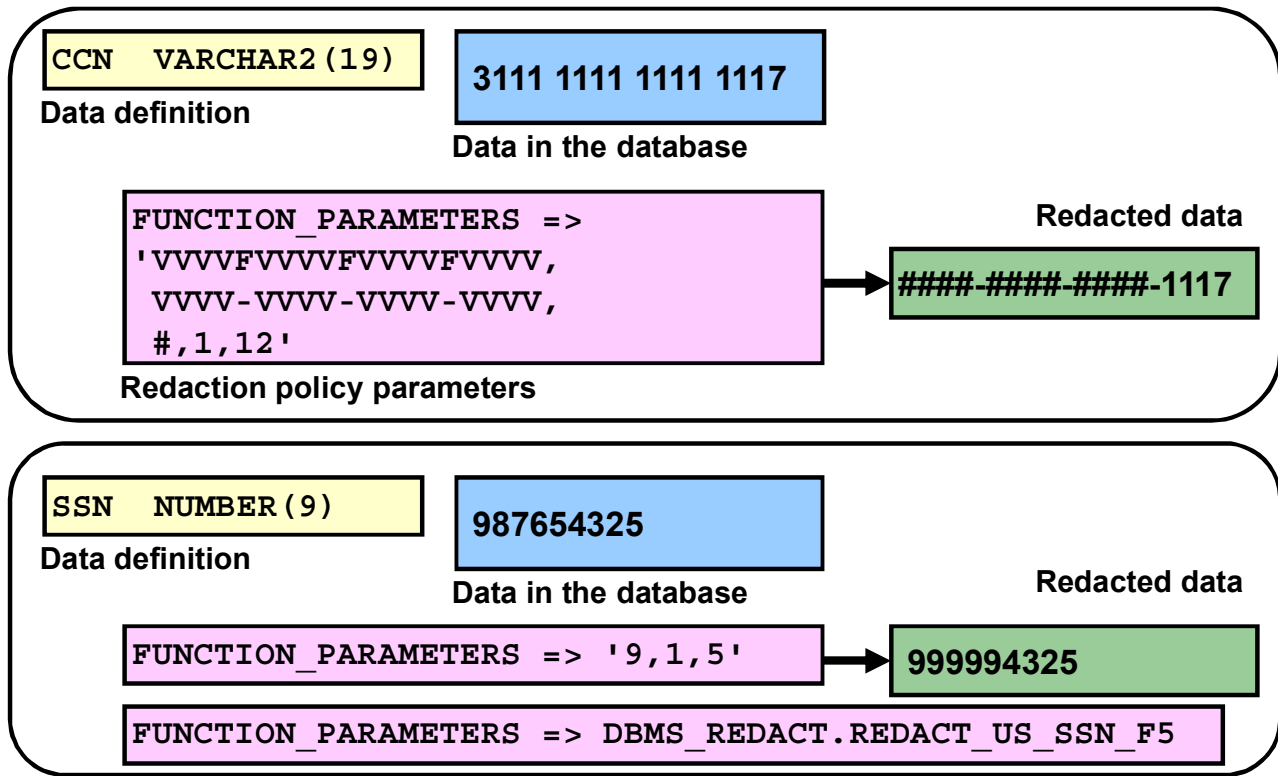
You use full redaction to redact the returned data to a fixed value. As shown in the examples:

- Character data is redacted to a single, blank space.
- Number data is redacted to a single zero.
- Datetime data is redacted to the first of January, 2001.

You specify full redaction by setting `FUNCTION_TYPE => DBMS_REDACT.FULL`. Full redaction is the default if you do not specify a redaction type in the redaction policy.

Use the `UPDATE_FULL_REDACTION_VALUES` procedure to modify the default displayed values for full redaction. After you modify a value, you must restart the database for it to take effect. You can find the current values by querying the `REDACTION_VALUES_FOR_TYPE_FULL` data dictionary view.

## Partial Redaction: Examples



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You specify partial redaction by setting `FUNCTION_TYPE => DBMS_REDACT.PARTIAL`. In addition, you specify parameters so that a portion of the data is redacted and part of the original data is preserved.

The graphic in the slide shows the use of an input parameter that specifies the format characters (F) and values (V) that may be redacted. The output format parameter specifies where to insert formatting characters. The other parameters indicate the redaction character that should be used and the values that should be redacted.

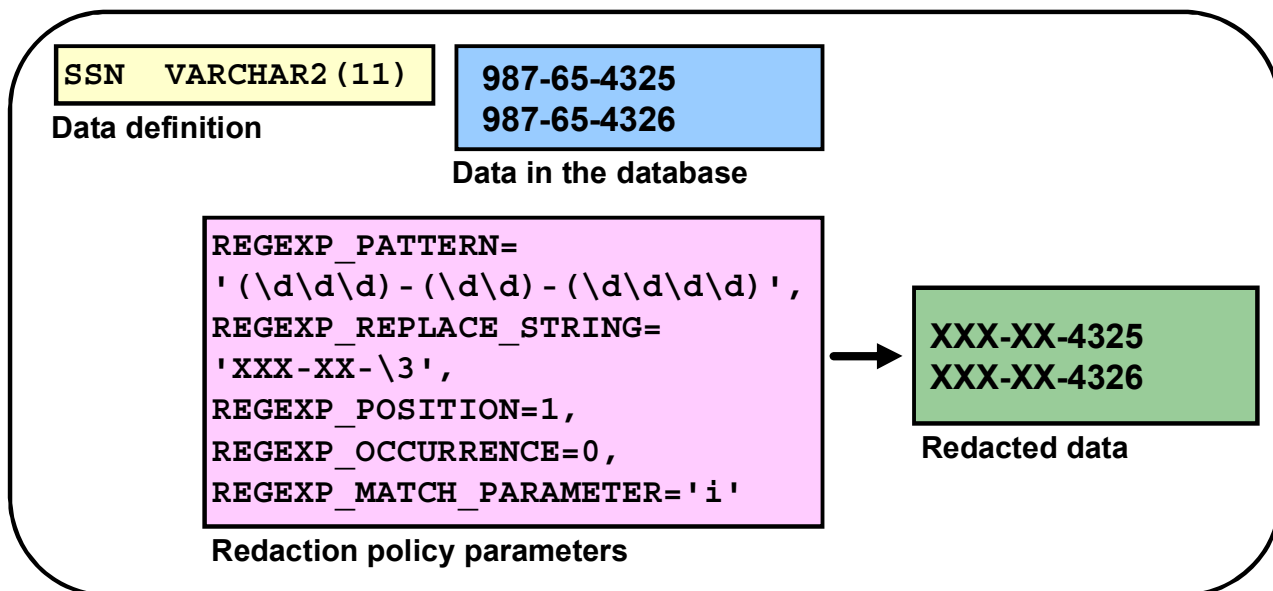
For a number data type, you can specify only numbers 0 through 9 as redaction characters.

For a date data type, lowercase letters indicate month, day, year, hour, minute, and second. The value following each letter specifies the redaction value. An uppercase letter indicates that no redaction should be performed for that element.

In the CCN example, the first 12 positions of the `VARCHAR2` column are changed to “#” and blanks are replaced with “-”, and the remaining four digits left as they are.

In the SSN numeric example, the first five positions get numeral 9.

# Regular Expression



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You specify regular expression redaction by setting `FUNCTION_TYPE => DBMS_REDACT.REGEXP`.

Use the following parameters to define regular expression redaction:

- `REGEXP_PATTERN` to specify a regular expression that represents the column data that will be redacted and that describes the pattern of data that must be matched. In the example in the slide, the `\d` indicates one digit, but not the letter "d." The whole pattern consists of three digits, then a "-", then two digits, then a "-", and then four digits.
- `REGEXP_REPLACE_STRING` to define how the data should be replaced. In the example in the slide, the `\3` indicates that the data in the third set of parenthesis should be preserved, but the first two sets of digits are replaced by `XXX-XX`.
- `REGEXP_POSITION` to indicate the starting position for the string search.
- `REGEXP_OCCURRENCE` to specify how the search and replace operation is to be performed. Use 0 to indicate that all occurrences of the match should be replaced. Use *n* to indicate that the *n*th occurrence of the match should be replaced.
- `REGEXP_MATCH_PARAMETER` to specify a text literal to change the default matching behavior of the function. The behavior of this parameter is the same as for the `REGEXP_REPLACE` SQL function. Refer to *Oracle Database SQL Language Reference* for detailed information.

## Modifying the Redaction Policy

Use the `DBMS_REDACT.ALTER_POLICY` procedure to alter an existing redaction policy as follows:

- Modify the policy expression.
- Modify the type of redaction for a specified column.
- Modify the function parameters for a specified column.
- Add a column to the redaction policy.
- Remove a column from the redaction policy.

```
DBMS_REDACT.ALTER_POLICY
( policy_name      => 'EMPSAL_POLICY',
  object_schema    => 'HR',
  object_name       => 'EMPLOYEES',
  action            => DBMS_REDACT.MODIFY_EXPRESSION,
  expression        =>
    'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') != ''HR-DIR'''
)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Use the `DBMS_REDACT.ALTER_POLICY` procedure to modify an existing redaction policy. Specify the type of modification by setting the `ACTION` parameter. Specify values for `EXPRESSION`, `COLUMN_NAME`, and `FUNCTION_PARAMETERS` or `REGEXP_*` depending on the value in `ACTION`.

Use the `DBMS_REDACT.ALTER_POLICY` procedure to:

- Modify the policy expression by specifying `DBMS_REDACT.MODIFY_EXPRESSION` for the `ACTION` parameter. Each policy can have only one policy expression. So when you modify the policy expression, you are replacing the existing policy expression.
- Modify the type of redaction for a specified column by specifying `DBMS_REDACT.MODIFY_COLUMN` for the `ACTION` parameter
- Modify the redaction function parameters for a specified column by specifying `DBMS_REDACT.MODIFY_COLUMN` for the `ACTION` parameter
- Add a column to the redaction policy by specifying `DBMS_REDACT.ADD_COLUMN` for the `ACTION` parameter. You must also specify the redaction type and any required parameters.
- Remove a column from the redaction policy by specifying `DBMS_REDACT.DROP_COLUMN` for the `ACTION` parameter

## Exempting Users from Redaction Policies

- Conditions included in policy expressions may allow users to see actual data.
- SYS is exempt from all redaction policies.
- Grant the EXEMPT REDACTION POLICY system privilege to exempt other users from all redaction policies.
- Best Practices:
  - Use default deny (white list) conditions in policy expressions.
  - Grant the EXEMPT REDACTION POLICY privilege judiciously to ensure that the redaction policies are enforced appropriately.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered on a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The SYS user is exempt from redaction policies and is able to view actual values for data.

If other users need access to the actual values, you must grant them the EXEMPT REDACTION POLICY system privilege. This privilege exempts the users from all redaction policies.

Users who are granted the DBA role are also exempt from redaction policies because the DBA role contains the EXP\_FULL\_DATABASE role, which is granted the EXEMPT REDACTION POLICY system privilege.

Because applications may need to perform CREATE TABLE AS SELECT operations that involve redacted source columns, you can grant the application the EXEMPT DDL REDACTION POLICY system privilege.

## Using Oracle Data Redaction with Other Oracle Database Security Solutions

- Serves different use cases
- Is complementary to other Oracle Database security solutions and can be used with them in tandem

|                                                                                    |                                                            |
|------------------------------------------------------------------------------------|------------------------------------------------------------|
|   | Oracle Advanced Security Transparent Data Encryption (TDE) |
|   | Oracle Database Vault                                      |
|   | Oracle Database Firewall                                   |
|   | Oracle Audit Vault                                         |
|   | Oracle Label Security (OLS)                                |
|   | Oracle Virtual Private Database (VPD)                      |
|   | Oracle Enterprise Manager Data Masking Pack                |
|  | Oracle Real Application Security (RAS)                     |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Redaction serves different use cases than other Oracle Database security solutions and should be viewed as complementary to them. Oracle Database security offerings can be paired with Oracle Data Redaction to provide a more secure environment.

# Oracle Database Security Features

| Oracle Database Security Product or Feature                | Description                                                                                                                                                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Oracle Data Masking Pack                                   | Physically alters and masks production data before you distribute copies of it to development and test environments                                                                                   |
| Oracle Virtual Private Database                            | Dynamically rewrites the SQL query to filter the data returned<br>Can prevent certain rows from being returned, but for columns, it is limited to setting them to NULL.                               |
| Oracle Advanced Security Transparent Data Encryption (TDE) | Transparently encrypts and decrypts data in storage without impacting applications<br>Does not alter the SQL query results<br>Requires managing the encryption keys outside of the database           |
| Real Application Security                                  | Powerful fine-grained features for controlling the data returned<br>Requires uptake of the programming framework and custom coding<br>Suitable for applications where you are able to modify the code |
| Oracle Database Vault                                      | Applies an additional layer of access control to highly privileged database user accounts<br>Disallows selected operations and returns an error code                                                  |



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Data Masking Pack is an optional feature of Oracle Enterprise Manager Cloud Control 12c. By irreversibly replacing original, sensitive data with fictitious data, it enables customers to safely share data with IT developers and business partners. It provides end-to-end secure automation for provisioning test databases from production in compliance with regulations.

You use Oracle Virtual Private Database (VPD) to create security policies that control database access at the row and column level. Essentially, VPD adds a dynamic `WHERE` clause to a SQL statement that is issued against the table, view, or synonym to which a VPD security policy was applied.

You can use VPD to redact column data, but only to change columns to NULL, which may cause errors in applications.

Oracle Data Redaction and VPD are complementary features that you can use together. For example, you can use VPD to filter the rows that are returned by the query, and you can use Oracle Data Redaction to redact the columns that are displayed.



Transparent Data Encryption (TDE) is a feature of Oracle Advanced Security. You use TDE to encrypt sensitive data that is stored in data files. It uses a TDE master encryption key that is stored in an Oracle Wallet or a Hardware Security Module (HSM).

For detailed information about TDE, refer to *Oracle Database Advanced Security Administrator's Guide 12c Release 1*.

Oracle Data Redaction and TDE are complementary features. They can be used together to protect the data that is stored in the database and to mask the data when it is returned to a query.

Oracle Database Real Application Security is a new feature of Oracle Database 12c. It is a database authorization model that supports declarative security policies and enables end-to-end security for multitier applications. It provides an integrated solution for securing the database and application user communities. Also, it advances the security architecture of Oracle Database to meet existing and emerging demands of applications that are developed for the Internet.

## Best Practices: Preventing Unauthorized Policy Modifications and Exemptions

- Limit the EXECUTE privilege on the DBMS\_REDACT package.
- Limit the EXEMPT REDACTION POLICY privilege.
- If you use Oracle Database Vault to restrict privileged user access, you can create realms to:
  - Further limit the execution of the DBMS\_REDACT package and granting of the EXEMPT REDACTION POLICY privilege
  - Limit access to the REDACTION\_POLICIES and REDACTION\_COLUMNS views

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a solid red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To prevent unauthorized policy modifications and exemptions, limit grants of the EXECUTE privilege on the DBMS\_REDACT package and granting of the EXEMPT REDACTION POLICY privilege.

Oracle Database Vault restricts access to certain areas of the database even for highly privileged users. If you use Database Vault, you can create realms to further limit execution of the DBMS\_REDACT package and granting of the EXEMPT REDACTION POLICY privilege. You can also use Database Vault to restrict access to the REDACTION\_POLICIES and REDACTION\_COLUMNS views.

For additional information about Oracle Database Vault, refer to the *Implementing Oracle Database Vault* course and *Oracle Database Vault Administrator's Guide 12c Release 1*.

## Best Practices: Considerations

- Choose the columns to redact selectively; redact only what is needed.
- Keep the policy expression logic as simple as possible.
- When you are defining regular expression policies, keep the regular expressions simple.
- Partial and full redaction policies generally provide better performance than regular expression policies that must be compiled each time they are used.
- You cannot perform a CTAS operation where any of the columns being selected is protected.
- You can apply only one policy on a table or view.

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, is positioned on the right side of a solid red horizontal bar.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To maximize performance, redact only the columns that are needed and keep the logic in policy expressions and regular expressions as simple as possible. Note that per-row processing time (CPU time) increases as more columns are redacted in a given table or view. Also, the evaluation of policy expressions occurs once for each query execution. So, applications that repeat queries to fetch one row of data at a time (rather than fetching multiple rows at once) may detect overhead due to redaction.

## Summary

In this lesson, you should have learned how to:

- Describe Oracle Data Redaction
- Manage redaction policies
- View redaction policy information in the data dictionary

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice 12: Overview

- 12-1: Using full data redaction
- 12-2: Using partial data redaction (*optional*)
- 12-3: Cleaning up redaction policies
- 12-4: Changing the default value for FULL redaction (*optional*)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# **Module**

## **High Availability**

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

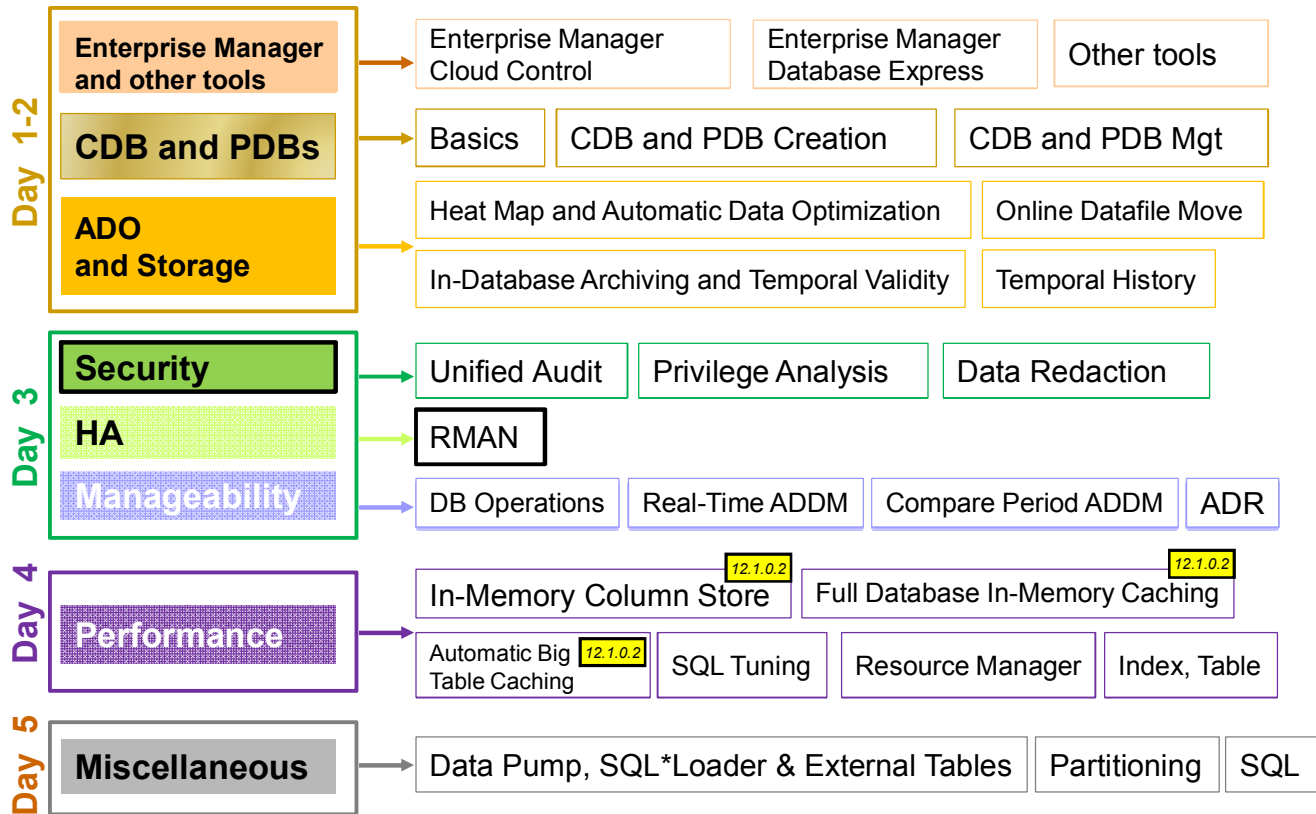
# Recovery Manager: New Features

13

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

There are several enhancements for **Recovery Manager**.



# Objectives

After completing this lesson, you should be able to:

- Describe the new RMAN enhancements
- Perform cross-platform data transport
- Use Table Recovery

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of the Oracle Recovery Manager, refer to the following guides in the Oracle documentation:

- *Oracle Database Administration Guide 12c Release 1 (12.1) – “Transporting Data”*
- *Oracle Database Backup and Recovery User’s Guide 12c Release 1 (12.1) – “Configuring Privilege and Role Authorization”*
- *Oracle Database Backup and Recovery Reference 12c Release 1 (12.1)*
- *Oracle Database Security Guide 12c Release 1 (12.1) – BACKUP, RESTORE and RECOVER*

Other sources of information:

- Oracle Learning Library: *Oracle Database 12c New Features Demo Series* demonstrations:
  - *Transporting Data*
  - *Table Recovery*
- MOS note: *“Reduce Transportable Tablespace Downtime using Cross Platform Incremental Backups (Doc ID 1389592.1)”*
- Oracle Technology Network Article: *Back Up a Thousand Databases Using Enterprise Manager Cloud Control 12c*

# Separation of DBA Duties

## RMAN Enhancements

SYSBACKUP  
 RMAN SQL  
 RMAN DESCRIBE  
 Multisection  
 Duplication  
 Third-party

Introducing task-specific and least-privileged administrative privilege **SYSBACKUP**

- Includes permissions for backup and recovery (connecting to a closed database)
- Does not include data access privileges such as `SELECT ANY TABLE`
- Is granted to the `SYSBACKUP` user that is created during database installation
- Can be explicitly used in RMAN connection by a `SYSBACKUP` privileged user



```
% rman TARGET ' "john@orcl AS SYSBACKUP" '
connected to target database: ORCL (DBID=1297344416)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Database 12c provides support for separation of database administration (DBA) duties for the Oracle database by introducing task-specific and least-privileged administrative privileges that do not require the `SYSDBA` administrative privilege. The new privilege to connect and execute commands in Recovery Manager is the `SYSBACKUP` privilege.

If you are working in a multitenant container database, each PDB of the CDB can create a local user and grant the user the `SYSBACKUP` privilege. The user can connect to the PDB and perform the backups for the PDB and only for the PDB he can connect to.

```
$ rman TARGET ' "tim/password@pdb1 AS SYSBACKUP" '
```

In this case, to backup the whole database, use the `BACKUP DATABASE` command with no database name. You are not allowed to use the `BACKUP PLUGGABLE DATABASE` command.

# Using SQL in RMAN

## RMAN Enhancements

SYSPBACKUP  
RMAN SQL  
RMAN DESCRIBE  
Multisection  
Duplication  
Third-party

- No SQL prefix or quotes required
- Provides SQL\*Plus DESCRIBE functionality
- Earlier releases:

```
RMAN> sql 'ALTER TABLESPACE users
          ADD DATAFILE '/testdata/users02.dbf' SIZE 10M';
```

- With Oracle Database 12c

```
RMAN> ALTER TABLESPACE users
      ADD DATAFILE '/testdata/users02.dbf' SIZE 10M;
```

```
RMAN> SELECT NAME, DBID, LOG_MODE FROM V$DATABASE;
```

```
RMAN> DESC tab1
```

| Name      | Null? | Type           |
|-----------|-------|----------------|
| -----     | ----- | -----          |
| TEST_NAME |       | VARCHAR2 (128) |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## What You Already Know

You can execute SQL commands and PL/SQL procedures from the RMAN command line. In versions earlier than Oracle Database 12.1, this required the SQL prefix and quotes as you see in the first example of adding a data file.

## What Is New

You can now issue most SQL commands in RMAN without preceding the command with the SQL keyword. As you see in the second example, this is easier to write.

An example of a simple SELECT command:

```
RMAN> select sysdate from dual;
```

```
SYSDATE
```

```
-----
```

```
TODAY
```

You can also use the abbreviated version DESC or the spelled-out DESCRIBE to list the column definitions of a table or view as shown in the slide.

Find the list of the SQL commands in *Oracle Database Backup and Recovery Reference 12c Release 1 Chapter RMAN Commands - SQL*

# Backing Up and Restoring Very Large Files

## RMAN Enhancements

SYSBACKUP  
RMAN SQL  
RMAN DESCRIBE  
Multisection  
Duplication  
Third-party

Multisection backups of a single data file:

- Are created by RMAN, with your specified size value
- Are for backup sets and **image copies**
- Are for full and **incremental backups**

Benefits:

- Reduce image copy creation time
- Are processed independently (serially or in parallel)
- Benefit Exadata

Requirements and restrictions:

- COMPATIBLE=12.0
- Not for control files or spfiles
- Not for a large value of parallelism

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## What you already know

Oracle data files can be up to 128 TB in size. By default, the smallest unit of an RMAN backup is an entire file. RMAN can optionally break up large files into sections and it can back up and restore these sections independently. You initiate the creation of multisection backups with the `SECTION SIZE` clause.

## How this can help

Multisection image copies reduce the image copy creation time for large data files, in particular benefiting Exadata environments. This can also reduce completion time for non-backup use cases. For example, copying a file as part of transportable tablespace procedure or creating a clone with the active database duplication.

Each section can be processed independently, either serially or in parallel. Backing up a file into separate sections can improve the performance of the backup operation, and it also allows large file backups to be restarted.

## Requirements and restrictions for multisection backups

- You can create multisection incremental backups only for data files, not for other database files, such as control files or spfiles.
- You should not apply large values of parallelism to back up a large file that resides on a small number of disks, because that would defeat the purpose of the parallel operation; multiple simultaneous accesses to the same disk device would be competing with each other.

# RMAN Duplication Enhancements

## RMAN Enhancements

SYSBACKUP  
RMAN SQL  
RMAN DESCRIBE  
Multisection  
Duplication  
Third-party

What is new for active database duplication: →

- Default use of backup sets for active duplication
- Choice of compression, section size, and encryption
- Option to end database duplication with database in mounted, but not opened, state
- Duplication of pluggable databases

ORACLE

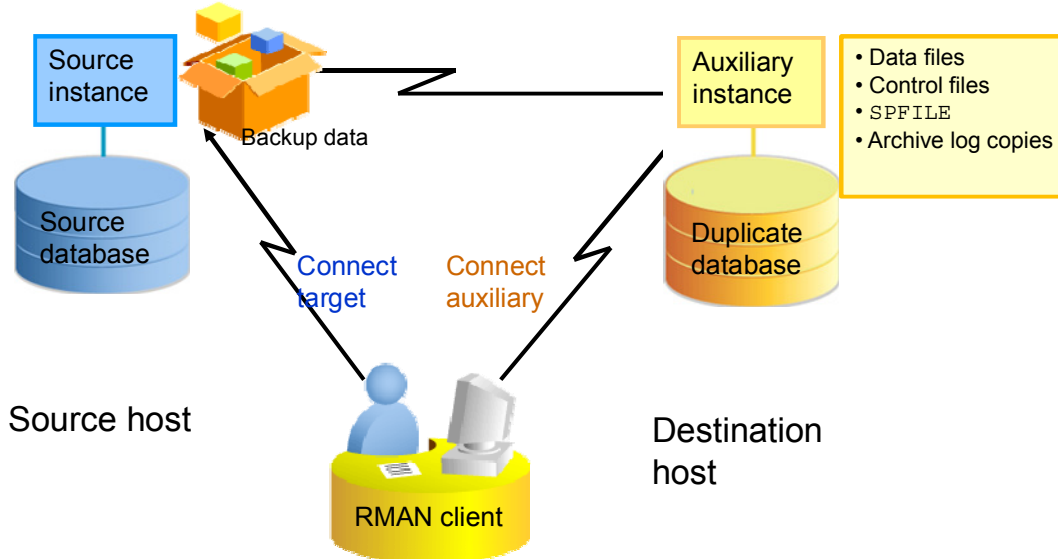
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The active database duplication process has additional options.

The RMAN `DUPLICATE` command allows to duplicate multitenant container databases and pluggable databases.

## Duplicating an Active Database

- With Enterprise Manager or RMAN command line
- With network (use the `FROM ACTIVE DATABASE` clause)
- With a customized `SPFILE`



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### What you already know about duplicating an active database

You can instruct the source database to send a “clone” of itself directly to the auxiliary instance by using Enterprise Manager or the `FROM ACTIVE DATABASE` clause of the RMAN `DUPLICATE` command.

Pre-existing backups are neither needed nor used for this operation. The online image copies are created by the source database and directly transmitted through Oracle Net (they are not written to disk) when you are using the `FROM ACTIVE DATABASE` clause of the RMAN `DUPLICATE` command. The source database can be open or mounted.

RMAN connects as `TARGET` to the source database instance and as `AUXILIARY` to the auxiliary instance (as shown in the slide).

The required files (data files, control files, `SPFILE`, and archive log copies) are copied from the source to an auxiliary instance through an inter-instance network connection. RMAN then uses a “memory script” (one that is contained only in memory) to complete recovery and open the database.

This method of active database duplication is referred to as the push-based method.

## What Is New?

Default use of backup sets for active duplication:

- Original “push” process based on image copies becomes “pull” process based on **backup sets**.
- Automatic “pull” process when a clause is used:
  - Compression
  - Section size
  - Encryption

Data is encrypted on the source database, before transmission.

```

RMAN> SET ENCRYPTION ...;
RMAN> DUPLICATE TARGET DATABASE TO orcl2
      FROM ACTIVE DATABASE
      [USING BACKUPSET]
      [SECTION SIZE ...]
      [USING COMPRESSED BACKUPSET] ...;
  
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The original “push” process is based on image copies.

With Oracle Database 12c, a “pull” (or restore) process is based on backup sets. A connection is first established with the source database. The auxiliary instance then retrieves the required database files from the source database as backup sets. A restore operation is performed from the auxiliary instance. Therefore, fewer resources are used on the source database. Both TNS connections are required on target and auxiliary instances.

Based on the `DUPLICATE` clauses, RMAN dynamically determines which process to use (push or pull). This ensures that existing customized scripts continue to function.

- When you specify `USING BACKUPSET`, RMAN uses the pull method.
- When you specify `SET ENCRYPTION` before the `DUPLICATE` command, RMAN automatically uses the pull method and creates backup sets. The backups sent to the destination are encrypted.
- The `SECTION SIZE` clause divides data files into subsections that are restored in parallel across multiple channels on the auxiliary database. For an effective use of parallelization, allocate more `AUXILIARY` channels.
- With the `USING COMPRESSED BACKUPSET` clause, the files are transferred as compressed backup sets. RMAN uses unused block compression while creating backups, thus reducing the size of backups that are transported over the network.

## NOOPEN Option

Option to end database duplication with database in mounted, but not opened, state:

- By default, it opens the duplicate database with the `RESETLOGS` option, and creates the online redo log files for the duplicate database.
- With `NOOPEN` option, the process finishes with the duplicate database in `MOUNT` mode.

Potential `NOOPEN` use cases:

- Moving the location of the database (for example, to ASM)
- Upgrading a database (where the database must not be open with resetlogs, prior to running upgrade scripts)

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You might duplicate a database with RMAN for various reasons. In earlier versions a recovered duplicated database was automatically opened. By default, this functionality continues with the Oracle Database 12c.

What is new is that you have an option to finish the duplication process with the database in a mounted, but not opened, state. This is useful when the attempt to open the database produce errors and when you want to modify initialization settings, which are otherwise quite difficult to modify. For example, you may want to move the location of the database to ASM. Also when you are performing an upgrade, where the database must not be open with resetlogs, prior to running upgrade scripts.

The `NOOPEN` option allows the duplication to create a new database as part of an upgrade procedure and leaves the database in a state ready for opening in upgrade mode and subsequent execution of upgrade scripts.



# Duplicating Multitenant Container Databases

- A single pluggable database:

```
RMAN> DUPLICATE DATABASE TO cdb1 PLUGGABLE DATABASE pdb1;
```

- Several pluggable databases:

```
RMAN> DUPLICATE DATABASE TO cdb1  
      PLUGGABLE DATABASE pdb1, pdb3;
```

- All pluggable databases except one:

```
RMAN> DUPLICATE DATABASE TO cdb1  
      SKIP PLUGGABLE DATABASE pdb3;
```

- A PDB and tablespaces of other PDBs:

```
RMAN> DUPLICATE DATABASE TO cdb1  
      PLUGGABLE DATABASE pdb1  
      TABLESPACE pdb2:users;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

RMAN enables you to duplicate multitenant container databases including all or individual PDBs by using the `DUPLICATE` command.

When you duplicate a pluggable database, by default, RMAN duplicates the root and the seed database of the multitenant container database that contains the pluggable database.

The first example shows how to duplicate a single pluggable database, the second one shows how to duplicate a set of pluggable databases, the third one shows how to duplicate all the databases in the multitenant container database, except a pluggable database, and the last one shows how to duplicate a set of tablespaces within a pluggable database.

You must be logged in to the root as a user who is granted the `SYSDBA` or `SYSBACKUP` role.

Find the whole procedure in *Oracle Database Backup and Recovery User's Guide 12c Release 1 Chapter Duplicating a Database*.

# Recovering Databases with Third-Party Snapshots

## RMAN Enhancements

SYSBACKUP  
SQL  
DESCRIBE  
Multisection  
Duplication  
Third-party ←

- Recover a database using the most recent snapshot:

```
RMAN> RECOVER DATABASE;
```

- Recover a database using a particular snapshot:

```
SQL> RECOVER DATABASE UNTIL TIME '02/15/2012 11:00:00'  
SNAPSHOT TIME '02/15/2012 10:00:00';
```

- Perform a partial recovery using archived redo log files:

```
SQL> RECOVER DATABASE UNTIL CANCEL SNAPSHOT TIME  
                                '02/15/2012 10:00:00';
```

- To be usable in recovery operations, the snapshots must conform to the following requirements:
  - The database is crash-consistent during the snapshot.
  - The snapshot preserves write order for each file.
  - The time stored is the time at which the snapshot is completed.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

You can use third-party technologies to take storage snapshots without putting the database or associated data files in BACKUP mode.

The snapshot technology stores the time at which the snapshot is completed.

## Recovering by Using Snapshots

To be usable in recovery operations, the snapshots must conform to the requirements described in the slide. Ensure that the database snapshots are usable. A snapshot for recovery is not usable if structural changes were made during the snapshot. Operations on data files and tablespaces like OFFLINE, READONLY, ONLINE, DROP, RENAME, SHRINK, and ADD make database structural changes and therefore invalidate a snapshot.

Use the RECOVER command using the new clause SNAPSHOT TIME to recover a snapshot to a consistent point in time. The SNAPSHOT TIME is used to inform the database that all of the files have been restored from a consistent snapshot.

- The first recovery example uses the most recent snapshot.
- The second example uses a specific snapshot, taken on February 15, 2012 at 10 AM. The UNTIL TIME clause can specify any time after the snapshot.
- The third example performs a partial recovery by using archived redo log files from a snapshot taken on February 15, 2012 at 10 AM.

The second and third examples showing the use of the UNTIL clause are valid only in SQL\*Plus.

## Quiz

In Oracle Database 12c, multisection backups can be used only for backup sets.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

# Transporting Data Across Platforms

Transporting databases, data files, and tablespaces across platforms:

- In earlier releases, by using image copies
- Oracle Database 12c, **by using backup sets or image copies**
- Allows **inconsistent tablespace** backups

Pre-12c

12c

Benefits:

- Reduced down time for platform migrations
- Choice of compression and multisection
- Not catalogued in control file, not used for regular restore operations

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## What You Already Know

RMAN enables you to transport databases, data files, and tablespaces across platforms. This includes transporting tablespaces across platforms with different endian formats. You can transport a whole database and thus convert a whole database on the destination host or source host as long as both the source and destination platforms share the same endian format. In earlier releases, cross-platform data transport uses only image copies, by using RMAN `CONVERT` command.

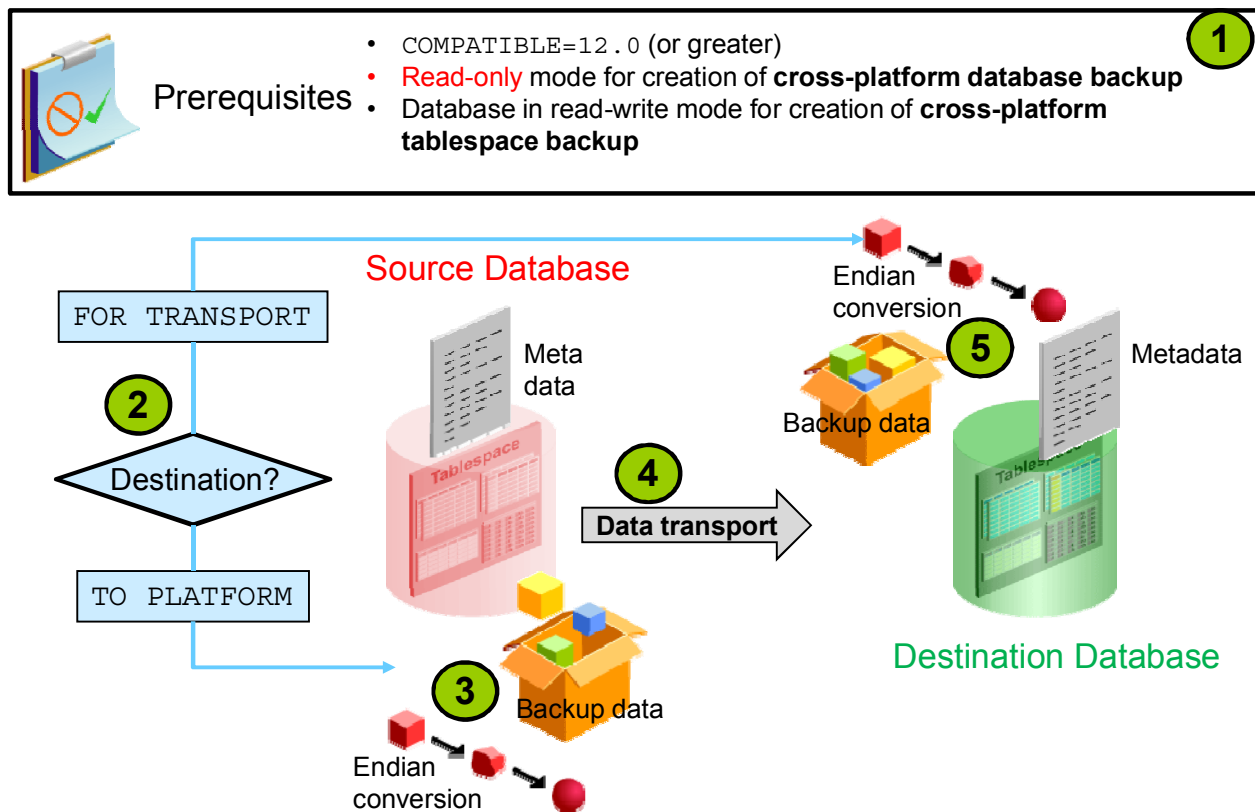
## What Is New

Starting with the Oracle Database 12c, you can transport data across platforms by using backup sets. You can also create cross-platform inconsistent tablespace backups by using image copies and backup sets. An inconsistent tablespace backup is one that is created when the tablespace is not in read-only mode. This method is allowed with the new clause `ALLOW INCONSISTENT`.

With the use of backup sets, you can choose compression and multisection options, that reduce the overall transport time.

**Note:** RMAN does not catalog backup sets created for cross-platform transport in the control file. This ensures that backup sets created for cross-platform transportation are not used during regular restore operations.

# Data Transport

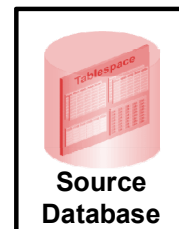


ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. Before you create a cross-platform data transportation backup set, the following prerequisites must be met:
  - COMPATIBLE parameter must be set to 12.0 or greater.
  - To transport an entire database, the source database must be open in read-only mode, because the SYS and SYSAUX tablespaces participate in the transport.
  - To transport only tablespaces, use the DATAPUMP clause. The database must be open in read-write mode, so that Data Pump can access the metadata.
2. There are two alternatives that affect the location of the endian conversion. The FOR TRANSPORT clause creates a cross-platform backup indicating that the backup set can be transported to any destination database. The TO PLATFORM clause indicates that the conversion performs on the source database for a specific platform and can be restored on that specific platform. If you transport tablespaces only, the DATAPUMP clause is required to create an export dump file for the transported tablespaces during the last incremental backup of the tablespace in read-only mode.
3. Use the BACKUP FOR TRANSPORT or TO PLATFORM command to create a cross-platform data backup set on the source host.
4. Use any operating system-utilities to transfer the created backup set (and dump file).
5. Use the RESTORE FOREIGN command to restore the cross-platform data backup set on the destination host.

# Transporting Database: Process Steps - 1



1. Verify the prerequisites:
  - COMPATIBLE: greater or equal 12.0
  - OPEN\_MODE: READ ONLY
2. Start an RMAN session to connect to the source database.
3. Query the exact name of the destination platform from V\$TRANSPORTABLE\_PLATFORM view.
4. Back up the source database, including control files:
  - Conversion on the source host

```
RMAN> BACKUP TO PLATFORM 'Linux x86 64-bit'
        FORMAT '/bkp_dir/transport.bck' DATABASE;
```

- Conversion at the destination host

```
RMAN> BACKUP FOR TRANSPORT FORMAT '/bkp_dir/transport.bck'
        DATABASE;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. Verify the prerequisites (the source database must be open in read-only mode if the whole database is transported).
2. Start an RMAN session and connect to the target instance.
3. For performing cross-platform database transport, you may need the exact name of the destination platform to which you are transporting data.
 

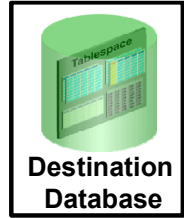
```
SELECT PLATFORM_ID, PLATFORM_NAME, ENDIAN_FORMAT
FROM V$TRANSPORTABLE_PLATFORM
WHERE UPPER(PLATFORM_NAME) LIKE '%LINUX%';
```
4. Back up the source database by using the BACKUP command with TO PLATFORM or FOR TRANSPORT. The FORMAT clause indicates the directory where the backup sets containing the data required for cross-platform database transportation are stored on the source host.
 

In the first example, the conversion will take place on the source host and the files stored in /bkp\_dir directory are converted for Linux x86 64-bit platform.

In the second example, the conversion will take place on the destination host during the restore command and the files stored in /bkp\_dir directory on the source host are not converted yet.

## Transporting Database: Process Steps - 2

5. Disconnect from the source database.
6. Move the backup sets to the destination host.
7. Connect to the destination host as TARGET.
8. Restore the full backupset with the RESTORE command.
  - When the conversion occurred on the source host



```
RMAN> RESTORE
      FROM PLATFORM 'Solaris Operating System (x86-64)'
      ALL FOREIGN DATAFILES
      FORMAT '/oradata/datafiles/df_%U'
      FROM BACKUPSET '/bkp_dir/transp.bck';
```

- When the conversion occurs at the destination host

```
RMAN> ALTER SYSTEM SET DB_CREATE_FILE_DEST='/oradata/orcl';
RMAN> RESTORE FOREIGN DATABASE TO NEW
      FORMAT '/oradata/datafiles/df_%U';
```

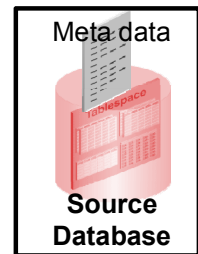
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Disconnect from the source database.
6. Move the backup sets created by the BACKUP command to the destination host. You can use operating system utilities to move the backup sets from the source host to the destination host.
7. Connect to the destination host, to which the database must be transported, as TARGET. Ensure that the destination database is not mounted.
8. Use the RESTORE command to restore the data in the destination database as described in the slide.

## Transporting Tablespace: Process Steps - 1

1. Verify the prerequisites.
2. Start an RMAN session in the source database.
3. Query the exact name of the destination platform.
4. Make the tablespace **read-only**.



```
RMAN> ALTER TABLESPACE test READ ONLY;
```

5. Perform a cross-transportable backup and a Data Pump export.

– Conversion on the source host

```
RMAN> BACKUP TO PLATFORM 'HP Tru64 UNIX'
      FORMAT '/bkp/test.bck'
      DATAPUMP FORMAT '/bkp/test.dmp' TABLESPACE test;
```

– Conversion on the destination host

```
RMAN> BACKUP FOR TRANSPORT FORMAT '/bkp/test.bck'
      DATAPUMP FORMAT '/bkp/test.dmp' TABLESPACE test;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. Verify the prerequisites. The source database must be opened in read-write mode.
2. Start an RMAN session and connect to the target instance.
3. For performing cross-platform tablespace transport, you may need the exact name of the destination platform to which you are transporting data.

```
SELECT PLATFORM_ID, PLATFORM_NAME, ENDIAN_FORMAT
FROM V$TRANSPORTABLE_PLATFORM
WHERE UPPER(PLATFORM_NAME) LIKE '%LINUX%';
```

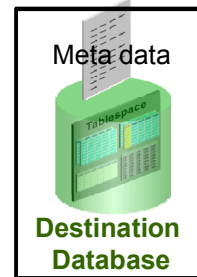
4. Make the tablespace read-only.
5. Back up the source tablespace using the BACKUP command with TO PLATFORM or FOR TRANSPORT to indicate where the conversion takes place. Use the DATAPUMP clause to indicate that an export dump file for the tablespaces must be created to export the tablespace metadata.

**Note:** A new BACKUP clause ALLOW INCONSISTENT enables you to back up tablespaces that are not in read-only mode. Although the backup is created, you cannot plug these tablespaces directly into the target database because they are inconsistent. You must later create an incremental backup of the tablespaces when they are in read-only mode. This incremental backup must contain the DATAPUMP clause that creates an export dump file of the tablespace metadata.



## Transporting Tablespace: Process Steps - 2

6. Move the backup sets and the Data Pump export dump file to the destination host.
7. Connect to the destination host to the TARGET.
8. Restore the cross-transportable backup and the Data Pump export.
  - When the conversion occurred on the source host



```

RMAN> RESTORE FOREIGN TABLESPACE test
      FORMAT '/oracle/test.dbf' FROM BACKUPSET '/bkp/test.bck'
      DUMP FILE FROM BACKUPSET '/bkp/test.dmp' ;
  
```

- When the conversion occurs at the destination host

```

RMAN> RESTORE FOREIGN TABLESPACE test TO NEW
      FROM BACKUPSET '/bkp/test.bck'
      DUMP FILE FROM BACKUPSET '/bkp/test.dmp' ;
  
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Disconnect from the source database and move the backup sets and the Data Pump export dump file to the destination host. You can use operating system utilities to move the backup sets and the dump file from the source host to the destination host.
7. Connect to the destination host, to which the tablespace must be transported, to the TARGET. Ensure that the destination database is opened in read-write mode.
8. Use the RESTORE command to restore the cross-transportable backup in the destination database as shown on the slide where FOREIGN TABLESPACE is the HP source data file. The FORMAT indicates where to restore the file retrieved from the backup set. Use the DUMP FILE FROM BACKUPSET to restore the metadata from the dump file required to plug the tablespace into the destination database.

**Note:** You can use a method that allows you to:

1. Create cross-platform inconsistent incremental backups with a new clause ALLOW INCONSISTENT

```

BACKUP INCREMENTAL FROM SCN=2720649 FOR TRANSPORT
ALLOW INCONSISTENT FORMAT '/home/u_incl.bkp' TABLESPACE users;
  
```

2. Restore inconsistent cross-platform tablespace backup by using RESTORE FOREIGN TABLESPACE command.
3. Recover restored data files copies with cross-platform incremental backups by using RECOVER FOREIGN DATAFILECOPY command.

## Quiz

When you transport data across platforms in Oracle Database 12c, you can use only image copies, not backup sets.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

# Table Recovery

- Table recovery with TSPITR, Flashback
- Table recovery from RMAN backups
  - Does not affect the remaining database objects
  - Reduces time and disk space compared to the earlier workflow like TSPITR

Pre-12c

12c

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Oracle database provides a number of different methods of how you can recover a table. It has done so in earlier versions as well. But the recovery scope might be much larger than what you want. The next interactive page provides an opportunity for you to review the major methods of:

- Database point-in-time recovery (**DBPITR**)
- Tablespace point-in-time recovery (**TSPITR**)
- Flashback Technology

## What Is New

- With Oracle Database 12c, RMAN enables you to recover one or more tables or table partitions to a specified point in time without affecting the remaining database objects. This is a table recovery by using RMAN backup data.
- This new functionality reduces time and disk space, compared to the earlier workflow:
  1. Recover tablespace in a separate disk location.
  2. Export desired table.
  3. Import into original database.

# Recovering Tables from Backups

When to recover tables and table partitions from RMAN backups:

- It is recommended to NOT use TSPITR for the following situations:
  - Small number of tables
  - Not in a self-contained tablespace
- It is not possible to flash back under these conditions:
  - Purged tables (no Flashback Drop)
  - Beyond the available undo (no Flashback Table)
  - After a structural DDL change (no Flashback Table)



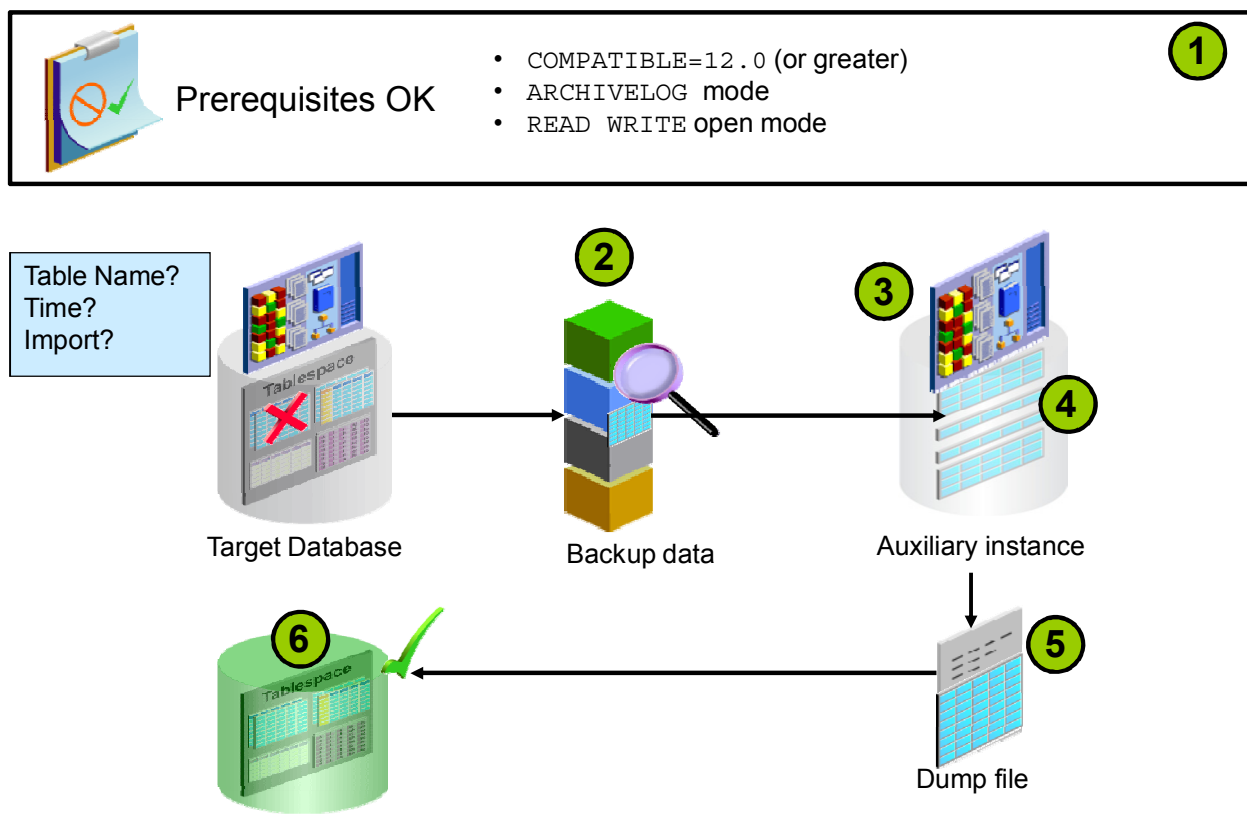
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

RMAN enables you to recover one or more tables or table partitions to a specified point in time without affecting the remaining database objects. Recovering tables and table partitions is useful in the following situations:

- You need to recover a very small number of tables to a particular point in time. In this situation, TSPITR is not the most effective solution because it moves all the objects in the tablespace to a specified point in time.
- You need to recover a tablespace that is not a self-contained to a particular point in time. TSPITR can be used only if the tablespace is self-contained.
- You need to recover tables that have either been corrupted or deleted with the PURGE option, so you cannot use the Flashback Drop functionality.
- You enabled logging for a Flashback Table, but the flashback target time or SCN is beyond the available undo.
- You want to recover data that is lost after a data definition language (DDL) operation has changed the structure of tables. You cannot use Flashback Table to rewind a table to before the point of a structural change, such as a truncate table operation.

# Table Recovery: Graphical Overview



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. RMAN uses backups that were previously created to recover tables and table partitions to a specified point in time. The prerequisites are fulfilled and you provide the following input with the `RECOVER` command:

- Names of tables or table partitions to be recovered
- Point in time to which the tables or table partitions need to be recovered
- Whether the recovered tables or table partitions must be imported into the target database

RMAN uses your input to automate the process of recovering the specified tables or table partitions.

2. RMAN determines the backup based on your specification.
3. RMAN creates an auxiliary instance.
4. RMAN recovers your tables or table partitions, up to the specified point in time, into this auxiliary instance.
5. RMAN creates a Data Pump export dump file that contains the recovered objects.
6. RMAN imports the recovered objects into the target database.
7. You can customize this process as you will see in the following slides.

## Specifying the Recovery Point-in-Time

Recover table and table partitions to the state they were in by specifying:

- `UNTIL SCN integer`: The system change number (SCN)
- `UNTIL TIME 'date_string'`: The time in the date format:
  - of the `NLS_LANG` and `NLS_DATE_FORMAT` environment variables, or:
  - Date constants, for example: `SYSDATE - 5`
- `UNTIL SEQUENCE integer (THREAD integer)`: The log sequence number and thread number

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

- RMAN recovers tables or table partitions to the state that they were at the time specified by the SCN. The SCN is an upper, noninclusive limit.
- RMAN recovers tables or table partitions to the state they were in at the specified time. Use the date format specified in the `NLS_LANG` and `NLS_DATE_FORMAT` environment variables. You can also use data constants such as `SYSDATE` to specify the time. (`SYSDATE - 5` means 5 days earlier than the system date).
- RMAN recovers tables or table partitions to the state they were at the time specified by the log sequence number and thread number. RMAN selects only files that it can use to restore or recover up to but not including the specified sequence number.

## Process Steps of Table Recovery - 1

1. Perform the planning tasks and start an RMAN session with the `CONNECT TARGET` command.
2. Enter the `RECOVER TABLE` command.
3. RMAN determines the backup based on your specification.
4. RMAN creates an auxiliary instance by using the `AUXILIARY DESTINATION` clause, if specified.
5. RMAN recovers your tables or table partitions, up to the specified point in time, into this auxiliary instance.
6. RMAN creates a Data Pump export dump file that contains the recovered objects with the `DUMP FILE=name` and `DATAPUMP DESTINATION=<OS path>`.

**Note:** If a file with the name specified by `DUMP FILE` exists in the location in which the dump file must be created, then the export fails.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

1. After verifying the prerequisites, start an RMAN session and connect to the target instance.
2. Enter the `RECOVER TABLE` command with your required clauses.
3. RMAN determines the backup that contains the data that needs to be recovered, based on the point in time specified for recovery.
4. RMAN creates an auxiliary instance. Optionally, you can specify the location of the auxiliary instance files with the `AUXILIARY DESTINATION` or `SET NEWNAME` clauses of the `RECOVER` command. `AUXILIARY DESTINATION` is the recommended clause, because when you use `SET NEWNAME` and you would forget just one data file name, the recovery would not happen.
5. RMAN recovers the specified tables or table partitions, up to the specified point in time, into this auxiliary instance.
6. RMAN creates a Data Pump export dump file that contains the recovered objects. You can optionally specify the name of the export dump file (with the `DUMP FILE` clause, default OS-specific name) that is used to store the metadata from the source database. You can also specify the location in which the export dump file is created with the `DATAPUMP DESTINATION` clause. The location is typically the path of the OS directory that stores the dump file. If omitted, the dump file is stored in the `AUXILIARY DESTINATION` location. If that is not specified, the dump file is stored in a default OS-specific location.

## Customization

### 7. You can customize as follows:

- RMAN imports the recovered objects into the target database unless you specified `NOTABLEIMPORT`.
- RMAN optionally renames the recovered tables or table partitions with the `REMAP TABLE` and the `REMAP TABLESPACE` clauses.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### 7. You can customize with the following options:

- By default, RMAN imports the recovered objects that are stored in the export dump file into the target database. However, you can choose not to import the recovered objects by using the `NOTABLEIMPORT` clause of the `RESTORE` command. If you choose not to import the recovered objects, RMAN recovers the tables or table partitions to the specified point and then creates the export dump file. However, this dump file is not imported into the target database. You must manually import this dump file into your target database, when required, by using the Data Pump Import utility.
- RMAN optionally renames the recovered tables or table partitions in the target database. When you recover tables or table partitions, you can either retain the existing names or rename the objects after they are imported into the target database. The `REMAP TABLE` option enables you to rename the recovered tables or table partitions in your target database. To import the recovered objects into a tablespace that is different from the one in which the objects originally existed, use the `REMAP TABLESPACE` clause of the `RECOVER` command. Only the tables or table partitions that are being recovered are remapped, the existing objects are not changed.



## Quiz

When you perform table recovery from backup sets in Oracle Database 12c, RMAN automatically creates, uses, and deletes an auxiliary database instance.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Summary

In this lesson, you should have learned how to:

- Describe the new RMAN enhancements
- Perform cross-platform data transport
- Use Table Recovery

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice 13: Overview

- 13-1: Using `SYSBACKUP` administrative privilege in RMAN
- 13-2: Recovering a table with RMAN

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# **Module**

## **Manageability**

**ORACLE**

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

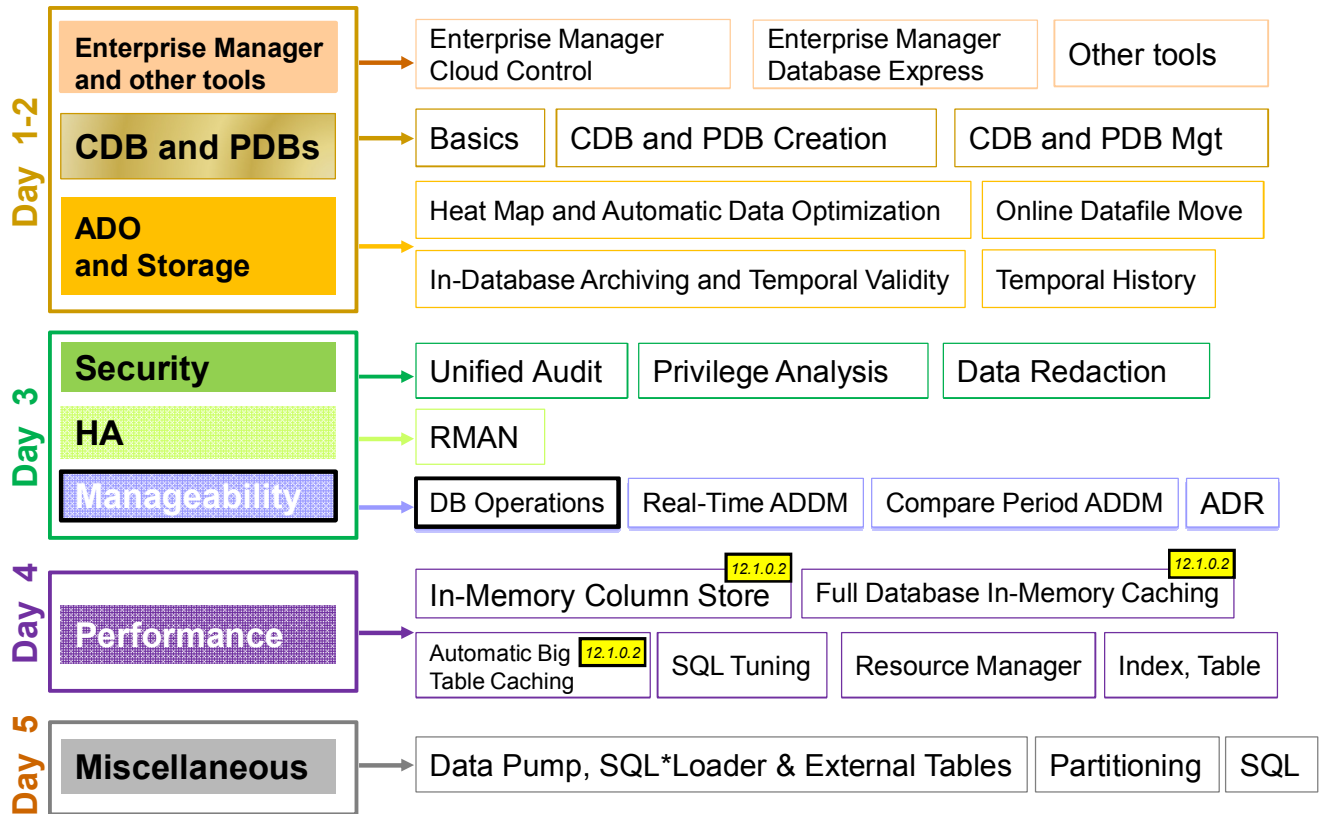
# 14

## Real-Time Database Operation Monitoring

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

# Oracle Database 12c New and Enhanced Features



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Manageability:** Managing databases implies monitoring database operations by using the new Real-Time Database Operation Monitoring feature.

# Objectives

After completing this lesson, you should be able to:

- Describe database operations
- Implement Real-Time Database Operation Monitoring

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

For a complete understanding of Real-Time Database Operation Monitoring new feature and usage, refer to the following guides in the Oracle documentation:

- *Oracle Database Performance Tuning Guide 12c Release 1 (12.1) – “Instance Tuning Using Performance Views – Real-Time Database Operation Monitoring Chapter”*
- *Oracle Database SQL Tuning 12c Release 1 (12.1) Chapter Monitoring Database Operations*

# Real-Time Database Operation Monitoring: Overview

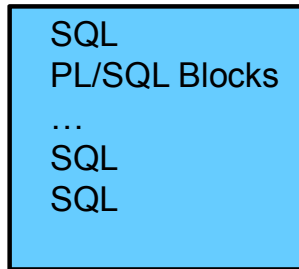
A database operation is:

- One or more SQL or PL/SQL statements
- A series of statements done from one or more sessions

A database operation can:

- Be monitored
- Produce active reports

**BEGIN\_OPERATION**



**END\_OPERATION**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Real-Time Database Operation Monitoring extends and generalizes Real-Time SQL Monitoring.

Real-Time SQL Monitoring helps determine where a currently executing SQL statement is in its execution plan and where the statement is spending its time.

You can use Real-Time Database Operation Monitoring for performance monitoring of active SQL statements, PL/SQL procedures, and functions. You can also see the breakdown of time and resources used for recently completed statements.

In databases where batch jobs run regularly, it can be very difficult to determine to which batch job an offending SQL statement belongs. In Decision Support Systems (DSS) it is not uncommon for many batch jobs to be running simultaneously. In some systems, hundreds of concurrent jobs could be running. Real-Time Database Operations Monitoring allows these jobs to be monitored while they are running.

Real-Time Database Operation Monitoring identifies an operation with a tag, determines what to monitor, and can generate reports. Real-Time Database Operation Monitoring monitors the progress of the operation, packages raw data for offline analysis, and creates active reports that do not require access to production systems.



# Use Cases

- Monitor batch jobs:
  - Set expected times; alert when exceeded.
  - Identify and tune expensive statements.
- Diagnose background processes:
  - Identify and tune expensive statements.
- Diagnose changes to jobs over time:
  - Compare previous executions of an operation.
  - Compare changes to jobs after an upgrade.

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In which situations would a DBA use Real-Time Database Operation Monitoring?

- A large retailer has a DSS with hundreds of batch jobs for Extraction, Transformation, and Loading (ETL). You want to monitor these jobs in real time. As an Enterprise Manager user, you experience a very long wait on a page load. You want to easily identify what is running on behalf of the page load. At times a PL/SQL job that is required to finish in 24 hours is taking up to 80 hours to run. You need to identify and tune the expensive statements in the job. You want to be alerted if the job is expected to run too long, so that it can be killed, fixed, and restarted before it consumes too many resources.
- After an upgrade a batch job went from 4.5 hours to 8 hours. You want to capture the performance statistics of the job steps, before and after the upgrade, then compare the two sets and identify the changes, for example, SQL execution plan, resource consumption, degree of parallelism.

## Current Tools

- **Real-Time SQL Monitor**
  - Is a subset of Database Operation Monitoring
  - Monitors **long-running** **single** SQL statements and PL/SQL functions
  - Displays statistics for recently completed statements
- **Active Session History**
  - 1-second sample
  - Useful for long running queries
- **SQL Trace**
  - Large overhead
  - Incident must be reproducible
- **DBMS\_MONITOR** and **trcsess** utility
- **V\$SESSION\_LONGOPS** and **DBMS\_APPLICATION\_INFO**

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

### Real-Time SQL Monitor

Oracle Database 11.1 introduced Real-Time SQL Monitor, a subset of Database Operation Monitoring. It helps DBAs know where a SQL statement is in its execution plan and where the time is being spent. It allowed users to monitor the performance of SQL statements while they are executing as well as see the breakdown of time and resources used for recently completed statements. Users can identify the expensive SQL statements in ETL, DDL, batch, reporting jobs, and check parallel-executed statements.

### Active session history

ASH contains samples of active sessions taken every second. It is on by default. Because samples are collected every second, ASH contains more information for long-running queries.

### SQL Trace

SQL Trace produces a lot of detailed information, which can be hard to interpret. The problem must be reproducible to enable SQL Trace to capture the details.

### Procedures and Views

- The **DBMS\_MONITOR** package enables the trace for a given database session, module, action, or service and **trcsess** command-line utility consolidates tracing information.
- **V\$SESSION\_LONGOPS** view and **DBMS\_APPLICATION\_INFO.SET\_SESSION\_LONGOPS** procedure are useful for long-running operations to mainly indicate how much work has been processed so far.

## Defining a DB Operation

Any set of database operations between two points in time identified by the user or application

Types of operations:

- Simple
  - One SQL or PL/SQL
- Composite
  - Multiple SQL statements and / or PL/SQL procedures and functions between two points in time
  - Single session
  - Multiple concurrent sessions

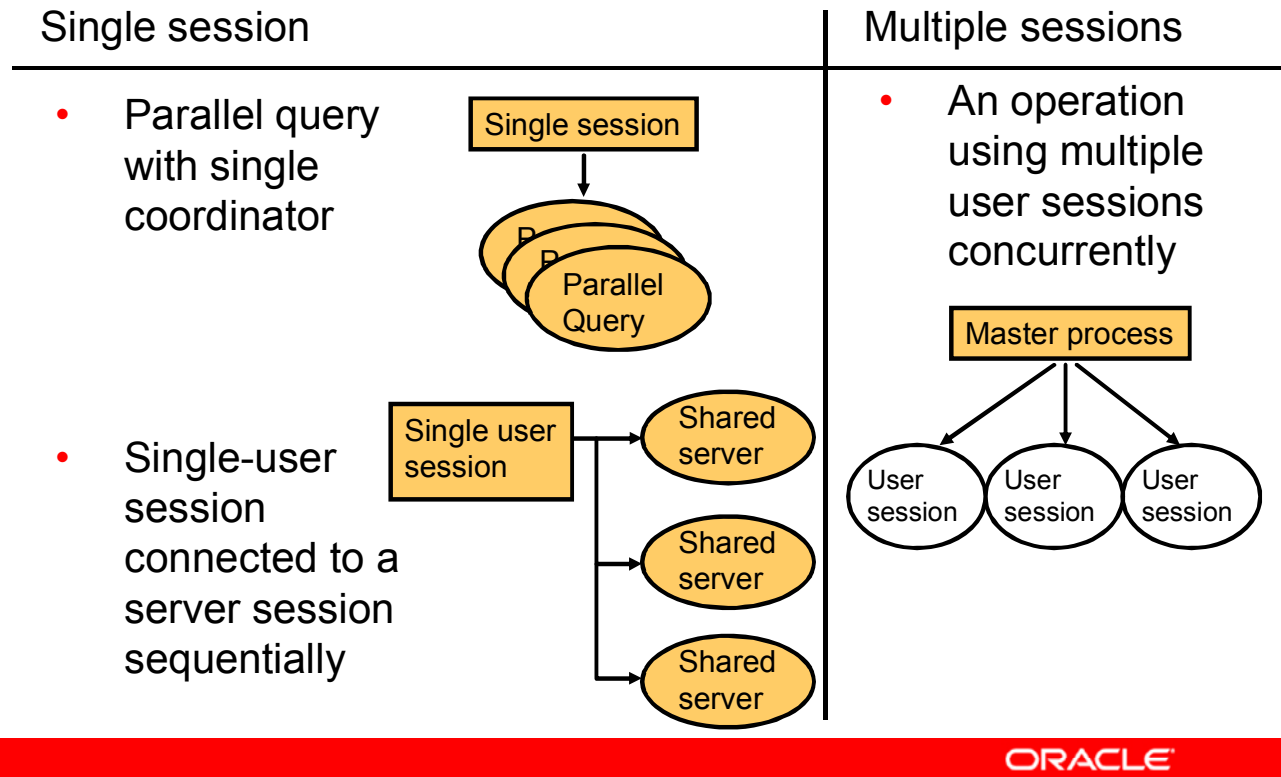
ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A DB operation is an operation that the database server runs to accomplish tasks. It can be either simple or composite.

- Simple database operations: Consist of a single SQL statement or a single PL/SQL function or procedure
- Composite database operations: Consist of the activity of one or more sessions between two points in time. SQL statements or PL/SQL procedures running in these sessions are part of the composite operations:
  - **Single sessions:** Single-session operations fall into two cases.
    - In one case, exactly one session exists for the duration of the database operation.
    - In the other case, multiple sessions exist in the database operation, but no more than one session runs at any given time. The application jumps from session to session.
  - **Multiple concurrent sessions:** In ETL, it is common for a job to involve multiple sessions running at the same time. You can define the entire ETL job as a single database operation.

# Scope of a Composite DB Operation



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A composite operation in the scope of a single session is composed of one of the following:

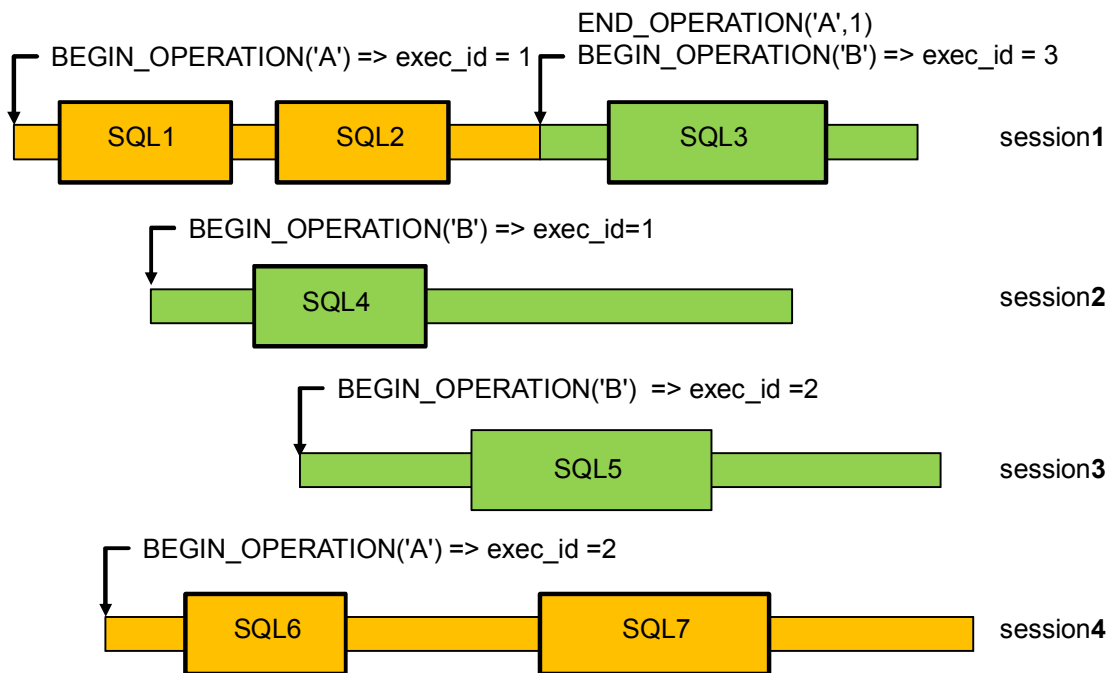
- SQL statements coordinated by the single session, such as parallel operations
- The active session, such as a user session in an application pool or shared server

Diagrams illustrating single-session and single-user session examples of a database operation are shown on the left side of the slide.

**Note:** These examples do not exclude a single session with statements that are neither nonparallel nor using shared servers.

A composite operation in the scope of multiple sessions is composed of SQL statements running concurrently such as during an ETL operation. This is shown in the diagram on the right side of the slide.

## Database Operation Concepts



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

In this illustration, there are four sessions. The first (upper) session shows that a session can belong to at the most one database operation at a time. It belongs to database operation A at first, then belongs to database operation B. The `END_OPERATION` and then `BEGIN_OPERATION` commands cause the session to change database operations.

Sessions 1, 2, and 3 show that the name and execution ID are taken together to uniquely identify the database operation B.

The time that a session belongs to a database operation but is not executing a SQL or PL/SQL statement is idle time.

# Identifying a Database Operation

- Naming **DBOP (Name)**

SQL  
 PL/SQL Blocks  
 ...  
 SQL  
 SQL
- Bracketing **DBMS\_SQL\_MONITOR.BEGIN\_OPERATION**

SQL  
 PL/SQL Blocks  
 ...  
 SQL  
 SQL

  
**DBMS\_SQL\_MONITOR.END\_OPERATION**

ORACLE

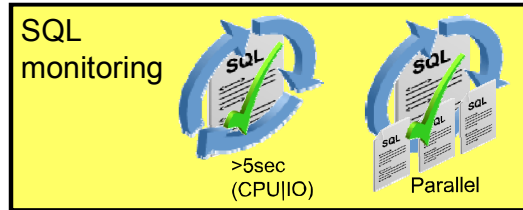
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

To monitor a database operation, the database operation must be given a name, a begin point, and an end point. There are two ways to name a database operation. The first is to insert a start point by using `DBMS_SQL_MONITOR.BEGIN_OPERATION` function and an end point by using `DBMS_SQL_MONITOR.END_OPERATION` procedure calls into the application. The second way is to set a tag (this is for JAVA and OCI applications). The tag can be set by using the OCI calls `OCIAttrSet` and `OCIAppCtxSet`, the Java call `setClientInfo`, or from a OS environment variable `ORA_DBOP`. The tag in Java or OCI piggybacks the next database call; so the begin point is easily distinguished.

The explicit calls provide a clear beginning and end of a database operation to provide the bracketing, but applications using tagging do not have an explicit method for ending an operation. Because starting a new operation ends the previous operation, the tag should be set to `NULL` and sent to the database.

# Enabling Monitoring of Database Operations

- At system level:
  - Set `STATISTICS_LEVEL` to `TYPICAL`
  - Set `CONTROL_MANAGEMENT_PACK_ACCESS` to `DIAGNOSTIC+TUNING`



- At database operation level:
  - Use `FORCED_TRACKING` attribute in `DBMS_SQL_MONITOR.BEGIN_OPERATION` function
- At statement level:
  - Use the `MONITOR` hint



MONITOR  
hint

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The SQL monitoring feature is enabled by default when the `STATISTICS_LEVEL` initialization parameter is either set to `TYPICAL` (the default value) or to `ALL`. Additionally, the `CONTROL_MANAGEMENT_PACK_ACCESS` parameter must be set to `DIAGNOSTIC+TUNING` (the default value) because SQL monitoring is a feature of the Oracle Database Tuning Pack. By default, SQL monitoring is started when a SQL command runs in parallel, or when it has consumed at least five seconds of the CPU or I/O time in a single execution. A SQL statement is also monitored when the `MONITOR` hint is explicitly declared in the statement.

When you are starting a database operation, you can force the database operation to be tracked when it starts. The default value of this parameter does not necessarily force any SQL statement or database operation to be tracked. The database operation is tracked only when it is sufficiently expensive. Use the `FORCED_TRACKING` attribute and set it to `Y` when starting the database operation with `DBMS_SQL_MONITOR.BEGIN_OPERATION` function:

```
DBMS_MONITOR.BEGIN_OPERATION (
    dbop_name IN VARCHAR2,
    dbop_eid IN NUMBER :=NULL,
    forced_tracking IN VARCHAR2 := NO_FORCE_TRACKING,
    attribute_list IN VARCHAR2 :=NULL)
RETURN NUMBER;
```

## Identifying, Starting, and Completing a Database Operation

1. Identify the database operation.
  - Operation Name
  - Execution ID
2. Start the database operation with  
DBMS\_SQL\_MONITOR.BEGIN\_OPERATION.
3. Complete the database operation with  
DBMS\_SQL\_MONITOR.END\_OPERATION.

```
SQL> VAR dbop_eid NUMBER;
SQL> EXEC :dbop_eid := DBMS_SQL_MONITOR.BEGIN_OPERATION
      ('ORA.MV.refresh', FORCED_TRACKING => 'Y')
SQL> SELECT ...
SQL> SELECT ...
SQL> EXEC DBMS_SQL_MONITOR.END_OPERATION
      ('ORA.MV.refresh', :dbop_eid)
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

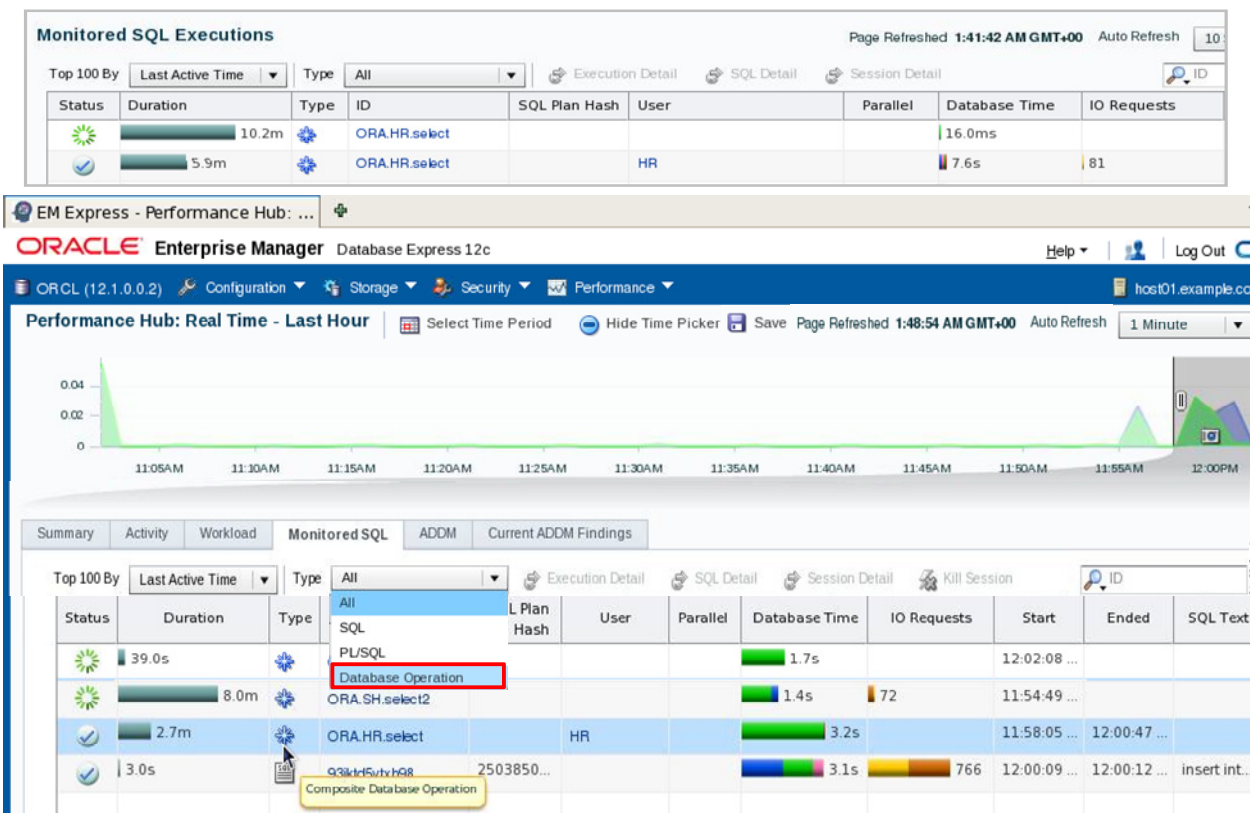
When monitoring, each database operation can be identified by the operation name and execution ID. Every DB operation has a name and an execution ID. The DB operation name and execution ID together uniquely identify a specific execution of the operation. The operation name is specific to a piece of code. The same code may be executed by multiple sessions or users at the same time. Each invocation will have the same operation name but different execution IDs. The execution ID is not unique between operations, that is, two DB operations with different names could have the same execution ID. The call to `BEGIN_OPERATION` returns the execution ID. If an execution ID is supplied to the call, it is returned. If the execution ID is not supplied, a unique execution ID is returned. In situations where a master process spawns several sessions that are associated with the same DB operation, an execution ID would be generated for the entire operation and each session would set the execution ID with a `BEGIN_OPERATION` call.

The database operation names are in a single namespace. A recommendation to prevent collisions is to use a format of `<component>.<subcomponent>.<operation_name>`. A suggestion is to use a component name of `ORA` for operations inside the database. The example in the slide is for an operation that performs a materialized view refresh.

Different executions of the same database operation (with the same name) can have different SQL statements or PL/SQL functions. This could be caused by different code paths being taken though conditions in PL/SQL. There is also nothing to prevent one operation name being assigned to multiple sets of code. This could cause confusion.



# Monitoring the Progress of a Database Operation



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Enterprise Manager Cloud Control and Enterprise Manager Database Express allow you to view database operations that are active and recently finished.

Using Enterprise Manager Cloud Control, log in to the target database and click the Performance menu, and then click the SQL Monitoring option.

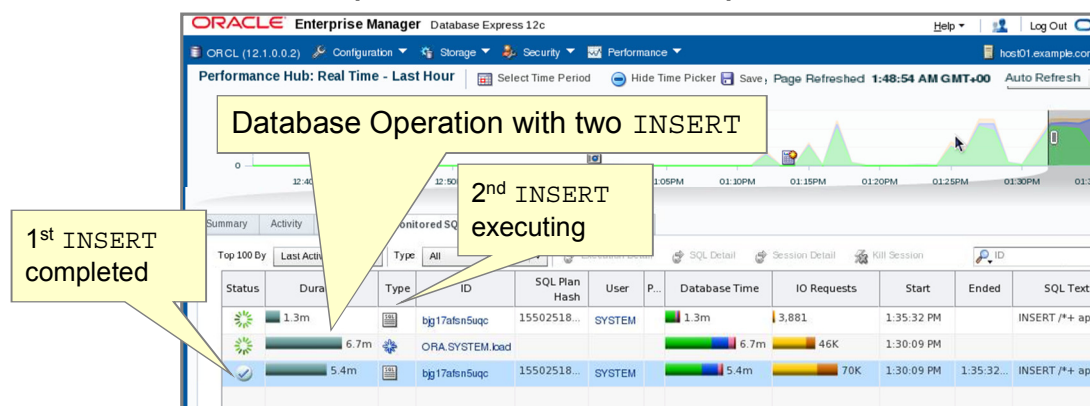
Using Enterprise Manager Database Express, in the Database Home page, the list of monitored SQL and database operations appears after going to Monitored SQL menu.

In the Performance menu, click the Performance Hub option to get the list of monitored SQL and database operations, as shown in the second screenshot of the slide. For details, click the database operation name. On the Details page, the execution ID is shown. By selecting the Database Operations filter above the list (SQL, PL/SQL and Database Operations), only database operations can be shown. Further filtering can be done on the operation name by using the search box on the right above the list.

# Monitoring Load Database Operations

```
SQL> VAR dbop_eid NUMBER;
SQL> EXEC :op_eid := DBMS_SQL_MONITOR.BEGIN_OPERATION
      ('ORA.SYSTEM.load', FORCED_TRACKING => 'Y')
SQL> INSERT /*+ APPEND */ * INTO tab1 SELECT ...;
SQL> INSERT /*+ APPEND */ * INTO tab2 SELECT ...;
SQL> EXEC DBMS_SQL_MONITOR.END_OPERATION
      ('ORA.SYSTEM.load', :op_eid)
```

One load is completed and the DB operation is still executing.



ORACLE

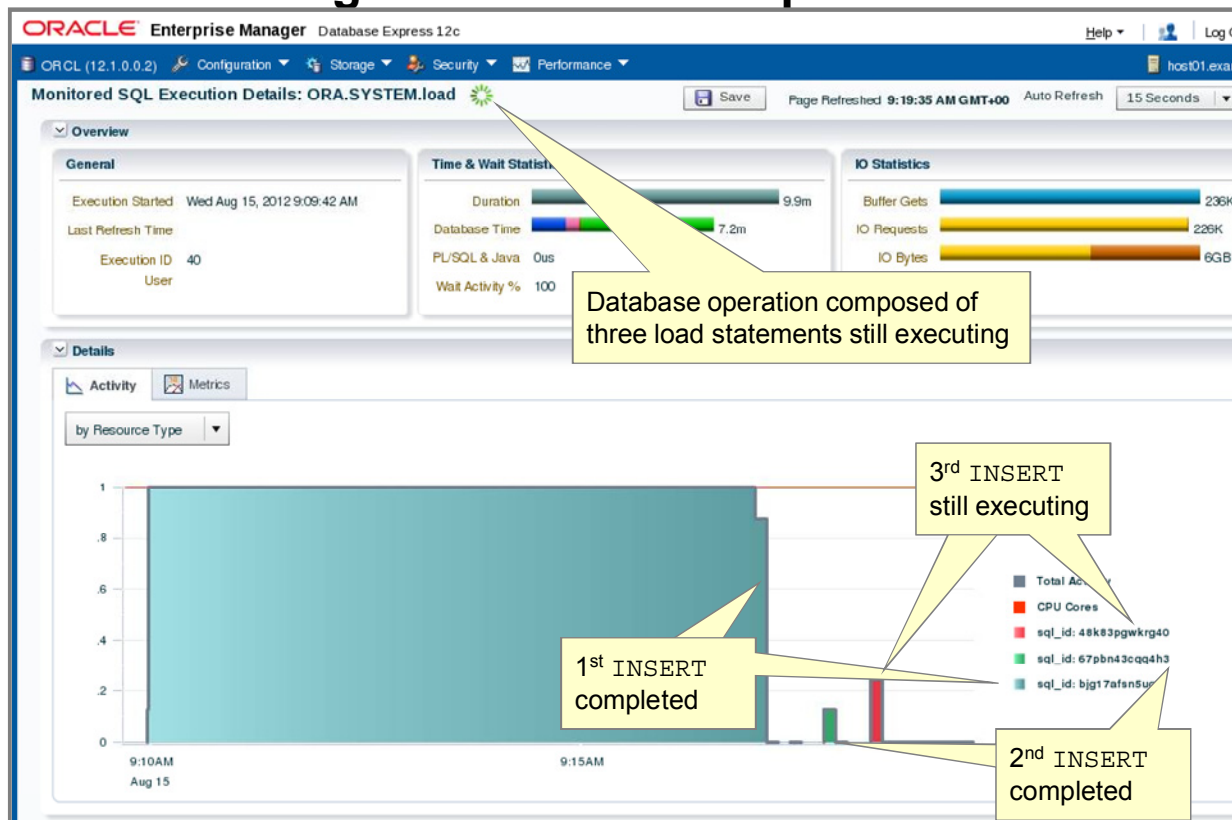
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Monitoring load operations can be performed. From the example shown in the slide, the database operation consists of two bulk load statements.

Using Enterprise Manager Database Express, from the Performance Hub page, you would find that the database load operation is still executing whereas the first bulk load statement is already completed. Another bulk load statement is still executing, being part of the same database load operation being monitored.

Using Enterprise Manager Cloud Control, log in to a target database, select Performance menu > SQL Monitoring option. You would see the same database load operation executing.

## Monitoring Load Database Operation Details



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The execution details page shows a summary at the top part of the screen and either the activity graph (shown), or the metrics graphs (not shown) at the bottom part of the screen. The `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR` and `DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST` functions are used to report execution details of real-time database operations.

## Reporting Database Operations Using Views

V\$SQL\_MONITOR view shows the list of executing and completed database operations through new columns.

```
SQL> SELECT DBOP_NAME, DBOP_EXEC_ID, STATUS
        FROM V$SQL_MONITOR
        WHERE DBOP_NAME IS NOT NULL;
```

| DBOP_NAME      | DBOP_EXEC_ID | STATUS    |
|----------------|--------------|-----------|
| ORA.HR.select  | 8            | EXECUTING |
| ORA.SH.select2 | 10           | EXECUTING |
| ORA.SH.select  | 3            | DONE      |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The V\$SQL\_MONITOR view reports the list of executing and completed database operations including global, high-level information about the top SQL statements in a database operation. Each monitored SQL statement has an entry in this view. Each row contains a SQL statement whose statistics are accumulated from multiple sessions and all of its executions in the operation.

The primary key is the combination of the columns DBOP\_NAME, DBOP\_EXEC\_ID, and SQL\_ID.

## Reporting Database Operations Using Views

- EM Cloud Control and EM Database Express use multiple views:
  - V\$SQL\_MONITOR, V\$ACTIVE\_SESSION\_HISTORY and new columns: DBOP\_NAME, DBOP\_EXEC\_ID
- AWR automatically captures SQL monitoring reports and stores active session history:
  - DBA\_HIST\_REPORTS, DBA\_HIST\_REPORTS\_DETAILS
  - DBA\_HIST\_ACTIVE\_SESS\_HISTORY

```
SQL> SELECT session_id, DBOP_NAME, DBOP_EXEC_ID
       FROM DBA_HIST_ACTIVE_SESS_HISTORY
       where DBOP_NAME is not NULL;
```

| SESSION_ID | DBOP_NAME       | DBOP_EXEC_ID |
|------------|-----------------|--------------|
| 30         | ORA.SYSTEM.load | 8            |
| 45         | ORA.HR.select   | 11           |

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

EM Cloud Control or EM Database Express summarize the activity for monitored statements and database operations collected from multiple views.

- Recently completed and active database operations from:
  - V\$SQL\_MONITOR, V\$ACTIVE\_SESSION\_HISTORY through new columns: DBOP\_NAME and DBOP\_EXEC\_ID
  - V\$SQL\_PLAN\_MONITOR, V\$SQL\_MONITOR\_SESSTAT, V\$SQL, V\$SQL\_PLAN, V\$SESSION\_LONGOPS
- Completed database operations and reports are stored in AWR and can be reported in the following views:
  - DBA\_HIST\_REPORTS, DBA\_HIST\_REPORTS\_DETAILS
  - DBA\_HIST\_ACTIVE\_SESS\_HISTORY

# Reporting Database Operations by Using Functions

DBMS\_SQL\_MONITOR package:

- REPORT\_SQL\_MONITOR\_LIST\_XML and REPORT\_SQL\_MONITOR\_XML functions report the list and details of database operations in XML format.
- REPORT\_SQL\_MONITOR\_LIST and REPORT\_SQL\_MONITOR functions report the list and details of database operations in a CLOB.

```
SQL> SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST_XML()
        FROM dual;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Reporting Using DBMS\_SQL\_MONITOR.REPORT\_SQL\_MONITOR\_\*\*\* Functions

```
SQL> select DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST_XML() from
dual;
```

```
DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST_XML()
```

```
-----
<report db_version="12.1.0.0.2" cpu_cores="1" hyperthread="N"
timezone_offset="0
" elapsed_time="0.00" cpu_time="0.00">
<report_id><![CDATA[/orarep/sqlmonitor/list?monitor_type=3]]></report_id>
  <sql_monitor_list_report version="1" sysdate="08/14/2012
10:33:04">
    <dbop detail="F" dbop_name="ORA.SH.select2"
dbop_exec_start="08/14/2012 10:26:11" dbop_exec_id="12">
      <status>EXECUTING</status>
      <first_refresh_time>08/14/2012 10:26:11</first_refresh_time>
```



```

<refresh_count>0</refresh_count>
  <inst_id>1</inst_id>
<session_id>43</session_id>
  <session_serial>229</session_serial>
  <user_id>0</user_id>
  <con_id>0</con_id>
    <module>SQL*Plus</module>
  <service>SYS$USERS</service>
  <program>sqlplus@host01.example.com (TNS V1-V3)</program>
  <plsql_entry_object_id>10531</plsql_entry_object_id>
  <plsql_entry_subprogram_id>6</plsql_entry_subprogram_id>
  <plsql_object_id>10375</plsql_object_id>
  <plsql_subprogram_id>63</plsql_subprogram_id>
  <is_cross_instance>N</is_cross_instance>
  <stats type="monitor">
    <stat name="duration">413</stat>
    <stat name="elapsed_time">3072887</stat>
    <stat name="cpu_time">1143825</stat>
    <stat name="user_io_wait_time">368882</stat>
    <stat name="application_wait_time">475</stat>
    <stat name="other_wait_time">1559705</stat>
    <stat name="buffer_gets">23</stat>
    <stat name="read_reqs">37</stat>
    <stat name="read_bytes">13033472</stat>
    <stat name="write_reqs">52</stat>
    <stat name="write_bytes">42041344</stat>
  </stats>
</dbop>

```

# Database Operation Tuning

- ADDM
- SQL Tuning Advisor
- SQL Performance Analyzer

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif font, centered within a red rectangular background.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The goal of tuning is to improve response time of a specific execution. Tuning database operation is more than just tuning SQL statements in the operation. But in Oracle Database 12c Release 1, you will consider that database operation tuning is to tune the Top SQL statements one by one.

Following are the tools you can use for SQL Tuning:

- The Automatic Database Diagnostic Monitor (ADDM): ADDM analyzes the information collected by the AWR for possible performance problems with Oracle Database, including high-load SQL statements.
- SQL Tuning Advisor and SQL Performance Analyzer: These tools can take SQL Tuning Sets or Top SQL statements as targets.  
To run SQL Tuning Advisor or SQL Performance Analyzer on SQL statements from a PDB, a common user must have the following privileges:
  - Common `SET CONTAINER` privilege or local `SET CONTAINER` privilege in the PDB
  - The privileges required to execute the SQL statements in the PDB



## Quiz

Monitoring database operations is allowed for sets of `SELECT` statements only.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: b**

## Quiz

You can define any set of SQL and PL/SQL statements (including multiple, concurrent sessions) between two points in time as a database operation to be monitored.

- a. True
- b. False

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

**Answer: a**

## Summary

In this lesson, you should have learned how to:

- Describe database operations
- Implement Real-Time Database Operation Monitoring

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

## Practice 14: Overview

- 14-1: Starting Enterprise Manager Database Express
- 14-2: Identifying and starting database operations (Using EM Database Express or EM Cloud Control)
- 14-3: Identifying and starting database load operations (*optional*)
- 14-4: Cleaning up

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

